

Yapay Zekâ Temelleri

Elektrik-Elektronik Mühendisliği

2026-2027 Güz Dönemi
Balıkesir Üniversitesi
Mühendislik Fakültesi

Dr. Öğr. Üyesi Sevdanur GENÇ

İçindekiler

İçindekiler	2
Yapay Zekâ Temelleri — Ders Notları	7
Dersin Amacı ve Öğrenme Çıktıları	7
Ön Koşullar ve Araçlar	7
Değerlendirme	7
15 Haftalık Ders Planı	8
Dönem Projesi	9
Kaynaklar	9
Hafta 1 — Yapay Zekâya Giriş	11
1.1 Kazanımlar	11
1.2 Yapay Zekâ Nedir?	11
1.3 Kısa Tarihçe	12
1.4 Klasik Programlama ile Makine Öğrenmesi Arasındaki Fark	13
1.5 Makine Öğrenmesinin Türleri (ilk bakış)	15
1.6 Bir Makine Öğrenmesi Projesinin Adımları	17
1.7 EEM'de Yapay Zekâ Uygulamaları	18
1.8 Sınırlar, Riskler ve Etik	19
1.9 Uygulama Saati — Google Colab'a İlk Adım	19
1.10 Sınıf İçi Sorular	21
1.11 Ödev / Hazırlık	21
Hafta 2 — Python ile Isınma	22
2.1 Kazanımlar	22
2.2 Neden Python?	22
2.3 Veri Tipleri ve Değişkenler	23
2.4 Koşullar ve Döngüler	24
2.5 Fonksiyonlar	25
2.6 NumPy: Sayısal Hesaplamanın Temeli	26
2.7 Hız Farkı: Döngü mü, Vektör mü?	30
2.8 Sık Yapılan Hatalar	30
2.9 Sınıf İçi Alıştırmalar	31
2.10 Ödev 1 — Python Isınma	31
Hafta 3 — Veri ile Tanışma: pandas ve Keşifsel Veri Analizi	32
3.1 Kazanımlar	32
3.2 Veri Nedir, Nasıl Görünür?	32
3.3 Tanımlayıcı İstatistik (Elle Örnekler)	33
3.4 pandas: Tablo Verisiyle Çalışmak	35
3.5 Eksik ve Aykırı Değerler	36
3.6 Görselleştirme ile Keşif	37

3.7 Ölçekleme: Neden Gerekli?	39
3.8 Nerede Veri Bulunur?	40
3.9 Sınıf İçi Alıştırmalar	41
3.10 Ödev 2 — EDA Raporu	41
Hafta 4 — Makine Öğrenmesinin Temel Kavramları	42
4.1 Kazanımlar	42
4.2 Öğrenme Türleri: Ayrıntılı Bakış.....	42
4.3 Genelleme ve Veri Ayrımı	43
4.4 Aşırı ve Eksik Öğrenme	44
4.5 Hata Ölçütleri (Regresyon)	46
4.6 Çapraz Doğrulama	48
4.7 Veri Kaçağı (Data Leakage)	48
4.8 Taban Model: Her Şeyden Önce	49
4.9 Sınıf İçi Sorular	50
4.10 Kısa Sınav 1 (15 dk, örnek sorular).....	51
4.11 Uygulama Saati ve Ödev	51
Hafta 5 — Doğrusal Regresyon.....	53
5.1 Kazanımlar	53
5.2 Model ve Maliyet Fonksiyonu	53
5.3 Gradyan İnişi.....	55
5.4 Çoklu Regresyon ve Matris Biçimi	56
5.5 Katsayıları Yorumlamak	57
5.6 Özellik Dönüşümü: Doğrusal Modelle Eğri Uydurmak	57
5.7 Düzenleştirme: Ridge ve Lasso	58
5.8 Uçtan Uca: Dersin Verisinde Taban Modeli Yenmek	59
5.9 Sınıf İçi Sorular	60
5.10 Ödev 3 — Fotovoltaik Üretim Tahmini	60
Hafta 6 — Sınıflandırma I: Lojistik Regresyon, k-NN ve Değerlendirme.....	63
6.1 Kazanımlar	63
6.2 Sınıflandırma Problemi	63
6.3 Lojistik Regresyon	63
6.4 k-En Yakın Komşu (k-NN)	65
6.5 Sınıflandırmayı Değerlendirmek	66
6.6 ROC Eğrisi ve Eşik Seçimi	68
6.7 Uygulama: Motor Arıza Verisi.....	69
6.8 Sınıf İçi Sorular	70
6.9 Kısa Sınav 2 (örnek sorular) ve Ödev	70
Hafta 7 — Sınıflandırma II ve Özellik Mühendisliği	72
7.1 Kazanımlar	72

7.2 Karar Ağaçları.....	72
7.3 Rastgele Orman	74
7.4 Destek Vektör Makineleri (SVM)	75
7.5 Kategorik Değişkenler: One-Hot Kodlama	76
7.6 Özellik Mühendisliği: Sinyalden Sayıya	77
7.7 Hiperparametre Arama ve Çok Sınıflı Değerlendirme	78
7.8 Dönem Projesi: Konu Bildirimi.....	79
7.9 Sınıf İçi Sorular	79
7.10 Ödev 4 — Rulman Arıza Sınıflandırma.....	80
Hafta 8 — Ara Sınav ve Denetimsiz Öğrenme	82
8.1 Ara Sınav	82
8.2 Kazanımlar	82
8.3 Denetimsiz Öğrenme Nedir?	83
8.4 k-Means Kümeleme.....	83
8.5 Hiyerarşik Kümeleme.....	85
8.6 Temel Bileşen Analizi (PCA)	85
8.7 Uygulama: Abone Yük Profilleri.....	87
8.8 Sınıf İçi Sorular	88
8.9 Uygulama Defteri ve Sonraki Hafta	89
Hafta 9 — Yapay Sinir Ağlarına Giriş.....	91
9.1 Kazanımlar	91
9.2 Biyolojik Esin ve Yapay Nöron.....	91
9.3 Perceptron ve Öğrenme Kuralı	92
9.4 Aktivasyon Fonksiyonları	93
9.5 XOR ve Gizli Katmanın Gücü	93
9.6 Çok Katmanlı Ağ ve İleri Yayılım	94
9.7 Evrensel Yaklaşım ve Mimari Seçimi.....	95
9.8 Uygulama.....	96
9.9 Sınıf İçi Sorular	97
9.10 Uygulama Defteri ve Sonraki Hafta	97
Hafta 10 — Geri Yayılım ve PyTorch'a Giriş.....	99
10.1 Kazanımlar	99
10.2 Zincir Kuralı ve Hesap Grafı.....	99
10.3 İki Katmanlı Ağda Geri Yayılım.....	100
10.4 Eğitim Döngüsü: Öğrenme Oranı, Batch, Epoch, Optimizasyon.....	101
10.5 PyTorch.....	103
10.6 Kaybolan Gradyanı Görmek.....	104
10.7 Sınıf İçi Sorular	105
10.8 Ödev 5 — Geri Yayılım ve PyTorch	105

Hafta 11 — Derin Öğrenme Pratiği.....	107
11.1 Kazanımlar	107
11.2 Çok Sınıflı Sınıflandırma: Softmax ve Çapraz Entropi	107
11.3 MNIST ve MLP.....	107
11.4 Öğrenme Eğrilerini Okumak	108
11.5 Düzenlileştirme Teknikleri	109
11.6 Başlangıç ve Batch Normalization	111
11.7 Hiperparametre Stratejisi	111
11.8 Uygulama Protokolü	112
11.9 Sınıf İçi Sorular	113
11.10 Kısa Sınav 3 (örnek) ve Uygulama.....	113
Hafta 12 — Evrişimli Sinir Ağları (CNN)	115
12.1 Kazanımlar	115
12.2 Görüntüde MLP Neden Yetersiz?	115
12.3 Konvolüsyon	115
12.4 Havuzlama	117
12.5 Bir CNN'in Anatomisi	117
12.6 Uygulama Verisi: PCB Kusur Tespiti.....	118
12.7 Veri Artırma ve Transfer Öğrenme	119
12.8 1-B Konvolüsyon: Sinyaller İçin CNN.....	121
12.9 Sınıf İçi Sorular	121
12.10 Ödev 6 — Görüntü Sınıflandırma	121
Hafta 13 — Zaman Serileri ve Tekrarlayan Sinir Ağları.....	123
13.1 Kazanımlar	123
13.2 Zaman Serisi: Bileşenler ve Otokorelasyon	123
13.3 Zaman Bölmesi, Sızıntı ve Naif Tabanlar.....	124
13.4 Gecikme Özellikleriyle Klasik Tahmin	125
13.5 Pencereleme: Diziden Tensöre.....	126
13.6 Tekrarlayan Sinir Ağları (RNN) ve Zaman Boyunca Geri Yayılım.....	127
13.7 LSTM ve GRU	127
13.8 Uygulama: Yük Tahmini — Naif → Ridge → Orman → LSTM.....	128
13.9 Artıklardan Anomali Tespiti	130
13.10 Hangi Model Ne Zaman?	130
13.11 Proje Ara Teslimi.....	131
13.12 Özet ve Gelecek Hafta	131
Hafta 14 — EEM'de Yapay Zekâ Uygulamaları, Gömülü YZ, Büyük Dil Modelleri, Güvenilirlik ve Etik	133
14.1 Kazanımlar	133
14.2 EEM'de YZ: Uygulama Haritası.....	133
14.3 Modelin Ömrü: Dağıtım, İzleme, Yeniden Eğitim	134

14.4 Gml YZ / TinyML: Modeli Mikrodenetleyiciye SıĖdırmak	135
14.5 Byk Dil Modelleri: Nasıl Çalıřır, Nasıl Kullanılır	137
14.6 Gvenilirlik: Kalibrasyon, Belirsizlik, Kayma, Dřmanca rnekler	138
14.7 Etik, Adillik, Gizlilik ve Dzenleme	140
14.8 zet ve Gelecek Hafta	141
Hafta 15 — Proje Sunumları, Dnem zeti ve Final Hazırlığı	142
15.1 Kazanımlar	142
15.2 Proje Sunumları	142
15.3 Dnem zeti: Kavram Haritası	143
15.4 Yntem Seim AĖacı	144
15.5 Forml ve lt zeti	145
15.6 Final Sınavı: Format ve rnek Sorular	145
15.7 Dersten Sonra: Yol Haritası	147
15.8 Ders zeti — Tek Sayfa	147

Yapay Zekâ Temelleri — Ders Notları

Bu belge, Elektrik-Elektronik Mühendisliği lisans öğrencileri için hazırlanmış 15 haftalık 'Yapay Zekâ Temelleri' dersinin müfredatını ve haftalık ders notlarını içerir. Ders haftada 3 saattir: ilk iki saat teori (tahtada elle çözülen sayısal örnekler dâhil), üçüncü saat bilgisayar başında Google Colab uygulamasıdır. Her hafta için bu notlara ek olarak bir Colab defteri (.ipynb) ve sade bir sunum dosyası (.pptx) bulunmaktadır; üçü birbirini tamamlayacak şekilde tasarlanmıştır: sunum derste anlatım için, notlar derinlemesine okuma için, defter ise uygulama ve ödev için.

Dersin temel felsefesi şudur: yapay zekâ, mühendislik problemlerini çözmek için bir araçtır; sihirli bir kara kutu değildir. Bu yüzden her yöntem önce küçük bir sayısal örnek üzerinde elle hesaplanır, sonra Python ile aynı hesap yapılır ve ancak ondan sonra hazır kütüphane çağrılır. Böylece öğrenci kütüphanenin içinde ne olduğunu bilerek kullanır.

Dersin Amacı ve Öğrenme Çıktıları

Bu dersin sonunda öğrenci:

1. Yapay zekâ, makine öğrenmesi ve derin öğrenme kavramlarını ayırt eder; bir mühendislik probleminin YZ ile çözülüp çözülemeyeceğini ve hangi tür öğrenmenin uygun olduğunu değerlendirir.
2. Python, NumPy, pandas ve matplotlib ile veri yükler, temizler, görselleştirir ve özetler.
3. Regresyon, sınıflandırma ve kümeleme algoritmalarının çalışma ilkesini elle küçük örnekler üzerinde hesaplar; scikit-learn ile uygular.
4. Yapay sinir ağlarının ileri ve geri yayılımını türetir; PyTorch ile basit bir ağ eğitir.
5. Bir modelin başarımını doğru ölçütlerle değerlendirir; aşırı öğrenme, veri kaçağı ve sınıf dengesizliği gibi tuzakları tanır ve önler.
6. EEM alanından (güç sistemleri, elektrik makineleri, sinyal işleme, kontrol, haberleşme, gömülü sistemler) bir problemi uçtan uca YZ ile çözen küçük bir proje geliştirir, raporlar ve sunar.

Ön Koşullar ve Araçlar

- Lineer cebir (vektör, matris çarpımı, transpoz, ters) ve temel olasılık/istatistik (ortalama, varyans, olasılık dağılımı) bilgisi — gerekli yerlerde hatırlatılacaktır.
- Programlama deneyimi ön koşul değildir; Python ilk üç haftada sıfırdan öğretilir. MATLAB bilenler için karşılaştırma tabloları verilir.
- Araçlar: Google Colab (ücretsiz, kurulum gerektirmez), Python 3, NumPy, pandas, matplotlib, scikit-learn, PyTorch.
- Öğrencilerin bir Google hesabı olması yeterlidir; kendi bilgisayarına kurulum isteyenler için Anaconda alternatifi 2. haftada anlatılır.

Değerlendirme

Bileşen	Ağırlık	Açıklama
Ödevler (6 adet)	%25	Colab defteri olarak teslim; her biri eşit ağırlıklı; 'Restart and run all' ile hatasız çalışmalı
Kısa sınavlar (3 adet)	%10	Ders başında 15 dakika; önceki iki haftanın kavramları ve küçük bir elle hesap

Bileşen	Ağırlık	Açıklama
Ara sınav	%25	8. hafta; kavramsal sorular + elle sayısal çözüm (regresyon, sınıflandırma ölçütleri)
Dönem projesi	%20	2-3 kişilik gruplar; EEM problemi, veri seti, rapor (6-8 sayfa) + 10 dakikalık sunum + çalışan kod
Final sınavı	%20	Tüm konular; ağırlık 9-14. haftalarda

15 Haftalık Ders Planı

Hafta	Konu	İçerik	Uygulama (Colab)	Değerlendirme
1	Yapay Zekâya Giriş	YZ/MÖ/DÖ tanımları, tarihçe, yaklaşımlar, EEM uygulama alanları, etik	Colab ortamına ilk adım: hesap, hücre, ilk kod	—
2	Python ile Isınma	Değişkenler, listeler, sözlükler, koşullar, döngüler, fonksiyonlar, NumPy dizileri	Sinyal dizisi üretme, RMS/ortalama hesabı, direnç hesapları, ilk grafik	Ödev 1 (Python)
3	Veri ile Tanışma	Veri türleri, tanımlayıcı istatistik, pandas, CSV yükleme, eksik/aykırı değer, keşifsel veri analizi (EDA), ölçekleme	Elektrik tüketimi veri setinde uçtan uca EDA	Ödev 2 (EDA raporu)
4	Makine Öğrenmesinin Temel Kavramları	Denetimli/denetimsiz/pekiştirmeli öğrenme, eğitim-doğrulama-test ayrımı, aşırı/eksik öğrenme, yanlılık-varyans, hata ölçütleri, veri kaçağı	train_test_split, taban (baseline) model, MAE/RMSE/R ² hesabı	Kısa sınav 1
5	Doğrusal Regresyon	En küçük kareler, maliyet fonksiyonu, normal denklem, gradyan inişi, çoklu regresyon, özellik dönüşümü, düzenleme (ridge/lasso)	Yük tahmini: elle çözüm → NumPy → scikit-learn	Ödev 3
6	Sınıflandırma I	Lojistik regresyon, sigmoid, karar sınırı, k-en yakın komşu, karışıklık matrisi, precision/recall/F1, ROC-AUC, sınıf dengesizliği	Arıza var/yok sınıflandırması	Kısa sınav 2
7	Sınıflandırma II ve Özellik Mühendisliği	Karar ağaçları, rastgele orman, SVM'e giriş, ölçekleme, one-hot kodlama, çapraz doğrulama, hiperparametre arama	Motor titreşim verisiyle arıza tipi sınıflandırma (CWRU)	Ödev 4 / Proje konusu bildirimi
8	Ara Sınav + Denetimsiz Öğrenme	k-means, dirsek yöntemi, hiyerarşik kümeleme, PCA ile boyut indirgeme ve görselleştirme	Yük profillerini kümeleme, PCA görselleştirme	ARA SINAV
9	Yapay Sinir Ağlarına Giriş	Biyolojik esin, perceptron, aktivasyon fonksiyonları, ileri yayılım (elle hesap), XOR problemi, evrensel yaklaşım	NumPy ile 2 katmanlı ağ ileri yayılımı	—
10	Geri Yayılım ve PyTorch'a Giriş	Zincir kuralı, gradyan hesabı (elle 2 nöronlu örnek), öğrenme oranı, tensörler, autograd, eğitim döngüsü	PyTorch ile ilk eğitim döngüsü	Ödev 5
11	Derin Öğrenme Pratiği	Kayıp fonksiyonları, SGD/Adam, mini-batch, dropout, batch norm, erken durdurma, öğrenme eğrileri, hiperparametreler	MNIST el yazısı rakam sınıflandırma	Kısa sınav 3

Hafta	Konu	İçerik	Uygulama (Colab)	Değerlendirme
12	Evrişimli Sinir Ağları (CNN)	Konvolüsyon (sinyal işleme bağlantısı), filtreler, padding/stride, havuzlama, klasik mimariler, transfer öğrenme	PCB/görüntü kusur tespiti veya CIFAR-10 küçük CNN	Ödev 6
13	Zaman Serileri ve Ardışık Veriler	Pencereleme, gecikmeli özellikler, RNN/LSTM/GRU sezgisi, sensör verisi, yük ve sıcaklık tahmini, 1-B CNN	LSTM ile saatlik tüketim tahmini	Proje ara teslimi
14	EEM'de YZ Uygulamaları, Etik ve Güncel Konular	Akıllı şebeke, kestirimci bakım, TinyML/gömülü YZ (nicemleme, budama), üretken YZ ve LLM'ler, güvenilirlik, açıklanabilirlik, etik	Edge cihaz için model küçültme (quantization) denemesi	—
15	Proje Sunumları ve Genel Tekrar	Dönem projelerinin sunumu, genel tekrar, final hazırlığı	Proje sunumları	PROJE TESLİMİ

Not: Haftalık 3 saatlik yapı: 1. ve 2. saat teori (tahtada elle çözümlü örnekler dâhil), 3. saat bilgisayar başında Colab uygulaması. İlk üç hafta bilinçli olarak sözel ağırlıklıdır ve öğrencileri Colab-veri seti-Python üçlüsüne ısındırır. Dönem boyunca aynı sentetik 'elektrik tüketimi' veri seti farklı yöntemlerle tekrar tekrar kullanılır; böylece öğrenci yöntemleri aynı problem üzerinde karşılaştırabilir.

Dönem Projesi

Proje, 7. haftada konu bildirimi, 13. haftada ara teslim ve 15. haftada sunum ile üç aşamada yürütülür. Öğrenciler kendi ölçümlerini toplayabilir (Arduino/ESP32 sensör kayıtları, osiloskop CSV çıktıları) ya da açık bir veri seti kullanabilir. Değerlendirme ölçütleri: problem tanımının netliği (%20), veri işleme ve keşif (%20), model seçimi ve doğru değerlendirme (%30), sonuçların mühendislik yorumu (%20), sunum ve kod kalitesi (%10).

Örnek proje konusu	Veri	Yöntem
Bir binanın saatlik yük tahmini	Sayaç + hava durumu	Regresyon / LSTM
Asenkron motor rulman arızası teşhisi	CWRU titreşim verisi	Özellik çıkarımı + SVM / 1-B CNN
Fotovoltaik üretim tahmini	Işınım, sıcaklık, geçmiş üretim	Rastgele orman / sinir ağı
Güç kalitesi bozukluğu sınıflandırma	Sentetik gerilim dalga şekilleri	CNN
ESP32 ile titreşim anomali tespiti (TinyML)	Kendi IMU kayıtları	Otokodlayıcı + nicemleme
PCB lehim kusuru tespiti	Kamera görüntüleri	Transfer öğrenme (CNN)

Kaynaklar

- Russell & Norvig, Artificial Intelligence: A Modern Approach (giriş bölümleri).
- Géron, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow.
- Goodfellow, Bengio, Courville, Deep Learning (ücretsiz çevrimiçi: deeplearningbook.org).
- Python resmi öğreticisi (docs.python.org/tr), NumPy ve pandas belgeleri, scikit-learn kullanıcı kılavuzu.

- Veri setleri: UCI Machine Learning Repository, Kaggle, scikit-learn gömülü veri setleri, CWRU rulman verisi, dersin kendi sentetik elektrik tüketimi veri seti.

Hafta 1 — Yapay Zekâya Giriş

Bölüm	Süre	İçerik
Saat 1	50 dk	Yapay zekâ nedir? Tanımlar, tarihçe, YZ ↔ MÖ ↔ DÖ ilişkisi
Saat 2	50 dk	Klasik programlama ile öğrenmenin farkı, öğrenme türleri, MÖ iş akışı, EEM'de uygulama alanları, sınırlar ve etik
Saat 3	50 dk	Uygulama: Google Colab'a ilk adım — hücreler, ilk Python komutları, hata okuma, defter kaydetme

Tablo 1.1 Haftanın ders planı

1.1 Kazanımlar

- Yapay zekâ, makine öğrenmesi ve derin öğrenme terimlerini doğru kullanır; birbirleriyle ilişkisini çizer.
- Kural tabanlı programlama ile veriden öğrenme arasındaki farkı bir örnekle açıklar; hangi problemin hangisine uygun olduğunu değerlendirir.
- Denetimli, denetimsiz ve pekiştirmeli öğrenmeyi ayırt eder.
- Bir makine öğrenmesi projesinin adımlarını sıralar ve zamanın nerede geçtiğini bilir.
- EEM alanında YZ'nin kullanıldığı en az beş uygulama sayar; her biri için girdi ve çıktığı söyler.
- Google Colab'da bir defter açar, kod hücresi çalıştırır, hata mesajı okur ve defteri kaydeder.

1.2 Yapay Zekâ Nedir?

Yapay zekâ (YZ), normalde insan zekâsı gerektirdiğini düşündüğümüz işleri — algılama, akıl yürütme, öğrenme, karar verme, dil kullanma — yapabilen bilgisayar sistemleri geliştirme çabasıdır. 'Zekâ' kelimesinin kendisi tartışmalı olduğundan, alanın kurucuları da tek bir tanım üzerinde uzlaşamamıştır. Russell ve Norvig'in ünlü ders kitabı, tarihsel tanımları iki eksenle dörde ayırır: sistemin insan gibi mi yoksa akılcı (rasyonel) mi olması hedefleniyor; ve düşünce süreci mi yoksa davranış mı ölçülüyor?

Yaklaşım	Ölçüt	Örnek
İnsan gibi davranan	Turing testi: bir yargıç, yazışmada makineyi insandan ayıramıyorsa	Sohbet botları, LLM tabanlı asistanlar
İnsan gibi düşünen	Bilişsel modelleme: insan zihninin işleyişini taklit	Bilişsel bilim modelleri, bazı sinir ağı yorumları
Akılcı düşünen	Mantık kuralları ile doğru çıkarım ('düşünce yasaları')	Uzman sistemler, teorem ispatlayıcılar
Akılcı davranan	Verilen bilgiyle hedefine en iyi ulaşan 'akılcı ajan'	Otonom araç, satranç programı, MPPT kontrolcüsü

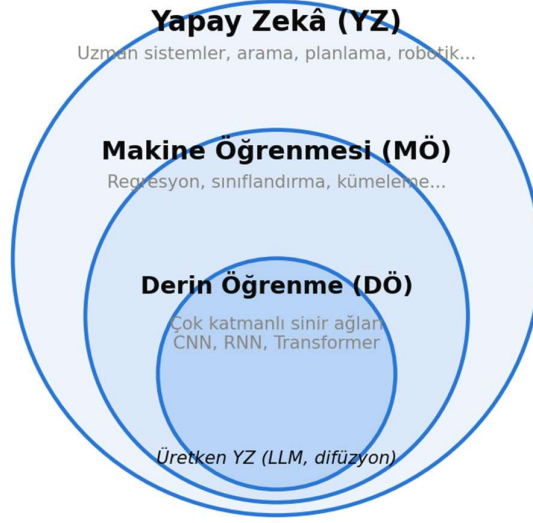
Tablo 1.2 Yapay zekâya dört bakış (Russell & Norvig)

Bu derste ağırlıklı olarak 'akılcı davranan ajan' bakışını benimseyeceğiz: bir sistemin zeki sayılması için insan gibi düşünmesi gerekmez; eldeki bilgiyle hedefine en iyi biçimde ulaşması yeterlidir. Bu bakış mühendislik için en verimlisidir, çünkü 'hedef' ve 'en iyi' kelimeleri ölçülebilir bir başarımlı ölçütüne (hata, doğruluk, maliyet) çevrilebilir. Makine öğrenmesi de tam olarak böyle tanımlanır: verilen bir görevde, deneyim (veri) arttıkça ölçülen başarımlı artan program.

Düşünelim: Turing testinin bir tuzağı vardır: insanı taklit etmek ile faydalı olmak aynı şey değildir. Bir uçak kuş gibi kanat çırpamaz; yine de uçar. Aynı şekilde bir yük tahmin modeli 'insan gibi' düşünmez; sadece daha iyi tahmin eder.

YZ, Makine Öğrenmesi ve Derin Öğrenme

Günlük dilde bu üç terim birbirinin yerine kullanılır; oysa iç içe kümelerdir. Yapay zekâ en geniş şemsiyedir ve içinde öğrenmeyen yöntemler de (arama algoritmaları, mantıksal çıkarım, planlama) vardır. Makine öğrenmesi, kuralların veriden çıkarıldığı alt kümedir. Derin öğrenme ise makine öğrenmesinin çok katmanlı yapay sinir ağlarını kullanan alt kümesidir; son on yılın görüntü, ses ve dil alanındaki sıçramaları buradan gelmiştir. Üretken yapay zekâ (LLM'ler, difüzyon modelleri) derin öğrenmenin en yeni dalıdır.



Şekil 1.1 İç içe kümeler: derin öğrenme, makine öğrenmesinin; makine öğrenmesi de yapay zekânın bir alt alanıdır.

Terim	Tanım	EEM örneği
Yapay Zekâ (YZ)	Zeki davranış üreten her tür yöntem: arama, mantık, planlama, öğrenme	Robot kol için yol planlama (A* araması)
Makine Öğrenmesi (MÖ)	Kuralları elle yazmak yerine örnek veriden öğrenen algoritmalar	Geçmiş ölçümlerden trafo arızasını tahmin
Derin Öğrenme (DÖ)	Çok katmanlı yapay sinir ağları ile öğrenme; ham veriden özellik çıkarır	Termal kamera görüntüsünden pano kusuru tespiti
Üretken YZ	Yeni metin, görüntü, ses üreten DÖ modelleri (LLM, difüzyon)	Devre şeması açıklaması yazan asistan; sentetik eğitim verisi üretimi

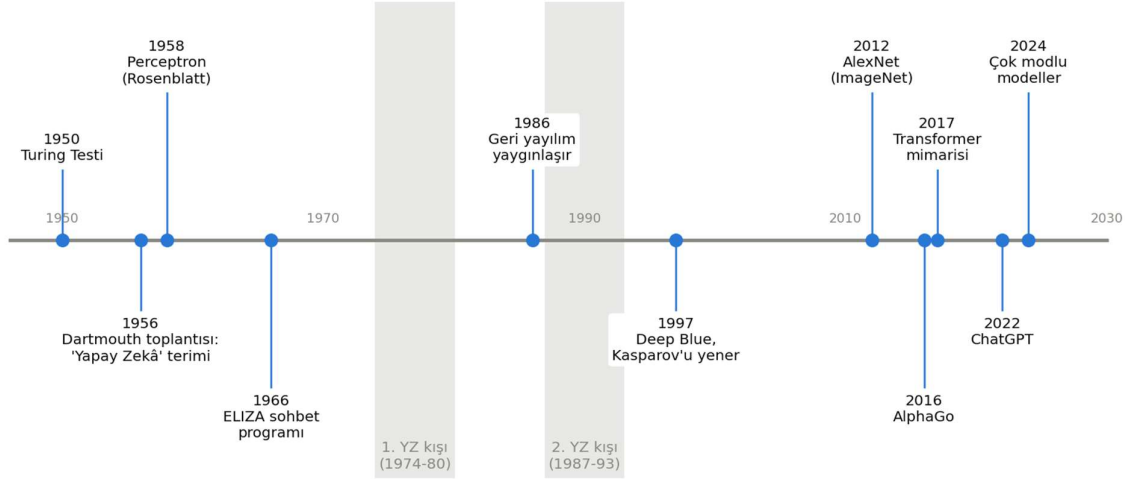
Tablo 1.3 Temel terimler

1.3 Kısa Tarihçe

Yapay zekânın tarihi, büyük vaatler ile hayal kırıklıklarının birbirini izlediği dalgalı bir hikâyedir. Alan 1956'da Dartmouth toplantısında isim aldı; iki yıl sonra Frank Rosenblatt'ın perceptron'u ilk öğrenen makine olarak manşetlere çıktı. 1960'larda ELIZA gibi basit sohbet programları insanları etkiledi. Ancak dönemin bilgisayarları yavaş, veri kıt ve algoritmalar sınırlıydı; 1969'da Minsky ve Papert perceptron'un XOR gibi basit problemleri çözemediğini gösterince tek katmanlı ağlara olan ilgi söndü ve ilk 'YZ kışı' başladı.

1980'lerde uzman sistemler (elle yazılmış binlerce 'eğer–ise' kuralı) sanayide ilk ticari uygulamaları buldu; aynı dönemde geri yayılım algoritması çok katmanlı ağların eğitilebileceğini gösterdi. Fakat uzman sistemlerin bakımı pahalı ve kırılgandı; ikinci kış geldi. 1990'lar ve 2000'ler istatistiksel öğrenmenin (destek

vektör makineleri, olasılıksal modeller) ve büyük veri altyapısının sessizce biriktiği yıllardı. 1997'de Deep Blue'nun Kasparov'u yenmesi arama gücünün, 2012'de AlexNet'in ImageNet yarışmasını ezici farkla kazanması ise derin öğrenmenin zaferiydi. O andan sonra GPU'lar, büyük etiketli veri setleri ve açık kaynak kütüphaneler bir araya geldi; 2017'deki Transformer mimarisi ve 2022'deki ChatGPT bu dalganın en görünür ürünleridir.



Şekil 1.2 Yapay zekânın kilometre taşları ve iki 'YZ kışı'

Dönem	Öne çıkanlar	Neden önemli?
1950-1970	Turing testi (1950), Dartmouth toplantısı (1956), perceptron (1958), ELIZA (1966)	Alan doğdu; ilk öğrenen makine ve ilk sohbet programı
1974-1980	1. YZ kışı	Beklentiler karşılanmadı; hesaplama gücü ve veri yetersizdi; XOR eleştirisi; fonlar kesildi
1980'ler	Uzman sistemler, geri yayılım algoritmasının yaygınlaşması (1986)	Sanayide ilk ticari uygulamalar; çok katmanlı ağlar eğitilebilir oldu
1987-1993	2. YZ kışı	Uzman sistemlerin bakım maliyeti ve kırılganlığı; özel donanım pazarının çöküşü
1997-2010	Deep Blue (1997), SVM, olasılıksal modeller, büyük veri	Veri + hesaplama gücü birikmeye başladı; internetten etiketli veri
2012-	AlexNet (2012), AlphaGo (2016), Transformer (2017), ChatGPT (2022), çok modlu modeller (2024)	GPU'lar ve büyük veri setleri derin öğrenme devrimini başlattı

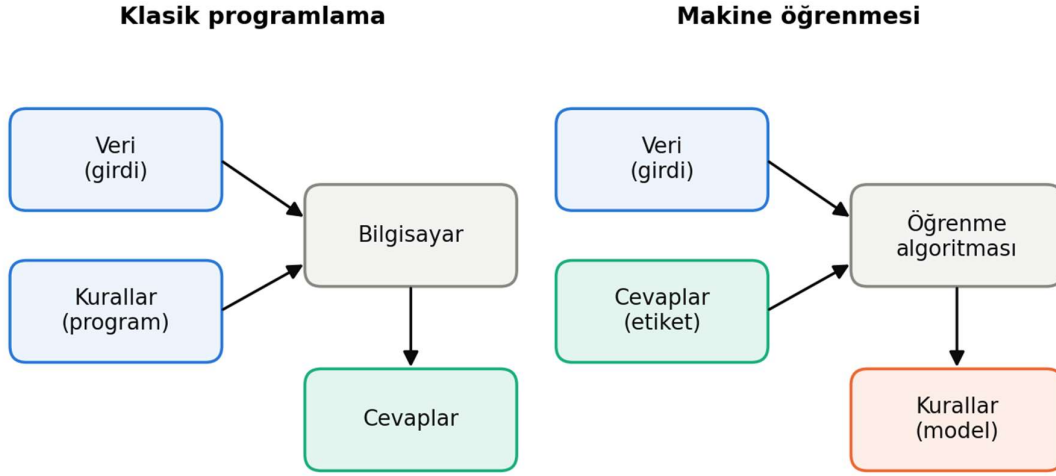
Tablo 1.4 Tarihçe özeti

Sınıfta tartışın: Tartışma sorusu: Perceptron 1958'de vardı, geri yayılım 1986'da biliniyordu. Derin öğrenme neden 2012'de 'patladı'? Cevap üç bileşenli: (1) veri — ImageNet'te 1.2 milyon etiketli görüntü, (2) donanım — oyun kartları (GPU) matris çarpımını 50-100 kat hızlandırdı, (3) algoritma — ReLU aktivasyonu, dropout gibi küçük ama kritik iyileştirmeler. Üçü birden gerekti. Bu ders için de mesaj aynı: iyi algoritma tek başına yetmez; veri ve hesaplama gücü de gerekir.

1.4 Klasik Programlama ile Makine Öğrenmesi Arasındaki Fark

Klasik programlamada mühendis problemi analiz eder, çözüm kurallarını yazar ve bilgisayar bu kuralları veriye uygulayarak cevabı üretir. Bir sigortanın çalışma mantığı budur: 'akım 16 amperi geçerse devreyi kes'. Kural açık, az sayıda ve sabittir. Makine öğrenmesinde ise akış tersine döner: bilgisayara veri ve o veriye ait

doğru cevaplar (etiketler) verilir; öğrenme algoritması veriden cevaba giden kuralları — yani modeli — kendisi bulur. Elde edilen model daha sonra klasik bir program gibi yeni veriye uygulanır.



Şekil 1.3 Klasik programlamada kuralları biz yazarız; makine öğrenmesinde kuralları (modeli) algoritma veriden çıkarır.

Bir e-posta spam filtresi düşünelim. Klasik yaklaşımda 'içinde "bedava" geçen ve 3'ten fazla ünlem içeren mesajlar spamdır' gibi kuralları elle yazarız. Spam gönderenler kuralları öğrenip 'b.e.d.a.v.a' yazınca kural bozulur; her yeni hile için yeni kural eklemek gerekir ve kural listesi bakımı imkânsız hâle gelir. Makine öğrenmesinde ise binlerce etiketli örnek (spam / değil) verilir; algoritma hangi kelime ve kalıpların spam ile ilişkili olduğunu kendisi öğrenir ve yeni örneklerle güncellenebilir. Aynı mantık EEM'e de taşınır: bir rulman arızasının titreşim sinyalinde nasıl görüldüğünü elle formüle etmek çok zordur; ama sağlam ve arızalı rulmanlardan yüzlerce kayıt varsa, model bu farkı öğrenebilir.

	Klasik programlama	Makine öğrenmesi
Girdi	Veri + kurallar	Veri + doğru cevaplar (etiketler)
Çıktı	Cevaplar	Kurallar (model)
Ne zaman uygun?	Kurallar açık, az sayıda ve sabit	Kurallar karmaşık, çok sayıda, değişken; ama bol örnek var
EEM örneği	Sigorta: $I > 16$ A ise devreyi kes; PID kontrol	Titreşim sinyalinden rulman arızasını erken tespit; yük tahmini
Zayıf yanı	Her durumu öngörmek imkânsız; kural bakımı zor	Veri kalitesine bağımlı; kararları açıklamak zor; veri dışına genelleme zayıf
Kesinlik	Belirlenimci: aynı girdi → aynı çıktı, gerekçe belli	İstatistiksel: tahminler hatalı olabilir, güven düzeyi verir

Tablo 1.5 İki yaklaşımın karşılaştırması

Bu iki yaklaşım rakip değil, tamamlayıcıdır. Gerçek sistemlerde makine öğrenmesi modeli genellikle klasik kurallarla sarmalanır: model bir arıza olasılığı üretir, ama devreyi kesen karar hâlâ belirlenimci bir koruma kuralıdır. Kural ne zaman yazılır, model ne zaman öğrenilir sorusunun pratik cevabı şudur: problemi bir sayfada kurallarla anlatabiliyorsanız kural yazın; anlatamıyorsanız ama bol örneğiniz varsa öğrenin.

Örnek 1.1 — Kural yazmak neden ölçeklenmez? (sayısal)

Soru: 28×28 piksellik siyah-beyaz bir görüntüde el yazısı rakamı tanıyan bir program yazmak istiyoruz. Her piksel 0 veya 1 olsun. Her olası görüntü için 'bu hangi rakam?' kuralını elle yazmak istesek kaç farklı görüntü ile karşılaşabiliriz?

Çözüm: Piksel sayısı $28 \times 28 = 784$. Her piksel 2 değer alabildiğinden olası görüntü sayısı $2^{784} \approx 10^{236}$. Evrendeki atom sayısının ($\approx 10^{80}$) çok çok üstünde. Dolayısıyla 'her durum için bir kural' yazmak imkânsızdır; genelleme yapabilen bir modele ihtiyaç vardır. Makine öğrenmesi tam olarak bunu yapar: sınırlı sayıda örnekten (MNIST veri setinde 60.000 görüntü) görmediği örneklerle genelleyen bir fonksiyon öğrenir. $60.000 / 10^{236}$ oranı sıfıra o kadar yakındır ki, modelin 'ezberlemek' yerine 'genellemek' zorunda olduğu açıktır.

Python ile doğrulama

```
piksel = 28 * 28          # 784
olası_goruntu = 2 ** piksel
print(piksel)
print(len(str(olası_goruntu)), "basamaklı bir sayı")    # 237 basamak
```

1.5 Makine Öğrenmesinin Türleri (ilk bakış)

Makine öğrenmesi yöntemleri, algoritmaya ne tür bir geri bildirim verildiğine göre üçe ayrılır. Denetimli öğrenmede her örneğin doğru cevabı (etiketi) bilinir ve model girdiden bu cevabı üretmeyi öğrenir; cevap sürekli bir sayıysa (kW, °C) regresyon, sınırlı sayıda kategoriye (arızalı/sağlam) sınıflandırma denir. Denetimsiz öğrenmede etiket yoktur; algoritma verinin içindeki yapıyı — benzer örneklerin kümelerini, verinin daha az boyutla anlatılabilecek özünü — bulur. Pekiştirmeli öğrenmede ise model bir ortamda eylemler yapar ve yalnızca gecikmeli bir ödül sinyali alır; deneme-yanılma ile ödülü en çoklayan stratejiyi öğrenir. Ayrıntılar 4. haftada; şimdilik üçünü tanımak ve ayırt etmek yeterli.

Tür	Ne verilir?	Ne öğrenilir?	EEM örneği
Denetimli öğrenme	Girdi + doğru çıktı (etiket)	Girdiden çıktıyı tahmin eden fonksiyon	Sıcaklık ve saatten elektrik yükü tahmini (regresyon); motor sesinden arıza var/yok (sınıflandırma)
Denetimsiz öğrenme	Sadece girdi	Verideki yapı: kümeler, düşük boyutlu temsil, anomaliler	Abonelerin tüketim profillerine göre gruplanması; alışılmadık sensör davranışının tespiti
Pekiştirmeli öğrenme	Ortam + ödül sinyali	Ödülü en çoklayan eylem stratejisi (politika)	Bir invertörün MPPT kontrolünü deneme-yanılma ile öğrenmesi; batarya şarj/deşarj planlama

Tablo 1.6 Öğrenme türleri

Denetimli öğrenmede veri bir tablo gibi düşünülür: her satır bir örnek (gözlem), her sütun bir özellik (feature), son sütun ise tahmin edilecek hedef (target). Aşağıdaki küçük tabloda üç özellik (sıcaklık, saat, hafta sonu) ve bir hedef (tüketim) var. Modelin işi, X sütunlarından y sütununa giden ilişkiyi bulmaktır.

Özellikler (girdi X)

Hedef (çıktı y)

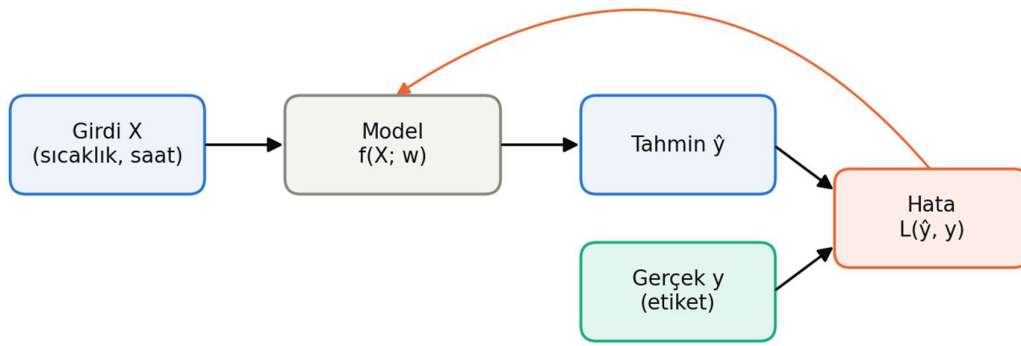
Sıcaklık (°C)	Saat	Hafta sonu	Tüketim (kW)
24.1	9	0	118.3
31.5	18	0	205.7
19.8	3	1	42.0
28.0	14	1	121.9

Her satır bir örnek (gözlem); her sütun bir özellik.

Şekil 1.4 Denetimli öğrenmede veri: satırlar örnek, sütunlar özellik; son sütun tahmin edilecek hedef

Peki 'öğrenme' fiilen nasıl olur? Model, ayarlanabilir parametrelere (ağırlıklara, w) sahip bir fonksiyondur: $\hat{y} = f(X; w)$. Başlangıçta parametreler rastgeledir ve tahminler kötüdür. Her örnek için tahmin ile gerçek değer karşılaştırılır, aradaki fark bir hata (kayıp) fonksiyonu ile ölçülür ve parametreler hatayı azaltacak yönde küçük adımlarla güncellenir. Bu döngü binlerce kez tekrarlanınca hata düşer; işte 'öğrenme' bu döngüdür. 5. ve 10. haftalarda bu döngünün matematiğini (gradyan inişi, geri yayılım) ayrıntılı göreceğiz.

Hatayı azaltacak yönde parametreleri (w) güncelle — 'öğrenme' budur



Şekil 1.5 Denetimli öğrenme döngüsü: tahmin et, hatayı ölç, parametreleri güncelle, tekrar et

Örnek 1.2 — 'Öğrenme' sezgisi: en yakın komşuya bakmak

Elimizde dört günlük ölçüm var. Yeni bir gün için sıcaklık 26 °C ve saat 15 iken tüketimi tahmin etmek istiyoruz. En basit 'öğrenme' yöntemi: geçmişte en benzer örneği bulup onun cevabını kullanmak. Bu yöntemin parametresi yoktur; 'model' verinin kendisidir. Yine de bir öğrenme yöntemidir çünkü veri arttıkça tahminler iyileşir.

Örnek	Sıcaklık (°C)	Saat	Tüketim (kW)
A	24	9	118
B	31	18	206
C	20	3	42
D	28	14	122

Benzerlik için Öklid uzaklığı kullanalım: $d = \sqrt{(\Delta \text{Sıcaklık})^2 + (\Delta \text{Saat})^2}$.

- A: $\sqrt{(26-24)^2 + (15-9)^2} = \sqrt{4 + 36} = \sqrt{40} \approx 6.32$

- B: $\sqrt{((26-31)^2 + (15-18)^2)} = \sqrt{(25 + 9)} = \sqrt{34} \approx 5.83$
- C: $\sqrt{((26-20)^2 + (15-3)^2)} = \sqrt{(36 + 144)} = \sqrt{180} \approx 13.42$
- D: $\sqrt{((26-28)^2 + (15-14)^2)} = \sqrt{(4 + 1)} = \sqrt{5} \approx 2.24 \leftarrow \text{en yakın}$

En yakın örnek D olduğundan tahmin ≈ 122 kW. Bu yöntem (k-en yakın komşu) 6. haftada ayrıntılı işlenecek. Dikkat: sıcaklık ve saat farklı birimlerde; sıcaklık Fahrenheit yazılıysaydı (68, 88, 68, 82 °F) sıcaklık farkları 1.8 kat büyür ve saat farkları önemsizleşirdi. Ölçekleme yapılmazsa bir özellik diğerini bastırabilir — 3. ve 7. haftada göreceğiz.

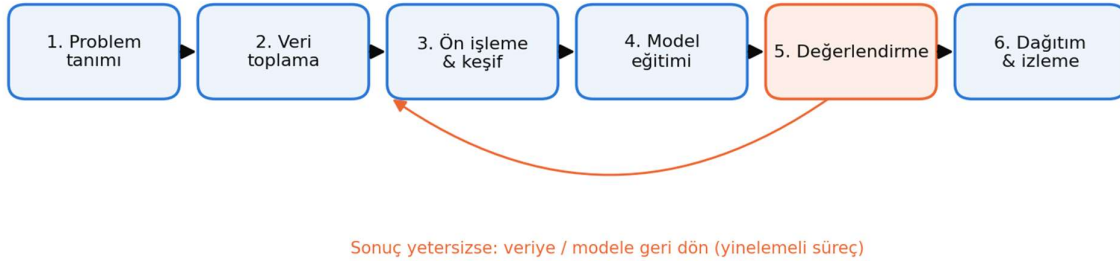
Aynı hesabın Python'u (2. haftadan sonra tamamen anlaşılacak)

```
import numpy as np
X = np.array([[24, 9], [31, 18], [20, 3], [28, 14]]) # sıcaklık, saat
y = np.array([118, 206, 42, 122]) # tüketim (kW)
yeni = np.array([26, 15])

uzakliklar = np.sqrt(((X - yeni) ** 2).sum(axis=1))
print("Uzaklıklar:", uzakliklar.round(2))
en_yakin = uzakliklar.argmin()
print("En yakın örnek:", "ABCD"[en_yakin], "-> tahmin:", y[en_yakin], "kW")
```

1.6 Bir Makine Öğrenmesi Projesinin Adımları

Bir MÖ projesi, algoritma seçmekle başlamaz; problemi ve başarı ölçütünü tanımlamakla başlar. 'Yük tahmini yapacağız' yeterli bir tanım değildir: hangi ufukta (bir saat sonrası mı, ertesi gün mü), hangi çözünürlükte, hangi hata kabul edilebilir, tahmin hangi kararı besleyecek? Bu sorular cevaplanmadan toplanan veri ve eğitilen model çoğu zaman boşa gider. Aşağıdaki akış, her projede az çok aynıdır ve yinelemelidir: değerlendirme sonucu yetersizse veriye ya da modele geri dönülür.



Şekil 1.6 MÖ iş akışı — süreç doğrusal değil, yinelemelidir

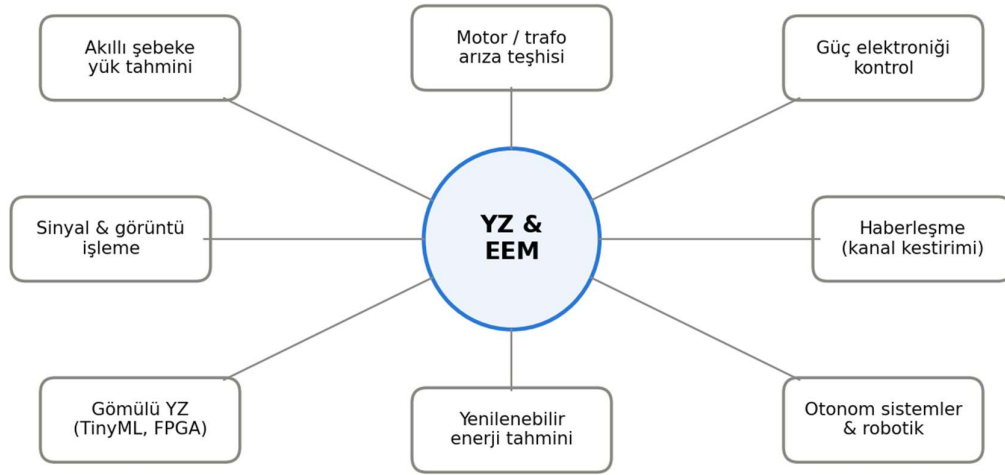
1. Problem tanımlama: Ne tahmin edilecek? Başarı nasıl ölçülecek? (Örn. saatlik yük tahmininde ortalama mutlak yüzde hata $< 5\%$.) Tahmin hangi kararı etkileyecek?
2. Veri toplama: Sensörler, SCADA kayıtları, sayaçlar, açık veri setleri. Etiket gerekiyorsa kim, nasıl etiketleyecek? Veri hangi dönemi kapsıyor; mevsimsellik var mı?
3. Ön işleme ve keşif: Eksik değerler, aykırı değerler, birim tutarlılığı, zaman damgası hizalama, görselleştirme (3. hafta). Zamanın büyük kısmı burada geçer.
4. Model eğitimi: Önce basit bir taban model (ortalama, geçen haftanın aynı saati), sonra uygun algoritma (5-13. haftalar). Basit modeli yenemeyen karmaşık model işe yaramaz.
5. Değerlendirme: Eğitimde görülmemiş test verisi üzerinde başarımla (4. hafta). Ölçüt problem tanımındaki ölçüt olmalı.

6. Dağıtım ve izleme: Modelin gömülü sisteme/sunucuya konması, gecikme ve bellek kısıtları, zamanla bozulmanın (veri kayması) izlenmesi ve yeniden eğitim (14. hafta).

Deneyimden: Uygulamada zamanın yaklaşık %60-80'i 2. ve 3. adımda (veri) geçer. 'Model' en havalı kısımdır ama en küçük kısımdır. Bu derste ki ödevlerin çoğunda da en çok uğraşacağınız şey veriyi doğru biçime getirmek olacak.

1.7 EEM'de Yapay Zekâ Uygulamaları

Elektrik-elektronik mühendisliği, makine öğrenmesi için son derece verimli bir alandır; çünkü sensörler, sayaçlar ve SCADA sistemleri sürekli veri üretir, fiziksel süreçler çoğu zaman karmaşık ve doğrusal olmayan ilişkiler içerir ve kararların ekonomik değeri yüksektir. Aşağıdaki şekil ve tablo, dersin ilerleyen haftalarında örnek olarak kullanılacak alanları özetler. Her satırda 'problem', 'yöntem' ve 'veri' sütunlarına dikkat edin: bir uygulamayı tanımlarken bu üçünü söyleyebilmek, problemi makine öğrenmesi diline çevirmenin ilk adımıdır.



Şekil 1.7 Elektrik-elektronik mühendisliğinde YZ uygulama alanları

Alan	Problem	YZ yöntemi	Veri kaynağı
Güç sistemleri	Saatlik/günlük yük tahmini; kararlılık değerlendirmesi	Regresyon, LSTM	Sayaç, hava durumu, takvim, PMU
Yenilenebilir enerji	Güneş/rüzgâr üretim tahmini; panel kirlenmesi tespiti	Regresyon, sinir ağı	Işınım, rüzgâr hızı, geçmiş üretim, kamera
Elektrik makineleri	Rulman/sargı arızası tespiti; kestirimci bakım	Sınıflandırma (SVM, CNN), otokodlayıcı	Titreşim, akım imzası (MCSA), sıcaklık
Güç elektroniği	Konvertör kontrol, MPPT, kondansatör yaşlanma tahmini	Pekiştirmeli öğrenme, sinir ağı	Gerilim/akım ölçümleri
Sinyal işleme	EKG/EEG sınıflandırma, gürültü giderme, konuşma tanıma	CNN, RNN, otokodlayıcı	Biyomedikal sinyaller, ses
Haberleşme	Kanal kestirimi, modülasyon tanıma, spektrum algılama	Derin öğrenme	I/Q örnekleri
Gömülü sistemler	Anahtar kelime algılama, anomali tespiti (TinyML)	Küçültülmüş sinir ağı	Mikrofon, IMU, akım sensörü

Alan	Problem	YZ yöntemi	Veri kaynağı
Üretim/test	PCB görsel kusur tespiti; otomatik test ölçüm yorumlama	CNN, transfer öğrenme	Kamera görüntüleri, test kayıtları
Elektronik tasarım	Devre parametre optimizasyonu, yerleşim	Bayeşçi optimizasyon, pekiştirmeli öğrenme	Simülasyon çıktıları

Tablo 1.7 EEM uygulamaları

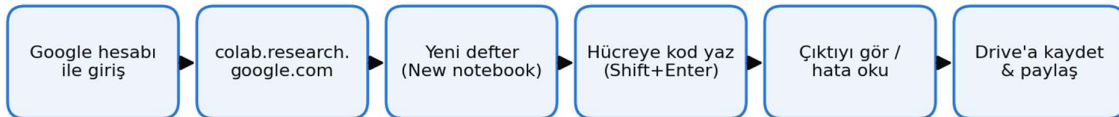
1.8 Sınırlar, Riskler ve Etik

Makine öğrenmesi güçlüdür ama sihir değildir; sınırlarını bilmeyen mühendis tehlikeli sistemler kurar. En temel sınır şudur: model, veride olanı öğrenir; veride olmayanı bilemez. Sadece yaz aylarında toplanan yük verisiyle eğitilen model kışın başarısızdır; sadece sağlıklı motorlardan veri toplanmışsa model arızayı tanıyamaz. Bu 'veri yanlılığı' problemi, toplumsal alanda (işe alım, kredi) ayrımcılığa; mühendislikte ise güvenlik açığına dönüşür.

- Veri yanlılığı: Model, verideki önyargıyı ve eksikliği aynen öğrenir. Eğitim verisinin kapsama alanı, modelin güvenle kullanılabileceği alanı belirler.
- Açıklanabilirlik: Derin ağların kararlarını gerekçelendirmek zordur; güvenlik-kritik sistemlerde (koruma röleleri, tıbbi cihazlar) bu ciddi bir sorundur. Bazı durumlarda daha az doğru ama açıklanabilir bir model tercih edilir.
- Sağlamlık: Küçük, insan gözünün fark etmeyeceği gürültü büyük hata üretebilir (adversarial örnekler). Sensör arızası ya da kalibrasyon kayması da modeli sessizce bozabilir.
- Veri kayması: Sistem zamanla değişir (yeni cihazlar, yeni tüketim alışkanlıkları); dün iyi olan model bugün kötü olabilir. İzleme ve yeniden eğitim şarttır.
- Enerji ve maliyet: Büyük modellerin eğitimi çok enerji tüketir; gömülü sistemde çalışmak için model küçültme (nicemleme, budama) gerekir (14. hafta).
- Sorumluluk ve etik: Otonom sistem hata yaparsa kim sorumlu? Kişisel tüketim verisi mahremiyet içerir. Mühendislik etiği YZ sistemleri için de geçerlidir; 'model öyle söyledi' bir mazeret değildir.

1.9 Uygulama Saati — Google Colab'a İlk Adım

Google Colab, tarayıcıda çalışan ücretsiz bir Python ortamıdır. Kurulum gerektirmez; NumPy, pandas, scikit-learn, PyTorch önceden yüklüdür ve ücretsiz GPU erişimi sunar (11. haftadan itibaren kullanacağız). Kod ve açıklamalar 'defter' (notebook, .ipynb) dosyasında hücreler hâlinde tutulur. Defter yaklaşımının ders için avantajı, kodu küçük parçalar hâlinde çalıştırıp her adımın çıktısını hemen görebilmektir; bir laboratuvar defteri gibi düşünün.



Şekil 1.8 Colab çalışma akışı

Hücre türü	Ne için?	Nasıl çalıştırılır?
Kod hücresi	Python kodu yazmak ve çalıştırmak; çıktı hücrenin altında görünür	Shift + Enter (çalıştır ve sonraki hücreye geç) veya soldaki ► düğmesi
Metin hücresi	Açıklama, başlık, formül (Markdown, LaTeX)	Çift tıkla düzenle, Shift + Enter ile görüntüle

Tablo 1.8 Colab hücreleri

Bilinmesi gereken önemli bir ayrıntı: hücreler yazıldıkları sırada değil, çalıştırıldıkları sırada etki eder. Bir hücrede tanımlanan değişken, o hücre çalıştırılmadıysa sonraki hücrede 'tanımsız' hatası verir. Bu yüzden ödev teslim etmeden önce her zaman 'Runtime → Restart and run all' ile defteri baştan sona temiz bir ortamda çalıştırın.

İlk hücre

```
# Bu bir yorum satırıdır; Python bunu çalıştırmaz.
print("Merhaba Yapay Zekâ Temelleri!")
```

Değişkenler ve basit aritmetik

```
# Değişken: bir değere isim vermek
gerilim = 230          # volt
akim = 2.5             # amper
guc = gerilim * akim   # watt
print("Güç =", guc, "W")
```

Aritmetik işlemler

```
# Python'u hesap makinesi gibi kullanmak
print(7 + 3)          # toplama
print(7 / 2)          # bölme -> 3.5 (ondalık)
print(7 // 2)         # tam bölme -> 3
print(7 % 2)          # kalan -> 1
print(2 ** 10)        # üs alma -> 1024
```

Hata mesajı okuma

```
# Hata okumayı öğrenmek en önemli beceridir. Bu hücre bilerek hata verir:
print(gerilim + " V")
# TypeError: unsupported operand type(s) for +: 'float' and 'str'
# Çözüm: sayıyı metne çevir -> print(str(gerilim) + " V") veya print(gerilim, "V")
```

Hata mesajı (traceback) kırmızı ve uzun görünür ama korkutucu değildir: en alttaki satır hatanın türünü ve nedenini söyler, üstteki satırlar hatanın hangi satırda oluştuğunu gösterir. Alışkanlık edinin: önce son satırı okuyun, sonra belirtilen satıra bakın. Hata mesajını olduğu gibi arama motoruna yapıştırmak da meşru ve yaygın bir yöntemdir.

Kütüphane kullanımı

```
# Kütüphane içe aktarma (import) – hazır araç kutuları
import math
print(math.sqrt(2))      # karekök
print(math.pi)          # 3.14159...
print(round(math.pi, 2)) # 2 basamağa yuvarla
```

Colab'da bilinmesi gereken kısayollar

İşlem	Kısayol / Yol
Hücreyi çalıştır	Shift + Enter
Üstüne kod hücresi ekle	Ctrl + M, A
Altına kod hücresi ekle	Ctrl + M, B
Hücreyi sil	Ctrl + M, D
Metin hücresine çevir	Ctrl + M, M
Tümünü çalıştır	Runtime → Run all
Ortamı sıfırla	Runtime → Restart runtime
Kaydet	Ctrl + S (Google Drive → Colab Notebooks klasörü)
Paylaş	Sağ üst 'Share' → bağlantıyı kopyala
Dosya yükle	Sol panel → Files → Upload

1.10 Sınıf İçi Sorular

1. 'Termostat bir yapay zekâ mıdır?' — Tablo 1.2'deki dört bakışa göre tartışın. Termostatın 'hedefi' nedir, 'akılcı' mıdır?
2. Aşağıdakilerden hangisi makine öğrenmesi için daha uygundur, neden? (a) Bir trafik ışığının süre çizelgesi (b) Bir güvenlik kamerasında insan tespiti (c) Ohm kanunu ile akım hesabı (d) Bir şehrin yarınki elektrik talebi.
3. Örnek 1.2'de sıcaklık Kelvin cinsinden yazılıysaydı en yakın komşu değişir miydi? Ya Fahrenheit? Hesaplayın ve neden farklı olduğunu açıklayın.
4. Tablo 1.7'den bir satır seçin: bu uygulama denetimli mi, denetimsiz mi, pekiştirmeli mi? Etiket kim üretir?
5. EEM'de kendi ilgi alanınızdan bir YZ uygulaması önerin; girdi verisi, tahmin edilecek çıktı ve başarı ölçütü ne olurdu?

1.11 Ödev / Hazırlık

- Google hesabı ile Colab'a girin, 'Hafta01_Giris_Colab.ipynb' defterini çalıştırın, sonunda istenen 3 alıştırmayı tamamlayıp Drive'a kaydedin ve bağlantısını paylaşın.
- Gelecek haftaya kadar: Python'da değişken, liste ve döngü kavramlarına göz atın (defterin son bölümü).
- Okuma: Russell & Norvig 1. bölüm veya 'AI Winter' başlıklı kısa bir makale; bir sonraki derste bir cümleyle özetlemeye hazır olun.

Hafta 2 — Python ile Isınma

Bölüm	Süre	İçerik
Saat 1	50 dk	Veri tipleri, değişkenler, listeler, sözlükler; koşullar (if) ve döngüler (for, while)
Saat 2	50 dk	Fonksiyonlar, NumPy dizileri, vektörel işlemler, basit istatistik, matris işlemleri
Saat 3	50 dk	Uygulama: Sinüs sinyali üretme, RMS hesabı, direnç fonksiyonları, ilk grafik

Tablo 2.1 Haftanın ders planı

2.1 Kazanımlar

- Temel veri tiplerini (int, float, str, bool, list, tuple, dict) ayırt eder ve uygun yerde kullanır.
- if/elif/else ve for/while yapıları ile basit algoritmalar yazar; girinti kuralını uygular.
- Parametre alan ve değer döndüren fonksiyon tanımlar; docstring yazar.
- NumPy dizileri oluşturur; toplama, çarpma, ortalama, standart sapma gibi işlemleri döngüsüz (vektörel) yapar; shape kavramını açıklar.
- Matris çarpımı ve lineer denklem çözümünü NumPy ile yapar.
- matplotlib ile bir sinyali çizdirir; eksen etiketi ve açıklama ekler.

2.2 Neden Python?

Python; okunabilir sözdizimi, devasa kütüphane ekosistemi (NumPy, pandas, scikit-learn, PyTorch) ve etkileşimli çalışma imkânı (Colab, Jupyter) sayesinde yapay zekânın fiili standart dilidir. Bir mühendis için asıl çekiciliği şudur: fikirden çalışan prototipe giden yol kısadır. C gibi derlenen dillere göre yavaştır; ama sayısal ağır işler NumPy ve PyTorch gibi C/CUDA ile yazılmış kütüphanelere devredildiği için pratikte hız sorunu yaşanmaz. Python'u 'yapıştırıcı' olarak düşünün: hızlı kütüphaneleri bir araya getiren, okunabilir bir üst katman.

MATLAB bilenler için geçiş kolaydır: NumPy, MATLAB'ın matris mantığını Python'a taşır. En sık karıştırılan fark indekslemedir — Python 0'dan başlar. İkinci fark, `*` operatörünün NumPy'da eleman bazlı çarpma, matris çarpımı için ise `@` olmasıdır.

Özellik	MATLAB	Python + NumPy
İndeksleme başlangıcı	1	0
Dizi tanımı	[1 2 3]	np.array([1, 2, 3])
Eleman bazlı çarpma	a .* b	a * b
Matris çarpımı	A * B	A @ B
Transpoz	A'	A.T
Ortalama	mean(x)	x.mean() veya np.mean(x)
Dizi boyutu	size(A)	A.shape
Yorum	% yorum	# yorum
Ücret	Lisanslı	Ücretsiz, açık kaynak

Tablo 2.2 MATLAB bilenler için hızlı karşılaştırma

2.3 Veri Tipleri ve Değişkenler

Python'da değişken tanımlamak için tip belirtmek gerekmez; `=` ile bir değere isim vermek yeterlidir ve tip değerden otomatik anlaşılır (dinamik tiplendirme). Yine de tipler vardır ve önemlidir: 7/2 işleminin sonucu her zaman ondalıklı (float) 3.5'tir; bir metin ile bir sayı toplanamaz; bir liste ile bir sayı çarpılamaz. `type()` fonksiyonu ile herhangi bir değerın tipini öğrenebilirsiniz. Aşağıdaki tablo bu derste kullanacağınız yedi temel tipi özetler.

Tip	Açıklama	Örnek	Kontrol
int	Tam sayı (sınırsız büyüklük)	n = 50	type(n) → int
float	Ondalık sayı (64 bit)	v = 230.0	type(v) → float
str	Metin (karakter dizisi)	ad = 'trafo'	len(ad) → 5
bool	Mantıksal (True/False)	acik = True	not acik → False
list	Sıralı, değiştirilebilir koleksiyon	olcum = [3.1, 2.9, 3.0]	olcum[0] → 3.1
tuple	Sıralı, değiştirilemez koleksiyon	boyut = (28, 28)	boyut[1] → 28
dict	Anahtar → değer eşlemesi	r = {'R1': 100, 'R2': 220}	r['R2'] → 220

Tablo 2.3 Temel veri tipleri

Tipler ve f-string

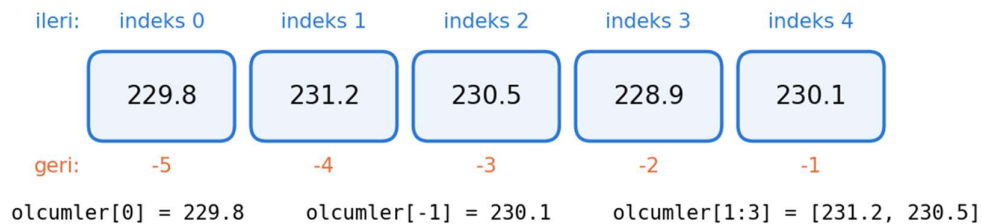
```
frekans = 50          # int
periyot = 1 / frekans # float -> 0.02 s
etiket = "şebeke"     # str
kritik = periyot < 0.05 # bool -> True

print(type(frekans), type(periyot), type(etiket), type(kritik))
print(f"Frekans {frekans} Hz için periyot {periyot*1000:.1f} ms") # f-string ile biçimli yazdırma
```

f-string (başında f olan tırnaklı metin), değişkenleri metnin içine yerleştirmenin en okunabilir yoludur. Süslü parantez içine bir ifade yazılır; iki noktadan sonra biçim belirtilir: `:.1f` bir ondalık basamak, `:6.2f` altı karakter genişlik iki ondalık, `:,` binlik ayırıcı. Rapor ve ödevlerde çıktıları düzenli yazdırmak için bunu kullanın.

Listeler

Liste, sıralı bir değer koleksiyonudur; ölçüm serileri, sensör okumaları gibi 'birden çok aynı türden şey' listede tutulur. Elemanlara sıfırdan başlayan indeksle erişilir; negatif indeks sondan sayar. Dilimleme (`[başlangıç:bitiş]`) bir alt liste verir ve bitiş indeksi dâhil değildir — bu kural Python'un her yerinde (range, NumPy, pandas) aynıdır.



Şekil 2.1 Liste indeksleme: ileri (0'dan) ve geri (-1'den) indeksler, dilimleme

Liste işlemleri

```
# Liste: birden fazla değeri sırayla tutar (0'dan başlayarak indekslenir)
olcumler = [229.8, 231.2, 230.5, 228.9, 230.1]
print(olcumler[0])      # ilk eleman -> 229.8
print(olcumler[-1])     # son eleman -> 230.1
print(olcumler[1:3])    # dilimleme: 1. ve 2. indeks -> [231.2, 230.5]
print(len(olcumler))    # eleman sayısı -> 5

olcumler.append(230.7)  # sona ekle
print(sum(olcumler) / len(olcumler)) # ortalama
print(max(olcumler), min(olcumler), sorted(olcumler))
```

Sözlükler

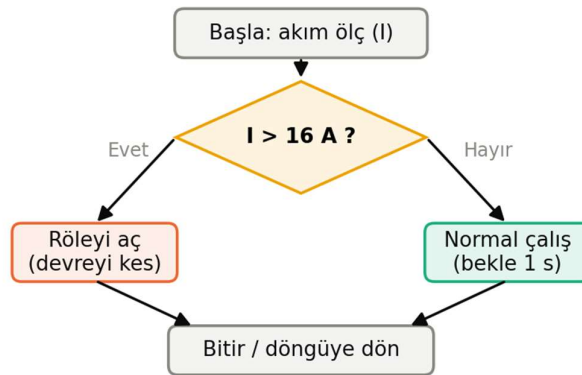
Sözlük (dict), değerlere sırayla değil isimle eriştiğiniz yapıdır: bir devredeki dirençler, bir sensörün kalibrasyon sabitleri, bir modelin hiperparametreleri sözlükte tutulur. Anahtar genellikle metindir, değer herhangi bir tip olabilir. Makine öğrenmesi kütüphaneleri ayarları çoğunlukla sözlük olarak alır; bu yüzden yapıya erken alışmak faydalıdır.

Sözlük (dict)

```
# Sözlük: isimle erişim
direncler = {"R1": 100, "R2": 220, "R3": 470}
print(direncler["R2"])
direncler["R4"] = 1000      # yeni eleman
for isim, deger in direncler.items():
    print(isim, "=", deger, "ohm")
print("R5" in direncler)    # anahtar var mı? -> False
```

2.4 Koşullar ve Döngüler

Programın akışını yönlendiren iki temel yapı vardır: koşul (bir şey doğruysa şunu yap) ve döngü (bir işi tekrar tekrar yap). Python'da blokların sınırını süslü parantez değil girinti (satır başındaki 4 boşluk) belirler; bu, kodun okunabilirliğini zorunlu kılan bilinçli bir tasarım tercihidir. Aşağıdaki akış diyagramı bir aşırı akım koruma kararını gösterir; altındaki kod aynı mantığın Python'udur.



Şekil 2.2 Bir aşırı akım koruma kararının akış diyagramı

if / elif / else — girinti (4 boşluk) bloğu belirler

```
akim = 18.2    # A

if akim > 16:
    print("Aşırı akım! Röle açılıyor.")
elif akim > 12:
    print("Uyarı: yük yüksek.")
else:
    print("Normal çalışma.")
```

Operatör	Anlamı	Örnek
==, !=	Eşit, eşit değil	akim == 0
>, <, >=, <=	Karşılaştırma	gerilim >= 220
and, or, not	Mantıksal bağlaçlar	(akim > 16) and (sure > 0.1)
in	Üyelik	'R2' in dirençler

Tablo 2.4 Karşılaştırma ve mantık operatörleri

`for` döngüsü bir koleksiyonun (liste, dizi, aralık) her elemanı için bloğu bir kez çalıştırır. `enumerate` hem indeksi hem değeri verir; `range(n)` 0'dan n-1'e sayılar üretir. `while` döngüsü ise bir koşul doğru kaldığı sürece devam eder; kaç tur süreceğini önceden bilmediğiniz durumlarda (bir değer eşige inene kadar, bir hata toleransın altına düşene kadar) kullanılır. Sayısal simülasyonların iskeleti tipik olarak bir `while` ya da `for` döngüsüdür.

for döngüsü

```
# for döngüsü: bir koleksiyonun her elemanı için tekrar
olcumler = [14.1, 15.8, 17.3, 12.0, 16.5]
for i, akim in enumerate(olcumler):    # enumerate: indeks + değer
    durum = "AŞIRI" if akim > 16 else "normal"
    print(f"Örnek {i}: {akim} A -> {durum}")

# range: sayı dizisi üretir
for k in range(5):                    # 0,1,2,3,4
    print(k, k**2)
```

while döngüsü — RC deşarjı

```
# while döngüsü: koşul doğru olduğu sürece tekrar
# Örnek: bir kondansatörün deşarjı, gerilim %5'in altına inene kadar
V0, R, C, dt = 5.0, 1000, 1e-3, 0.001    # V, ohm, F, s
V, t = V0, 0.0
while V > 0.05 * V0:
    V = V - (V / (R * C)) * dt            # dV/dt = -V/(RC) Euler adımı
    t += dt
print(f"%5'e inme süresi ≈ {t:.3f} s    (teorik 3RC = {3*R*C:.3f} s)")
```

Mühendislik notu: Bu küçük örnek aslında bir sayısal integrasyondur (Euler yöntemi). Adım büyüklüğü dt küçüldükçe sonuç teorik 3RC değerine yaklaşır; dt = 0.05 s alırsanız hata belirgin şekilde büyür. Makine öğrenmesindeki 'öğrenme oranı' da (5. hafta) tam olarak böyle bir adım büyüklüğüdür; çok büyük seçilirse çözüm ıraksar.

2.5 Fonksiyonlar

Fonksiyon, tekrar kullanılabilir bir işlem paketidir: girdi (parametre) alır, çıktı (return) verir. Aynı hesabı iki kez yazıyorsanız bir fonksiyon yazmalısınız. Fonksiyonlar kodu okunabilir kılar (isim, ne yaptığını söyler), test edilebilir kılar (küçük örneklerle doğrulanır) ve hatayı tek yerde düzeltmeyi sağlar. İlk satırdaki üç tırnaklı açıklama (docstring) fonksiyonun ne yaptığını anlatır; `help(fonksiyon)` bunu gösterir.

Fonksiyon tanımı — direnç eşdeğerleri

```
def seri(*direncler):
    """Seri bağlı dirençlerin eşdeğeri: toplam."""
    return sum(direncler)

def paralel(*direncler):
    """Paralel bağlı dirençlerin eşdeğeri: 1 / (1/R1 + 1/R2 + ...)."""
    return 1 / sum(1 / R for R in direncler)

print(seri(100, 220, 470))          # 790
print(paralel(100, 100))            # 50.0
print(round(paralel(100, 220, 470), 2)) # 58.68
```

Parametre adıyla çağırma da mümkündür

```
def gerilim_bolucu(vin, r1, r2):
    """Vout = Vin * R2 / (R1 + R2)"""
    return vin * r2 / (r1 + r2)

print(gerilim_bolucu(12, 10_000, 4_700))          # 3.84 V (alt çizgi: okunabilirlik)
print(gerilim_bolucu(vin=12, r1=1e4, r2=4.7e3))  # parametre adıyla çağırma
```

`\*direncler` yazımı, fonksiyonun istediği kadar parametre kabul etmesini sağlar (değişken sayıda argüman). Fonksiyon içinde tanımlanan değişkenler dışarıdan görünmez (yerel kapsam); bu, farklı fonksiyonların birbirinin değişkenlerini bozmasını engeller. Bir fonksiyon `return` satırına gelmeden biterse `None` döndürür — sık yapılan bir hata, `print` ile `return`'ü karıştırmaktır: `print` ekrana yazar, `return` değeri çağırana verir.

Örnek 2.1 — Direnç renk kodu çözücü

Soru: 4 bantlı bir direncin ilk üç bantının renklerini alıp değeri hesaplayan bir fonksiyon yazın. (Kahverengi-Siyah-Kırmızı → $10 \times 10^2 = 1000 \Omega$)

Çözüm fikri: İlk iki bant iki basamaklı bir sayı, üçüncü bant 10'un kuvveti. Renk → rakam eşlemesi bir sözlükte tutulur; fonksiyon üç rengi alıp $(10 \cdot b_1 + b_2) \times 10^{b_3}$ döndürür.

```
RENK = {"siyah": 0, "kahverengi": 1, "kırmızı": 2, "turuncu": 3, "sarı": 4,
        "yeşil": 5, "mavi": 6, "mor": 7, "gri": 8, "beyaz": 9}

def direnc_degeri(b1, b2, b3):
    return (10 * RENK[b1] + RENK[b2]) * 10 ** RENK[b3]

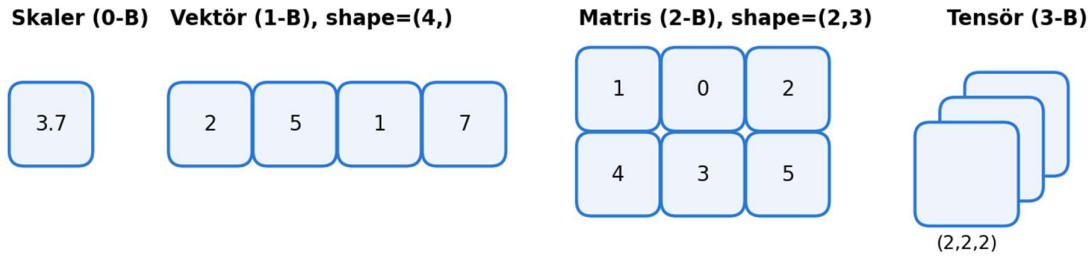
print(direnc_degeri("kahverengi", "siyah", "kırmızı")) # 1000
print(direnc_degeri("sarı", "mor", "turuncu"))         # 47000
```

Çözümde sözlük (dict), fonksiyon ve üs alma bir arada kullanıldı. Dördüncü bant (tolerans: altın %5, gümüş %10) için fonksiyonu genişletmek ve sonucu (değer, tolerans) ikilisi olarak döndürmek sınıf içi alıştırma.

2.6 NumPy: Sayısal Hesaplamanın Temeli

NumPy (Numerical Python), çok boyutlu dizi (ndarray) veri yapısını ve bu diziler üzerinde hızlı, döngüsüz işlem yapmayı sağlar. Python listesinden iki temel farkı vardır: (1) tüm elemanlar aynı tiptedir ve bellekte ardışık durur, bu yüzden işlemler C hızında yapılır; (2) aritmetik operatörler tüm elemanlara birden uygulanır (vektörel işlem). Makine öğrenmesinde veri neredeyse her zaman NumPy dizisi (veya ondan türeyen pandas DataFrame'i ya da PyTorch tensörü) olarak tutulur; bu haftanın en önemli konusu budur.

Bir dizinin `shape` özelliği boyutlarını verir: `(4,)` dört elemanlı bir vektör, `(2, 3)` iki satır üç sütunlu bir matris, `(2, 2, 2)` üç boyutlu bir tensör. Makine öğrenmesinde veri matrisi geleneksel olarak (örnek sayısı, özellik sayısı) biçimindedir: 720 saatlik ölçüm ve 3 özellik varsa $X.shape = (720, 3)$. Hataların büyük çoğunluğu shape uyumsuzluğundan kaynaklanır; şüphelendiğinizde `print(x.shape)` yazın.



Şekil 2.3 Skaler, vektör, matris, tensör ve shape kavramı

Dizi oluşturma ve indeksleme

```
import numpy as np

a = np.array([2, 5, 1, 7])          # 1-B dizi (vektör)
M = np.array([[1, 0, 2], [4, 3, 5]]) # 2-B dizi (matris)
print(a.shape, M.shape)           # (4,) (2, 3)
print(a.dtype)                    # int64
print(M[1, 2])                    # 2. satır 3. sütun -> 5
print(M[:, 0])                    # tüm satırlar, 1. sütun -> [1 4]
print(M[0, :])                    # 1. satır -> [1 0 2]
```

Vektörel işlemler ve mantıksal maske — Ohm kanunu

```
# Eleman bazlı (vektörel) işlemler — döngü yok!
I = np.array([0.5, 1.0, 1.5, 2.0]) # A
R = 100                             # ohm
V = I * R                           # her elemana uygulanır
P = V * I
print(V)                            # [ 50. 100. 150. 200.]
print(P)                            # [ 25. 100. 225. 400.]
print(V.mean(), V.std(), V.max(), V.argmax())
print(V > 120)                      # mantıksal maske -> [False False True True]
print(V[V > 120])                   # maske ile seçim -> [150. 200.]
```

Mantıksal maske (son iki satır) çok kullanışlı bir kalıptır: bir koşul, dizi ile aynı boyutta True/False dizisi üretir ve bu dizi indeks olarak kullanılıncaya yalnızca koşulu sağlayan elemanlar seçilir. 'Eşiği aşan ölçümler', 'hafta sonu satırları', 'aykırı değerler' hep bu kalıpla bulunur; pandas'ta da aynı mantık geçerlidir.

```
# Hazır dizi üreticiler
print(np.zeros(3))          # [0. 0. 0.]
print(np.ones((2, 2)))      # 2x2 birler
print(np.arange(0, 1, 0.25)) # [0. 0.25 0.5 0.75] (bitiş dâhil değil)
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1. ] (5 nokta, bitiş dâhil)
rng = np.random.default_rng(0) # tekrarlanabilir rastgelelik (seed=0)
print(rng.normal(0, 1, 4))    # ortalama 0, std 1 olan 4 gürültü örneği
print(rng.uniform(0, 10, 3)) # 0-10 arası düzgün dağılım
```

Tekrarlanabilirlik: `default\_rng(0)` içindeki 0 'tohum' (seed) değeridir: aynı tohumla her çalıştırmada aynı rastgele sayılar üretilir. Makine öğrenmesinde deneyleri tekrarlanabilir kılmak için tohum sabitlemek standart uygulamadır; ödevlerinizde de bunu yapın ki sonuçlarınız kontrol edilebilsin.

```
# Matris işlemleri
A = np.array([[2, 1], [1, 3]])
b = np.array([5, 10])
print(A @ b)          # matris-vektör çarpımı -> [20 35]
print(A.T)            # transpoz
print(np.linalg.solve(A, b)) # A x = b çöz -> [1. 3.]
print(np.linalg.inv(A))   # ters matris
print(A * A)           # DİKKAT: eleman bazlı kare, matris çarpımı değil
```

Örnek 2.2 — Düğüm gerilimleri: $A \cdot x = b$

Soru: İki düğümlü bir DC devrenin düğüm denklemleri $2V_1 + V_2 = 5$ ve $V_1 + 3V_2 = 10$ olsun. Düğüm gerilimlerini bulun.

Çözüm (elle): İlk denklemden $V_2 = 5 - 2V_1$; ikincide yerine koy: $V_1 + 15 - 6V_1 = 10 \rightarrow -5V_1 = -5 \rightarrow V_1 = 1$ V, $V_2 = 3$ V. NumPy'da aynı iş `np.linalg.solve(A, b)` ile tek satırdır (yukarıdaki kod). Determinant sıfırsa (bağımlı denklemler) `solve` hata verir; bu, devrede düğüm seçiminin hatalı olduğunu gösterir.

Makine öğrenmesiyle bağlantı: 5. haftada göreceğimiz en küçük kareler regresyonu da sonuçta bir $A \cdot x = b$ çözümüdür (normal denklem). Yani bu satırı bugün öğrenmek, üç hafta sonra regresyonun içini görmek demektir.

Örnek 2.3 — RMS değeri: elle ve NumPy ile

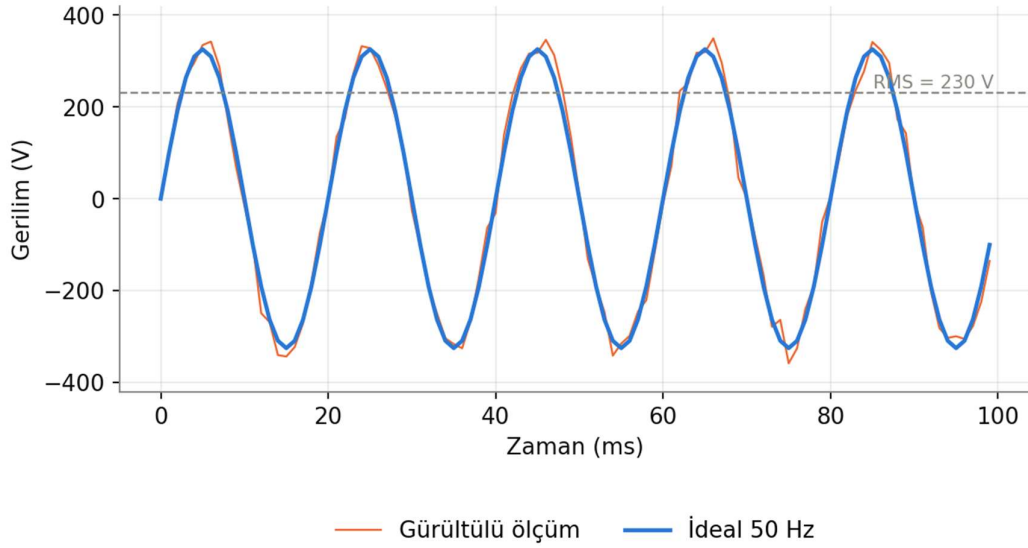
Soru: Bir sinyalden 4 örnek alındı: $x = [3, -1, 2, -2]$. RMS (etkin) değerini hesaplayın.

Çözüm (elle): $RMS = \sqrt{(1/N) \sum x_i^2} = \sqrt{(9 + 1 + 4 + 4) / 4} = \sqrt{18/4} = \sqrt{4.5} \approx 2.121$

```
x = np.array([3, -1, 2, -2])
rms = np.sqrt(np.mean(x ** 2))
print(round(rms, 3))          # 2.121
```

Üç NumPy işlemi (kare, ortalama, karekök) formülün üç adımına birebir karşılık gelir; döngü yazmaya gerek yoktur. Şimdi gerçek bir şebeke sinyaline uygulayalım: tepe değeri 230V₂ V olan 50 Hz sinüsün RMS'i 230 V olmalı.

Sinyal üretmek için önce bir zaman eksenı gerekir: örnekleme frekansı $f_s = 1000$ Hz ise örnekler 1 ms aralıktır ve 0.1 saniyede 100 örnek olur. `np.arange(0, 0.1, 1/fs)` bu eksenı üretir; `np.sin(2π·50·t)` her zaman noktası için sinüsü tek seferde hesaplar. Gürültü, aynı uzunlukta normal dağılımlı bir dizi eklenerek modellenir.



Şekil 2.4 NumPy ile üretilen 50 Hz ideal sinüs ve gürültülü ölçüm

Sinyal üretme ve çizdirme

```
import numpy as np
import matplotlib.pyplot as plt

fs = 1000                                # örnekleme frekansı (Hz)
t = np.arange(0, 0.1, 1/fs)              # 0..0.1 s, 100 örnek
v = 230*np.sqrt(2)*np.sin(2*np.pi*50*t)  # ideal 50 Hz
rng = np.random.default_rng(0)
v_olcum = v + rng.normal(0, 25, size=t.size) # gürültü eklenmiş ölçüm

print("İdeal RMS :", np.sqrt(np.mean(v**2)).round(2)) # ~230.0
print("Ölçüm RMS :", np.sqrt(np.mean(v_olcum**2)).round(2)) # biraz büyük (gürültü gücü eklenir)

plt.figure(figsize=(8, 3))
plt.plot(t*1000, v_olcum, label="ölçüm", lw=1)
plt.plot(t*1000, v, label="ideal", lw=2)
plt.xlabel("Zaman (ms)"); plt.ylabel("Gerilim (V)"); plt.legend(); plt.grid(alpha=.3)
plt.show()
```

Mühendislik notu: Gürültülü sinyalin RMS'i ideal olandan büyük çıkar: $RMS^2 = \text{sinyal gücü} + \text{gürültü gücü}$ (bağımsız bileşenlerin güçleri toplanır). Gürültü std'si $\sigma = 25$ V ise gürültü gücü $\sigma^2 = 625$ V², beklenen $RMS \approx \sqrt{230^2 + 625} \approx 231.4$ V. Bu, 3. haftada 'varyans' kavramıyla yeniden karşımıza çıkacak: varyans bir sinyalin ortalama etrafındaki gücüdür.

matplotlib ile grafik: temel kalıp

Her grafik aynı beş adımdan oluşur: figür oluştur, veriyi çiz (plot/scatter/hist), eksenleri etiketle, gerekiyorsa açıklama (legend) ve ızgara ekle, göster. Eksen etiketi ve birim olmayan grafik mühendislik raporunda kabul edilmez; bu alışkanlığı ilk günden edinin.

Fonksiyon	Ne çizer?	Örnek
plt.plot(x, y)	Çizgi grafiği (zaman serisi, sinyal)	plt.plot(t, v, lw=2, label='ideal')
plt.scatter(x, y)	Saçılım (iki değişken ilişkisi)	plt.scatter(sicaklik, tuketim, s=8)
plt.hist(x, bins)	Histogram (dağılım)	plt.hist(tuketim, bins=30)
plt.bar(kat, deger)	Çubuk (kategorik karşılaştırma)	plt.bar(['R1', 'R2'], [100, 220])
plt.subplots(1, 3)	Yan yana birden çok eksen	fig, ax = plt.subplots(1, 3, figsize=(12, 3))

Tablo 2.5 Temel matplotlib fonksiyonları

2.7 Hız Farkı: Döngü mü, Vektör mü?

Örnek 2.4 — Bir milyon elemanın karesi

```
import time, numpy as np
x = np.arange(1_000_000, dtype=float)

t0 = time.time()
kare_liste = [xi**2 for xi in x]      # Python döngüsü (liste üretici)
t1 = time.time()
kare_np = x**2                      # NumPy vektörel
t2 = time.time()
print(f"Döngü: {t1-t0:.3f} s   NumPy: {t2-t1:.4f} s   hızlanma ≈ {(t1-t0)/(t2-t1):.0f}x")
```

Tipik sonuç: NumPy 30-200 kat hızlıdır (donanıma göre değişir). Fark, Python döngüsünün her elemanda tip kontrolü ve nesne oluşturma yükü taşımasından, NumPy'ın ise derlenmiş C kodunda ardışık bellek üzerinde çalışmasından kaynaklanır. Kural: Sayısal veride mümkün olduğunca döngü yerine dizi işlemi kullanın. 60.000 görüntülü bir veri setinde bu fark, saniyeler ile saatler arasındaki farktır.

2.8 Sık Yapılan Hatalar

Programlamaya yeni başlayanların hatalarının büyük çoğunluğu aşağıdaki türlere girer. Mesajı okuyup tabloda bulmak, çoğu zaman sorunu 30 saniyede çözer.

Hata mesajı	Sebebe	Çözüm
IndentationError	Girinti (boşluk) tutarsız	Blok içinde 4 boşluk kullanın; Tab/boşluk karıştırmayın
NameError: name 'x' is not defined	Değişken tanımlanmadan kullanıldı ya da hücre çalıştırılmadı	Önceki hücreleri sırayla çalıştırın; yazım hatası kontrol edin
IndexError: list index out of range	Olmayan indekse erişim	Python 0'dan başlar; son eleman len-1
TypeError: can't multiply sequence by non-int	Liste ile sayı çarpıldı	Listeyi np.array'e çevirin
ValueError: operands could not be broadcast	Dizi boyutları uyumsuz	shape'leri yazdırıp kontrol edin; reshape gerekebilir
ZeroDivisionError	Sıfıra bölme	Paydayı kontrol edin (paralel dirençte R=0?)
KeyError: 'R5'	Sözlükte olmayan anahtar	'R5' in dirençler ile kontrol edin; .get() kullanın

Hata mesajı	Sebebe	Çözüm
SyntaxError	Eksik parantez, iki nokta, tırnak	Belirtilen satırın bir üstüne de bakın

Tablo 2.6 Hata mesajları rehberi

2.9 Sınıf İçi Alıştırmalar

1. Bir listede verilen gerilim ölçümlerinden 225 V altındakileri ve 235 V üstündekileri sayan kod yazın (for + if). Sonra aynı işi NumPy mantıksal maske ile tek satırda yapın.
2. Bir sayının asal olup olmadığını dönen fonksiyon yazın (while veya for); 2-50 arası asalları listeleyin.
3. np.linspace ile 0- 2π arası 200 nokta üretip sin ve cos'u aynı grafikte, eksen etiketli ve açıklamalı çizin.
4. Dört bantlı direnç için tolerans bandını da ekleyerek direnc_degeri fonksiyonunu genişletin; (değer, tolerans) döndürsün.
5. Örnek 2.3'teki ölçümün RMS'ini gürültü standart sapması 0, 10, 25, 50 V için hesaplayıp tabloya yazın; $\sqrt{(230^2 + \sigma^2)}$ ile karşılaştırın.
6. Seri RLC devresinde $R = 10 \Omega$, $L = 50 \text{ mH}$, $C = 100 \mu\text{F}$ için 10-1000 Hz arasında $|Z| = \sqrt{R^2 + (\omega L - 1/\omega C)^2}$ eğrisini çizin; rezonans frekansını grafikten ve $1/(2\pi\sqrt{LC})$ formülünden karşılaştırın.

2.10 Ödev 1 — Python Isınma

- Hafta02 defterindeki 6 alıştırmayı tamamlayın (fonksiyon yazma, döngü, NumPy dizileri, grafik).
- Bonus: Kendi elinizle 10 ölçüm alın (ör. telefon ışık/ivme sensörü, bir çevrimiçi veri) ve ortalama, std, min, max hesaplayın; bir cümleyle yorumlayın.
- Teslim: Colab paylaşım bağlantısı; defter baştan sona hatasız çalışmalı (Runtime → Restart and run all).

Hafta 3 — Veri ile Tanışma: pandas ve Keşifsel Veri Analizi

Bölüm	Süre	İçerik
Saat 1	50 dk	Veri nedir? Veri türleri, veri seti yapısı, tanımlayıcı istatistik (ortalama, medyan, std, çeyrekler, korelasyon) — elle örnekler
Saat 2	50 dk	pandas DataFrame, CSV yükleme, filtreleme, gruplama; eksik ve aykırı değerler; ölçekleme
Saat 3	50 dk	Uygulama: Elektrik tüketimi veri setinde uçtan uca keşifsel veri analizi (EDA) ve grafikler

Tablo 3.1 Haftanın ders planı

3.1 Kazanımlar

- Sayısal/kategorik, sürekli/kesikli veri türlerini ayırt eder ve her birinin nasıl işleneceğini söyler.
- Ortalama, medyan, standart sapma, çeyrekler ve Pearson korelasyonunu elle ve pandas ile hesaplar; hangi ölçünün ne zaman yanıltıcı olduğunu bilir.
- CSV dosyasını pandas ile yükler; satır/sütun seçer, filtreler, gruplar, yeni sütun türetir.
- Eksik ve aykırı değerleri tespit eder; uygun stratejiyi gerekçesiyle seçer.
- Histogram, saçılım, kutu ve çizgi grafiklerini üretir ve yorumlar; korelasyon matrisini okur.
- Min-max ve z-skor ölçeklemenin neden gerektiğini açıklar ve uygular.

3.2 Veri Nedir, Nasıl Görünür?

Makine öğrenmesinde veri genellikle bir tablodur: her satır bir örnek (gözlem, kayıt), her sütun bir özellik (feature, değişken). Denetimli öğrenmede sütunlardan biri hedef (target, etiket) olur. Bu tablo yapısı, EEM'de karşılaşılan pek çok veri kaynağıyla doğrudan örtüşür: bir SCADA kaydında her satır bir zaman damgası, sütunlar ölçüm noktalarıdır; bir test raporunda her satır bir cihaz, sütunlar ölçülen parametrelerdir. Görüntü ve ses gibi 'yapısal olmayan' veriler de sonunda sayısal dizilere dönüştürülür; farkları, özelliklerin önceden tanımlı olmaması ve derin öğrenme ile çıkarılmasıdır.

Bir sütunun türü, onunla ne yapabileceğinizi belirler. Sayısal sütunların ortalaması alınır, ölçeklenir, uzaklık hesaplanır. Kategorik sütunlar (motor tipi, arıza sınıfı) sayıya çevrilmeden modele giremez; nominal (sırasız) kategoriler one-hot kodlanır, ordinal (sıralı) kategoriler tam sayı ile temsil edilebilir. Zaman damgası özel bir durumdur: doğrudan sayı olarak kullanmak yerine saat, haftanın günü, hafta sonu gibi anlamlı özelliklere ayrıştırılır — dersin veri setinde tam olarak bu yapılmıştır.

Veri türü	Alt tür	Örnek	Nasıl işlenir?
Sayısal	Sürekli	Sıcaklık 23.7 °C, gerilim 229.8 V	Doğrudan kullanılır; genellikle ölçeklenir
Sayısal	Kesikli	Saat 0-23, arıza sayısı	Doğrudan kullanılır; bazen kategorik gibi ele alınır
Kategorik	Nominal (sırasız)	Motor tipi: asenkron / senkron / DC	One-hot kodlama (7. hafta)
Kategorik	Ordinal (sıralı)	Yük seviyesi: düşük < orta < yüksek	Sıralı tam sayı kodlama
İkili	0/1	Hafta sonu mu? Arızalı mı?	Doğrudan kullanılır (0/1)
Zaman	Zaman damgası	2026-09-22 14:00	Saat, gün, hafta sonu, mevsim özelliklerine ayrıştırılır

Veri türü	Alt tür	Örnek	Nasıl işlenir?
Yapısal olmayan	Metin, görüntü, ses	Termal kamera görüntüsü	Derin öğrenme (12-13. hafta)

Tablo 3.2 Veri türleri

Dersin veri seti: elektrik_tuketimi.csv

Bu hafta ve sonraki haftalarda kullanacağımız sentetik veri seti, bir binanın 30 gün boyunca saatlik elektrik tüketimini içerir (720 satır). Veri, gerçekçi bir yapıya sahip olacak şekilde üretilmiştir: gün içinde tipik bir yük profili (gece düşük, akşam saatlerinde tepe), hafta sonu %25 daha düşük tüketim, 24 °C üzerinde sıcaklıkla artan klima yükü ve rastgele gürültü. Öğretim amacıyla bilerek 12 eksik sıcaklık değeri ve 1 aykırı tüketim değeri (950 kW) eklenmiştir. Verinin 'nasıl üretildiğini' bilmemiz bir avantajdır: bulduğumuz kalıpların doğru olup olmadığını kontrol edebiliriz. Gerçek hayatta bu lüks yoktur; kalıpları fiziksel bilgiyle doğrulamak gerekir.

Sütun	Tür	Açıklama
gun	Kesikli (1-30)	Ayın günü
saat	Kesikli (0-23)	Günün saati
hafta_sonu	İkili (0/1)	1 ise Cumartesi/Pazar
sicaklik_C	Sürekli	Dış sıcaklık (°C); 12 değer eksik (NaN)
tuketim_kW	Sürekli	Saatlik ortalama güç (kW) — hedef değişken; 1 aykırı değer

Tablo 3.3 Veri seti sütunları

3.3 Tanımlayıcı İstatistik (Ele Örnekler)

Bir veri setine bakmanın ilk yolu birkaç sayıyla özetlemektir: merkez nerede (ortalama, medyan), yayılım ne kadar (varyans, standart sapma, çeyrekler açıklığı), değişkenler birlikte nasıl hareket ediyor (korelasyon). Bu ölçüler basit görünse de doğru yorumlamak beceri ister. Ortalama, aykırı değerlerden şiddetle etkilenir; medyan etkilenmez. Standart sapma, verinin ortalama etrafındaki 'gücünü' ölçer — sinyal işlemedeki AC gücü ile aynı fikirdir. Korelasyon yalnızca doğrusal ilişkiyi yakalar; U biçimli güçlü bir ilişki sıfıra yakın korelasyon verebilir.

Ölçü	Formül	Ne söyler?
Ortalama (μ , $\bar{x}$)	$\bar{x} = (1/N) \sum x_i$	Merkez; aykırı değerlere duyarlı
Medyan	Sıralı verinin ortası	Merkez; aykırı değerlere dayanıklı
Mod	En sık görülen değer	Kategorik veride merkez
Varyans (σ^2)	$\sigma^2 = (1/N) \sum (x_i - \bar{x})^2$	Yayılım (kare birim); gürültü gücü ile aynı fikir
Standart sapma (σ)	$\sigma = \sqrt{\sigma^2}$	Yayılım (orijinal birim)
Çeyrekler (Q1, Q3), IQR	$IQR = Q3 - Q1$	Orta %50'nin genişliği; aykırı değer tespiti
Pearson korelasyonu (r)	$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$	İki değişken arasındaki doğrusal ilişki, -1..+1

Tablo 3.4 Temel istatistikler

Dikkat: pandas .std() ve .var() varsayılan olarak N yerine N-1'e böler (örneklem tahmini, ddof=1). NumPy ise varsayılan olarak N'e böler (ddof=0). Küçük veride fark belirgindir (N=5 için %12); büyük veride ihmal edilebilir. Elle çözdüğümüz örneklerde N'e böleceğiz; sınavda hangisinin istendiğine dikkat edin.

Örnek 3.1 — Ortalama, medyan, std ve aykırı değerin etkisi

Soru: Beş saatlik tüketim ölçümü (kW): 100, 110, 120, 130, 140. Ortalama, medyan ve standart sapmayı (N'e bölerek) hesaplayın. Sonra 140 yerine hatalı okunan 950 girilirse ne değişir?

Çözüm: $\bar{x} = (100+110+120+130+140)/5 = 600/5 = 120$. Medyan = 120 (sıralı ortadaki). Sapmalar: -20, -10, 0, 10, 20 → kareler: 400, 100, 0, 100, 400 → toplam 1000 → $\sigma^2 = 1000/5 = 200$ → $\sigma = \sqrt{200} \approx 14.14$.

Aykırı değerle: $\bar{x} = (100+110+120+130+950)/5 = 1410/5 = 282$. Medyan hâlâ 120! Sapmalar: -182, -172, -162, -152, 668 → kareler toplamı = 33124 + 29584 + 26244 + 23104 + 446224 = 558280 → $\sigma^2 = 111656$ → $\sigma \approx 334.1$. Tek bir hatalı değer ortalamayı 2.3 kat, standart sapmayı 24 kat büyüttü; medyan hiç değişmedi. Bu yüzden aykırı değer kontrolü modelden önce yapılır ve raporlarda ortalama ile birlikte medyan da verilir.

```
import numpy as np
x = np.array([100, 110, 120, 130, 140])
x2 = np.array([100, 110, 120, 130, 950])
for v in (x, x2):
    print(f"ort={v.mean():.1f}  medyan={np.median(v):.1f}  std={v.std():.2f}")
```

Örnek 3.2 — Pearson korelasyonu elle

Soru: Dört gözlem için sıcaklık x = [20, 24, 28, 32] ve tüketim y = [80, 90, 110, 140]. Korelasyon katsayısını hesaplayın.

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$
20	80	-6	-25	150	36	625
24	90	-2	-15	30	4	225
28	110	2	5	10	4	25
32	140	6	35	210	36	1225
$\bar{x}=26$	$\bar{y}=105$			$\Sigma=400$	$\Sigma=80$	$\Sigma=2100$

$r = 400 / \sqrt{(80 \times 2100)} = 400 / \sqrt{168000} = 400 / 409.88 \approx 0.976$. Güçlü pozitif doğrusal ilişki: sıcaklık arttıkça tüketim (klima yükü) artıyor. Korelasyon nedensellik değildir; ama tahmin için yararlı bir özellik olduğunu gösterir. r'nin karesi (≈ 0.95), y'deki değişimin yaklaşık %95'inin x ile doğrusal olarak açıklanabildiğini söyler — 5. haftada bu sayıya R^2 diyeceğiz.

```
x = np.array([20, 24, 28, 32]); y = np.array([80, 90, 110, 140])
print(np.corrcoef(x, y)[0, 1].round(3))  # 0.976
```

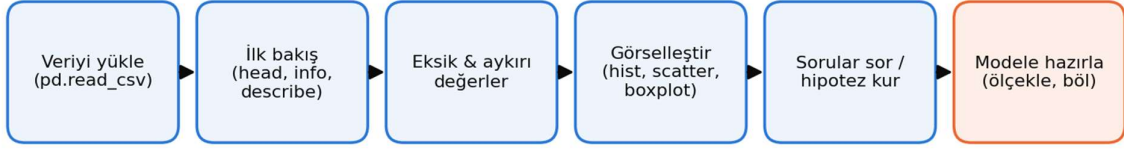
r aralığı	Yorum	Örnek
0.0 - 0.2	İlişki yok / çok zayıf	Hafta sonu ile saat (bağımsız)
0.2 - 0.5	Zayıf-orta	Sıcaklık ile tüketim (tüm gün)

r aralığı	Yorum	Örnek
0.5 - 0.8	Orta-güçlü	Saat ile tüketim
0.8 - 1.0	Güçlü	Sıcaklık ile tüketim (sadece öğleden sonra)

Tablo 3.5 Korelasyon büyüklüğünün kaba yorumu (alan bağımlıdır)

3.4 pandas: Tablo Verisiyle Çalışmak

pandas, tablo verisi için DataFrame (tablo) ve Series (tek sütun) yapılarını sunar. NumPy'ın üstüne kurulmuştur; sütun isimleriyle çalışma, eksik veri yönetimi, gruplama ve CSV/Excel okuma gibi işleri kolaylaştırır. Bir DataFrame'i, her sütunu bir NumPy dizisi olan ve sütunların isimle çağrıldığı bir tablo olarak düşünün; `df.values` ile altındaki NumPy dizisine her zaman ulaşabilirsiniz. Keşifsel veri analizi (EDA) dediğimiz süreç, modele geçmeden önce veriyi pandas ve grafiklerle 'tanıma' sürecidir: sürprizleri (eksik değer, aykırı değer, beklenmedik ilişki) modelden önce yakalamak amaçlanır.



Şekil 3.1 Keşifsel veri analizi (EDA) adımları

Veri yükleme

```
import pandas as pd

df = pd.read_csv("elektrik_tuketimi.csv") # Colab'da dosyayı sol panelden yükleyin ya da Drive'a
bağlanın
print(df.shape) # (720, 5): 720 satır, 5 sütun
df.head() # ilk 5 satır
```

İlk bakış

```
df.info() # sütun tipleri ve eksik olmayan değer sayıları
df.describe() # sayısal sütunlar için özet istatistik (count, mean, std, min, %25, %50, %75, max)
df.isna().sum() # sütun başına eksik değer sayısı
```

`describe()` çıktısı ilk şüpheleri uyandıracak yerdir. Dersin veri setinde tüketim_kW için max ≈ 950 iken %75'lik değer ≈ 135 'tir: en büyük değer, verinin dörtte üçünün 7 katı. Bu, ya bir ölçüm hatası ya da olağanüstü bir olay demektir; her iki durumda da araştırılmalıdır. `info()` ise sıcaklık_C sütununda 708 dolu değer gösterir — 720'den 12 eksik. Bu iki satır çıktıyı okumak, saatlerce süren model hatasından kurtarabilir.

İşlem	Kod	Sonuç
Sütun seç	df['tuketim_kW']	Series
Birden çok sütun	df[['saat', 'tuketim_kW']]	DataFrame
Satır filtrele	df[df['hafta_sonu'] == 1]	Sadece hafta sonu satırları

İşlem	Kod	Sonuç
Çoklu koşul	<code>df[(df.saat >= 8) & (df.saat <= 18)]</code>	Mesai saatleri (& ve kullanılır; and/or değil)
Konumla seç	<code>df.iloc[0:5, 1:3]</code>	İlk 5 satır, 2-3. sütun
Etiketle seç	<code>df.loc[df.saat == 12, 'tuketim_kW']</code>	Öğlen tüketimleri
Sırala	<code>df.sort_values('tuketim_kW', ascending=False)</code>	Büyükten küçüğe
Yeni sütun	<code>df['tuketim_W'] = df.tuketim_kW * 1000</code>	Türetilmiş özellik
Gruplama	<code>df.groupby('saat')['tuketim_kW'].mean()</code>	Saatlik ortalama profil
Çapraz özet	<code>df.pivot_table('tuketim_kW', 'saat', 'hafta_sonu')</code>	Saat × gün tipi tablosu
Kaydet	<code>df.to_csv('temiz.csv', index=False)</code>	Temizlenmiş veriyi dosyaya yaz

Tablo 3.6 Sık kullanılan pandas işlemleri

Gruplama (`groupby`), EDA'nın en güçlü aracıdır: veriyi bir kategoriye göre parçalara ayırır, her parçaya bir özet fonksiyon uygular ve sonuçları birleştirir. 'Her saat için ortalama tüketim' sorusu tek satırdır ve sonucu doğrudan günlük yük profilidir. `agg` ile birden çok özet aynı anda alınabilir.

Gruplama ile özet

```
# Saatlik ortalama tüketim profili
profil = df.groupby("saat")["tuketim_kW"].mean()
print(profil.round(1))

# Hafta içi / hafta sonu farkı
print(df.groupby("hafta_sonu")["tuketim_kW"].agg(["mean", "median", "std", "count"]))
```

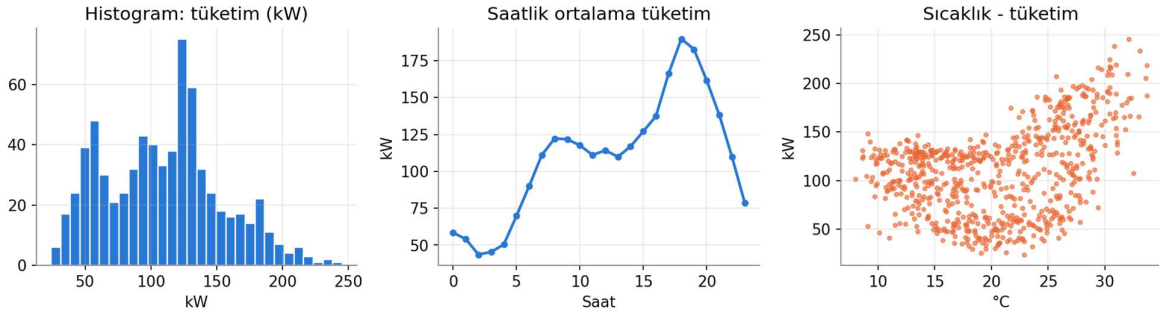
3.5 Eksik ve Aykırı Değerler

Gerçek veri hiçbir zaman temiz değildir. Sensörler kopar, iletişim kesilir, sayaç okuma hatası olur, birimler karışır. Eksik değer (pandas'ta NaN) ve aykırı değer (diğerlerinden çok uzak gözlem) iki temel kirlilik türüdür ve modelden önce mutlaka ele alınmalıdır; çoğu algoritma NaN ile hiç çalışmaz, aykırı değer ise ortalamayı ve dolayısıyla modeli çarpıtır (Örnek 3.1). Ama dikkat: 'temizlemek' her zaman 'silme' demek değildir. Zaman serisinde eksik bir saati silmek serinin sürekliliğini bozar; komşu değerlerden interpolasyon daha doğrudur. Aykırı değer de bazen tam aradığınız şeydir — arıza tespiti projesinde aykırı değerler etikettir, gürültü değil.

Durum	Tespit	Seçenekler	Ne zaman?
Eksik değer (NaN)	<code>df.isna().sum()</code> , <code>df.info()</code>	Satırı sil (<code>dropna</code>) / ortalama-medyan ile doldur (<code>fillna</code>) / zaman serisinde önceki-sonraki değerle doldur (<code>ffill</code> , <code>interpolate</code>) / eksikliği ayrı bir özellik olarak işaretle	Az sayıda ve rastgele eksik → sil; zaman serisi → interpolasyon; eksiklik bilgi taşıyorsa (sensör arızası) → işaretle
Aykırı değer	<code>describe()</code> , kutu grafiği, IQR kuralı, z-skor ($ z > 3$), fiziksel sınırlar	Sil / sınırla (<code>clip</code>) / medyanla değiştir / olduğu gibi bırak (gerçek olay olabilir) / dayanıklı model kullan	Ölçüm hatası ise düzelt veya sil; gerçek olay ise (arıza, fırtına) belki tam da aradığınız şeydir!

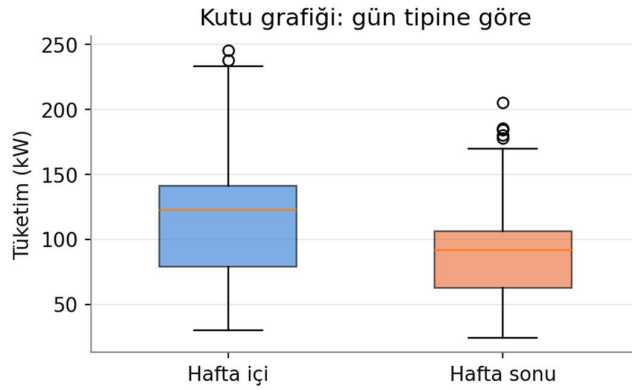
Tablo 3.7 Eksik ve aykırı değer stratejileri

'iki değişken birlikte nasıl hareket ediyor?', kutu grafiği 'gruplar arasında fark var mı ve aykırı değerler nerede?'. Şekil 3.3'teki üç grafik dersin veri setinde bu soruları cevaplar.



Şekil 3.3 Histogram, saatlik profil (çizgi) ve sıcaklık-tüketim saçılım grafiği

Histogram, tüketimin sağa çarpık (uzun sağ kuyruklu) olduğunu gösterir: çoğu saat 50-150 kW arasında, az sayıda saat 200 kW üstünde. Saatlik profil, sabah 6'dan itibaren yükselen, öğlen platoya ulaşan ve 18-19'da tepe yapan tipik bir yük eğrisidir; 4. haftadan itibaren 'geçen günün aynı saati' bu profile dayanan basit ama güçlü bir taban model olacak. Saçılım grafiği ise en öğretici olanıdır: 24 °C'ye kadar tüketim sıcaklıktan neredeyse bağımsızken, 24 °C üstünde belirgin biçimde artar — klima yükü devreye giriyor. Bu ilişki doğrusal değil, kırık çizgi biçimindedir.



Şekil 3.4 Kutu grafiği: medyan (çizgi), Q1-Q3 (kutu), bıyıklar (1.5 IQR) ve aykırı noktalar

Kutu grafiği, Örnek 3.3'teki IQR kuralının görsel hâlidir: kutu Q1-Q3 aralığını, ortadaki çizgi medyanı, bıyıklar 1.5-IQR sınırlarını, dışarıdaki noktalar aykırı adayları gösterir. Hafta içi ve hafta sonu kutularının medyanları belirgin biçimde farklı; bu, hafta_sonu sütununun tahmin için değerli bir özellik olduğunu grafik üzerinden söyler.

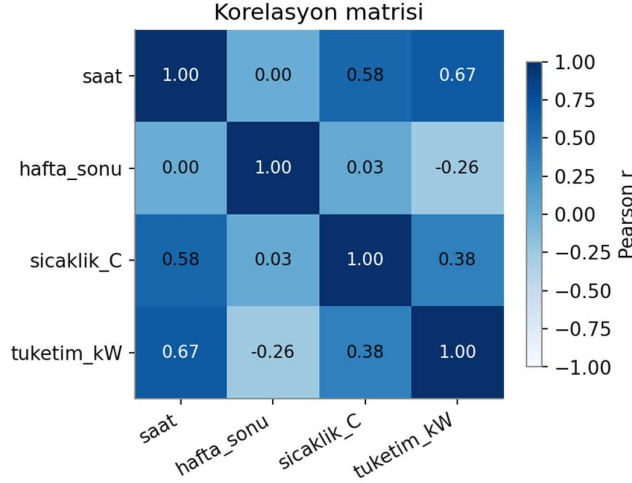
Grafik	Ne zaman?	matplotlib / pandas
Histogram	Tek değişkenin dağılımı (çarpıklık, çoklu tepe)	df.tuketim_kW.hist(bins=30)
Çizgi	Zamana bağlı değişim, profiller	profil.plot()
Saçılım (scatter)	İki sayısal değişken arasındaki ilişki	plt.scatter(df.sicaklik_C, df.tuketim_kW, s=8)
Kutu (boxplot)	Gruplar arası karşılaştırma, aykırı değerler	df.boxplot('tuketim_kW', by='hafta_sonu')
Isı haritası	Korelasyon matrisi, saat × gün tabloları	plt.imshow(df.corr()) veya seaborn.heatmap

Tablo 3.8 Temel grafik türleri

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 3, figsize=(13, 3.5))
temiz["tuketim_kW"].hist(bins=30, ax=ax[0]); ax[0].set_title("Tüketim dağılımı")
temiz.groupby("saat")["tuketim_kW"].mean().plot(ax=ax[1], marker="o"); ax[1].set_title("Saatlik profil")
ax[2].scatter(temiz["sicaklik_C"], temiz["tuketim_kW"], s=8); ax[2].set_title("Sıcaklık - tüketim")
plt.tight_layout(); plt.show()

print(temiz[["saat", "hafta_sonu", "sicaklik_C", "tuketim_kW"]].corr().round(2))
```



Şekil 3.5 Korelasyon matrisi ısı haritası: hedef (tuketim_kW) ile en güçlü ilişki saat sütununda

Korelasyon matrisi, her değişken çiftinin r değerini tek bir tabloda gösterir; köşegen her zaman 1'dir. Hedef sütununa (tuketim_kW) bakın: saat ile 0.67, sıcaklık ile 0.38, hafta sonu ile -0.26. Saatin en güçlü çıkması, günlük profilin baskınlığını; hafta sonunun negatif çıkması, hafta sonu tüketiminin düşük olmasını yansıtır. Sıcaklığın 0.38 gibi 'orta' çıkması ise yanıltıcıdır: saçılım grafiğinde gördüğümüz kırık çizgi ilişkisini Pearson korelasyonu tam yakalayamaz. İşte bu yüzden korelasyon tablosu tek başına yetmez; grafiklerle birlikte okunmalıdır.

Grafikten çıkarım: Şekil 3.3'teki saçılımda ilişki 24 °C civarına kadar düz, sonra artan bir eğri: klima yükü. Doğrusal olmayan bu ilişki 5. haftada 'özellik dönüşümü' (ör. max(T-24, 0) adında yeni bir sütun) ile doğrusal regresyona sığdırılacak. EDA'da fark edilen her doğrusal olmayan kalıp, ileride bir özellik mühendisliği fırsatıdır.

3.7 Ölçekleme: Neden Gerekli?

Örnek 1.2'de sıcaklık (0-40) ve saat (0-23) farklı aralıklardaydı. Bir özelliğin aralığı 0-1000 iken diğeri 0-1 ise uzaklık tabanlı (k-NN, k-means) ve gradyan tabanlı (regresyon, sinir ağı) yöntemler büyük olanı 'daha önemli' sanır; küçük ölçekli özellik fiilen yok sayılır. Bu yüzden özellikler modelden önce ortak bir ölçeğe getirilir. Ağaç tabanlı yöntemler (7. hafta) ölçeğe duyarlıdır; ama alışkanlık olarak her zaman ölçeklemek zarar vermez.

Yöntem	Formül	Sonuç aralığı	Ne zaman?
Min-max	$x' = (x - \min) / (\max - \min)$	[0, 1]	Sınırlar biliniyorsa; sinir ağı girişleri, görüntü pikselleri

Yöntem	Formül	Sonuç aralığı	Ne zaman?
Z-skor (standartlaştırma)	$x' = (x - \mu) / \sigma$	ortalama 0, std 1	Genel amaçlı; aykırı değerlere min-max'tan dayanıklı
Robust	$x' = (x - \text{medyan}) / \text{IQR}$	$\approx [-1, 1]$ çoğunluk	Ağır aykırı değer varsa
Log dönüşümü	$x' = \log(1 + x)$	Sıkıştırılmış	Sağa çarpık, birkaç büyüklük mertebesi yayılan veri (güç, gelir)

Tablo 3.9 Ölçekleme yöntemleri

Örnek 3.4 — Min-max ve z-skor elle

Soru: Sıcaklık ölçümleri [20, 24, 28, 32] için min-max ve z-skor (N'e bölen σ ile) ölçekli değerleri bulun.

Min-max: min = 20, max = 32, aralık = 12 $\rightarrow [0, 4/12, 8/12, 12/12] = [0, 0.333, 0.667, 1]$.

Z-skor: $\mu = 26$; sapmalar $-6, -2, 2, 6$; $\sigma^2 = (36+4+4+36)/4 = 20$; $\sigma = 4.472 \rightarrow [-1.342, -0.447, 0.447, 1.342]$. Z-skor'un yorumu sezgiseldir: 32 °C, ortalamanın 1.34 standart sapma üstündedir.

```
x = np.array([20, 24, 28, 32], dtype=float)
minmax = (x - x.min()) / (x.max() - x.min())
z = (x - x.mean()) / x.std()
print(minmax.round(3), z.round(3))

# scikit-learn ile aynı iş (4. haftadan itibaren böyle yapacağız)
from sklearn.preprocessing import MinMaxScaler, StandardScaler
print(StandardScaler().fit_transform(x.reshape(-1, 1)).ravel().round(3))
```

İleriye dönük uyarı: Kritik kural (4. haftada tekrar): ölçekleme parametreleri (min, max, μ , σ) yalnızca eğitim verisinden hesaplanır ve test verisine aynen uygulanır. Test verisini de katarak hesaplırsanız modele gelecekteki bilgi sızdırmış olursunuz ('veri kaçağı'). scikit-learn'de bu, 'fit' ile 'transform'ın ayrı olmasının nedenidir.

3.8 Nerede Veri Bulunur?

Ödev ve projeler için veri bulmak eskisinden çok kolaydır; asıl zorluk, verinin belgelenmiş ve probleme uygun olmasıdır. Aşağıdaki kaynaklar EEM projeleri için iyi başlangıç noktalarıdır. Bir veri setini kullanmadan önce üç soruyu cevaplayın: veri nasıl toplanmış (sensör, örnekleme hızı, süre), sütunlar ne anlama geliyor (birimler!), lisansı kullanımınıza izin veriyor mu?

Kaynak	İçerik	Erişim
scikit-learn gömülü setler	iris, diabetes, california_housing, digits — küçük, temiz, öğretim amaçlı	from sklearn.datasets import load_iris
UCI ML Repository	Individual Household Electric Power Consumption, Combined Cycle Power Plant, Appliances Energy, Steel Plates Faults	archive.ics.uci.edu
Kaggle	Binlerce veri seti; ör. Smart Grid, wind turbine SCADA, solar power generation	kaggle.com/datasets (hesap gerekir)
Case Western (CWRU)	Rulman arıza titreşim verisi (7. hafta)	engineering.case.edu
NASA Prognostics	Batarya yaşlanma, turbofan motor kalan ömür (RUL) verileri	nasa.gov/intelligent-systems-division

Kaynak	İçerik	Erişim
Kendi ölçümlerinizi	Arduino/ESP32 sensör kayıtları, osiloskop CSV çıktıları, telefon sensörleri (Phyphox)	CSV olarak kaydedip Colab'a yükleyin

Tablo 3.10 Veri kaynakları

Colab'da dosya erişimi

```
# Colab'a dosya yüklemenin üç yolu
# 1) Sol panel -> Files -> Upload (oturum kapanınca silinir)
# 2) Kod ile yükleme penceresi:
from google.colab import files
yuklenen = files.upload() # bilgisayardan seç
# 3) Google Drive'ı bağlama (kalıcı):
from google.colab import drive
drive.mount("/content/drive")
df = pd.read_csv("/content/drive/MyDrive/YZ_Temelleri/elektrik_tuketimi.csv")
```

3.9 Sınıf İçi Alıştırmalar

1. Veri setinde en yüksek 5 tüketim saatini (gün + saat) listeleyin; bunların sıcaklık ve gün tipini yorumlayın.
2. Hafta içi ve hafta sonu için ayrı ayrı saatlik profil çizin (aynı eksen, iki çizgi, açıklamalı). Tepe saatler farklı mı?
3. Sıcaklık ile tüketim korelasyonunu sadece saat 12-18 arası için hesaplayın; tüm veriyle karşılaştırın ve farkı Şekil 3.3 ile açıklayın.
4. Eksik sıcaklıkları (a) sütun ortalaması (b) interpolasyon ile doldurun; iki yöntemin sonuçlarını eksik noktalarda karşılaştırın.
5. Örnek 3.1'deki gibi, 5 elemanlı kendi seçtiğiniz bir veriyle medyanın aykırı değere dayanıklılığını gösterin.
6. `tuketim_kW` için z-skoru hesaplayıp $|z| > 3$ olan satırları bulun; IQR kuralıyla bulduklarınızla karşılaştırın.

3.10 Ödev 2 — EDA Raporu

- UCI 'Combined Cycle Power Plant' veri setini (AT, V, AP, RH → PE: ortam sıcaklığı, vakum, basınç, nem → net elektrik üretimi) veya dersin veri setini yükleyin.
- Veriyi tanıttın (satır/sütun, türler, birimler), eksik/aykırı değer analizi yapın ve kararınızı gerekçelendirin.
- En az 4 grafik üretin (histogram, saçılım, kutu, korelasyon ısı haritası) ve her grafiğin altına metin hücresinde 1-2 cümle yorum yazın.
- Hedef değişkenle en yüksek korelasyona sahip iki özelliği bulun ve neden mantıklı olduğunu fiziksel olarak açıklayın (ör. sıcaklık arttıkça gaz türbini verimi neden düşer?).
- Teslim: Colab defteri (metin hücreleriyle raporlaştırılmış), 'Run all' ile hatasız çalışmalı.

Hafta 4 — Makine Öğrenmesinin Temel Kavramları

Bölüm	Süre	İçerik
Saat 1	50 dk	Öğrenme türleri ayrıntılı; model, parametre, hiperparametre; eğitim-doğrulama-test ayrımı; genelleme
Saat 2	50 dk	Aşırı/eksik öğrenme, yanlılık-varyans, hata ölçütleri (elle), çapraz doğrulama, veri kaçağı
Saat 3	50 dk	Uygulama: train_test_split, taban modeller, MAE/RMSE/R ² hesabı, polinom derecesiyle aşırı öğrenme deneyi

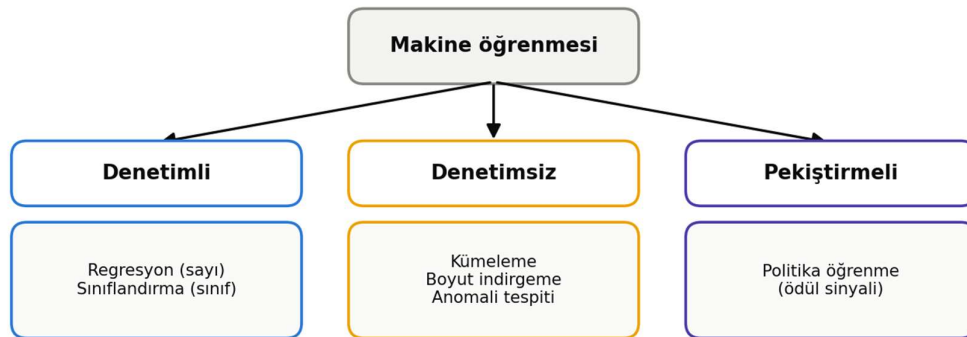
Tablo 4.1 Haftanın ders planı

4.1 Kazanımlar

- Denetimli, denetimsiz ve pekiştirmeli öğrenmeyi; regresyon ile sınıflandırmayı ayırt eder ve bir problemi bunlardan birine eşler.
- Model, parametre ve hiperparametre kavramlarını örneklerle açıklar.
- Veriyi eğitim / doğrulama / test olarak neden ve nasıl ayırdığını söyler; zaman serisinde rastgele ayırmanın neden yanlış olduğunu açıklar.
- Aşırı ve eksik öğrenmeyi öğrenme eğrisinden teşhis eder; yanlılık-varyans dengesini yorumlar.
- MAE, MSE, RMSE, MAPE ve R² ölçütlerini elle hesaplar ve hangisinin ne zaman uygun olduğunu bilir.
- k-katlı çapraz doğrulamayı uygular; veri kaçağını tanır ve önler.
- Her modelden önce bir taban (baseline) model kurar ve karşılaştırır.

4.2 Öğrenme Türleri: Ayrıntılı Bakış

İlk hafta üç öğrenme türünü tanıımıştık; bu hafta her birinin ne tür bir 'çıktı' ürettiğine ve nasıl değerlendirildiğine odaklanıyoruz. Denetimli öğrenmede iki alt tür vardır: hedef sürekli bir sayıysa (yük kW, sıcaklık °C, kalan ömür saat) regresyon; hedef sonlu sayıda sınıftan biriye (arızalı/sağlam, hangi arıza tipi, hangi modülasyon) sınıflandırma. Aynı problem iki şekilde de kurulabilir: 'rulmanın kalan ömrü kaç saat?' regresyon, 'rulman bir hafta içinde arızalanır mı?' sınıflandırmadır. Hangisini seçeceğiniz, tahminin besleyeceği karara bağlıdır.



Şekil 4.1 Makine öğrenmesi türleri ve alt türleri

Tür	Alt tür	Çıktı	Değerlendirme	Bu derste
Denetimli	Regresyon	Sürekli sayı	MAE, RMSE, R ²	5. hafta

Tür	Alt tür	Çıktı	Değerlendirme	Bu derste
Denetimli	Sınıflandırma	Sınıf etiketi / olasılık	Doğruluk, precision, recall, F1, ROC	6-7. hafta
Denetimsiz	Kümeleme	Grup numarası	Küme içi dağılım, silüet	8. hafta
Denetimsiz	Boyut indirgeme	Daha az sayıda yeni özellik	Açıklanan varyans	8. hafta
Denetimsiz	Anomali tespiti	Normal / anormal skoru	Bilinen anomalilerde recall	14. hafta
Pekiştirmeli	—	Eylem politikası	Toplam ödül	14. hafta (kısası)

Tablo 4.2 Öğrenme türleri ve değerlendirme

Model, parametre, hiperparametre

Model, girdiden çıktıya giden ve içinde ayarlanabilir sayılar barındıran bir fonksiyondur: $\hat{y} = f(X; w)$. Buradaki w 'ler parametredir ve veriden öğrenilir — doğrusal regresyonda katsayılar, sinir ağında ağırlıklar.

Hiperparametre ise öğrenme başlamadan önce mühendisin seçtiği ayarlardır: polinom derecesi, k-NN'deki k , ağın katman sayısı, öğrenme oranı. Parametreler eğitim verisiyle, hiperparametreler doğrulama verisiyle belirlenir; bu ayırım, bu haftanın en önemli fikri olan veri ayırımının nedenidir.

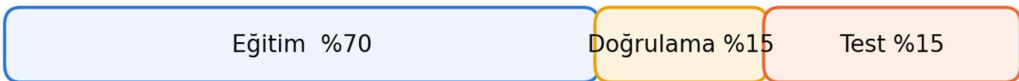
Model	Parametreler (öğrenilir)	Hiperparametreler (seçilir)
Doğrusal regresyon	Katsayılar w , sabit b	Düzenleştirme katsayısı λ , özellik dönüşümleri
k-NN	Yok (veri saklanır)	k , uzaklık ölçüsü
Karar ağacı	Bölme kuralları	Maksimum derinlik, yaprak başına min örnek
Sinir ağı	Ağırlıklar ve sapmalar	Katman/nöron sayısı, öğrenme oranı, epoch, dropout

Tablo 4.3 Parametre ve hiperparametre örnekleri

4.3 Genelleme ve Veri Ayırımı

Bir modelin amacı, eğitimde gördüğü örneklerde iyi olmak değil, görmediği yeni örneklerde iyi olmaktır; buna genelleme denir. Eğitim verisindeki hata, genelleme hakkında yanıltıcı bir bilgidir: model veriyi ezberlemişse eğitim hatası sıfıra yakın ama gerçek hayattaki hata büyüktür. Bu yüzden veri, model görmeden önce üçe ayrılır. Eğitim kümesi parametreleri öğrenmek, doğrulama kümesi hiperparametreleri seçmek ve modeller arasında karar vermek, test kümesi ise en sonda, yalnızca bir kez, nihai başarıyı raporlamak içindir. Test kümesine bakarak model ayarlamak, sınav sorularını görüp çalışmaya benzer: puan yüksek çıkar ama hiçbir şey öğrenilmemiştir.

Rastgele ayırım (bağımsız örnekler)



Zamana göre ayırım (zaman serisi — geleceği geçmişle tahmin et)



Şekil 4.2 Rastgele ayırım (bağımsız örnekler) ve zamana göre ayırım (zaman serisi)

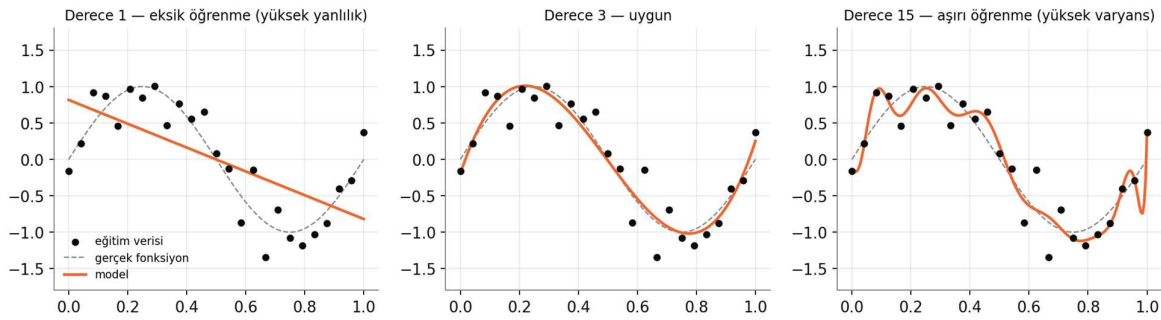
Ayırımın nasıl yapılacağı verinin yapısına bağlıdır. Örnekler birbirinden bağımsızsa (farklı motorların ölçümleri, farklı PCB görüntüleri) rastgele karıştırıp bölmek doğrudur. Ama veri bir zaman serisiyse (saatlik yük, sensör kaydı) rastgele ayırım ciddi bir hata olur: test kümesindeki bir saatin bir önceki ve bir sonraki saati eğitim kümesinde olur, model komşuları 'kopyalayarak' gerçekte olmayan bir başarı gösterir. Zaman serisinde kural, geleceği geçmişle tahmin etmektir: ilk günler eğitim, son günler test. Benzer bir dikkat 'gruplu' verilerde de gerekir: aynı motordan alınan 100 titreşim kaydı hem eğitimde hem testte varsa, model motoru tanır, arızayı değil.

Veri yapısı	Doğru ayırım	Yanlış ayırım ve sonucu
Bağımsız örnekler	Rastgele karıştır, %70/15/15 böl (stratify: sınıf oranlarını koru)	—
Zaman serisi	Zamana göre: ilk %80 eğitim, son %20 test; ileriye dönük doğrulama	Rastgele → komşu saatler sızar, iyimser hata
Gruplu veri (motor, hasta, cihaz)	Grupları böl (GroupKFold): bir motor ya eğitimde ya testte	Kayıt bazında rastgele → model motoru ezberler
Az veri (< birkaç yüz)	k-katlı çapraz doğrulama	Tek ayırım → sonuç şansa bağlı

Tablo 4.4 Veri yapısına göre ayırım stratejisi

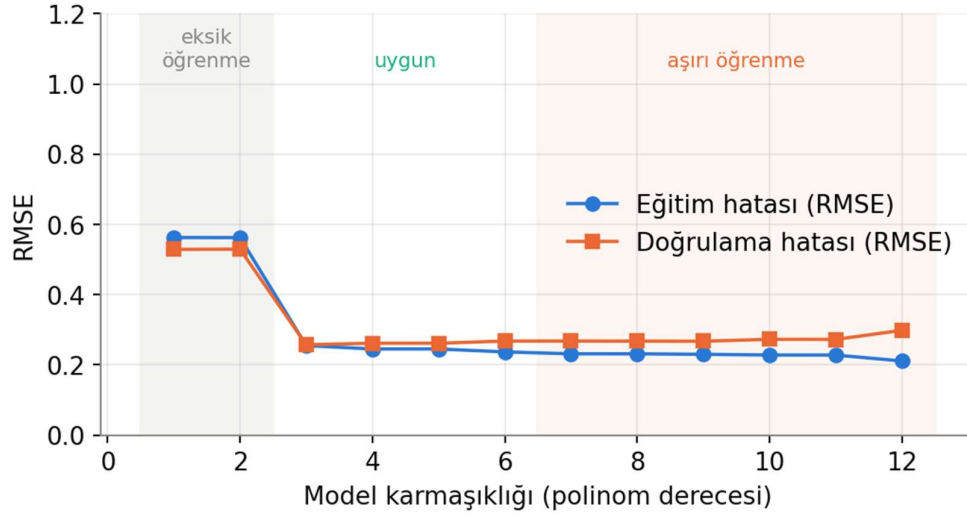
4.4 Aşırı ve Eksik Öğrenme

Model karmaşıklığı ile genelleme arasında bir denge vardır. Çok basit bir model veriyi anlatamaz: doğrusal bir çizgi, sinüs biçimli bir ilişkiyi yakalayamaz; eğitim hatası da test hatası da yüksektir. Buna eksik öğrenme (underfitting) denir ve modelin yanlışlığı (bias) yüksektir. Çok karmaşık bir model ise eğitim verisindeki gürültüyü de öğrenir: 25 noktadan geçen 15. derece polinom her noktayı tutturur ama aralarda çılgınca salınır; eğitim hatası sıfıra yakın, test hatası büyüktür. Buna aşırı öğrenme (overfitting) denir ve modelin varyansı yüksektir — eğitim verisi biraz değişse model tamamen değişirdi.



Şekil 4.3 Aynı 25 noktaya uydurulan 1., 3. ve 15. derece polinomlar

Bu dengeyi görmenin en iyi yolu, model karmaşıklığına karşı eğitim ve doğrulama hatasını birlikte çizmektir. Eğitim hatası karmaşıklık arttıkça hep düşer; doğrulama hatası önce düşer, sonra tekrar yükselir. İki eğrinin ayrıştığı nokta aşırı öğrenmenin başladığı yerdir; en iyi model doğrulama hatasının en düşük olduğu karmaşıklıktadır. Aynı grafiği epoch sayısına (sinir ağı) veya ağaç derinliğine karşı da çizebilirsiniz; şekil hep aynıdır.



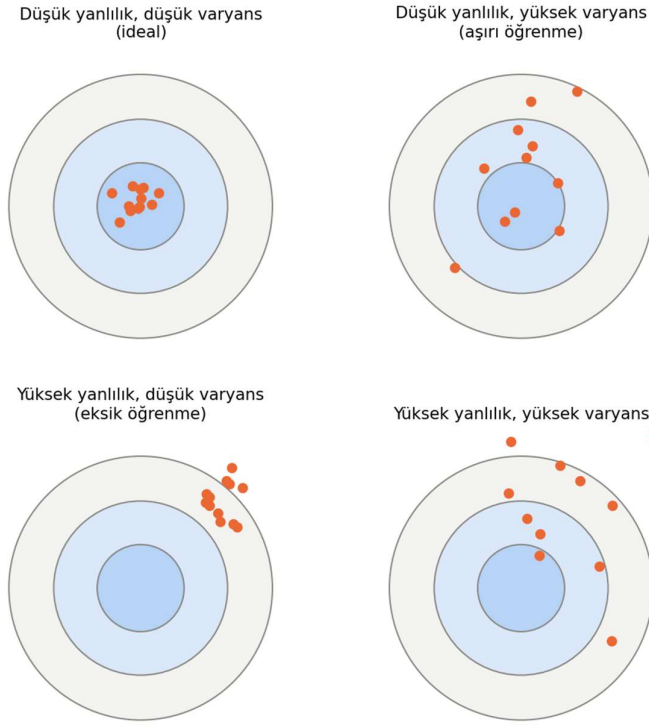
Şekil 4.4 Karmaşıklık arttıkça eğitim hatası hep düşer; doğrulama hatası U biçimlidir

Belirti	Teşhis	Çare
Eğitim hatası yüksek, doğrulama hatası yüksek ve yakın	Eksik öğrenme (yüksek yanlılık)	Daha karmaşık model, daha iyi özellikler, daha uzun eğitim, düzenlileştirmeyi azalt
Eğitim hatası düşük, doğrulama hatası çok daha yüksek	Aşırı öğrenme (yüksek varyans)	Daha fazla veri, daha basit model, düzenlileştirme, erken durdurma, özellik azaltma
Her ikisi düşük ve yakın	İyi genelleme	Test kümesinde raporla
Doğrulama hatası eğitimden düşük	Şüpheli: veri kaçağı veya ayırım hatası	Ayrımı ve ön işleme sırasını kontrol et

Tablo 4.5 Öğrenme eğrisi teşhis tablosu

Yanlılık–varyans dengesi

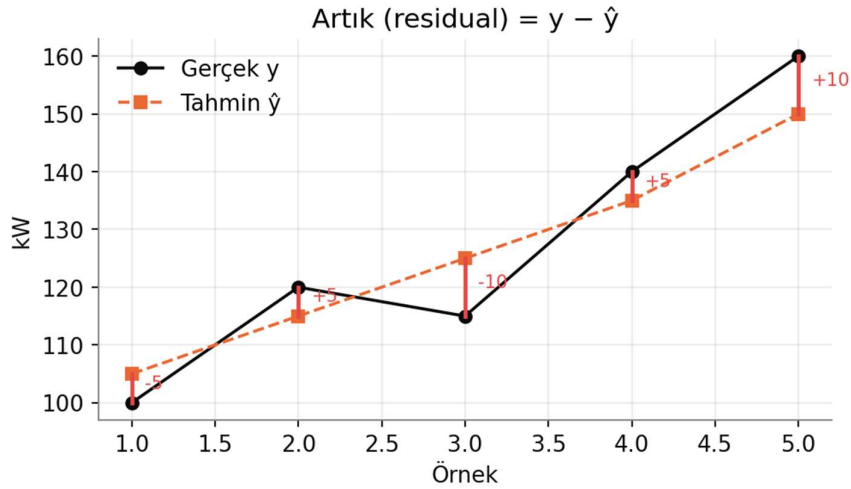
Bir modelin test hatası üç bileşene ayrılır: yanlılık² + varyans + indirgenemez gürültü. Yanlılık, modelin sistematik hatasıdır (yanlış varsayım: 'ilişki doğrusaldır'); varyans, modelin eğitim verisinin rastgeleliğine duyarlılığıdır; gürültü ise ölçümün doğasındaki, hiçbir modelin gideremeyeceği kısımdır. Karmaşıklık arttıkça yanlılık düşer ama varyans artar; toplam hata U biçimlidir. Aşağıdaki hedef tahtası benzetmesi bunu sezgisel anlatır: her atış, farklı bir eğitim verisiyle eğitilmiş modelin tahminidir.



Şekil 4.5 Yanlılık (merkezden sapma) ve varyans (dağınıklık)

4.5 Hata Ölçütleri (Regresyon)

Bir tahmin ne kadar iyi? Cevap, hangi ölçütle baktığınıza bağlıdır ve doğru ölçütü seçmek mühendisin işidir. Tüm regresyon ölçütleri artık (residual) $e = y - \hat{y}$ üzerinden tanımlanır. MAE (ortalama mutlak hata) 'ortalama kaç kW şaşıyoruz?' sorusunu cevaplar, birimi hedefle aynıdır ve her hatayı eşit sayar. MSE ve karekökü RMSE büyük hataları karesiyle cezalandırır: 10 kW'lık bir hata, 1 kW'lık on hatadan daha kötüdür; bu, büyük sapmaların pahalı olduğu durumlarda (şebeke dengeleme) istenen davranıştır. MAPE yüzde cinsinden konuşur ama hedef sıfıra yakinken patlar (gece tüketimi küçükse yüzde hata şişer). R^2 ise modelin, hedefteki değişimin ne kadarını açıkladığını söyler: 1 mükemmel, 0 'ortalamayı tahmin etmek kadar iyi', negatif ise ortalamadan bile kötü.



Şekil 4.6 Artık = gerçek - tahmin; ölçütler bu farklardan türetilir

Ölçüt	Formül	Birim	Ne zaman?
MAE	$(1/N) \sum y_i - \hat{y}_i $	y ile aynı	Hataların eşit maliyetli olduğu, aykırılara dayanıklı ölçüm istenen durumlar
MSE	$(1/N) \sum (y_i - \hat{y}_i)^2$	y^2	Eğitimde kayıp fonksiyonu (türevi kolay)
RMSE	$\sqrt{\text{MSE}}$	y ile aynı	Büyük hataların pahalı olduğu durumlar; raporlama
MAPE	$(100/N) \sum y_i - \hat{y}_i / y_i $	%	Farklı ölçekli hedefleri karşılaştırma; $y \approx 0$ ise KULLANMA
R^2	$1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$	birimsiz	Modelin açıkladığı varyans oranı; 0 = ortalama kadar

Tablo 4.6 Regresyon ölçütleri

Örnek 4.1 — Beş tahmin için MAE, MSE, RMSE, MAPE ve R^2 (elle)

Soru: Beş saat için gerçek yük $y = [100, 120, 115, 140, 160]$ kW, model tahmini $\hat{y} = [105, 115, 125, 135, 150]$. Beş ölçütü hesaplayın.

i	y_i	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$	$ e_i $	e_i^2	$ e_i /y_i$	$(y_i - \bar{y})^2$
1	100	105	-5	5	25	0.050	729
2	120	115	5	5	25	0.042	49
3	115	125	-10	10	100	0.087	144
4	140	135	5	5	25	0.036	169
5	160	150	10	10	100	0.063	1089
Σ	635			35	275	0.277	2180

$\bar{y} = 635/5 = 127$. $\text{MAE} = 35/5 = 7.0$ kW. $\text{MSE} = 275/5 = 55$ kW². $\text{RMSE} = \sqrt{55} \approx 7.42$ kW. $\text{MAPE} = 100 \times 0.277/5 \approx \%5.5$. $R^2 = 1 - 275/2180 = 1 - 0.126 = 0.874$.

Yorum: Model ortalama 7 kW şaşıyor; $\text{RMSE} > \text{MAE}$ olması hataların eşit olmadığını (10'lar var) gösterir — ikisi eşitse tüm hatalar aynı büyüklüktedir. $R^2 = 0.87$: yükteki değişimin %87'si açıklanıyor. Şebeke için %5.5'lik MAPE kabul edilebilir bir ilk sonuçtur.

```
import numpy as np
y = np.array([100, 120, 115, 140, 160]); yp = np.array([105, 115, 125, 135, 150])
e = y - yp
print("MAE :", np.abs(e).mean()) # 7.0
print("MSE :", (e**2).mean()) # 55.0
print("RMSE:", np.sqrt((e**2).mean()).round(2)) # 7.42
print("MAPE:", (100*np.abs(e)/y).mean().round(1)) # 5.5
print("R2 :", (1 - (e**2).sum()/((y-y.mean())**2).sum()).round(3)) # 0.874

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
print(mean_absolute_error(y, yp), round(np.sqrt(mean_squared_error(y, yp)), 2), round(r2_score(y, yp), 3))
```

Örnek 4.2 — Hangi model? (doğrulama hatasıyla seçim)

Soru: Üç model aynı veride eğitildi; eğitim ve doğrulama RMSE'leri tabloda. Hangisini seçersiniz, neden?

Model	Eğitim RMSE (kW)	Doğrulama RMSE (kW)
A: doğrusal (3 özellik)	18.2	18.9
B: polinom derece 3	11.4	12.1
C: polinom derece 12	3.1	27.5

Çözüm: C en düşük eğitim hatasına sahip ama doğrulama hatası eğitimin 9 katı: klasik aşırı öğrenme, elenir. A'da iki hata yakın ama ikisi de yüksek: eksik öğrenme. B hem düşük hem birbirine yakın: seçilir. Sonra ve yalnızca bir kez B'nin test RMSE'si raporlanır. Eğitim hatasına bakarak C'yi seçen bir mühendis sahada 27.5 kW'lık hatayla karşılaşır.

4.6 Çapraz Doğrulama

Tek bir eğitim/doğrulama ayrımı şansa bağlıdır: doğrulama kümesine tesadüfen 'kolay' örnekler düşmüşse model olduğundan iyi görünür. Az veride bu risk büyüktür. k-katlı çapraz doğrulama, veriyi k eşit parçaya böler; sırayla her parçayı doğrulama, kalanını eğitim yapar ve k hata değerinin ortalamasını (ve standart sapmasını) raporlar. Böylece her örnek tam bir kez doğrulamada kullanılır ve sonuç tek ayrımın şansına bağlı kalmaz. Standart sapma da bilgi taşır: iki modelin ortalama hataları yakınsa ama birinin sapması çok büyükse o model kararsızdır. Zaman serisinde k-katlı yerine 'ileriye dönük' (TimeSeriesSplit) kullanılır: her katta geçmişle eğit, sonraki dilimde doğrula.



Sonuç: $\text{ortalama}(\text{hata}_1.. \text{hata}_5) \pm \text{std}$ — tek bir ayrımın şansına bağlı kalmaz

Şekil 4.7 5-katlı çapraz doğrulama: her kat bir kez doğrulama olur

Örnek 4.3 — 5 katın ortalaması ve sapması

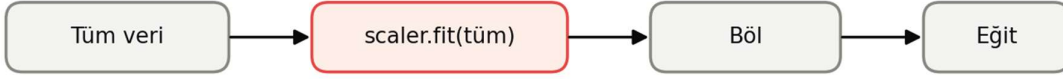
Soru: 5-katlı çapraz doğrulamada RMSE değerleri 12.1, 13.4, 11.8, 14.9, 12.8 kW çıktı. Ortalama ve standart sapmayı bulun; 'modelin hatası 12 kW' demek doğru mu?

Çözüm: Ortalama = $(12.1 + 13.4 + 11.8 + 14.9 + 12.8)/5 = 65.0/5 = 13.0$ kW. Sapmalar: -0.9, 0.4, -1.2, 1.9, -0.2 → kareler: 0.81, 0.16, 1.44, 3.61, 0.04 → toplam 6.06 → $\sigma = \sqrt{(6.06/5)} \approx 1.10$ kW. Doğru rapor: 'RMSE = 13.0 ± 1.1 kW'. Tek bir ayırmada 11.8 çıkıp 'hata 12 kW' demek iyimser olurdu; en kötü kat 14.9 kW'dır ve sahada bu da mümkündür.

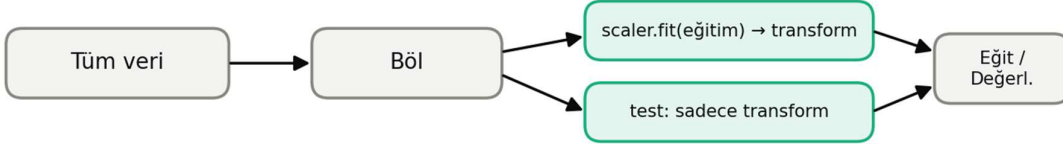
4.7 Veri Kaçağı (Data Leakage)

Veri kaçağı, test verisine ait bilginin herhangi bir yoldan eğitime sızmasıdır ve makine öğrenmesindeki en sinsi hatadır: model her şey yolunda görünürken sahada çöker. En yaygın biçimi ön işlemedir. Ölçekleyici (StandardScaler) tüm veriye uydurulursa test verisinin ortalaması ve standart sapması eğitime karışır. Eksik değerler tüm verinin ortalamasıyla doldurulursa aynı şey olur. Doğru sıra: önce böl, sonra tüm ön işlemeyi yalnızca eğitim kümesine 'fit' et, test kümesine sadece 'transform' uygula. scikit-learn'ün Pipeline yapısı bu sırayı otomatik olarak korur; bu yüzden ödevlerde Pipeline kullanımı teşvik edilir.

YANLIŞ: ölçekleyici tüm veriye uydurulur (test bilgisi sızar)



DOĞRU: önce böl, ölçekleyici yalnızca eğitim verisine uydurulur



Şekil 4.8 Ölçekleyicinin yanlış ve doğru kullanımı

Kaçak türü	Örnek	Önlem
Ön işleme kaçağı	Scaler / imputer tüm veriye fit	Böl → fit(eğitim) → transform(test); Pipeline
Hedef kaçağı	'Fatura tutarı' sütunuyla tüketimi tahmin etmek (tutar tüketimden hesaplanır)	Tahmin anında bilinemeyecek sütunları at
Zaman kaçağı	Zaman serisinde rastgele ayırım; gelecekteki değerlerle özellik üretmek	Zamana göre ayırım; sadece geçmiş pencereden özellik
Grup kaçağı	Aynı motorun kayıtları hem eğitimde hem testte	GroupKFold; gruba göre ayır
Test kümesine bakarak ayar	Hiperparametreyi test hatasına göre seçmek	Doğrulama kümesi / CV; test bir kez

Tablo 4.7 Veri kaçağı türleri

Örnek 4.4 — Kaçağın sayısal etkisi

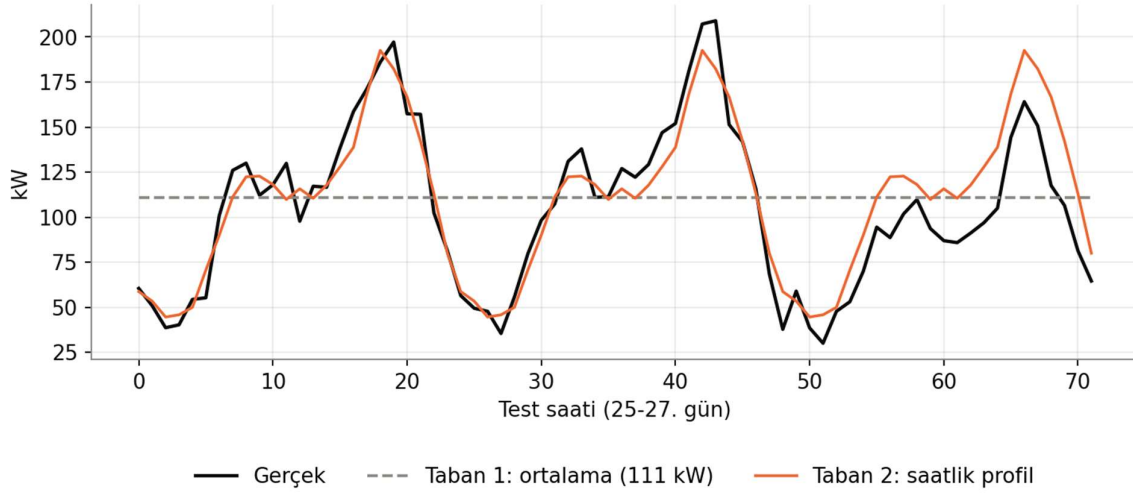
Soru: Sıcaklık sütununun eğitim kümesinde ortalaması 20 °C, std'si 6; test kümesi (son günler, sıcak hava) ortalaması 28 °C. Test örneği 28 °C'nin z-skoru, (a) doğru yöntemle (eğitim parametreleriyle) (b) tüm veriye fit edilmiş scaler ile (tüm veri: $\mu = 22$, $\sigma = 7$) ne olur?

Çözüm: (a) $z = (28 - 20)/6 = 1.33$. (b) $z = (28 - 22)/7 = 0.86$. Yanlış yöntem test örneğini olduğundan 'daha normal' gösterir; model, gerçekte hiç görmediği kadar sıcak bir günü sıradan sanır ve klima yükünü düşük tahmin eder. Sahada ise scaler eğitim parametreleriyle çalışacağından değerlendirmede görülen başarı gerçekleşmez.

4.8 Taban Model: Her Şeyden Önce

Karmaşık bir modele başlamadan önce mutlaka basit bir referans kurun. Yük tahmininde iki doğal taban vardır: eğitim ortalamasını tahmin etmek ($R^2 = 0$ 'ın tanımı) ve saatlik profili kullanmak — yani 'bu saatte genellikle ne tüketiliyorsa onu söyle'. Zaman serilerinde 'dünün aynı saati' (naif tahmin) de güçlü bir

tabandır. Şaşırtıcı biçimde, bu basit yöntemler çoğu zaman iyi sonuç verir ve karmaşık modelin gerçekten katkı sağlayıp sağlamadığını ancak onlarla karşılaştırdığınızda anlarsınız. Taban modeli yenemeyen sinir ağı, yanlış kurulmuş bir problemin işaretidir.



Şekil 4.9 Dersin veri setinde iki taban model: ortalama ve saatlik profil (test: 25-27. günler)

Taban modeller — 5. haftada regresyon bunları yenmeye çalışacak

```
import pandas as pd, numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("elektrik_tuketimi.csv")
df = df[df.tuketim_kw < 300].copy()
df["sicaklik_C"] = df["sicaklik_C"].interpolate()

# Zamana göre ayırım: 1-24. gün eğitim, 25-30. gün test
egitim, test = df[df.gun <= 24], df[df.gun > 24]

def rapor(ad, y, yp):
    print(f"{ad:28s} MAE={mean_absolute_error(y, yp):6.2f} RMSE={np.sqrt(mean_squared_error(y, yp)):6.2f} R2={r2_score(y, yp):6.3f}")

# Taban 1: eğitim ortalaması
rapor("Taban: ortalama", test.tuketim_kw, np.full(len(test), egitim.tuketim_kw.mean()))
# Taban 2: saatlik profil (eğitimden)
profil = egitim.groupby("saat")["tuketim_kw"].mean()
rapor("Taban: saatlik profil", test.tuketim_kw, test.saat.map(profil))
# Taban 3: saat x gün tipi profili
profil2 = egitim.groupby(["hafta_sonu", "saat"])[ "tuketim_kw"].mean()
rapor("Taban: saat x gün tipi", test.tuketim_kw, [profil2[(h, s)] for h, s in zip(test.hafta_sonu, test.saat)])
```

4.9 Sınıf İçi Sorular

1. Bir rulman için 'kalan ömür (saat)' ve 'bir hafta içinde arıza (evet/hayır)' problemlerinden hangisi regresyon, hangisi sınıflandırmadır? Bakım ekibi için hangisi daha kullanışlıdır?
2. Tabloda eğitim RMSE 2, doğrulama RMSE 20 olan bir model görüyorsunuz. Teşhis? Üç çare sayın.
3. $y = [50, 200]$ ve $\hat{y} = [60, 190]$ için MAE ve MAPE hesaplayın. MAPE'nin ilk örneği neden daha çok cezalandırdığını açıklayın.
4. Saatlik yük verisini rastgele %80/20 böldünüz ve $R^2 = 0.98$ çıktı. Bu sonuca neden güvenmemelisiniz?
5. Ölçekleyiciyi tüm veriye fit etmek neden kaçıktır; hangi parametreler sızır?

4.10 Kısa Sınav 1 (15 dk, örnek sorular)

1. YZ, MÖ ve DÖ arasındaki ilişkiyi bir şekilde gösterin ve her biri için bir EEM örneği verin. (3 p)
2. $y = [10, 20, 30]$, $\hat{y} = [12, 18, 33]$ için MAE ve RMSE hesaplayın. (4 p)
3. Aşırı öğrenmeyi tanımlayın; öğrenme eğrisinden nasıl anlaşılır? (3 p)

4.11 Uygulama Saati ve Ödev

- Hafta04 defteri: veri ayrımı (zamana göre vs rastgele — farkı ölçün), üç taban model, ölçütleri elle ve sklearn ile hesaplama, polinom derecesiyle aşırı öğrenme deneyi, 5-katlı CV.
- Alıştırmalar defterin sonunda; çözümler eğitmen sürümünde.
- Gelecek hafta: doğrusal regresyon — taban modelleri yenen ilk 'gerçek' model.

Hafta 5 — Doğrusal Regresyon

Bölüm	Süre	İçerik
Saat 1	50 dk	Basit doğrusal regresyon: model, maliyet fonksiyonu, en küçük kareler (elle çözüm), katsayı yorumu, R^2
Saat 2	50 dk	Gradyan inişi (elle iterasyon), öğrenme oranı; çoklu regresyon ve matris biçimi (normal denklem); özellik dönüşümü; düzenleme (ridge/lasso)
Saat 3	50 dk	Uygulama: elle $\rightarrow$ NumPy $\rightarrow$ scikit-learn; dersin veri setinde taban modeli yenmek; PV üretim verisi

Tablo 5.1 Haftanın ders planı

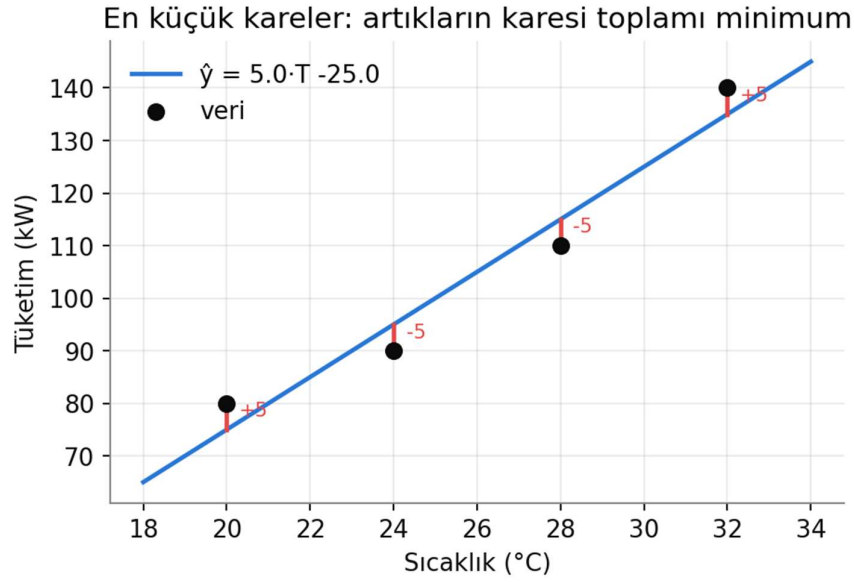
5.1 Kazanımlar

- Doğrusal regresyon modelini ve MSE maliyet fonksiyonunu yazar; en küçük kareler çözümünü elle türetir ve küçük veride hesaplar.
- Gradyan inişini elle birkaç iterasyon uygular; öğrenme oranının etkisini açıklar.
- Çoklu regresyonu matris biçiminde yazar; normal denklemini NumPy ile çözer.
- Katsayıları fiziksel birimlerle yorumlar ($\text{kW}/^\circ\text{C}$); ölçeklemenin katsayı yorumuna etkisini bilir.
- Doğrusal olmayan ilişkileri özellik dönüşümüyle (kırık çizgi, sin/cos, polinom) doğrusal modele sığdırır.
- Ridge ve lasso düzenlemesinin ne yaptığını ve λ 'nın nasıl seçildiğini açıklar.

5.2 Model ve Maliyet Fonksiyonu

Doğrusal regresyon, makine öğrenmesinin 'merhaba dünya'sıdır; ama önemi bir alıştırma olmasından çok öte: hem sinir ağlarının tek nöronu hem de kestirim teorisinin temelidir. Model şudur: tek özellik için $\hat{y} = w \cdot x + b$; çok özellik için $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$. w 'ler ağırlık (eğim), b ise sabit (kesim noktası). Modelin 'doğrusal' olması, parametrelere göre doğrusal olması demektir; özellikler x 'in karesi ya da sinüsü olabilir — bu ayrım 5.6'da işimize yarayacak.

Öğrenme, parametreleri tahminlerin gerçeğe en yakın olacağı şekilde seçmektir. 'En yakın'ı ölçmek için bir maliyet (kayıp) fonksiyonu gerekir; doğrusal regresyonda standart seçim ortalama kare hatadır: $J(w, b) = (1/N) \sum (\hat{y}_i - y_i)^2$. Kare almanın üç nedeni var: negatif ve pozitif hatalar birbirini götürmez, büyük hatalar daha çok cezalandırılır ve türevi her yerde tanımlı olduğundan minimum hem kapalı formülle hem gradyanla bulunabilir. Bu maliyeti en küçükleyen (w, b) çiftine en küçük kareler çözümü denir.



Şekil 5.1 En küçük kareler: dikey artıkların karesi toplamını en küçükleyen doğru

Basit regresyonun kapalı çözümü

Tek özellikli durumda J'nin w ve b 'ye göre türevlerini sıfıra eşitleyince iki denklem çıkar ve çözüm kapalı biçimde yazılır: $w = \sum(x_i - \bar{x})(y_i - \bar{y}) / \sum(x_i - \bar{x})^2$, $b = \bar{y} - w \cdot \bar{x}$. Payı 3. haftadaki korelasyon formülünden tanıyorsunuz: aslında $w = r \cdot (\sigma_y / \sigma_x)$. Yani eğim, korelasyonun birim dönüştürülmüş hâlidir; korelasyon 'ne kadar doğrusal', eğim 'x bir birim artınca y kaç birim artar' sorusunu cevaplar.

Örnek 5.1 — Dört noktaya doğru uydurma (elle)

Soru: Sıcaklık $x = [20, 24, 28, 32]$ °C, tüketim $y = [80, 90, 110, 140]$ kW. En küçük kareler doğrusunu bulun; 30 °C için tahmin yapın; R^2 'yi hesaplayın.

Çözüm: 3. haftada (Örnek 3.2) hesaplamıştık: $\bar{x} = 26$, $\bar{y} = 105$, $\sum(x - \bar{x})(y - \bar{y}) = 400$, $\sum(x - \bar{x})^2 = 80$. Eğim $w = 400/80 = 5.0$ kW/°C. Sabit $b = 105 - 5 \times 26 = -25$. Model: $\hat{y} = 5T - 25$.

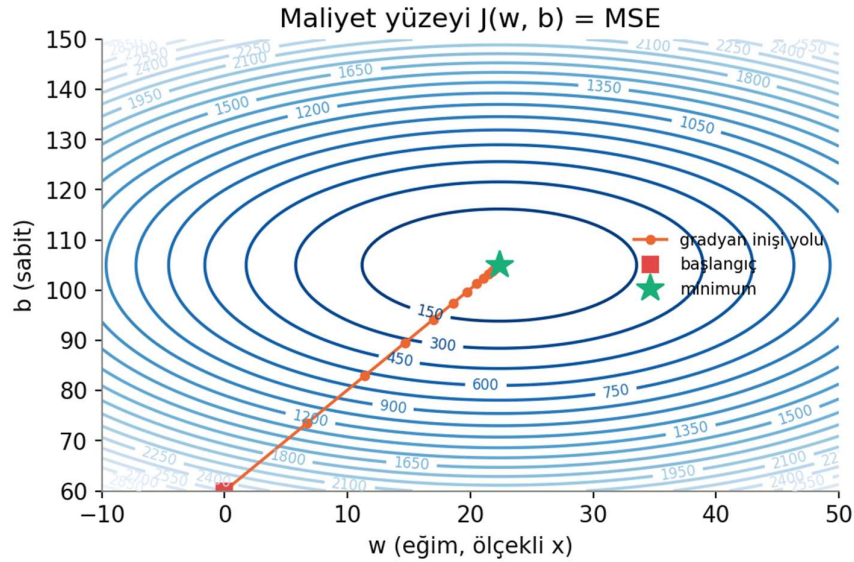
T	y	$\hat{y} = 5T - 25$	artık	artık ²
20	80	75	5	25
24	90	95	-5	25
28	110	115	-5	25
32	140	135	5	25
			Σ	100

$MSE = 100/4 = 25$ kW² $\rightarrow$ $RMSE = 5$ kW. $\sum(y - \hat{y})^2 = 2100$ (Örnek 3.2) $\rightarrow R^2 = 1 - 100/2100 = 0.952$. (Kontrol: $r = 0.976$, $r^2 = 0.952$ — basit regresyonda $R^2 = r^2$.) 30 °C için $\hat{y} = 5 \times 30 - 25 = 125$ kW. Yorum: her 1 °C artış $\approx$ 5 kW ek yük (klima). Sabit -25 kW'ın fiziksel anlamı yoktur (0 °C verinin dışında); ekstrapolasyona dikkat.

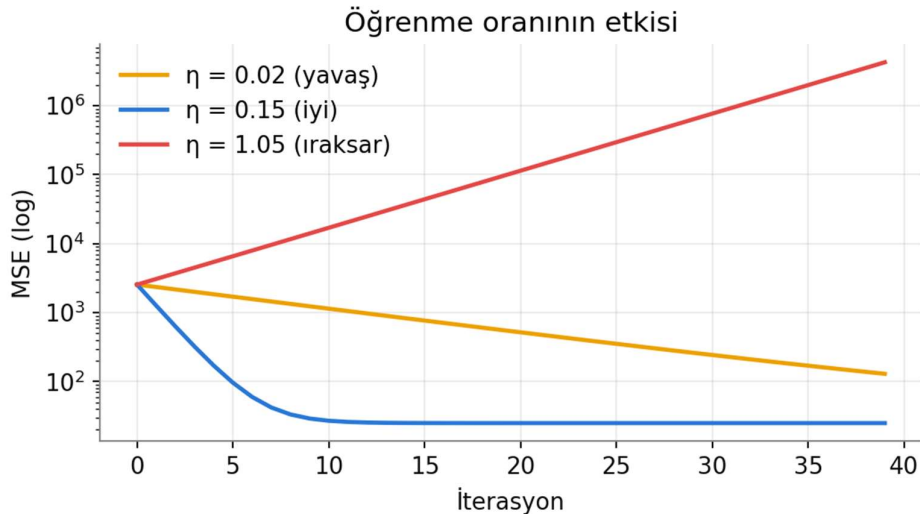
```
import numpy as np
x = np.array([20, 24, 28, 32.]); y = np.array([80, 90, 110, 140.])
w = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean())**2).sum()
b = y.mean() - w * x.mean()
yp = w*x + b
print(w, b, "R2 =", 1 - ((y-yp)**2).sum()/((y-y.mean())**2).sum()) # 5.0 -25.0 0.952
print("30 °C ->", w*30 + b) # 125.0
```

5.3 Gradyan İnişi

Kapalı çözüm güzeldir ama her modelde yoktur (sinir ağlarında yoktur). Genel yöntem gradyan inişidir: parametrelere rastgele bir değer ver, maliyetin parametrelere göre türevini (gradyanı) hesapla, parametreleri gradyanın tersi yönünde küçük bir adım güncelle, tekrarla. Gradyan, maliyetin en hızlı arttığı yönü gösterir; tersi yönde yürümek maliyeti azaltır. Doğrusal regresyonda türevler basittir: $\partial J/\partial w = (2/N) \sum (\hat{y}_i - y_i) x_i$ ve $\partial J/\partial b = (2/N) \sum (\hat{y}_i - y_i)$. Güncelleme: $w \leftarrow w - \eta \partial J/\partial w$, $b \leftarrow b - \eta \partial J/\partial b$. Buradaki η öğrenme oranıdır ve 2. haftadaki RC simülasyonunun dt'si gibi bir adım büyüklüğüdür.



Şekil 5.2 Maliyet yüzeyi (eş yükselti eğrileri) ve gradyan inişinin minimuma giden yolu



Örnek 5.2 — İki iterasyon elle

Soru: Ölçeklenmiş veride $x = [-1, 1]$, $y = [1, 3]$ olsun. $w = 0$, $b = 0$ 'dan başlayıp $\eta = 0.1$ ile iki gradyan inişi adımı atın.

İterasyon 1: $\hat{y} = [0, 0]$; hatalar $\hat{y} - y = [-1, -3]$. $\partial J / \partial w = (2/2)[(-1)(-1) + (-3)(1)] = 1 - 3 = -2$. $\partial J / \partial b = (2/2)(-1 - 3) = -4$. $w = 0 - 0.1(-2) = 0.2$; $b = 0 - 0.1(-4) = 0.4$.

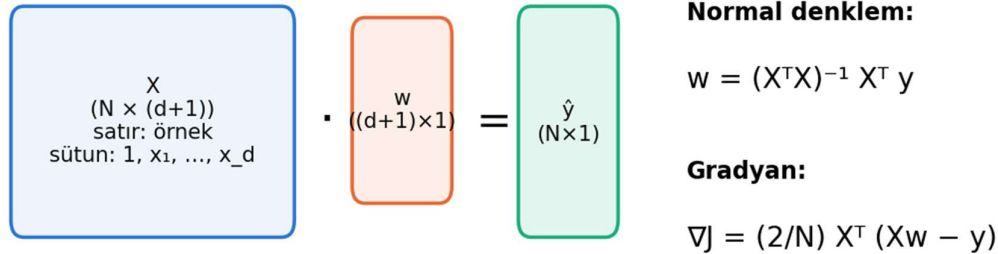
İterasyon 2: $\hat{y} = [0.2(-1) + 0.4, 0.2(1) + 0.4] = [0.2, 0.6]$; hatalar $[-0.8, -2.4]$. $\partial J / \partial w = (-0.8)(-1) + (-2.4)(1) = 0.8 - 2.4 = -1.6$; $\partial J / \partial b = -0.8 - 2.4 = -3.2$. $w = 0.2 + 0.16 = 0.36$; $b = 0.4 + 0.32 = 0.72$. Kapalı çözüm $w = 1$, $b = 2$ 'ye doğru ilerliyor; $\eta = 0.1$ ile yaklaşık 40 iterasyonda ulaşır. $\eta = 1.1$ seçilseydi her adımda hata büyür, ıraksardı.

```
x = np.array([-1., 1.]); y = np.array([1., 3.]); w = b = 0.0; eta = 0.1
for it in range(1, 41):
    yp = w*x + b
    gw = 2*np.mean((yp - y)*x); gb = 2*np.mean(yp - y)
    w -= eta*gw; b -= eta*gb
    if it in (1, 2, 10, 40): print(it, round(w, 3), round(b, 3), "MSE =", round(np.mean((yp-y)**2), 4))
```

Ölçekleme ve gradyan: Ölçekleme neden şart? Gradyan inişinde özelliklerin ölçeği farklıysa maliyet yüzeyi dar bir vadi olur: bir yönde büyük, diğerinde küçük adım gerekir ve tek bir η ikisini birden karşılayamaz; iniş zikzak çizer ya da ıraksar. Özellikleri standartlaştırmak yüzeyi 'yuvarlaklaştırır'. Kapalı çözüm (normal denklem) ölçekten etkilenmez; ama katsayıların yorumu değişir (5.5).

5.4 Çoklu Regresyon ve Matris Biçimi

Birden çok özellik olunca formüller matrislerle yazılır. X , her satırı bir örnek, her sütunu bir özellik olan $N \times d$ matrisidir; sabiti de katsayı gibi ele almak için X 'e birlerden oluşan bir sütun eklenir ($N \times (d+1)$). Parametreler w vektörü, tahminler $\hat{y} = Xw$. Maliyet $J = (1/N)\|Xw - y\|^2$. Gradyan $\nabla J = (2/N) X^T(Xw - y)$. Gradyanı sıfıra eşitleyince normal denklem çıkar: $X^T X w = X^T y \rightarrow w = (X^T X)^{-1} X^T y$. 2. haftada `np.linalg.solve` ile çözdüğümüz $A \cdot x = b$ problemi tam olarak budur: $A = X^T X$, $b = X^T y$.



Şekil 5.4 Çoklu regresyonun matris biçimi, normal denklem ve gradyan

Örnek 5.3 — Normal denklem ile üç nokta (elle)

Soru: $x = [0, 1, 2]$, $y = [1, 3, 4]$ için $\hat{y} = w_0 + w_1 x$ modelini normal denklemle çözün.

$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$. $X^T X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$; $X^T y = [8, 11]$. Denklem: $3w_0 + 3w_1 = 8$; $3w_0 + 5w_1 = 11$. Çıkarınca $2w_1 = 3 \rightarrow w_1 = 1.5$; $w_0 = (8 - 4.5)/3 = 1.167$. Model $\hat{y} = 1.167 + 1.5x$. Kontrol: tahminler $[1.17, 2.67, 4.17]$, artıklar $[-0.17, 0.33, -0.17]$, toplamı sıfır (normal denklemin bir sonucu).

```
X = np.c_[np.ones(3), [0, 1, 2]]; y = np.array([1, 3, 4.])
w = np.linalg.solve(X.T @ X, X.T @ y); print(w) # [1.167 1.5]
print(np.linalg.lstsq(X, y, rcond=None)[0]) # aynı sonuç, sayısal olarak daha kararlı
```

Pratikte $(X^T X)^{-1}$ hesaplamak yerine `np.linalg.lstsq` ya da scikit-learn `LinearRegression` kullanılır; iki özellik neredeyse aynıysa (eş-doğrusallık) $X^T X$ tekil olmaya yaklaşıp ve tersi patlar — düzenlileştirmenin (5.7) bir nedeni de budur. Özellik sayısı çok büyükse (on binler) normal denklem pahalıdır ((d^3) işlem); o zaman gradyan inişi tercih edilir.

5.5 Katsayıları Yorumlamak

Doğrusal regresyonun en değerli yanı yorumlanabilirliğidir: w_j , 'diğer her şey sabitken x_j bir birim artınca $\hat{y}$ kaç birim değişir' demektir. Ders verisinde sıcaklık katsayısı 3.2 kW/°C ise, klima yükü her derece için 3.2 kW artıyor demektir; hafta sonu katsayısı -28 kW ise hafta sonu ortalama 28 kW daha az tüketiliyor. Bu yorum iki şeye bağlıdır: (1) katsayı birimi hedef birimi / özellik birimi cinsindedir; özellikler standartlaştırılmışsa katsayı 'bir standart sapma artış' başınadır ve o zaman katsayılar büyüklük olarak karşılaştırılabilir — hangi özellik daha etkili sorusuna cevap verir; (2) özellikler birbirine bağlıysa (eş-doğrusallık) 'diğerleri sabitken' varsayımı bozulur ve katsayılar güvenilmez olur.

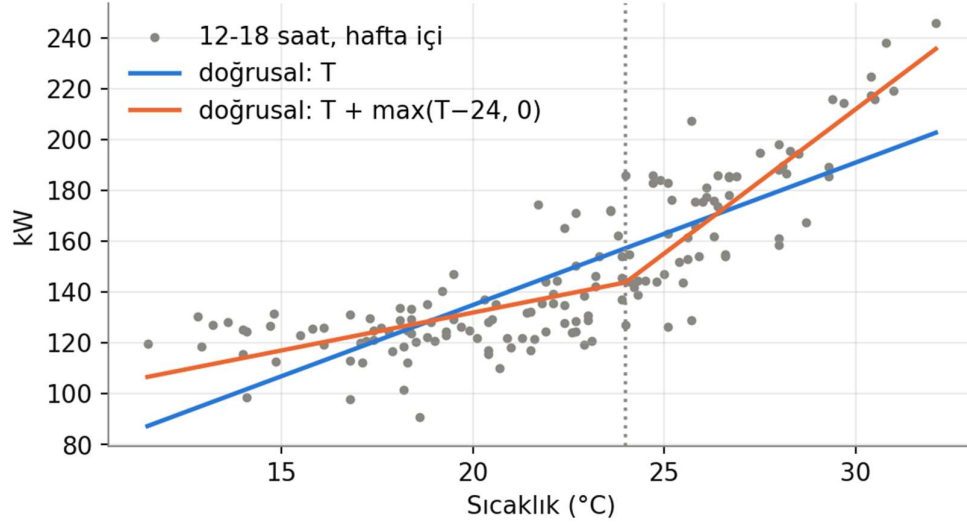
Özellik	Katsayı (ham birim)	Yorum	Katsayı (standartlaştırılmış)
sicaklik_C	$\approx 0.4 \text{ kW/}^\circ\text{C}$	24 °C altında 1 °C artış $\approx 0.4 \text{ kW}$ (küçük)	$\approx 2 \text{ kW}/\sigma$
klima = max(T-24,0)	$\approx 6.6 \text{ kW/}^\circ\text{C}$	24 °C üstündeki her derece ek 6.6 kW	$\approx 15 \text{ kW}/\sigma$
hafta_sonu	$\approx -28 \text{ kW}$	Hafta sonu 28 kW düşük	$\approx -12 \text{ kW}/\sigma$
$\sin(2\pi h/24)$, $\cos(\dots)$	-26, -32 kW	Günlük profilin fazı ve genliği	$\approx -18, -23 \text{ kW}/\sigma$ — en büyük, profil baskın

Tablo 5.2 Ders verisinde tipik katsayılar (yaklaşık; defterde hesaplanır)

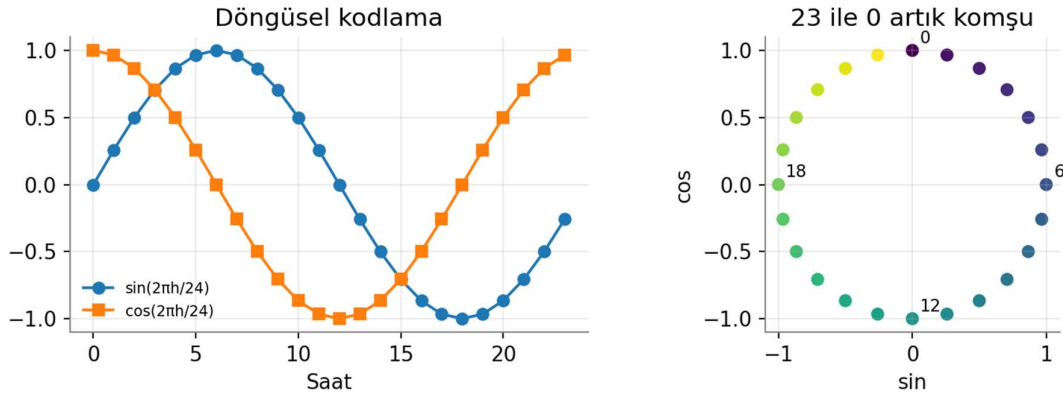
5.6 Özellik Dönüşümü: Doğrusal Modelle Eğri Uydurmak

'Doğrusal' model eğri ilişkileri de yakalayabilir; yeter ki eğriliği özelliklere koyalım. 3. haftada sıcaklık-tüketim ilişkisinin 24 °C'de kırıldığını görmüştük. Tek bir T özelliği bu kırığı yakalayamaz; ama T'nin yanına $\text{klima} = \max(T - 24, 0)$ eklenince model iki parçalı bir doğru öğrenir: 24 °C altında eğim w_T , üstünde $w_T + w_{\text{klima}}$. Aynı şekilde saat 0-23 sayısını doğrudan kullanmak yanıltır (23 ile 0 komşudur ama sayı olarak en uzaktır ve profil tepelidir); $\sin(2\pi h/24)$ ve $\cos(2\pi h/24)$ çifti saati bir çember üzerine koyar, ikinci harmonik (4π) sabah/akşam çift tepeyi verir. Polinom terimleri (x^2 , x^3) de bir dönüşümdür; ama derece arttıkça 4. haftadaki aşırı öğrenme gelir.

Özellik dönüşümü ile kırık çizgi



Şekil 5.5 Aynı doğrusal model, klima özelliği eklenince kırık çizgiyi öğrenir



Şekil 5.6 Saatin döngüsel (sin/cos) kodlaması

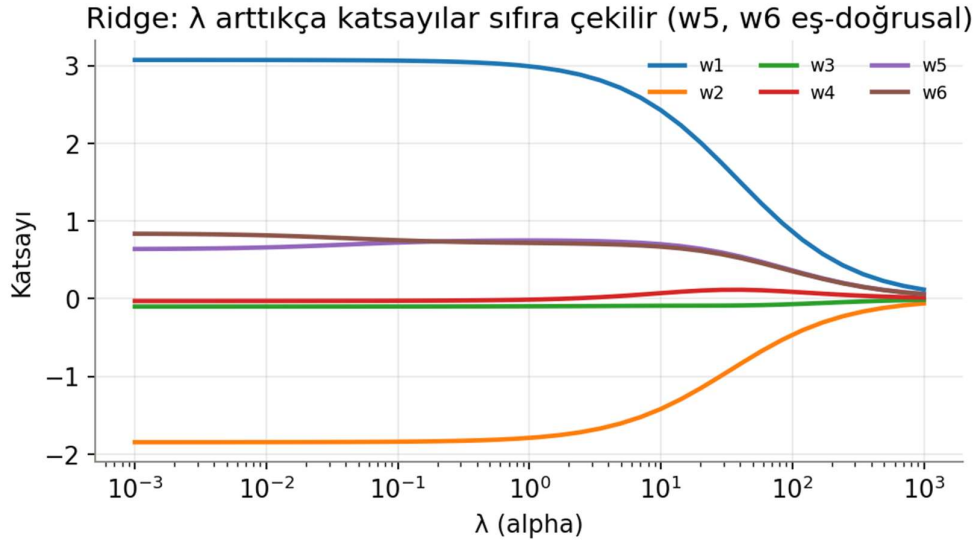
İlişki	Dönüşüm	Ne zaman?
Kırık çizgi (eşik)	$\max(x - c, 0)$ (ReLU!)	Klima yükü, doyma, eşikli fiziksel olaylar
Döngüsel (saat, gün, açı)	$\sin(2\pi x/P)$, $\cos(2\pi x/P)$	Zaman ve açı özellikleri
Kategorik	One-hot (her sınıfa bir 0/1 sütun)	Motor tipi, bölge (7. hafta)
Üstel / güç yasası	$\log(x)$, $\log(y)$	Aşınma, ömür, güç-frekans
Etkileşim	$x_1 \cdot x_2$	Sıcaklık etkisi gündüz daha güçlüyse: T·gunduz
Polinom	x^2 , x^3	Düz eğrilik; derece ≤ 3 tutun

Tablo 5.3 Sık kullanılan özellik dönüşümleri

İleriye bakış: $\max(x - c, 0)$ fonksiyonu, 9. haftada sinir ağlarının en yaygın aktivasyonu olan ReLU'nun ta kendisidir. Sinir ağı, bu tür kırık çizgileri kendisi öğrenen bir 'otomatik özellik dönüştürücü' olarak düşünülebilir.

5.7 Düzenleştirme: Ridge ve Lasso

Çok özellik (özellikle polinom terimleri) eklenince model karmaşıklaşır ve aşırı öğrenme riski artar; eş-doğrusal özellikler katsayıları kararsızlaştırır. Düzenileştirme, maliyete katsayıların büyüklüğünü cezalandıran bir terim ekler: Ridge (L2) $J + \lambda \sum w_j^2$, Lasso (L1) $J + \lambda \sum |w_j|$. $\lambda = 0$ sıradan regresyondur; λ büyüdükçe katsayılar sıfıra çekilir, model 'yumuşar' ve varyansı düşer (yanlılık biraz artar). Lasso'nun ek bir özelliği, bazı katsayıları tam sıfır yapmasıdır: otomatik özellik seçimi. λ bir hiperparametredir ve 4. haftada öğrendiğimiz gibi doğrulama/çapraz doğrulama ile seçilir (RidgeCV, LassoCV). Sabit terim b cezalandırılmaz ve özellikler mutlaka ölçeklenir; aksi hâlde büyük ölçekli özellik daha az cezalanır.



Şekil 5.7 Ridge katsayı yolu: λ arttıkça katsayılar küçülür; eş-doğrusal w5 ve w6 önce 'paylaşıyor' sonra birlikte sıfıra gider

Örnek 5.4 — Ridge'in tek katsayılı etkisi

Soru: Tek özellikli, ortalanmış veride $\sum x_i^2 = 80$ ve $\sum x_i y_i = 400$ (Örnek 5.1). Sıradan çözüm $w = 400/80 = 5$. Ridge çözümü $w = \sum x_i y_i / (\sum x_i^2 + \lambda)$ olduğuna göre $\lambda = 20$ ve $\lambda = 80$ için w'yi bulun.

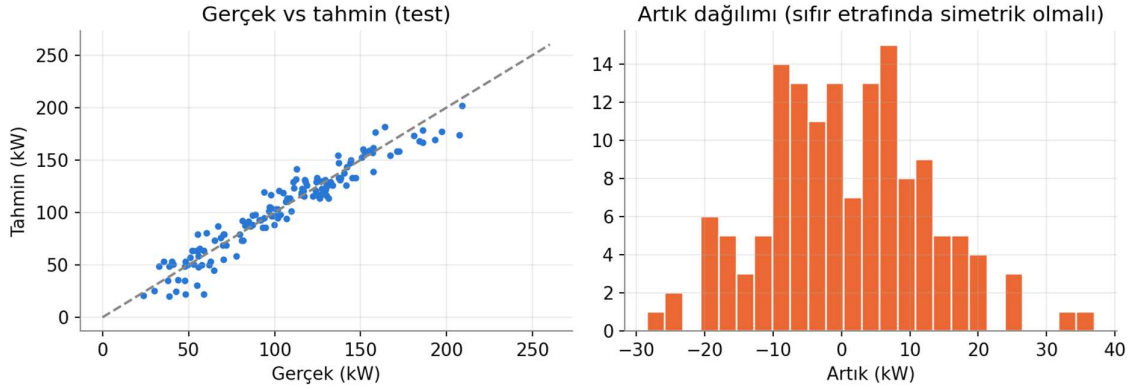
$\lambda = 20$: $w = 400/100 = 4.0$. $\lambda = 80$: $w = 400/160 = 2.5$. Katsayı λ ile 'büzülür'; model daha az agresif tahmin yapar. Bu tek özellikte pek yararlı görünmez; yararı, onlarca eş-doğrusal veya gürültülü özellikte gürültüye uyan katsayıları bastırmasıdır. λ 'ı tek başına büyütmek her zaman iyi değildir: fazla λ eksik öğrenmeye götürür; doğru λ doğrulama hatasının minimumundadır.

	Ridge (L2)	Lasso (L1)
Ceza	$\lambda \sum w^2$	$\lambda \sum w $
Etkisi	Tüm katsayıları küçültür, sıfır yapmaz	Bazı katsayıları tam sıfır yapar (özellik seçimi)
Eş-doğrusal özellikler	Etkiyi paylaştırır (kararlı)	Birini seçer, diğerini atar
Çözüm	Kapalı: $(X^T X + \lambda I)^{-1} X^T y$	Yinelemeli (koordinat inisi)
Ne zaman?	Çok özellik, hepsi az çok yararlı	Çok özellik, çoğu gereksiz; yorumlanabilir model istenir

Tablo 5.4 Ridge ve lasso karşılaştırması

5.8 Uçtan Uca: Dersin Verisinde Taban Modeli Yenmek

4. haftada 'saat x gün tipi' profili $R^2 = 0.92$, $RMSE \approx 12$ kW vermişti; doğrusal model saatle düz çizgi çekip 0.54'te kalmıştı. Bu hafta özellik mühendisliğiyle aynı doğrusal model adım adım iyileşir: klima özelliği 0.54, bir harmonik sin/cos 0.72, iki harmonik 0.905, üç harmonik 0.92 (taban yakalandı); ikinci derece etkileşimler ve ridge ile 0.955 ($RMSE$ 8.8 kW) — taban belirgin biçimde geçilir. Bu sıra öğreticidir: hiçbir adımda algoritma değişmedi, yalnızca özellikler değişti. Aşağıdaki şekil test kümesinde gerçek-tahmin saçılımını ve artık dağılımını gösterir; iyi bir modelde noktalar köşegene yakın, artıklar sıfır etrafında simetrik ve yapısızdır (artıkta kalıp varsa model bir şeyi kaçıırıyordur). Sonucu ve karşılaştırmayı defterde yapacağız.



Şekil 5.8 Test kümesinde gerçek-tahmin saçılımı ve artık histogramı

Özellik mühendisliği + doğrusal regresyon

```
import numpy as np, pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv("elektrik_tuketimi.csv"); df = df[df.tuketim_kw < 300].copy(); df["sicaklik_C"] = df["sicaklik_C"].interpolate()
def ozellik(d):
    h = 2*np.pi*d.saat/24
    return np.c_[np.sin(h), np.cos(h), np.sin(2*h), np.cos(2*h), np.sin(3*h), np.cos(3*h),
                  d.hafta_sonu, d.sicaklik_C, np.clip(d.sicaklik_C - 24, 0, None)]
egitim, test = df[df.gun <= 24], df[df.gun > 24]
model = LinearRegression().fit(ozellik(egitim), egitim.tuketim_kw)
tahmin = model.predict(ozellik(test))
print("RMSE:", np.sqrt(mean_squared_error(test.tuketim_kw, tahmin)).round(2), "R2:",
      r2_score(test.tuketim_kw, tahmin).round(3))
# Taban (saat x gün tipi): RMSE 12.1, R2 0.916 -> bu model: RMSE ~11.8, R2 ~0.92
# 2. derece etkileşimler + ridge (defterde): RMSE ~8.8, R2 ~0.955
```

5.9 Sınıf İçi Sorular

1. $x = [1, 2, 3]$, $y = [2, 4, 5]$ için w ve b 'yi elle bulun; R^2 'yi hesaplayın.
2. Gradyan inişinde η 'yı 10 katına çıkardınız ve maliyet her adımda büyüyor. Ne oldu, ne yaparsınız?
3. Sıcaklık katsayısı standartlaştırılmış veride 9, ham veride 1.5 çıktı. Hangisi 'sıcaklık daha etkili' sorusuna cevap verir, hangisi 'derece başına kaç kW' sorusuna?
4. Saat 0-23'ü doğrudan sayı olarak kullanmanın iki sakıncası nedir?
5. İki özellik neredeyse aynıysa (ör. $^{\circ}C$ ve $^{\circ}F$ cinsinden sıcaklık) normal denklemde ne olur? Ridge bunu nasıl çözer?

5.10 Ödev 3 — Fotovoltaik Üretim Tahmini

Veri: pv_uretim.csv — 60 gün boyunca saatlik ışınlm (W/m^2), hava sıcaklığı ($^{\circ}C$), rüzgâr hızı (m/s), bulutluluk (0-1) ve 10 kW'lık bir çatı PV sisteminin üretimi (kW). Fizik: üretim ışınlmıla doğru orantılıdır; hücre sıcaklığı arttıkça verim düşer (yaklaşık $-0.4/^{\circ}C$); rüzgâr paneli soğutur.

- Veriyi zamana göre bölün (ilk 45 gün eğitim, son 15 gün test); yalnızca gündüz saatlerini (ışınlm > 0) kullanmayı düşünün ve kararınızı gerekçelendirin.
- Taban model kurun (saatlik profil). Sonra sırasıyla (a) yalnız ışınlm, (b) ışınlm + sıcaklık, (c) ışınlm + sıcaklık + rüzgâr + etkileşim terimi ışınlm $\times$ sıcaklık ile doğrusal regresyon eğitin; test RMSE ve R^2 tablosu yapın.
- Katsayıları fiziksel birimle yorumlayın: ışınlm katsayısı $kW/(W/m^2)$ olarak ne anlama gelir? Sıcaklık katsayısının işareti fizikle uyumlu mu?
- Ridge ile λ 'yı 5-katlı CV ile seçin (ölçeklemeyi unutmayın); sıradan regresyonla karşılaştırın.
- Gerçek–tahmin saçılımı ve artık histogramı çizin; artıktaki kalıp var mı (ör. öğle saatlerinde sistematik hata)? Yorumlayın.
- Teslim: Colab defteri, 'Restart and run all' hatasız; her sonucun altında 1-2 cümle yorum.

Hafta 6 — Sınıflandırma I: Lojistik Regresyon, k-NN ve Değerlendirme

Bölüm	Süre	İçerik
Saat 1	50 dk	Sınıflandırma problemi; lojistik regresyon: sigmoid, karar sınırı, log-loss; k-en yakın komşu
Saat 2	50 dk	Karışıklık matrisi; doğruluk, precision, recall, F1 (elle); sınıf dengesizliği; ROC-AUC; eşik seçimi ve maliyet
Saat 3	50 dk	Uygulama: motor arıza verisinde lojistik regresyon ve k-NN; ölçütler; eşik ayarı

Tablo 6.1 Haftanın ders planı

6.1 Kazanımlar

- Sınıflandırma ile regresyon arasındaki farkı çıktı türü ve kayıp fonksiyonu açısından açıkla.
- Lojistik regresyonun sigmoid ile olasılık ürettiğini, karar sınırının doğrusal olduğunu bilir; z 'den olasılığı elle hesaplar.
- Log-loss (çapraz entropi) kaybını hesaplar ve neden MSE yerine kullanıldığını söyler.
- k-NN sınıflandırmasını elle uygular; k 'nın ve ölçeklemenin etkisini açıklar.
- Karışıklık matrisinden doğruluk, precision, recall, özgülük ve F1'i elle hesaplar; dengesiz sınıfta doğruluğun tuzağını bilir.
- ROC eğrisini ve AUC'yi yorumlar; maliyete göre karar eşiği seçer.

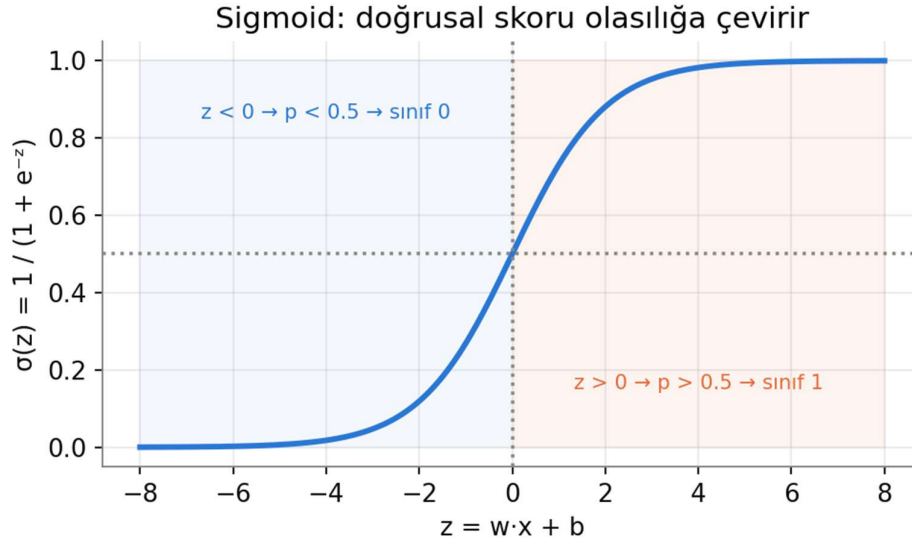
6.2 Sınıflandırma Problemi

Regresyonda hedef bir sayıydı; sınıflandırmada hedef sonlu sayıda sınıftan biridir: motor arızalı mı (evet/hayır), hangi arıza tipi (iç bilezik / dış bilezik / bilye / sağlam), hangi modülasyon (BPSK/QPSK/...). İki sınıf varsa ikili, daha çok varsa çok sınıflı sınıflandırma denir. İyi bir sınıflandırıcı yalnızca bir etiket değil, bir olasılık üretir: 'bu motorun arızalı olma olasılığı 0.83'. Olasılık, kararı ertelemeyi (bakım ekibine öncelik listesi vermek) ve eşiği maliyete göre ayarlamayı mümkün kılar. Bu hafta ikili sınıflandırmaya odaklanıyoruz; çok sınıflı durum 7. haftada.

Neden doğrusal regresyon kullanmıyoruz? $y \in \{0, 1\}$ için $\hat{y} = w \cdot x + b$ uydurulabilir ama çıktı 0-1 aralığında kalmaz (-0.4 ya da 1.7 gibi 'olasılıklar'), uç örnekler doğruyu gereksiz yere çeker ve MSE bu problem için yanlış kayıptır. Çözüm, doğrusal skoru bir 'ezici' fonksiyondan geçirmektir.

6.3 Lojistik Regresyon

Lojistik regresyon, adına rağmen bir sınıflandırma yöntemidir ve doğrusal regresyonun doğal uzantısıdır. Önce doğrusal bir skor hesaplanır: $z = w \cdot x + b$. Sonra bu skor sigmoid fonksiyonuyla $(0, 1)$ aralığına sıkıştırılır: $p = \sigma(z) = 1 / (1 + e^{-z})$. $z = 0$ iken $p = 0.5$; z büyük pozitifken $p \rightarrow 1$; büyük negatifken $p \rightarrow 0$. Karar kuralı $p \geq 0.5$ (yani $z \geq 0$) ise sınıf 1'dir; dolayısıyla karar sınırı $w \cdot x + b = 0$ hiperdüzlemdir — doğrusal. Sigmoidin tersi 'logit' ya da log-odds'tur: $\log(p/(1-p)) = z$; yani lojistik regresyon, olasılık oranının logaritmasını doğrusal modeller. Katsayı w_j 'nin yorumu: x_j bir birim artınca log-odds w_j kadar, odds ise e^{w_j} kat artar.



Şekil 6.1 Sigmoid fonksiyonu ve karar eşiği

Örnek 6.1 — Skordan olasılığa

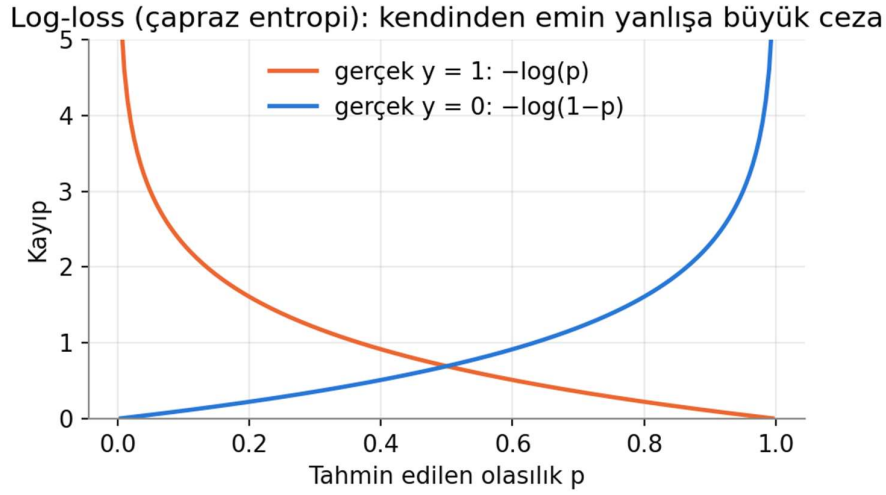
Soru: Ölçeklenmiş özelliklerle lojistik model $z = 1.8 \cdot \text{rms} + 1.1 \cdot \text{sıcaklık} - 0.5$. Bir motor için $\text{rms}(z) = 1.2$, $\text{sıcaklık}(z) = 0.4$ ise arıza olasılığı? Karar?

$z = 1.8 \times 1.2 + 1.1 \times 0.4 - 0.5 = 2.16 + 0.44 - 0.5 = 2.10$. $p = 1 / (1 + e^{-2.10}) = 1 / (1 + 0.1225) = 0.891$. $p > 0.5 \rightarrow$ arızalı. Odds = $p / (1-p) = 8.2$: arıza 'sağlam olmaktan 8 kat olası'. Aynı motorda rms bir standart sapma düşük olsaydı (0.2): $z = 0.30$, $p = 0.574$ — hâlâ arızalı ama kararsız; olasılık işte bu nüansı verir.

```
import numpy as np
z = 1.8*1.2 + 1.1*0.4 - 0.5
p = 1/(1 + np.exp(-z)); print(round(z, 2), round(p, 3), "odds =", round(p/(1-p), 2))
```

Kayıp fonksiyonu: log-loss (çapraz entropi)

Parametreler nasıl öğrenilir? MSE burada kötü bir seçimdir (sigmoid ile birleşince dışbükey olmayan, düz bölgeleri olan bir yüzey verir). Doğru kayıp, olasılıksal bakıştan çıkar: modelin gerçek etikete verdiği olasılığın logaritmasını en büyükmek, eşdeğer olarak $L = -[y \log p + (1-y) \log(1-p)]$ kaybını en küçükmek. $y = 1$ iken kayıp $-\log p$ 'dir: $p = 0.9 \rightarrow 0.105$, $p = 0.5 \rightarrow 0.693$, $p = 0.01 \rightarrow 4.6$. Yani kendinden emin bir yanlış çok pahalıdır. Bu kaybın gradyanı şaşırtıcı biçimde doğrusal regresyondakine benzer: $\partial L / \partial w = (p - y) \cdot x$. Bu yüzden eğitim yine gradyan inişidir; scikit-learn bunu daha hızlı yöntemlerle (L-BFGS) yapar. Düzenleştirme ($C = 1/\lambda$) varsayılan olarak açıktır.



Şekil 6.2 Log-loss: $y = 1$ için $-\log p$, $y = 0$ için $-\log(1-p)$

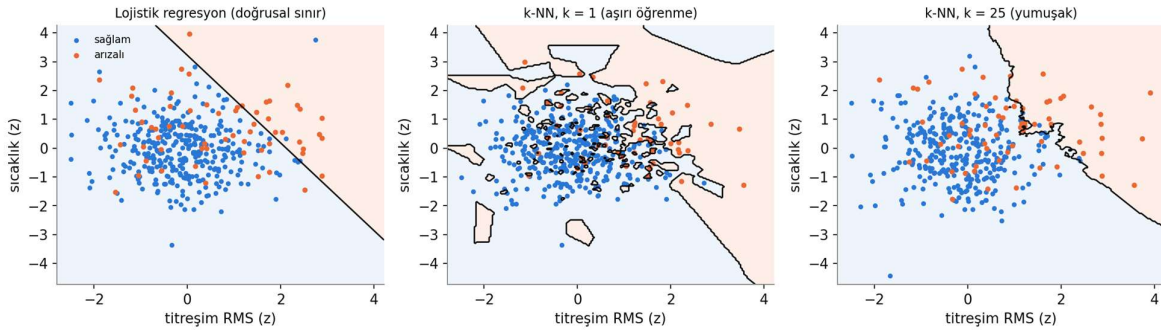
Örnek 6.2 — Üç tahmin için log-loss

Soru: Üç motor için gerçek etiketler $y = [1, 0, 1]$, model olasılıkları $p = [0.9, 0.2, 0.3]$. Ortalama log-loss? Hangi tahmin en pahalı?

Kayıplar: $-\log 0.9 = 0.105$; $-\log(1 - 0.2) = -\log 0.8 = 0.223$; $-\log 0.3 = 1.204$. Ortalama = $(0.105 + 0.223 + 1.204)/3 = 0.511$. Üçüncü motor arızalıyken model %30 demiş: en pahalı. Doğruluk ölçütü bu üç tahmine $2/3 = 0.67$ der ama 'ne kadar emindi' bilgisini atar; log-loss onu korur. Karşılaştırma: her şeye 0.5 diyen model 0.693 verir; 0.511 ondan iyidir.

6.4 k-En Yakın Komşu (k-NN)

1. haftada tek bir en yakın komşuya bakmıştık; k-NN bunun genelidir: yeni örnek için eğitim kümesindeki en yakın k örneği bul, sınıflandırmada çoğunluk oyu (olasılık = oy oranı), regresyonda ortalama al. Eğitim aşaması yoktur (veri saklanır); tahmin aşaması tüm uzaklıkları hesaplamayı gerektirir, bu yüzden büyük veride yavaştır. Karar sınırı doğrusal değildir; k küçükken sınır her noktayı sararak aşırı öğrenir ($k = 1$ 'de eğitim doğruluğu %100), k büyüdükçe sınır yumuşar ve sonunda her şeyi çoğunluk sınıfına atar (eksik öğrenme). k bir hiperparametredir, CV ile seçilir; ikili sınıflamada tek sayı seçmek beraberliği önler.

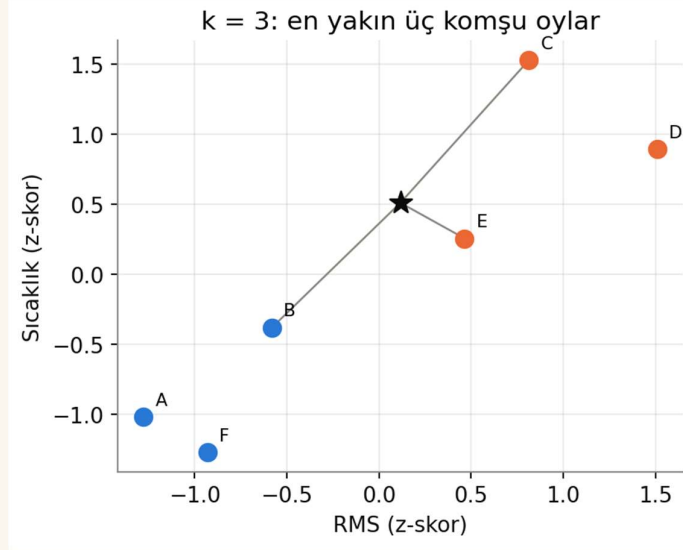


Şekil 6.3 Motor verisinde karar sınırları: lojistik (doğrusal), $k = 1$ (aşırı), $k = 25$ (yumuşak)

Ölçekleme: k-NN uzaklığa dayandığı için ölçekleme zorunludur. Çalışma saati (100-20000) ile RMS (0-8) birlikte ölçeklenmeden kullanılırsa uzaklık tamamen çalışma saatinin kontrolüne girer ve RMS yok sayılır — 1. haftadaki Fahrenheit tartışmasının ta kendisi.

Örnek 6.3 — $k = 3$ ile elle sınıflandırma

Soru: Altı motorun ölçeklenmiş (RMS, sıcaklık) değerleri ve etiketleri şekildeki gibi; yeni motor (yıldız) için $k = 3$ sonucu?



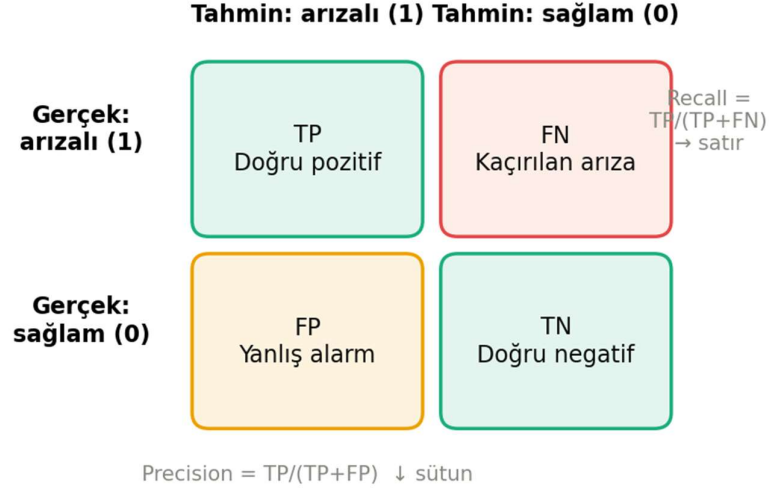
Şekil 6.4 $k = 3$: en yakın üç komşu ve oyları

Örnek	RMS (z)	Sıcaklık (z)	Etiket	Yeni örneğe uzaklık
A	-1.34	-0.94	0	1.86
B	-0.62	-0.31	0	1.10
C	0.83	1.57	1	1.03
D	1.55	0.94	1	1.47
E	0.47	0.31	1	0.32
F	-0.98	-1.19	0	1.72
Yeni	0.11	0.57	?	—

En yakın üç: E (0.32), C (1.03), B (1.10). Oylar: 1, 1, 0 $\rightarrow$ çoğunluk 1 $\rightarrow$ arızalı; olasılık $2/3 \approx 0.67$. $k = 1$ olsaydı sadece E $\rightarrow$ 1. $k = 5$ olsaydı E, C, B, D, F $\rightarrow$ 1, 1, 0, 1, 0 $\rightarrow$ yine 1 ama $3/5 = 0.6$. k , kararı ve güveni değiştirir.

6.5 Sınıflandırmayı Değerlendirmek

Regresyonda hata bir sayıydı; sınıflandırmada hata türleri vardır ve hepsi eşit maliyetli değildir. Bir arıza dedektörü için arızayı kaçırmak (yanlış negatif) ile yanlış alarm (yanlış pozitif) çok farklı sonuçlar doğurur: ilki plansız duruş ve hasar, ikincisi gereksiz bir bakım ziyaretidir. Karışıklık matrisi bu dört durumu sayar; tüm ölçütler bu dört sayıdan türetilir.



Şekil 6.5 Karışıklık matrisi: precision sütuna, recall satıra bakar

Ölçüt	Formül	Soru	Ne zaman öncelikli?
Doğruluk (accuracy)	$(TP + TN) / N$	Kaç karar doğru?	Sınıflar dengeli ve hatalar eşit maliyetliyse
Precision (kesinlik)	$TP / (TP + FP)$	'Arızalı' dediklerimin kaç gerçekten arızalı?	Yanlış alarm pahalıysa (gereksiz bakım)
Recall (duyarlılık, TPR)	$TP / (TP + FN)$	Gerçek arızaların kaçını yakaladım?	Kaçırmak pahalıysa (güvenlik, hasar)
Özgüllük (TNR)	$TN / (TN + FP)$	Sağlamların kaçını sağlam dedim?	ROC'nin x eksen: $1 - \text{özgüllük} = \text{FPR}$
F1	$2 \cdot P \cdot R / (P + R)$	Precision ve recall'un harmonik ortalaması	Tek sayı istenince; dengesiz sınıfta doğruluk yerine
ROC-AUC	Eğri altı alan	Rastgele bir pozitif, rastgele bir negatiften daha yüksek skor alma olasılığı	Eşikten bağımsız model karşılaştırması

Tablo 6.2 Sınıflandırma ölçütleri

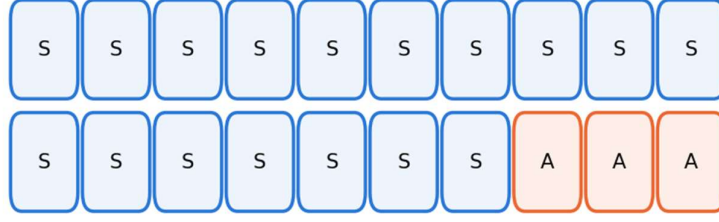
Örnek 6.4 — Karışıklık matrisinden ölçütler (elle)

Soru: 1000 motorluk test kümesinde 150 arızalı var. Model: TP = 120, FN = 30, FP = 40, TN = 810. Doğruluk, precision, recall, özgüllük, F1 hesaplayın. 'Hepsi sağlam' diyen model ile karşılaştırın.

Doğruluk = $(120 + 810) / 1000 = 0.930$. Precision = $120 / (120 + 40) = 0.750$. Recall = $120 / (120 + 30) = 0.800$. Özgüllük = $810 / (810 + 40) = 0.953$. F1 = $2 \times 0.75 \times 0.80 / (0.75 + 0.80) = 1.2 / 1.55 = 0.774$.

'Hepsi sağlam' modeli: TP = 0, FN = 150, FP = 0, TN = 850 → doğruluk 0.850, recall 0, precision tanımsız (0/0), F1 = 0. Doğruluk %85 ile 'fena değil' görünür ama tek bir arızayı yakalamaz. Dengesiz sınıflarda doğruluk yerine recall/precision/F1 ve AUC raporlanır; bu tuzak 'doğruluk paradoksu' olarak bilinir.

1000 motor: 850 sağlam, 150 arızalı (%15). 'Hepsi sağlam' diyen model → doğruluk %85!



Ama recall = 0: hiçbir arızayı yakalamaz. Dengesiz sınıfta doğruluk yanıltır.

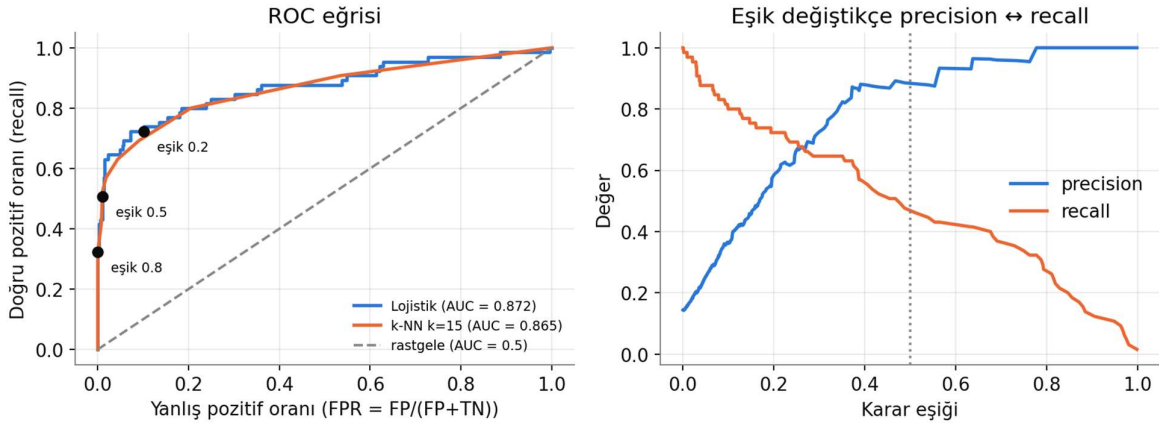
Şekil 6.6 Doğruluk paradoksu: %15 arıza oranında 'hepsi sağlam' demek %85 doğruluk verir

Sınıf dengesizliğiyle başa çıkmak

- Doğru ölçüt: doğruluk yerine recall, precision, F1, PR-AUC; ayrımı stratify ile yapın (sınıf oranları her kümede aynı kalsın).
- Sınıf ağırlığı: `class_weight='balanced'` — azınlık sınıfının hatalarını daha çok cezalandırır.
- Yeniden örnekleme: azınlığı çoğalt (oversampling, SMOTE) ya da çoğunluğu azalt; yalnızca eğitim kümesinde!
- Eşik ayarı: 0.5 yerine maliyete göre eşik (aşağıda).
- Anomali tespiti bakışı: pozitif çok azsa (%1) sınıflandırma yerine 'normalden sapma' modelleri (14. hafta).

6.6 ROC Eğrisi ve Eşik Seçimi

Lojistik regresyon ve k-NN olasılık üretir; 0.5 eşiği bir varsayılan, bir zorunluluk değildir. Eşiği düşürürseniz daha çok örneğe 'arızalı' dersiniz: recall artar, precision düşer; yükseltirseniz tersi olur. ROC eğrisi, tüm eşikler için (FPR, TPR) çiftlerini çizer; sol üst köşeye yakın eğri iyidir, köşegen rastgele tahmindir. Eğri altındaki alan (AUC) eşikten bağımsız tek bir kalite sayısıdır: 0.5 rastgele, 1.0 mükemmel. AUC'nin yorumu: rastgele seçilmiş bir arızalı motorun skoru, rastgele seçilmiş bir sağlamıkinden daha yüksek olma olasılığı. Dengesiz sınıfta ROC iyimser görünebilir; PR eğrisi (precision-recall) o zaman daha bilgilendiricidir.



Şekil 6.7 Motor verisinde ROC eğrileri (sol) ve eşiğe göre precision/recall (sağ)

Örnek 6.5 — Üç eşikten ROC noktaları

Soru: 10 motor için model skorları ve etiketleri: (0.95,1) (0.85,1) (0.70,0) (0.65,1) (0.55,1) (0.40,0) (0.35,0) (0.30,1) (0.20,0) (0.10,0). Eşik 0.5, 0.6 ve 0.9 için (FPR, TPR) hesaplayın. Pozitif sayısı 5, negatif 5.

Eşik	Pozitif denener	TP	FP	TPR = TP/5	FPR = FP/5
0.9	0.95	1	0	0.2	0.0
0.6	0.95, 0.85, 0.70, 0.65	3	1	0.6	0.2
0.5	+ 0.55	4	1	0.8	0.2
0.25	+ 0.40, 0.35, 0.30	5	3	1.0	0.6

Eşik düştükçe TPR artar ama FPR de artar; eğri (0,0)'dan (1,1)'e sol üstten kıvrılarak gider. Bu örnekte AUC'yi kabaca hesaplamak için tüm pozitif–negatif çiftlerine bakılır: 25 çiftten pozitifin skoru negatiften büyük olan sayısı 21 → AUC = 0.84.

Örnek 6.6 — Maliyete göre eşik

Soru: Kaçırılan bir arıza (FN) 10 000 TL, yanlış alarm (FP) 500 TL. Tablo, üç eşikte 1000 motorluk test kümesi sonuçlarını veriyor. Hangi eşik?

Eşik	TP	FN	FP	TN	Maliyet = 10000·FN + 500·FP
0.7	95	55	12	838	550 000 + 6 000 = 556 000
0.5	120	30	40	810	300 000 + 20 000 = 320 000
0.3	140	10	110	740	100 000 + 55 000 = 155 000
0.15	148	2	260	590	20 000 + 130 000 = 150 000

En düşük toplam maliyet 0.15 eşliğindedir (150 000 TL) — doğruluğu (%73.8) 0.5 eşliğinden (%93) çok düşük olmasına rağmen. Kaçırma maliyeti yüksekse eşik düşürülür, recall öncelenir. Karar eşiği bir mühendislik/ekonomi kararıdır, istatistik kararı değil; ölçüt de doğruluk değil beklenen maliyettir. Not: bu eşik doğrulama kümesinde seçilir, test kümesinde raporlanır.

6.7 Uygulama: Motor Arıza Verisi

Dersin ikinci veri seti motor_ariza.csv, 1500 motorun bakım kayıtlarından oluşan sentetik bir tablodur: motor tipi (kategorik), çalışma saati, titreşim RMS (mm/s), titreşim basıklığı (kurtosis — darbeli arızaların imzası), sargı sıcaklığı (°C), akım dengesizliği (%) ve etiket (arıza 0/1, ~%15). Defterde lojistik regresyon ve k-NN eğitilecek, ölçütler elle doğrulanacak, ROC çizilecek ve maliyete göre eşik seçilecek. Kategorik sütun (motor_tipi) bu hafta atlanır, 7. haftada one-hot ile eklenir.

Sütun	Tür	Açıklama
motor_tipi	Kategorik	asenron / senkron / dc (7. haftada kullanılacak)
calisma_saati	Sürekli	Toplam çalışma süresi (saat)
titresim_rms_mms	Sürekli	Titreşim hız RMS (mm/s) — ISO 10816 sınıfları

Sütun	Tür	Açıklama
titresim_kurtosis	Sürekli	Basıklık: darbeli (rulman) arızalarda yükselir
sicaklik_C	Sürekli	Sargı sıcaklığı
akim_dengesizlik_pct	Sürekli	Faz akımı dengesizliği (%) — rotor/sargı arızası imzası
ariza	İkili	Hedef: 1 = arızalı (%15)

Tablo 6.3 motor_ariza.csv

Lojistik regresyon — Pipeline ile ölçekleme kaçıksız

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

m = pd.read_csv("motor_ariza.csv")
ozellik = ["calisma_saati", "titresim_rms_mms", "titresim_kurtosis", "sicaklik_C",
"akim_dengesizlik_pct"]
X, y = m[ozellik], m["ariza"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y) # stratify:
oranları koru

model = make_pipeline(StandardScaler(), LogisticRegression()).fit(Xtr, ytr)
p = model.predict_proba(Xte)[: , 1]
print(confusion_matrix(yte, p >= 0.5)) # [[TN FP], [FN TP]]
print(classification_report(yte, p >= 0.5, digits=3))
print("AUC:", round(roc_auc_score(yte, p), 3))
```

6.8 Sınıf İçi Sorular

1. $z = -1.5$ için sigmoid değerini hesaplayın. Bu örneğe hangi sınıf verilir? Eşik 0.2 olsaydı?
2. TP = 30, FN = 20, FP = 5, TN = 945: doğruluk, precision, recall, F1. Bu model 'iyi' mi? Hangi ölçüt sizi ikna eder?
3. k-NN'de $k = N$ (tüm veri) seçilirse model ne yapar? $k = 1$ seçilirse eğitim doğruluğu kaçtır?
4. AUC = 0.5 olan bir modelin ROC eğrisi nasıldır? AUC = 0.3 çıkarsa ne düşünürsünüz?
5. Yanlış alarm çok ucuz, kaçırma çok pahalıysa eşiği hangi yöne çekersiniz? Hangi ölçüt artar, hangisi düşer?

6.9 Kısa Sınav 2 (örnek sorular) ve Ödev

1. Örnek 5.1'deki veriyle 26 °C için tahmini ve artığı hesaplayın. (3 p)
2. TP = 40, FN = 10, FP = 20, TN = 130 için precision, recall, F1. (4 p)
3. Sigmoid neden gerekli; karar sınırı neden doğrusal? (3 p)
 - Uygulama defteri: motor verisinde lojistik regresyon ve k-NN; ölçütleri elle doğrulama; k taraması; ROC; maliyete göre eşik.
 - Ödev yok (Kısa sınav 2 haftaya). Gelecek hafta: karar ağaçları, rastgele orman, SVM, one-hot, çapraz doğrulama ile hiperparametre arama; rulman titreşim verisi.

Hafta 7 — Sınıflandırma II ve Özellik Mühendisliği

Bölüm	Süre	İçerik
Saat 1	50 dk	Karar ağaçları: bölme kuralı, Gini/entropi (elle), derinlik ve aşırı öğrenme; rastgele orman (bagging), özellik önemi
Saat 2	50 dk	Destek vektör makineleri: marj, C, RBF çekirdeği; kategorik kodlama (one-hot); çapraz doğrulama ile hiperparametre arama; çok sınıflı ölçütler
Saat 3	50 dk	Uygulama: rulman titreşim sinyallerinden özellik çıkarımı, ağaç/orman/SVM karşılaştırması, GridSearchCV

Tablo 7.1 Haftanın ders planı

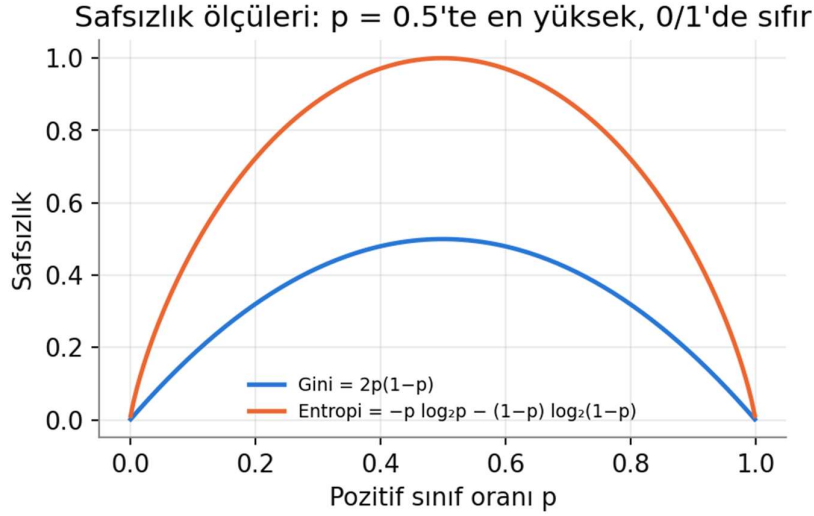
7.1 Kazanımlar

- Bir karar ağacının bölme kuralını Gini/entropi ile elle hesaplar; derinliğin aşırı öğrenmeye etkisini açıklar.
- Bagging ve rastgele ormanın neden tek ağaçtan iyi genellediğini söyler; özellik önemini yorumlar.
- SVM'nin marj fikrini, C ve gamma hiperparametrelerinin rolünü açıklar.
- Kategorik değişkenleri one-hot ile kodlar; sıralı kodlamanın tuzağını bilir.
- Ham titreşim sinyalinden zaman ve frekans alanı özellikleri çıkarır (RMS, kurtosis, crest, bant enerjileri).
- GridSearchCV ile hiperparametre seçer; çok sınıflı karışıklık matrisi ve makro-F1'i yorumlar; dönem projesi konusunu belirler.

7.2 Karar Ağaçları

Karar ağacı, veriyi 'eğer-ise' sorularıyla ardışık olarak bölen bir modeldir: 'kurtosis > 3.6 mı?' evetse sola, hayırsa sağa; her dalda yeni bir soru; yapraklarda sınıf kararı. İnsan gibi düşündüğü için yorumlanabilirliği en yüksek yöntemlerden biridir ve bir bakım teknisyenine 'neden arızalı?' sorusunun cevabı olarak gösterilebilir. Ölçek gerektirmez (yalnızca eşiklerle çalışır), kategorik ve sayısal özellikleri birlikte ele alır, doğrusal olmayan sınırlar üretir. Zayıflığı, tek başına kararsız ve aşırı öğrenmeye yatkın olmasıdır: veri biraz değişse ağaç tamamen değişebilir.

Ağaç nasıl büyür? Her düğümde algoritma tüm özellikleri ve tüm olası eşikleri dener; düğümdeki 'karışıklığı' (safsızlığı) en çok azaltan bölmeyi seçer. Safsızlık ölçüsü olarak Gini ($1 - \sum p_k^2$) ya da entropi ($-\sum p_k \log_2 p_k$) kullanılır; ikisi de tüm örnekler tek sınıftan olunca sıfır, sınıflar eşit karışınca en büyüktür. Bir bölmenin kazancı = ebeveyn safsızlığı - çocukların ağırlıklı ortalama safsızlığı. Bölme, durma koşuluna (maks derinlik, yaprakta min örnek, kazanç eşiği) kadar özinelemeli tekrarlanır.



Şekil 7.1 Gini ve entropi: $p = 0.5$ 'te maksimum, saf düğümde sıfır

Örnek 7.1 — En iyi bölme Gini ile seçmek (elle)

Soru: 10 motorluk bir düğümde 6 sağlam (S), 4 arızalı (A) var. İki aday bölme: (1) kurtosis > 3.5 : sol {2S, 4A}, sağ {4S, 0A}; (2) sıcaklık > 65 : sol {3S, 3A}, sağ {3S, 1A}. Hangi bölme daha iyi?

Ebeveyn Gini = $1 - (0.6^2 + 0.4^2) = 1 - 0.52 = 0.48$.

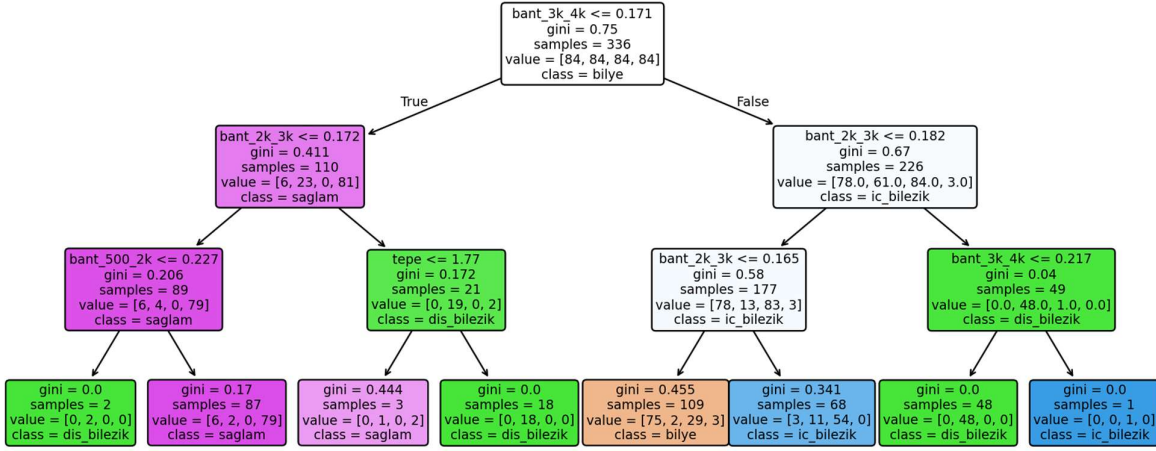
Bölme 1: sol (6 örnek): $p = 2/6, 4/6 \rightarrow$ Gini = $1 - (0.111 + 0.444) = 0.444$; sağ (4 örnek): saf $\rightarrow 0$. Ağırlıklı = $(6/10)(0.444) + (4/10)(0) = 0.267$. Kazanç = $0.48 - 0.267 = 0.213$.

Bölme 2: sol (6): $p = 0.5, 0.5 \rightarrow$ Gini 0.5; sağ (4): $p = 0.75, 0.25 \rightarrow 1 - (0.5625 + 0.0625) = 0.375$. Ağırlıklı = $0.6 \times 0.5 + 0.4 \times 0.375 = 0.45$. Kazanç = 0.03.

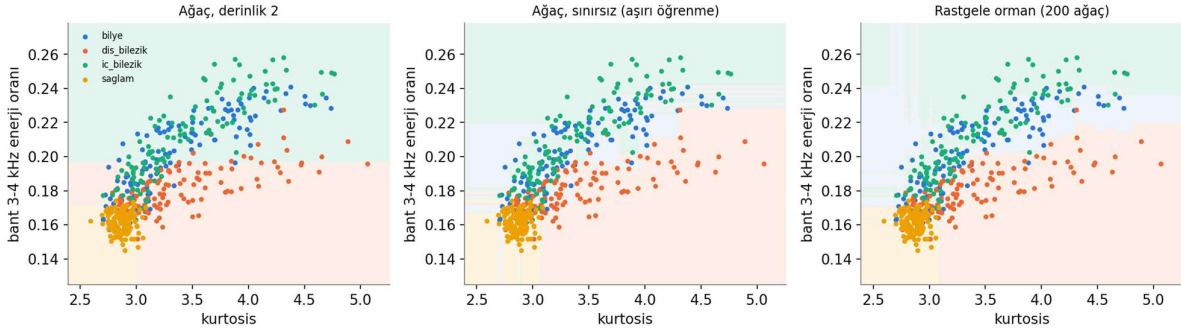
Bölme 1 çok daha iyi (kazanç $0.213 > 0.03$): kurtosis, sağ dalda 4 motoru kesin sağlam olarak ayırıyor. Ağaç, kök düğüme bu soruyu koyar; sonra sol daldaki {2S, 4A} için aynı işlemi tekrarlar.

```
def gini(sayilar):
    p = np.array(sayilar) / sum(sayilar); return 1 - (p**2).sum()
ebeveyn = gini([6, 4])
for ad, sol, sag in (("kurtosis>3.5", [2, 4], [4, 0]), ("sicaklik>65", [3, 3], [3, 1])):
    agirlikli = sum(sol)/10*gini(sol) + sum(sag)/10*gini(sag)
    print(f"{ad:14s} ağırlıklı Gini={agirlikli:.3f} kazanç={ebeveyn - agirlikli:.3f}")
```

Aşağıdaki ağaç, rulman özellik verisinde derinlik 3 ile öğrenildi. Kök düğümdeki soru en ayırt edici özelliktir; her kutuda Gini, örnek sayısı, sınıf dağılımı ve çoğunluk sınıfı yazar. Derinliği sınırsız bırakırsanız ağaç her eğitim örneğini kendi yaprağına koyana kadar büyür: eğitim doğruluğu %100, test doğruluğu düşer (Şekil 7.3, orta panel).



Şekil 7.2 Rulman verisinde derinlik-3 karar ağacı



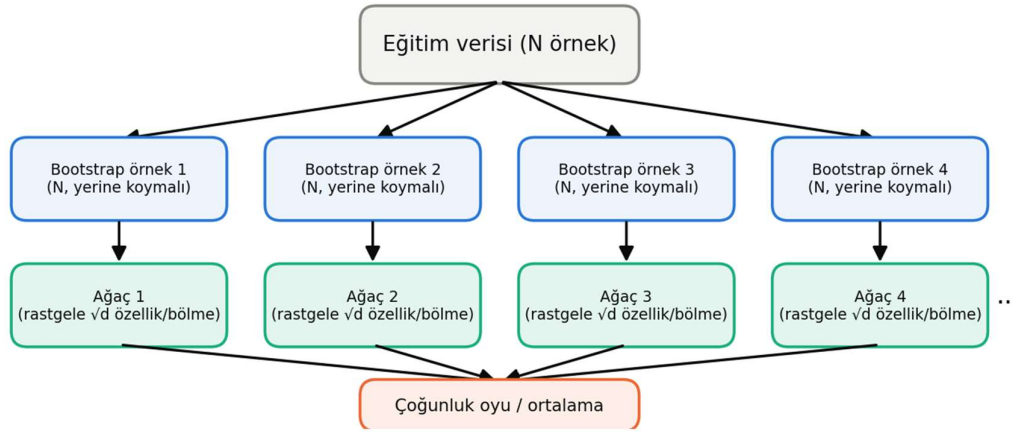
Şekil 7.3 İki özelliikle karar bölgeleri: sıf ağaç, sınırsız ağaç (aşırı öğrenme), rastgele orman

Hiperparametre	Etkisi	Tipik aralık
max_depth	Küçük → eksik öğrenme; büyük → aşırı öğrenme	3-10; None = sınırsız
min_samples_leaf	Yaprakta en az örnek; büyük → daha yumuşak	1-20
min_samples_split	Bölmek için gereken örnek	2-50
criterion	gini / entropy — sonuç genelde benzer	gini
ccp_alpha	Maliyet-karmaşıklık budaması	0-0.05

Tablo 7.2 Karar ağacı hiperparametreleri

7.3 Rastgele Orman

Tek ağaç yüksek varyanslıdır. Varyansı düşürmenin klasik yolu ortalamadır: aynı dağılımdan bağımsız B tahmin alıp ortalarsanız varyans B kat düşer. Bagging (bootstrap aggregating) bunu uygular: eğitim kümesinden yerine koymalı N örnek çekerek B farklı 'bootstrap' kümesi oluşturur, her birinde bir ağaç büyütür ve tahminleri oylar. Rastgele orman buna bir rastgelelik daha ekler: her bölmede tüm özelliklere değil rastgele seçilmiş bir alt kümeye (tipik olarak $\sqrt{d}$) bakılır. Bu, ağaçların birbirine benzemesini (aynı güçlü özelliği hep kök yapmasını) engeller ve ortalamanın etkisini artırır. Sonuç: tek ağaçtan belirgin biçimde daha iyi genelleyen, çok az ayar isteyen, tablo verisi için hâlâ en güçlü yöntemlerden biri.



Şekil 7.4 Rastgele orman: bootstrap örnekler + rastgele özellik alt kümeleri + oylama

Ormanın iki yan ürünü değerlidir. (1) Özellik önemi: her özelliğin ağaçlardaki bölmelerde sağladığı toplam safsızlık azalması; hangi ölçümün arızayı ayırt ettiğini söyler ve sensör seçimine (hangi sensörden vazgeçilebilir?) yardım eder. Dikkat: eş-doğrusal özellikler önemi paylaşır, sürekli özellikler kategoriklere göre şişirilir; daha güvenilir alternatif 'permütasyon önemi'dir. (2) Torba dışı (OOB) hata: her ağaç için bootstrap'e girmeyen örneklerde ölçülen hata, ücretsiz bir doğrulama tahmini verir.



Şekil 7.5 Rulman verisinde özellik önemleri

Örnek 7.2 — Ortalamanın varyans etkisi

Soru: Tek bir ağacın test hatasının standart sapması 6 puan olsun. Birbirinden bağımsız 25 ağacın ortalamasının sapması ne olur? Ağaçlar arasında korelasyon $\rho = 0.4$ varsa?

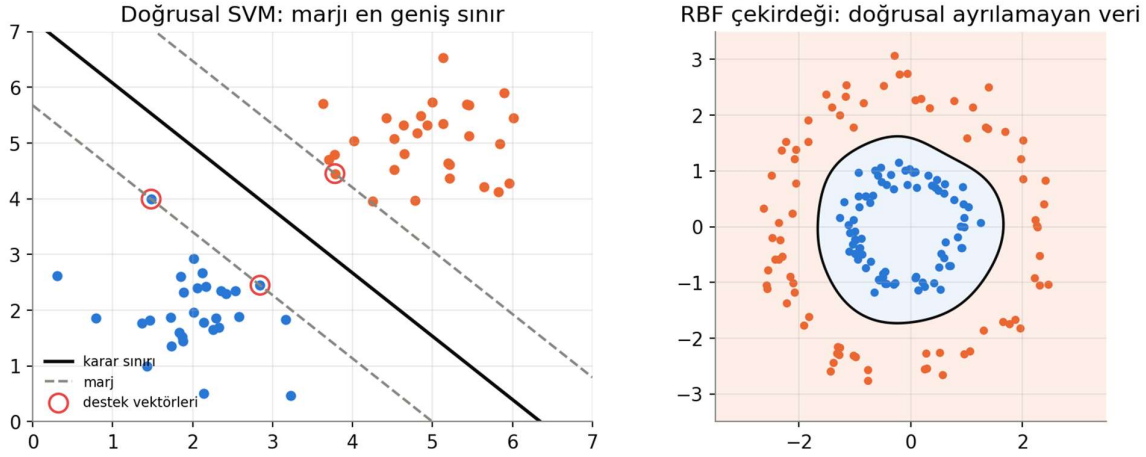
Bağımsız: $\sigma/\sqrt{B} = 6/\sqrt{25} = 1.2$ puan. Korelasyonlu: $\text{Var} = \rho\sigma^2 + (1-\rho)\sigma^2/B = 0.4 \times 36 + 0.6 \times 36/25 = 14.4 + 0.864 = 15.26 \rightarrow \sigma \approx 3.9$ puan. Korelasyon, ortalamanın kazancını sınırlar; rastgele özellik alt kümesinin amacı tam olarak ρ 'yu düşürmektir. B'yi 25'ten 1000'e çıkarmak korelasyonlu durumda 3.9'u ancak 3.8'e indirir: ağaç sayısını artırmak bir yerden sonra işe yaramaz, ağaçları çeşitlendirmek yarar.

7.4 Destek Vektör Makineleri (SVM)

İki sınıfı ayıran sonsuz sayıda doğru varken hangisi en iyisidir? SVM'nin cevabı: iki sınıfa en uzak olan, yani marjı (sınır ile en yakın örnekler arasındaki mesafeyi) en geniş olan. Marjı belirleyen o en yakın örneklerle

destek vektörleri denir; sınır yalnızca onlara bağlıdır, diğer örnekler silinse sınır değişmez. Veri tam ayıramıyorsa 'yumuşak marj' kullanılır: bazı örneklerin marjı ihlal etmesine izin verilir, ihlal C katsayısıyla cezalandırılır. C küçükse geniş marj, çok ihlal (basit model); C büyükse dar marj, az ihlal (karmaşık model, aşırı öğrenme riski).

Doğrusal ayıramayan veri için çekirdek (kernel) hilesi devreye girer: veriyi daha yüksek boyutlu bir uzaya haritalayıp orada doğrusal sınır çizmek, ama haritayı hiç hesaplamadan yalnızca iç çarpımları bir çekirdek fonksiyonuyla hesaplamak. En yaygın çekirdek RBF'dir: $K(x, x') = \exp(-\gamma \|x - x'\|^2)$. γ büyükse her örnek yalnızca çok yakın çevresini etkiler (karmaşık, kıvrımlı sınır); küçükse etki geniştir (yumuşak sınır). C ve γ birlikte CV ile seçilir. SVM ölçeklemeye duyarlıdır (uzaklık kullanır) ve orta boy veride (birkaç bin örnek) çok güçlüdür; çok büyük veride yavaşlar.



Şekil 7.6 Sol: doğrusal SVM, marj ve destek vektörleri. Sağ: RBF çekirdeği halka biçimli sınıfı ayırır

Yöntem	Sınır	Ölçekleme	Yorumlanabilirlik	Güçlü yanı	Zayıf yanı
Lojistik regresyon	Doğrusal	Gerekli	Yüksek (katsayı)	Hızlı, olasılık, taban model	Doğrusal olmayan ilişki
k-NN	Yerel, eğri	Zorunlu	Düşük	Basit, eğitimsiz	Yavaş tahmin, boyut laneti
Karar ağacı	Eksen-paralel dikdörtgenler	Gerekmez	Çok yüksek	Karma özellik, kural çıkarımı	Kararsız, aşırı öğrenir
Rastgele orman	Eğri (oylama)	Gerekmez	Orta (özellik önemi)	Az ayar, güçlü, tablo verisi	Büyük model, ekstrapolasyon yok
SVM (RBF)	Eğri (çekirdek)	Zorunlu	Düşük	Orta boy veride çok güçlü	C, γ ayarı; büyük veride yavaş

Tablo 7.3 Sınıflandırıcıların karşılaştırması

7.5 Kategorik Değişkenler: One-Hot Kodlama

Motor tipi (asenkon/senkron/dc) gibi sırasız kategorik bir sütun sayıya çevrilmeden çoğu modele giremez. Akla ilk gelen 0, 1, 2 atamak yanlıştır: model 'dc, senkronun iki katı' gibi olmayan bir sıralama ve mesafe varsayar. Doğru yol one-hot kodlamadır: her kategori için bir 0/1 sütunu. Üç kategori üç sütun olur; doğrusal modellerde sabit terimle eş-doğrusallığı önlemek için bir sütun düşürülebilir (drop='first'). Ağaç tabanlı modeller sıralı kodlamayı da tolere eder ama one-hot güvenli tercihtir. Kategori sayısı çok büyükse

(yüzlerce cihaz ID'si) one-hot patlar; o zaman hedef kodlama ya da gömme (embedding, 12. hafta) kullanılır. Ordinal (sıralı) kategoriler — düşük/orta/yüksek — sıralı tam sayı ile kodlanabilir, çünkü sıra gerçektir.

motor_tipi		tip_asenkron	tip_senkron	tip_dc
asen kron		1	0	0
sen kron	one-hot →	0	1	0
dc		0	0	1
asen kron		1	0	0

Yanlış: asenkron=0, senkron=1, dc=2 → 'dc, senkronun iki katı' anlamı çıkar

Şekil 7.7 One-hot kodlama

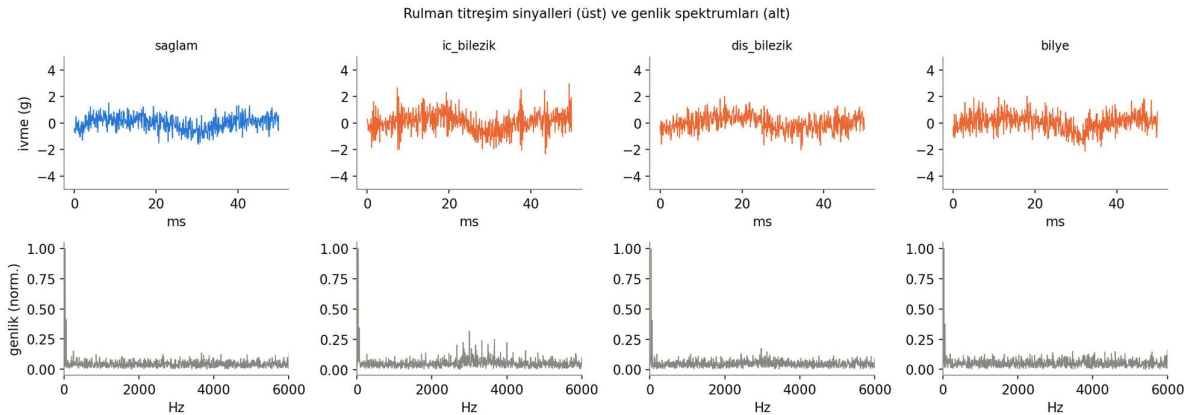
ColumnTransformer: sayısal ölçek, kategoriğe one-hot — hepsi tek Pipeline'da, kaçıksız

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression

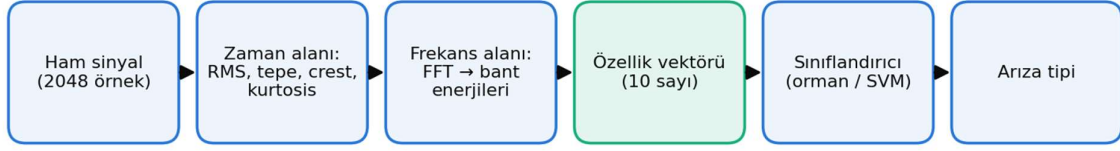
sayisal = ["calisma_saati", "titresim_rms_mms", "titresim_kurtosis", "sicaklik_C",
"akim_dengesizlik_pct"]
kategorik = ["motor_tipi"]
on_isleme = ColumnTransformer([("num", StandardScaler(), sayisal),
("kat", OneHotEncoder(handle_unknown="ignore"), kategorik)])
model = make_pipeline(on_isleme, LogisticRegression(max_iter=1000))
model.fit(Xtr, ytr) # Xtr: hem sayısal hem kategorik sütunları içeren DataFrame
```

7.6 Özellik Mühendisliği: Sinyalden Sayıya

Bu haftanın veri seti ham titreşim sinyalleridir: 12 kHz'de örneklenmiş 2048 noktalık 480 parça, dört sınıf (sağlam, iç bilezik, dış bilezik, bilye arızası). Ham 2048 sayıyı doğrudan bir ağaca vermek işe yaramaz (her örnek noktası ayrı bir 'özellik' olur, faz kaymasıyla anlamını yitirir). Mühendislik bilgisi burada devreye girer: rulman arızaları periyodik darbeler üretir; darbeler zaman alanında basıklığı (kurtosis) ve tepe/RMS oranını (crest faktörü) yükseltir, frekans alanında ise darbelerin uyardığı yüksek frekans bantlarında (2-6 kHz) enerji artışı yaratır. On sayılı bir özellik vektörü, 2048 sayılı sinyalin arıza bilgisini damıtır. Bu adıma özellik mühendisliği denir ve klasik makine öğrenmesinin en emek yoğun, en değerli kısmıdır. 12. haftada CNN'in bu özellikleri ham sinyalden kendisinin öğrendiğini göreceğiz — ama sinyal işleme bilgisi orada da veri hazırlamada gerekecek.



Şekil 7.8 Dört sınıftan örnek sinyaller ve spektrumları: arızalı sinyallerde periyodik darbeler ve 2-4 kHz civarında enerji



Özellik mühendisliği: alan bilgisi (sinyal işleme) modelin işini kolaylaştırır — 12. haftada CNN bunu ham sinyalden kendisi öğrenecek

Şekil 7.9 Özellik çıkarımı akışı

Özellik	Formül / tanım	Neyi yakalar?
RMS	$\sqrt{\text{mean}(x^2)}$	Genel titreşim seviyesi (ISO 10816)
Tepe	$\max x $	En büyük darbe
Crest faktörü	tepe / RMS	Darbelilik; erken arızada yükselir
Kurtosis (basıklık)	$\text{mean}((x-\mu)^4)/\sigma^4$	Darbe yoğunluğu; sağlam ≈ 3 , arızalı > 4
Çarpıklık	$\text{mean}((x-\mu)^3)/\sigma^3$	Asimetri
Bant enerji oranı	$\sum X(f) \text{ (} f \in \text{bant) } / \sum X(f) $	Arıza darbelerinin uyardığı rezonans bantları
Zarf spektrumu tepesi (ileri)	Hilbert zarfının FFT'sinde arıza frekansı	Arıza tipini doğrudan söyler (BPFO, BPFI, BSF)

Tablo 7.4 Titreşim sinyali özellikleri

Örnek 7.3 — Kurtosis elle

Soru: Sekiz örneklik iki sinyal: sağlam $s = [1, -1, 1, -1, 1, -1, 1, -1]$ ve darbeli $d = [0, 0, 0, 4, 0, 0, 0, -4]$. Kurtosis değerlerini karşılaştırın (ortalama sıfır).

$s: \sigma^2 = \text{mean}(s^2) = 1; \text{mean}(s^4) = 1 \rightarrow \text{kurtosis} = 1/1^2 = 1$. $d: \sigma^2 = (16 + 16)/8 = 4; \text{mean}(d^4) = (256 + 256)/8 = 64 \rightarrow \text{kurtosis} = 64/16 = 4$. Aynı RMS'e sahip olmasalar da ($s: 1, d: 2$) kurtosis darbeliliği ayırt eder: enerji birkaç örnekte toplanmışsa dördüncü moment şişer. Gauss gürültüsü için kurtosis 3'tür; sağlam rulman ≈ 3 , darbeli arıza 4-10 arası tipiktir.

```

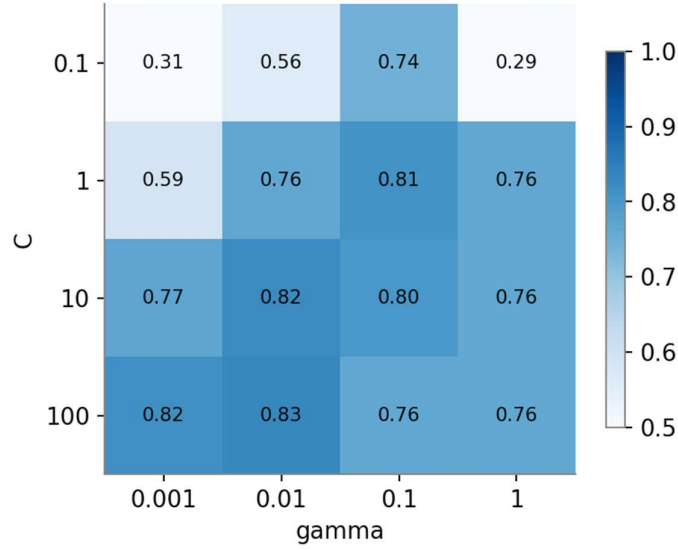
s = np.array([1, -1, 1, -1, 1, -1, 1, -1]); d = np.array([0, 0, 0, 4, 0, 0, 0, -4.])
kurt = lambda x: np.mean((x - x.mean())**4) / np.var(x)**2
print(kurt(s), kurt(d)) # 1.0 4.0
from scipy.stats import kurtosis; print(kurtosis(d, fisher=False)) # aynı (Pearson tanımı)

```

7.7 Hiperparametre Arama ve Çok Sınıflı Değerlendirme

Ağaç derinliği, orman ağaç sayısı, SVM'nin C ve γ 'sı: hepsi hiperparametre ve hepsi doğrulama verisiyle seçilmeli. GridSearchCV bunu sistematik yapar: verilen değer ızgarasındaki her kombinasyon için k-katlı CV skoru hesaplar, en iyisini seçer ve tüm eğitim verisiyle yeniden eğitir. Izgara büyürse ($4 \times 4 \times 4 = 64$ kombinasyon $\times 5$ kat = 320 eğitim) maliyet artar; RandomizedSearchCV rastgele örnekler, genellikle çok daha az denemeye benzer sonuç verir. Kural: arama Pipeline üzerinde yapılır ki ölçekleme her katta yalnızca eğitim parçasına fit olsun; test kümesi en sonda bir kez kullanılır.

GridSearchCV: 5-katlı CV doğruluğu (en iyi {'svc_C': 100, 'svc_gamma': 0.01})



Şekil 7.10 SVM için C-γ ızgarasında 5-katlı CV doğruluğu

Dört sınıflı problemde karışıklık matrisi 4×4 olur; köşegen doğru sınıflandırmalar, köşegen dışı hangi sınıfın hangisiyle karıştığını gösterir (ör. iç bilezik ile bilye). Precision/recall her sınıf için ayrı hesaplanır; tek sayı için makro-F1 (sınıfların F1 ortalaması, her sınıf eşit ağırlıklı) ya da ağırlıklı F1 (sınıf büyüklüğüne göre) raporlanır. Sınıflar dengesizse makro-F1 azınlık sınıfını görür kılar.

Örnek 7.4 — 3×3 karışıklık matrisinden makro-F1

Gerçek \ Tahmin	sağlam	iç	dış
sağlam	45	3	2
iç	4	38	8
dış	1	6	43

sağlam: TP 45, FP = 4 + 1 = 5, FN = 3 + 2 = 5 → P = 0.90, R = 0.90, F1 = 0.90. iç: TP 38, FP = 3 + 6 = 9, FN = 4 + 8 = 12 → P = 0.809, R = 0.760, F1 = 0.784. dış: TP 43, FP = 2 + 8 = 10, FN = 1 + 6 = 7 → P = 0.811, R = 0.860, F1 = 0.835. Makro-F1 = (0.90 + 0.784 + 0.835)/3 = 0.840. Doğruluk = 126/150 = 0.84. En çok karışan çift: iç ↔ dış (8 + 6 = 14 hata) — iki bilezik arızasının darbe imzaları benzer; ayırt etmek için zarf spektrumu (arıza frekansı) özelliği eklenmeli.

7.8 Dönem Projesi: Konu Bildirimi

Bu hafta proje konunuzu (2-3 kişi) bir sayfalık öneriyle bildirin: problem tanımı (girdi, çıktı, başarı ölçütü), veri kaynağı (kendi ölçümünüz ya da açık veri), planlanan yöntemler (taban + en az iki model), riskler. Değerlendirme ölçütleri ve örnek konular giriş bölümünde. 13. haftada ara teslim (EDA + taban model), 15. haftada sunum.

7.9 Sınıf İçi Sorular

1. {5S, 5A} düğümü için Gini ve entropi hesaplayın. {10S, 0A} için? Bölme kazancı ne zaman sıfırdır?

2. Rastgele ormanda ağaç sayısını 100'den 10 000'e çıkarmak aşırı öğrenmeye yol açar mı? Neden?
3. SVM'de $C \rightarrow \infty$ ne demektir? $\gamma \rightarrow 0$ iken RBF sınırı neye benzer?
4. Bir sütunda 500 farklı cihaz ID'si var; one-hot yaparsanız ne olur? Alternatif?
5. Sağlam rulmanın kurtosisi neden ≈ 3 ? Hangi arıza aşamasında crest faktörü düşmeye başlar (ipucu: ileri aşama, çok sayıda darbe)?

7.10 Ödev 4 — Rulman Arıza Sınıflandırma

- rulman_titresim.csv'den kendi özellik çıkarım fonksiyonunuzu yazın (en az Tablo 7.4'teki 6 özellik + bir yeni özellik: ör. zarf spektrumu tepe frekansı ya da farklı bant bölmesi).
- Stratified %70/30 ayırım; lojistik regresyon, karar ağacı, rastgele orman ve SVM (RBF) eğitin; GridSearchCV ile ağaç derinliği ve SVM C/ γ seçin; makro-F1 ve 4×4 karışıklık matrisi raporlayın.
- Özellik önemini çizin; en önemsiz iki özelliği atınca skor değişiyor mu?
- Sağlamlık testi: test sinyallerine ek beyaz gürültü ($\sigma = 0.3$) ekleyip skorları yeniden hesaplayın; hangi model daha dayanıklı? Hangi özellik gürültüden en çok etkileniyor?
- Teslim: defter + yarım sayfa yorum; proje konu bildirimi ayrıca.

Hafta 8 — Ara Sınav ve Denetimsiz Öğrenme

Bölüm	Süre	İçerik
Saat 1	50 dk	ARA SINAV (1-7. haftalar): kavramsal sorular + elle sayısal çözüm
Saat 2	50 dk	Denetimsiz öğrenme: k-means (elle iterasyon), dirsek ve silüet, hiyerarşik kümeleme; PCA: fikir, açıklanan varyans, bileşen yorumu
Saat 3	50 dk	Uygulama: 300 abonenin yük profillerini kümeleme; PCA ile görselleştirme; küme merkezlerini yorumlama

Tablo 8.1 Haftanın ders planı

8.1 Ara Sınav

Ara sınav ilk 7 haftayı kapsar; kitap ve not kapalı, hesap makinesi serbest. Ağırlıklar: kavramlar (%40: YZ/MÖ/DÖ, öğrenme türleri, aşırı öğrenme, veri kaçağı, ölçüt seçimi), elle hesap (%45: ortalama/std/korelasyon, MAE/RMSE/R², en küçük kareler doğrusu, gradyan inişi adımı, sigmoid ve log-loss, karışıklık matrisi ölçütleri, Gini kazancı), kısa kod okuma (%15: verilen bir scikit-learn Pipeline'in ne yaptığını açıklamak, hatayı bulmak). Örnek sorular:

1. Bir yük tahmin modelinde eğitim RMSE 3 kW, test RMSE 21 kW. Teşhis, iki neden ve üç çare yazın. (10 p)
2. $x = [1, 2, 3, 4]$, $y = [3, 5, 6, 8]$ için en küçük kareler doğrusunu bulun; R^2 'yi hesaplayın; $x = 5$ için tahmin. (15 p)
3. $TP = 60$, $FN = 40$, $FP = 10$, $TN = 890$: doğruluk, precision, recall, F1. Kaçırma maliyeti 20 000 TL, yanlış alarm 1 000 TL ise eşiği hangi yöne çekersiniz, neden? (15 p)
4. $z = 0.8$ için sigmoid; $y = 1$ ise log-loss. (5 p)
5. Aşağıdaki kodda veri kaçağı var mı? Varsa satırı işaretleyin ve düzeltin: `scaler.fit(X)`; `Xs = scaler.transform(X)`; `Xtr, Xte = train_test_split(Xs)`. (10 p)
6. Bir düğümde {4S, 4A}; iki aday bölme: (a) {4S, 1A} | {0S, 3A}, (b) {2S, 3A} | {2S, 1A}. Gini kazançlarını hesaplayıp seçin. (10 p)
7. Saat 0-23'ü doğrudan sayı olarak kullanmak yerine sin/cos kodlamasının iki gerekçesi. (5 p)
8. Rastgele orman neden tek karar ağacından daha iyi genelleştirir? İki mekanizma. (10 p)
9. Zaman serisinde rastgele train/test ayrımı neden yanlıştır? Doğru ayrım nasıl yapılır? (10 p)
10. Bir motor arıza modeli için 'doğruluk %96' raporlandı; arıza oranı %4. Bu sonuç neden yetersizdir, hangi ölçütleri istersiniz? (10 p)

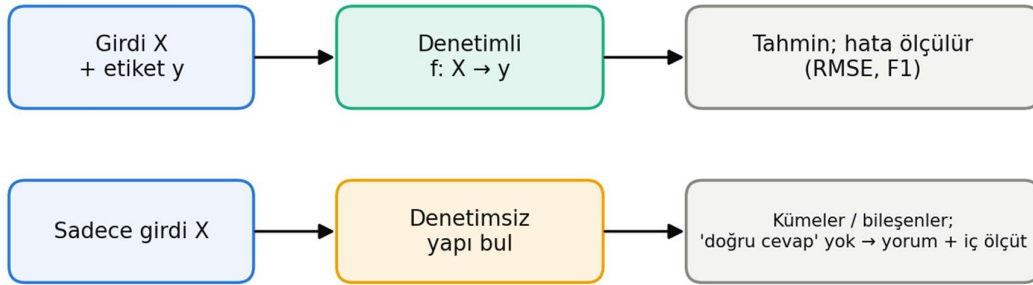
8.2 Kazanımlar

- Denetimsiz öğrenmenin denetimliden farkını ve 'doğru cevap' olmadan değerlendirmenin nasıl yapıldığını açıkla.
- k-means algoritmasını elle birkaç iterasyon uygula; başlangıç duyarlılığını ve k seçimini (dirsek, silüet) bil.
- Hiyerarşik kümelemeyi ve dendrogramı yorumla; k-means ile farkını söyle.
- PCA'nın ne yaptığını (maksimum varyans yönleri) geometrik olarak açıkla; açıklanan varyansı ve bileşen yüklerini yorumla.
- Yük profillerini kümeler, küme merkezlerini mühendislik diliyle yorumla (konut, ofis, sanayi...).

8.3 Denetimsiz Öğrenme Nedir?

Şimdiye kadar her örneğin doğru cevabı vardı ve model o cevaba yaklaşmayı öğrendi. Denetimsiz öğrenmede etiket yoktur; elimizde yalnızca girdi vardır ve soru 'bu veride ne tür bir yapı var?' sorusudur. İki ana görev vardır. Kümeleme: benzer örnekleri gruplamak — 300 aboneyi tüketim alışkanlıklarına göre birkaç 'tarife grubuna' ayırmak, sensör kayıtlarında normal çalışma rejimlerini bulmak. Boyut indirgeme: çok sayıda özelliği, bilgiyi olabildiğince koruyan az sayıda yeni özellik ile temsil etmek — 24 saatlik profili 2-3 sayıyla anlatmak, 100 sensörlü bir sistemi görselleştirmek, gürültüyü atmak. Üçüncü bir görev olan anomali tespiti (normalden sapanı bulmak) 14. haftada.

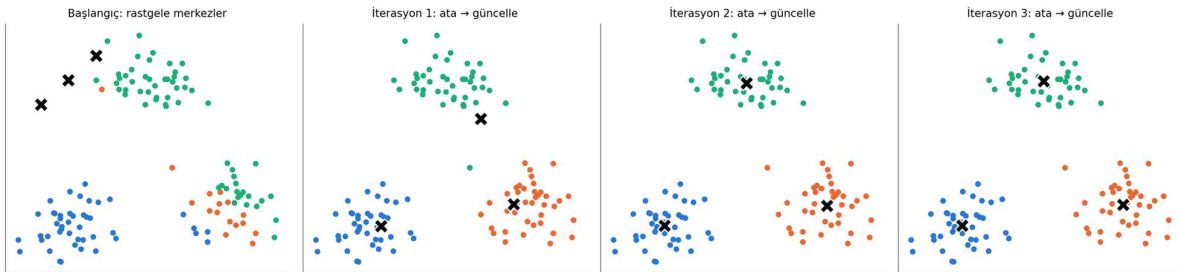
Değerlendirme denetimsizde zordur, çünkü karşılaştırılacak doğru cevap yoktur. İki yol izlenir: iç ölçütler (kümeler ne kadar sıkı ve birbirinden ayık — inertia, silüet) ve alan bilgisiyle yorum (bulunan kümeler anlamlı mı, mühendis onları isimlendirebiliyor mu?). Bazen bir 'dış' etiket sonradan elde edilir (bu haftaki veri setinde gerçek abone tipi ayrı bir dosyada tutulur) ve kümeler onunla karşılaştırılır; ama bu bir doğrulama lüksüdür, algoritma etiketi görmez.



Şekil 8.1 Denetimli ve denetimsiz öğrenmenin akışı

8.4 k-Means Kümeleme

k-means, en yaygın kümeleme algoritmasıdır ve fikri basittir: k küme merkezi seç; her örneği en yakın merkeze ata; her merkezi kendi örneklerinin ortalamasına taşı; atamalar değişmeyene kadar tekrarla. Algoritma, küme içi kareler toplamını (her örneğin kendi merkezine uzaklık karesi — inertia) her adımda azaltır ve bir yerel minimuma yakınsar. 'Yerel' kelimesi önemlidir: sonuç başlangıç merkezlerine bağlıdır; bu yüzden pratikte algoritma farklı rastgele başlangıçlarla birkaç kez çalıştırılır ve en düşük inertia'lı sonuç alınır (n_{init}), ya da akıllı başlangıç (k-means++) kullanılır. k-means küresel, benzer boyutlu kümeler varsayar; uzun ince ya da iç içe kümelerde başarısız olur. Uzaklık kullandığı için ölçekleme şarttır.



Şekil 8.2 k-means: rastgele başlangıç, ata-güncelle iterasyonları

Örnek 8.1 — k-means elle (1-B)

Soru: Altı abonenin günlük ortalama yükü (kW): 2, 3, 4, 10, 11, 25. $k = 2$, başlangıç merkezleri $m_1 = 2$, $m_2 = 25$. İterasyonları yapın.

İterasyon 1 — atama: her nokta en yakın merkeze: 2, 3, 4, 10, 11 $\rightarrow m_1$ (uzaklıklar 0, 1, 2, 8, 9 vs 23, 22, 21, 15, 14); 25 $\rightarrow m_2$. Güncelleme: $m_1 = (2+3+4+10+11)/5 = 6.0$; $m_2 = 25$.

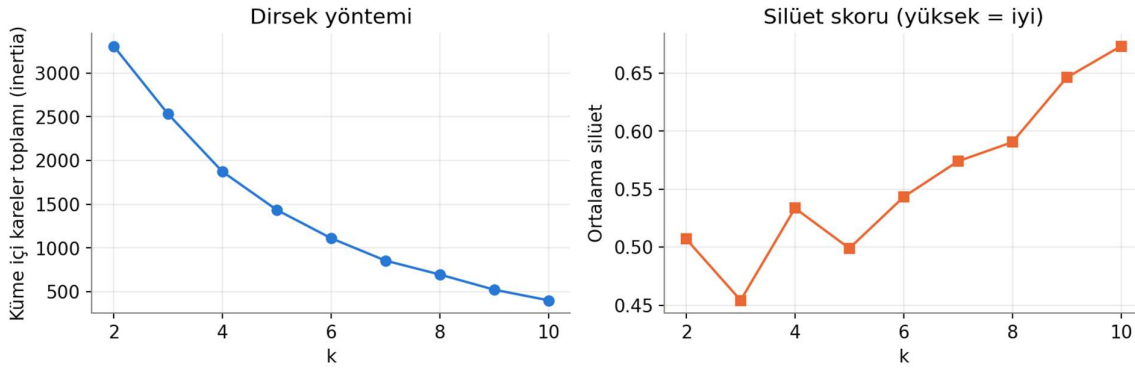
İterasyon 2 — atama: 2, 3, 4 $\rightarrow m_1$ ($|x-6| < |x-25|$); 10 $\rightarrow m_1$ ($4 < 15$); 11 $\rightarrow m_1$ ($5 < 14$); 25 $\rightarrow m_2$. Atamalar değişmedi $\rightarrow$ yakınsadı. Sonuç: {2, 3, 4, 10, 11} ve {25}. Inertia = $(16 + 9 + 4 + 16 + 25) + 0 = 70$.

Farklı başlangıç: $m_1 = 3$, $m_2 = 10$. İt. 1: 2, 3, 4 $\rightarrow m_1$; 10, 11, 25 $\rightarrow m_2 \rightarrow m_1 = 3$, $m_2 = 15.33$. İt. 2: 2, 3, 4 $\rightarrow m_1$; 10, 11, 25 $\rightarrow m_2$ ($|10-15.33| = 5.3 < 7$). Yakınsadı: {2, 3, 4} ve {10, 11, 25}; inertia = $(1 + 0 + 1) + (28.4 + 18.8 + 93.4) = 142.6$. İlk çözüm daha iyi ($70 < 142.6$); başlangıç sonucu değiştirdi — n_init 'in nedeni. Ayrıca 25'in tek başına küme olması aykırı değer etkisini gösterir: k-means aykırılara duyarlıdır.

```
x = np.array([2, 3, 4, 10, 11, 25])
def kmeans1d(x, m, it=10):
    for _ in range(it):
        lab = np.argmax(np.abs(x[:, None] - m[None]), 1); yeni = np.array([x[lab == k].mean() for k
        in range(len(m))])
        if np.allclose(yeni, m): break
        m = yeni
    return m, lab, sum((x - m[lab])**2)
for m0 in ([2., 25.], [3., 10.]): print(kmeans1d(x, np.array(m0)))
```

k nasıl seçilir?

k bir hiperparametredir ama doğrulama hatası yoktur; iki iç ölçüt kullanılır. Dirsek yöntemi: inertia'yı k'ya karşı çizin; k arttıkça inertia her zaman düşer ($k = N$ 'de sıfır), ama bir noktadan sonra kazanç azalır — eğrinin 'dirseği' makul k'dır. Silüet skoru: her örnek için $(b - a)/\max(a, b)$, a = kendi kümesindeki ortalama uzaklık, b = en yakın diğer kümeye ortalama uzaklık; +1 mükemmel ayırık, 0 sınırda, negatif yanlış küme. Ortalama silüeti en yüksek yapan k seçilir. İki ölçüt her zaman aynı k'yı vermez; son söz alan bilgisindedir: 'beş tarife grubu yönetilebilir, on iki çok fazla'.



Şekil 8.3 Abone verisinde dirsek (inertia) ve silüet eğrileri

Örnek 8.2 — Silüet elle

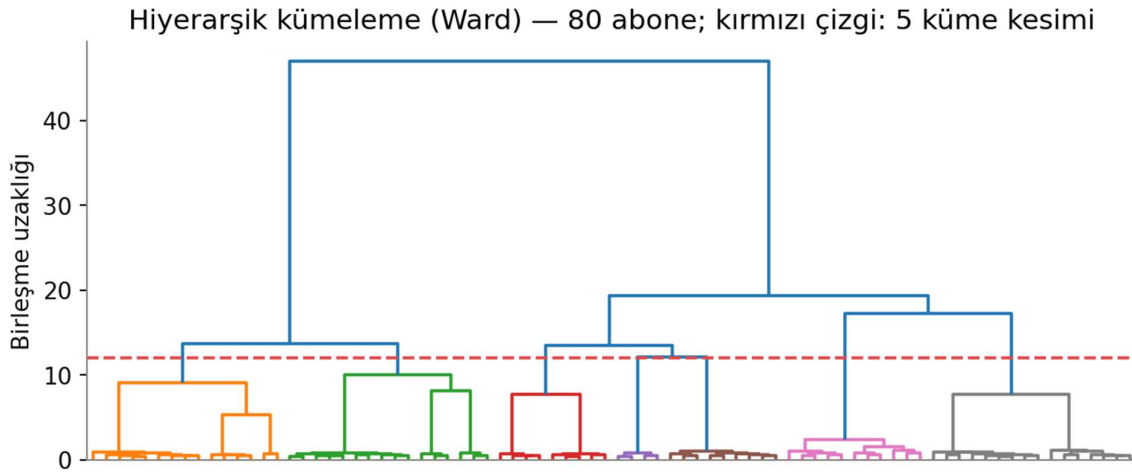
Soru: 1-B veride kümeler {2, 3, 4} ve {10, 11, 25}. $x = 4$ ve $x = 10$ noktalarının silüetini hesaplayın.

$x = 4$: $a = \text{ortalama}(|4-2|, |4-3|) = 1.5$; $b = \text{ortalama}(6, 7, 21) = 11.33 \rightarrow s = (11.33 - 1.5)/11.33 = 0.868$ (iyi yerleşmiş). $x = 10$: $a = \text{ortalama}(1, 15) = 8.0$; $b = \text{ortalama}(8, 7, 6) = 7.0 \rightarrow s = (7 - 8)/8 = -0.125$

(negatif: 10 aslında diğer kümeye daha yakın; 25 aykırı değeri kümeyi bozuyor). Ortalama silüet düşükse ya k yanlışdır ya da aykırı değer/ölçekleme sorunu vardır.

8.5 Hiyerarşik Kümeleme

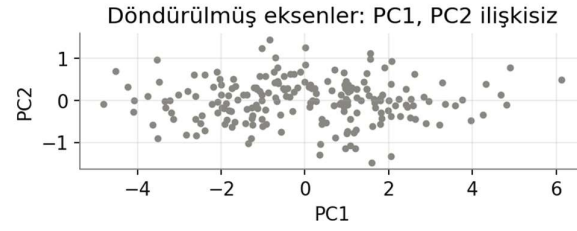
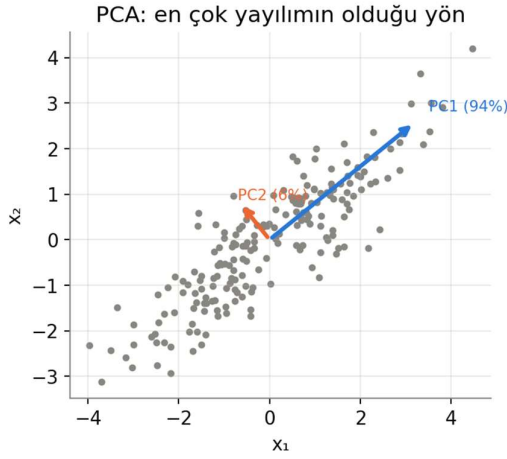
k-means k'yı önceden ister; hiyerarşik (aglomeratif) kümeleme istemez: her örnek kendi kümesi olarak başlar, en yakın iki küme birleştirilir, tek küme kalana kadar sürer. Sonuç bir ağaçtır (dendrogram): yatay bir çizgiyle kesildiğinde istenen sayıda küme elde edilir ve birleşme yükseklikleri kümelerin ne kadar 'uzak' olduğunu gösterir. 'İki kümenin uzaklığı' nasıl tanımlanır sorusunun cevabı bağlantı (linkage) kuralıdır: tek (en yakın çift), tam (en uzak çift), ortalama, Ward (birleşince inertia artışı en küçük — k-means'e en yakın). Hiyerarşik kümeleme küçük-orta veride (birkaç bin) kullanışlıdır; N^2 bellek ister. Avantajı, k'yı dendrogramdan sonradan seçmek ve iç içe yapıyı (alt kümeler) görmektir.



Şekil 8.4 80 abonenin Ward dendrogramı; kesim çizgisi 5 küme verir

8.6 Temel Bileşen Analizi (PCA)

24 saatlik bir yük profili 24 boyutlu bir vektördür; 300 aboneyi 24 boyutlu uzayda göremeyiz. PCA (Principal Component Analysis), veriyi en çok yayılımın (varyansın) olduğu yönlerde döndürür: birinci temel bileşen (PC1) verinin en çok değiştiği doğrultudur, PC2 ona dik ve kalan varyansı en çok açıklayan doğrultudur, böyle devam eder. Bileşenler, orijinal özelliklerin doğrusal birleşimleridir; ilk birkaçı çoğu zaman varyansın büyük kısmını taşır ve geri kalanı gürültü gibi atılabilir. Matematiksel olarak PCA, kovaryans matrisinin özvektörlerini bulur; özdeğerler her bileşenin açıkladığı varyanstır. Uygulamada `PCA(n\_components)` çağrılır; ama önce standartlaştırma şarttır, yoksa büyük ölçekli özellik PC1'i tek başına belirler.



Şekil 8.5 PCA geometrisi: en çok yayılım yönü PC1, ona dik PC2; döndürülmüş eksenlerde bileşenler ilişkisiz

Örnek 8.3 — 2x2 kovaryansın özdeğerleri (elle)

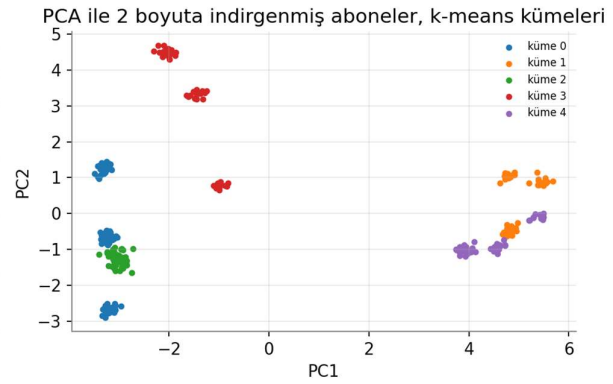
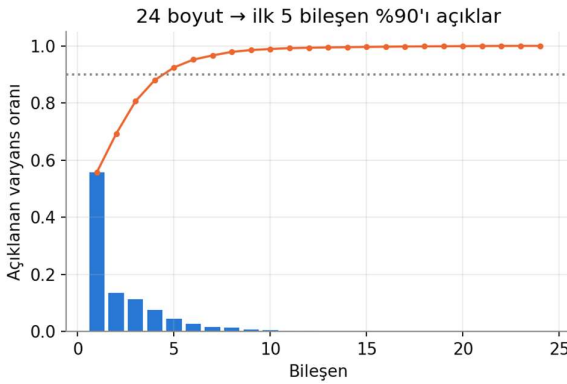
Soru: Standartlaştırılmış iki özelliğin kovaryans matrisi $C = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$. Temel bileşenleri ve açıklanan varyans oranlarını bulun.

Özdeğerler: $\det(C - \lambda I) = (1 - \lambda)^2 - 0.64 = 0 \rightarrow 1 - \lambda = \pm 0.8 \rightarrow \lambda_1 = 1.8, \lambda_2 = 0.2$. Özvektörler: λ_1 için $(C - 1.8I)v = 0 \rightarrow -0.8v_1 + 0.8v_2 = 0 \rightarrow v = (1, 1)/\sqrt{2}$ (45° yönü: 'iki özellik birlikte artar'); λ_2 için $v = (1, -1)/\sqrt{2}$ ('fark').

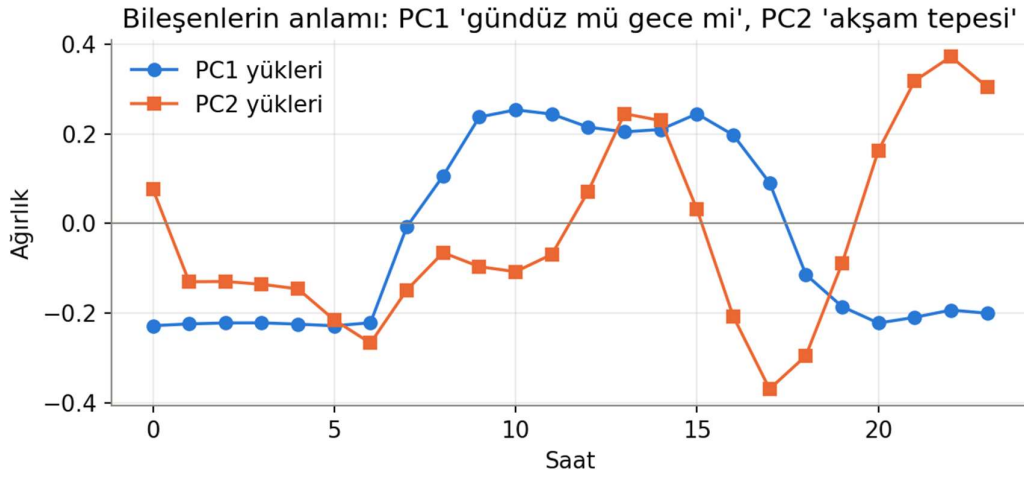
Toplam varyans = $1.8 + 0.2 = 2$. PC1 oranı = $1.8/2 = \%90$; PC2 = $\%10$. Yorum: korelasyon 0.8 olan iki özellik neredeyse tek boyutludur; PC1 tek başına $\%90$ 'ı taşır — boyut 2'den 1'e inerken bilginin $\%10$ 'u kaybolur.

```
C = np.array([[1, .8], [.8, 1]]); lam, V = np.linalg.eigh(C)
print("özdeğerler:", lam[::-1], " oranlar:", (lam/lam.sum())[::-1].round(2)); print("PC1 yönü:", V[:, -1].round(3))
```

PCA'nın çıktıları üç şekilde okunur. (1) Açıklanan varyans grafiği: kaç bileşen yeterli? ($\%90$ kuralı yaygındır.) (2) Skorlar: her örneğin PC1, PC2 değerleri — saçılım grafiği çizmek ve kümeleri görmek için. (3) Yükler: her bileşenin orijinal özelliklere verdiği ağırlık — bileşene anlam vermek için. Abone verisinde PC1 gündüz saatlerine pozitif, geceye negatif ağırlık veriyorsa PC1 'gündüz-gece dengesi' eksenidir; ofisler bir uçta, gece çalışan tesisler diğer uçtadır.



Şekil 8.6 Sol: açıklanan varyans — ilk 5 bileşen $\%90$, ilk 2 bileşen $\%70$. Sağ: PC1–PC2 düzleminde aboneler ve k-means kümeleri



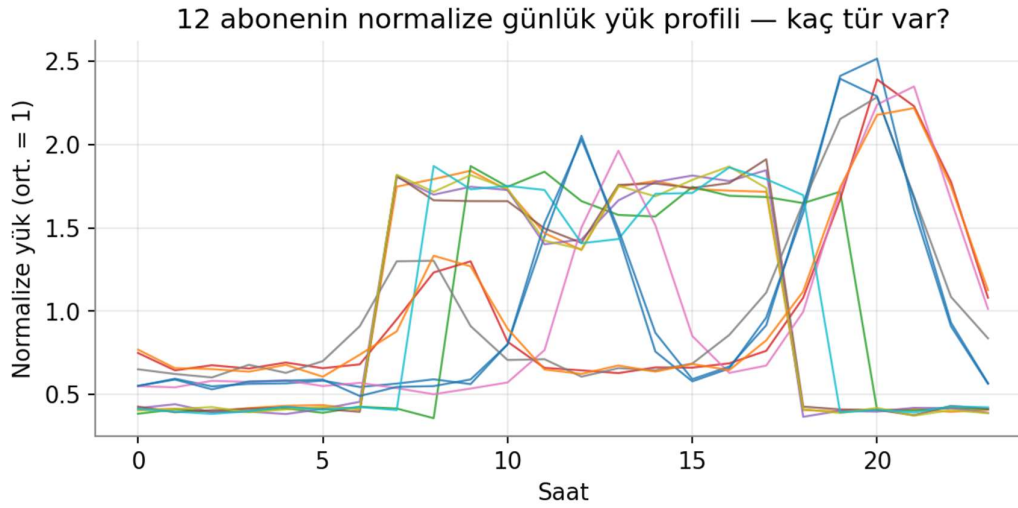
Şekil 8.7 Bileşen yükleri: PC1 gündüz/gece, PC2 akşam tepesi

Yöntem	Ne yapar?	Girdi	Çıktı	Dikkat
k-means	k küresel küme; ata-güncelle	Ölçekli X, k	Etiket + merkezler	Başlangıç (n_init), aykırılar, küresel varsayım
Hiyerarşik	Alt-üst küme ağacı	Ölçekli X, bağlantı kuralı	Dendrogram	N ² bellek; k sonradan
DBSCAN (ek)	Yoğunluk tabanlı; gürültüyü ayırır	eps, min_samples	Etiket (-1 = gürültü)	Farklı yoğunluklarda zor
PCA	Maks varyans yönleri; doğrusal	Ölçekli X, bileşen sayısı	Skorlar, yükler, oranlar	Doğrusal; varyans ≠ önem
t-SNE / UMAP (ek)	Doğrusal olmayan görselleştirme	X, perplexity	2-B harita	Yalnızca görselleştirme; uzaklıklar anlamsız

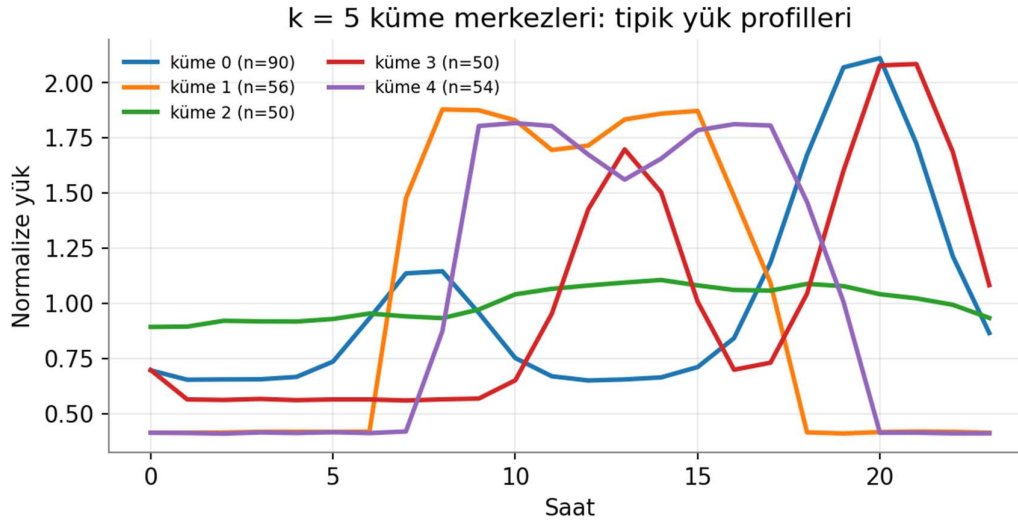
Tablo 8.2 Denetimsiz yöntemler

8.7 Uygulama: Abone Yük Profilleri

Veri seti abone_profilleri.csv: 300 abonenin 24 saatlik ortalama günlük yük profili (kW). Aboneler farklı büyüklüktedir (2-40 kW); şekle göre kümelemek için önce her profil kendi ortalamasına bölünür (normalize), sonra standartlaştırılır. Amaç, dağıtım şirketinin tarife/talep yönetimi için tipik profil gruplarını bulmak ve her gruba mühendislik adı vermektir. Gerçek abone tipleri ayrı bir dosyada (abone_profilleri_etiket.csv) verilmiştir; yalnızca son adımda, bulunan kümelerin anlamlı olup olmadığını kontrol etmek için kullanılır.



Şekil 8.8 12 abonenin normalize profili



Şekil 8.9 k = 5 küme merkezleri: her biri bir 'tipik gün'

Kümeleme + PCA iskeleti

```
import pandas as pd, numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

ab = pd.read_csv("abone_profilleri.csv"); P = ab.drop(columns="abone_id").values
Pn = P / P.mean(1, keepdims=True) # şekil: her profilin ortalaması 1
Z = StandardScaler().fit_transform(Pn)
for k in range(2, 9):
    km = KMeans(k, n_init=10, random_state=0).fit(Z)
    print(k, "inertia", round(km.inertia_), "silüet", round(silhouette_score(Z, km.labels_), 3))
km = KMeans(5, n_init=10, random_state=0).fit(Z)
merkez = km.cluster_centers_ # ölçekli uzayda; yorum için inverse_transform
T = PCA(2).fit_transform(Z) # 2-B görselleştirme
```

8.8 Sınıf İçi Sorular

1. k-means'te k = N seçilirse inertia kaçır? Bu 'en iyi' model midir?

2. Ölçeklemeden k-means: çalışma saati (0-20000) ve RMS (0-8) özelliklerinde kümeler neye göre oluşur?
3. Bir dendrogramda iki alt kümenin çok yüksekte birleşmesi ne anlama gelir?
4. PCA'da PC1 %40, PC2 %35, PC3 %5 açıklıyor. Kaç bileşen tutarsınız? PC3'ü atmak ne kaybettirir?
5. Standartlaştırılmamış veride bir özellik 1000 kat büyük ölçekliyse PC1 ne olur?

8.9 Uygulama Defteri ve Sonraki Hafta

- Defter: k-means elle → sklearn; dirsek/silüet; hiyerarşik + dendrogram; PCA (açıklanan varyans, skorlar, yükler); küme merkezlerini isimlendirme; etiketle karşılaştırma (ARI).
- Ödev yok (ara sınav haftası). Proje konu bildirimleri geri bildirimle iade edilir.
- Gelecek hafta: yapay sinir ağlarına giriş — perceptron, aktivasyon, ileri yayılım elle, XOR.

Hafta 9 — Yapay Sinir Ağlarına Giriş

Bölüm	Süre	İçerik
Saat 1	50 dk	Biyolojik esin, yapay nöron, perceptron öğrenme kuralı (elle), aktivasyon fonksiyonları
Saat 2	50 dk	XOR problemi ve gizli katman; çok katmanlı ağ (MLP); ileri yayılım matris biçiminde (elle); evrensel yaklaşım; mimari seçimi
Saat 3	50 dk	Uygulama: NumPy ile 2-3-1 ağın ileri yayılımı; perceptron eğitimi; sklearn MLP ile motor verisi

Tablo 9.1 Haftanın ders planı

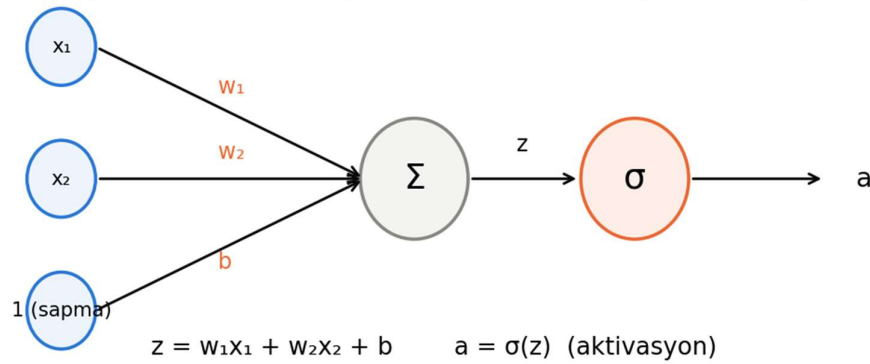
9.1 Kazanımlar

- Yapay nöronun hesabını (ağırlıklı toplam + aktivasyon) yazar ve elle hesaplar.
- Perceptron öğrenme kuralını elle birkaç adım uygular; neden yalnızca doğrusal ayrılabilen problemleri çözdüğünü açıklar.
- Sigmoid, tanh, ReLU aktivasyonlarını ve türevlerini karşılaştırır; ReLU'nun neden tercih edildiğini söyler.
- XOR'un tek katmanla çözilemeyip gizli katmanla çözüldüğünü gösterir.
- Çok katmanlı ağın ileri yayılımını matris biçiminde yazar; küçük bir ağ için elle hesaplar; parametre sayısını bulur.
- Evrensel yaklaşım fikrini ve derinlik/genişlik seçimini açıklar.

9.2 Biyolojik Esin ve Yapay Nöron

Beyindeki bir nöron, dendritlerinden gelen sinyalleri toplar; toplam bir eşiği aşarsa akson boyunca bir darbe ('ateşleme') gönderir. Sinapsların gücü öğrenmeyle değişir. 1943'te McCulloch ve Pitts bu davranışı matematiksel bir birime indirgedi; 1958'de Rosenblatt bu birimin ağırlıklarını veriden öğrenen perceptron'u yaptı. Yapay nöron iki adımdır: girdilerin ağırlıklı toplamı $z = \sum w_i x_i + b$ ve bu toplamın doğrusal olmayan bir aktivasyon fonksiyonundan geçirilmesi $a = f(z)$. İlk adımı tanıyorsunuz — 5. haftanın doğrusal regresyonu. İkinci adım da tanıdık: sigmoid seçilirse nöron tam olarak 6. haftanın lojistik regresyonudur. Yapay sinir ağı, bu nöronlardan çok sayıda ve katmanlar hâlinde bağlanmış olanıdır; 'derin öğrenme' katman sayısının çokluğuna gönderme yapar.

Yapay nöron: ağırlıklı toplam + doğrusal olmayan aktivasyon



Şekil 9.1 Yapay nöron: ağırlıklı toplam ve aktivasyon

Biyolojik	Yapay	Bu derste karşılığı
Dendrit girdileri	Özellikler $x_1 \dots x_d$	Sıcaklık, saat, titreşim RMS
Sinaps gücü	Ağırlık w_i	Regresyon katsayısı
Eşik	Sapma b (bias)	Sabit terim
Ateşleme	Aktivasyon $f(z)$	Sigmoid $\rightarrow$ olasılık; ReLU $\rightarrow$ kırık çizgi (5. hafta klima özelliği!)
Öğrenme (sinaptik plastisite)	Ağırlık güncelleme	Gradyan inişi (5. hafta) $\rightarrow$ geri yayılım (10. hafta)

Tablo 9.2 Benzetme ve daha önce gördüklerimizle bağlantı

9.3 Perceptron ve Öğrenme Kuralı

Perceptron, basamak aktivasyonlu tek nöronudur: $z \geq 0$ ise 1, değilse 0. Öğrenme kuralı son derece basittir ve türev gerektirmez: her örnek için tahmin yap; yanlışsa ağırlıkları $w \leftarrow w + \eta(y - \hat{y})x$ ve $b \leftarrow b + \eta(y - \hat{y})$ ile güncelle. Yani yanlış negatifte ($y = 1, \hat{y} = 0$) ağırlıklar x yönünde artar, yanlış pozitifte azalır; doğru tahminde hiçbir şey değişmez. Rosenblatt'ın teoremi, veri doğrusal ayrılabilirse bu kuralın sonlu adımda hatasız bir ayırıcı bulacağını garanti eder. Sınırı da buradadır: ayrılamıyorsa asla yakınsamaz ve karar sınırı her zaman bir doğrudur (hiperdüzlem).

Örnek 9.1 — Perceptron ile AND kapısı (elle)

Soru: AND fonksiyonunu öğrenmek istiyoruz: $(0,0) \rightarrow 0, (0,1) \rightarrow 0, (1,0) \rightarrow 0, (1,1) \rightarrow 1$. $w = (0, 0)$, $b = 0$, $\eta = 1$ ile başlayın; örnekleri sırayla gezip güncellemeleri yapın.

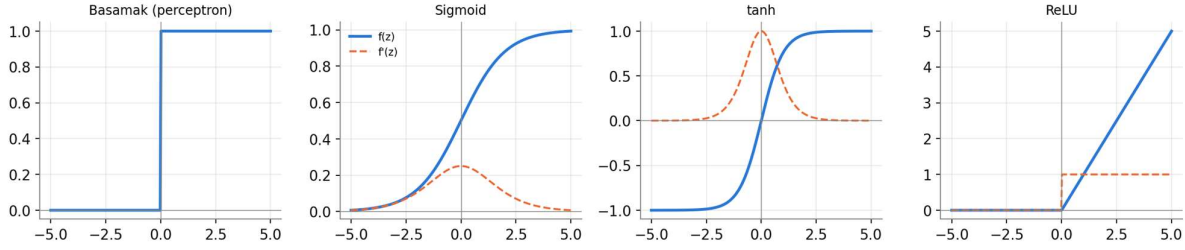
Örnek	x	y	$z = w \cdot x + b$	$\hat{y}$	$y - \hat{y}$	Güncelleme $\rightarrow (w_1, w_2, b)$
1	(0,0)	0	0	1	-1	(0, 0, -1)
2	(0,1)	0	-1	0	0	değişmez
3	(1,0)	0	-1	0	0	değişmez
4	(1,1)	1	-1	0	+1	(1, 1, 0)
1'	(0,0)	0	0	1	-1	(1, 1, -1)
2'	(0,1)	0	0	1	-1	(1, 0, -2)
3'	(1,0)	0	-1	0	0	değişmez
4'	(1,1)	1	-1	0	+1	(2, 1, -1)

($z \geq 0 \rightarrow \hat{y} = 1$ kuralı.) Üçüncü turda: $(0,0)$: $z = -1 \rightarrow 0 \checkmark$; $(0,1)$: $z = 0 \rightarrow 1 \times \rightarrow (2, 0, -2)$; $(1,0)$: $z = 0 \rightarrow 1 \times \rightarrow (1, 0, -3)$; $(1,1)$: $z = -2 \rightarrow 0 \times \rightarrow (2, 1, -2)$. Dördüncü tur: $(0,0)$ -2 $\checkmark$, $(0,1)$ -1 $\checkmark$, $(1,0)$ 0 $\rightarrow 1 \times \rightarrow (1, 1, -3)$; $(1,1)$ -1 $\times \rightarrow (2, 2, -2)$. Beşinci tur: -2 $\checkmark$, 0 $\times \rightarrow (2, 1, -3)$; $(1,0)$: -1 $\checkmark$; $(1,1)$: 0 $\checkmark$. Altıncı tur: -3, -2, -1, 0 $\rightarrow$ hepsi doğru; yakınsadı: $2x_1 + x_2 - 3 \geq 0$ doğrusu AND'i ayırır. Elle uzun ama mekanik; defterde 10 satır kod.

```
X = np.array([[0,0],[0,1],[1,0],[1,1]]); y = np.array([0,0,0,1]); w = np.zeros(2); b = 0.0
for tur in range(10):
    hata = 0
    for xi, yi in zip(X, y):
        yp = int(w @ xi + b >= 0); w += (yi - yp) * xi; b += (yi - yp); hata += int(yp != yi)
    print(tur + 1, w, b, "hata:", hata)
    if hata == 0: break
```

9.4 Aktivasyon Fonksiyonları

Aktivasyon fonksiyonu ağı doğrusal olmayanlık katar; onsu kaç katman koyarsanız koyun ağı tek bir doğrusal dönüşüme eşdeğer olur (doğrusal fonksiyonların bileşimi doğrusaldır). Basamak fonksiyonu perceptron için uygundur ama türevi her yerde sıfırdır; gradyanla öğrenilemez. Sigmoid (0,1) aralığında yumuşaktır, çıktı katmanında olasılık için idealdir; ama türevi en çok 0.25'tir ve $|z|$ büyüyünce sıfıra iner — çok katmanlı ağıda gradyanlar katman katman küçülür ('kaybolan gradyan'). tanh sıfır merkezli olduğu için sigmoidden biraz iyidir ama aynı doygunluk sorununu taşır. ReLU = $\max(0, z)$ 2012 sonrası standarttır: hesabı ucuz, pozitif bölgede türevi 1 (gradyan kaybolmaz), seyrek aktivasyon üretir. Kusuru, negatif bölgede türevin sıfır olması ('ölü nöron'); Leaky ReLU, ELU, GELU gibi türevleri bunu yumuşatır.



Şekil 9.2 Aktivasyon fonksiyonları ve türevleri

Fonksiyon	Formül	Aralık	Türev	Kullanım
Basamak	$1[z \geq 0]$	$\{0, 1\}$	0 (öğrenilemez)	Perceptron (tarihî)
Sigmoid	$1/(1+e^{-z})$	$(0, 1)$	$\sigma(1-\sigma) \leq 0.25$	İkili çıktı katmanı
tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	$(-1, 1)$	$1 - \tanh^2$	RNN kapıları; eski gizli katmanlar
ReLU	$\max(0, z)$	$[0, \infty)$	1 ($z > 0$), 0 ($z < 0$)	Gizli katmanlar (varsayılan)
Leaky ReLU	$\max(0.01z, z)$	$(-\infty, \infty)$	1 veya 0.01	Ölü nöron sorununa
Softmax	$e^{z_i} / \sum e^{z_j}$	olasılık dağılımı	—	Çok sınıflı çıktı katmanı

Tablo 9.3 Aktivasyon fonksiyonları

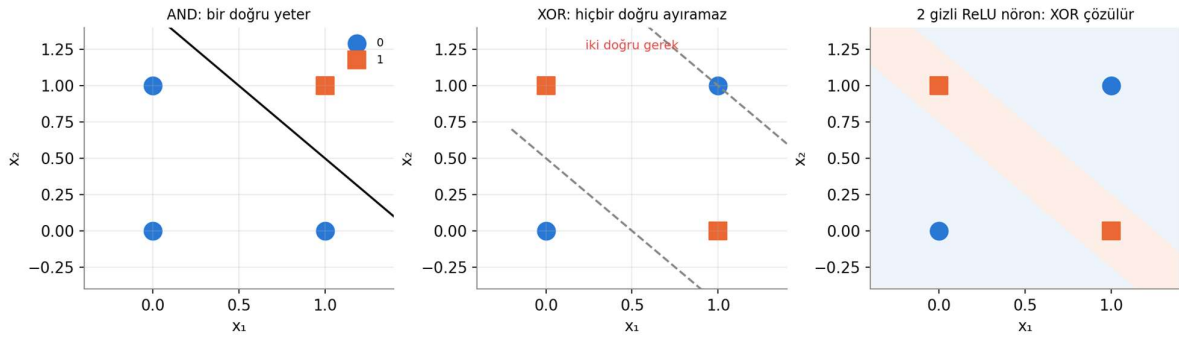
Örnek 9.2 — Softmax elle

Soru: Üç sınıflı bir ağı çıktısı skorları $z = [2.0, 1.0, -1.0]$. Softmax olasılıklarını hesaplayın.

$e^2 = 7.389$, $e^1 = 2.718$, $e^{-1} = 0.368$; toplam 10.475. $p = [0.705, 0.259, 0.035]$. Toplam 1; en büyük skor en büyük olasılığı alır ama diğerleri sıfır olmaz. Skorlara sabit eklemek sonucu değiştirmez ($z - \max(z)$ ile hesaplamak sayısal taşmayı önler). İkili durumda softmax sigmoide indirgenir.

9.5 XOR ve Gizli Katmanın Gücü

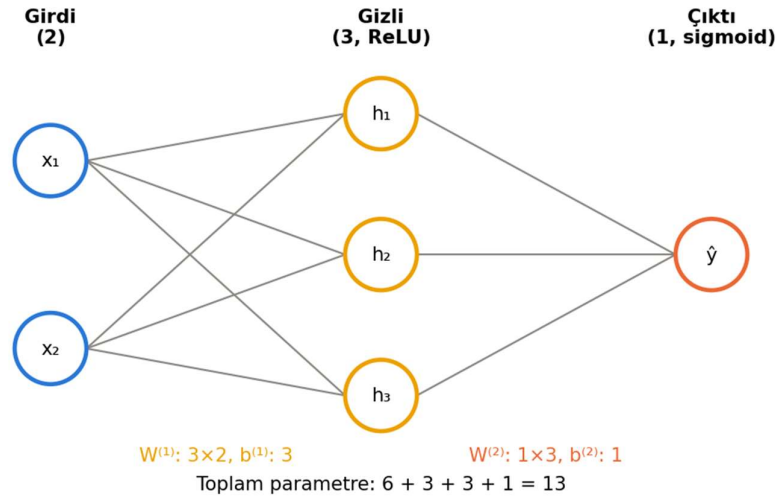
1969'da Minsky ve Papert'in gösterdiği gibi perceptron XOR'u öğrenemez: $(0,0) \rightarrow 0$, $(1,1) \rightarrow 0$ ama $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$ noktalarını tek bir doğru ayıramaz. Çözüm, araya bir 'gizli' katman koymaktır. İki gizli ReLU nöron düşünün: $h_1 = \text{ReLU}(x_1 + x_2 - 0.5)$ ve $h_2 = \text{ReLU}(x_1 + x_2 - 1)$. Girdiler $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$ için $h_1 = 0, 0.5, 0.5, 1.5$ ve $h_2 = 0, 0, 0, 1$. Çıktı $\hat{y} = 1[h_1 - 2h_2 > 0.25]$: $0 \rightarrow 0 \checkmark$, $0.5 \rightarrow 1 \checkmark$, $0.5 \rightarrow 1 \checkmark$, $1.5 - 2 = -0.5 \rightarrow 0 \checkmark$. İkinci nöron yalnızca 'her ikisi de 1' durumunda açılır ve birincinin katkısını bastırır. Gizli katman girdi uzayını yeni bir uzaya (h_1, h_2) haritalar ve orada problem doğrusal ayrılabilir olur. Bu, sinir ağlarının temel fikridir: gizli katmanlar özellik dönüşümünü (5. haftada elle yaptığımız klima = $\max(T-24, 0)$ gibi!) kendileri öğrenir.



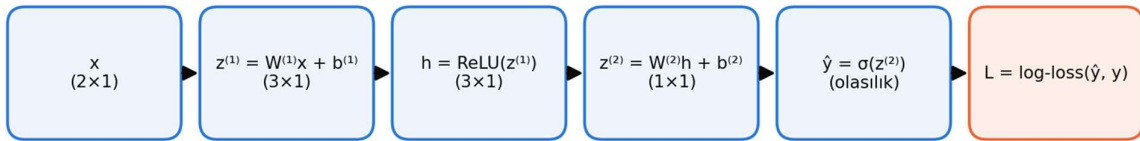
Şekil 9.3 AND bir doğruyla ayrılır; XOR ayrılamaz; iki gizli ReLU nöron XOR'u çözer

9.6 Çok Katmanlı Ağ ve İleri Yayılım

Çok katmanlı algılayıcı (MLP), tam bağlı katmanların ardışık dizilişidir: her katman önceki katmanın tüm çıktılarını alır, ağırlıklı toplam + aktivasyon uygular. Katman l için $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ ve $a^{(l)} = f(z^{(l)})$; ilk katmanın girdisi $a^{(0)} = x$, son katmanın çıktısı $\hat{y}$. $W^{(l)}$ matrisinin boyutu (bu katmandaki nöron sayısı $\times$ önceki katmandaki nöron sayısı). Girdiden çıktıya bu hesaba ileri yayılım denir; tahmin böyle yapılır, eğitim ise 10. haftadaki geri yayılımla. Parametre sayısı hızla büyür: 2-3-1 ağda 13, 784-128-10 ağda (MNIST) 101 770 parametre vardır.



Şekil 9.4 2-3-1 ağ: 13 parametre



Şekil 9.5 İleri yayılım: matris çarpımı, aktivasyon, tekrar

Örnek 9.3 — 2-3-1 ağda ileri yayılım (elle)

Soru: $W^{(1)} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}$, $b^{(1)} = [0, -1, -1]$; gizli aktivasyon ReLU. $W^{(2)} = [1, -2, 1]$, $b^{(2)} = 0.5$; çıktı sigmoid. Girdi $x = (1, 0.5)$ için $\hat{y}$? Gerçek $y = 1$ ise log-loss?

$z^{(1)} = W^{(1)}x + b^{(1)} = [1 \cdot 1 + (-1) \cdot 0.5 + 0, 0 \cdot 1 + 2 \cdot 0.5 - 1, 1 \cdot 1 + 1 \cdot 0.5 - 1] = [0.5, 0, 0.5]$. $h = \text{ReLU}(z^{(1)}) = [0.5, 0, 0.5]$. $z^{(2)} = 1 \cdot 0.5 + (-2) \cdot 0 + 1 \cdot 0.5 + 0.5 = 1.5$. $\hat{y} = \sigma(1.5) = 1/(1 + e^{-1.5}) = 1/(1 + 0.223) = 0.818$. Log-loss = $-\log(0.818) = 0.201$. İkinci gizli nöron bu girdi için 'kapalı' (ReLU sıfır): gradyan ona akmayacak — 10. haftada göreceğiz.

```
W1 = np.array([[1, -1], [0, 2], [1, 1]]); b1 = np.array([0, -1, -1.]); W2 = np.array([1, -2, 1.]);
b2 = 0.5
x = np.array([1, 0.5]); relu = lambda z: np.maximum(z, 0); sig = lambda z: 1/(1+np.exp(-z))
h = relu(W1 @ x + b1); yp = sig(W2 @ h + b2); print(h, round(yp, 3), "log-loss:", round(-np.log(yp),
3))
```

Örnek 9.4 — Parametre sayısı

Soru: Girdi 10 özellik, gizli katmanlar 64 ve 32 nöron, çıktı 4 sınıf (softmax). Kaç parametre? Aynı ağda gizli katman aktivasyonu olmasa ne olurdu?

Katman 1: 64×10 ağırlık + 64 sapma = 704. Katman 2: $32 \times 64 + 32 = 2080$. Katman 3: $4 \times 32 + 4 = 132$. Toplam 2916. Karşılaştırma: 10 özellikli lojistik regresyon 4 sınıf için 44 parametre. Aktivasyon olmasaydı: $\hat{y} = W_3(W_2(W_1x + b_1) + b_2) + b_3 = (W_3W_2W_1)x + \text{sabit}$ — tek bir 4×10 doğrusal dönüşüm; 2916 parametre 44'ün yaptığından fazlasını yapamazdı. Aktivasyon, parametrelerin 'işe yaramasını' sağlar.

9.7 Evrensel Yaklaşım ve Mimari Seçimi

Evrensel yaklaşım teoremi: tek gizli katmanlı, yeterince geniş bir ağ, herhangi bir sürekli fonksiyonu istenen hassasiyetle yaklaşabilir. ReLU ile sezgisi basittir: her gizli nöron bir 'kırılma noktası' ekler, ağ parçalı doğrusal bir fonksiyon üretir; nöron sayısı arttıkça kırılma noktaları sıklaşır ve eğri daha iyi izlenir. Teorem 'öğrenilebilir' değil 'var' der: doğru ağırlıkları bulmak gradyan inişinin işidir ve genişlik yerine derinlik çoğu zaman aynı işi çok daha az parametreyle yapar — derin katmanlar hiyerarşik özellikler kurar (görüntüde kenar → doku → parça → nesne). Pratik kural: 2-3 gizli katman, 16-256 nöronla başlayın; 4. haftanın öğrenme eğrileriyle büyütün ya da küçültün; düzenleştirmeyi (11. hafta) unutmayın.



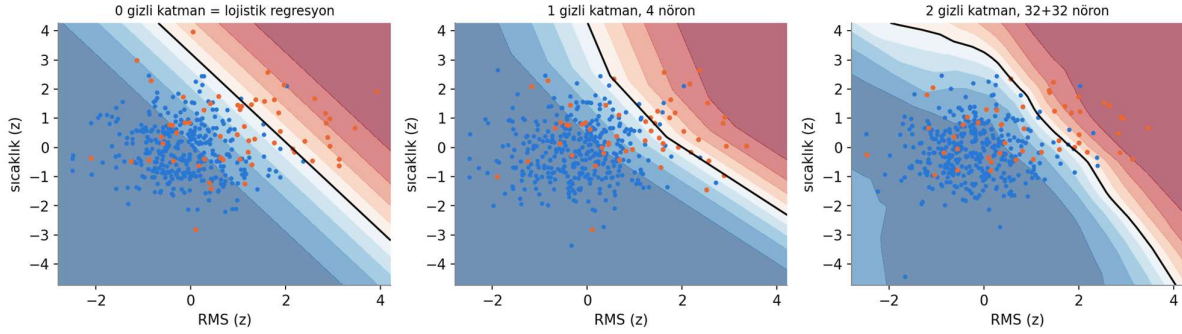
Şekil 9.6 ReLU ağı parçalı doğrusal: nöron sayısı arttıkça eğriyi daha iyi izler

Siğ ve geniş
1 gizli katman, çok nöron
→ her şeyi öğrenebilir (teorik)
ama verimsiz

Derin
çok katman, az nöron
→ hiyerarşik özellikler
(kenar → şekil → nesne)

Pratik kural
2-3 gizli katman, 16-256 nöron
ile başla; doğrulama
hatasına göre büyüt/küçült

Şekil 9.7 Siğ-geniş, derin ve pratik kural



Şekil 9.8 Motor verisinde karar sınırları: gizli katman yokken doğrusal, 4 nöronla eğri, 32+32 ile daha esnek

Sorun	Ne kadar gizli katman/nöron?	Çıktı katmanı	Kayıp
İkili sınıflandırma	1-2 katman, 16-64	1 nöron, sigmoid	Log-loss (BCE)
Çok sınıflı (K)	1-2 katman, 32-128	K nöron, softmax	Çapraz entropi
Regresyon	1-2 katman, 16-64	1 nöron, doğrusal (aktivasyon yok)	MSE / MAE
Görüntü	CNN (12. hafta)	softmax	Çapraz entropi
Zaman serisi	RNN/LSTM veya 1-B CNN (13. hafta)	doğrusal	MSE

Tablo 9.4 Problem türüne göre mimari

9.8 Uygulama

Bu haftanın defterinde ağı kendimiz kurarız: NumPy ile 2-3-1 ağı ileri yayılımı (Örnek 9.3), perceptron kuralı ile AND/OR ve XOR'un başarısızlığı, elle ayarlanmış gizli katmanla XOR çözümü, sonra scikit-learn MLPClassifier ile motor arıza verisinde gizli katman boyutunun etkisi. Geri yayılım henüz yok; sklearn bunu içeride yapar ve gelecek hafta kendimiz yazacağız.

```
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

for gizli in ((), (4,), (16,), (32, 32)):
    if gizli:
        model = make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=gizli,
max_iter=2000, random_state=0))
    else:
        model = make_pipeline(StandardScaler(), LogisticRegression())
    model.fit(Xtr, ytr); p = model.predict_proba(Xte)[: , 1]
    print(gizli, "AUC:", round(roc_auc_score(yte, p), 3))
```

9.9 Sınıf İçi Sorular

1. Tek nöron + sigmoid ile lojistik regresyon arasındaki fark nedir? (Cevap: yoktur.)
2. Neden aktivasyonsuz 10 katmanlı ağ, 1 katmanlı doğrusal modele eşdeğerdir? Matris çarpımıyla gösterin.
3. Sigmoid gizli katmanlı 20 katmanlı bir ağda ilk katmanın gradyanı neden çok küçük olur? (Türev ≤ 0.25 , 20 kez çarpılır.)
4. 3 girdi, 5 gizli, 2 çıktı ağı: kaç parametre? Girdi 1000 olsaydı?
5. OR kapısı için perceptron ağırlıklarını kafadan yazın ($w = (1,1)$, $b = -0.5$ çalışır mı?).

9.10 Uygulama Defteri ve Sonraki Hafta

- Defter: perceptron eğitimi (AND/OR/XOR), NumPy ileri yayılım, XOR'u elle çözmek, MLPClassifier ile mimari karşılaştırması, evrensel yaklaşım deneyi.
- Ödev yok bu hafta; Ödev 5 gelecek hafta (geri yayılım + PyTorch).
- Gelecek hafta: zincir kuralı, geri yayılım (elle 2-nöronlu örnek), PyTorch tensörleri ve autograd, ilk eğitim döngüsü.

Hafta 10 — Geri Yayılım ve PyTorch'a Giriş

Bölüm	Süre	İçerik
Saat 1	50 dk	Zincir kuralı ve hesap grafi; tek nöronda gradyan (elle); iki katmanlı ağda geri yayılım (elle); ağırlık güncelleme
Saat 2	50 dk	Öğrenme oranı, mini-batch, epoch; SGD ve Adam; PyTorch: tensör, autograd, nn.Module, optimizer, eğitim döngüsü
Saat 3	50 dk	Uygulama: NumPy ile geri yayılımı yazmak ve autograd ile doğrulamak; PyTorch ile yük tahmini ağı

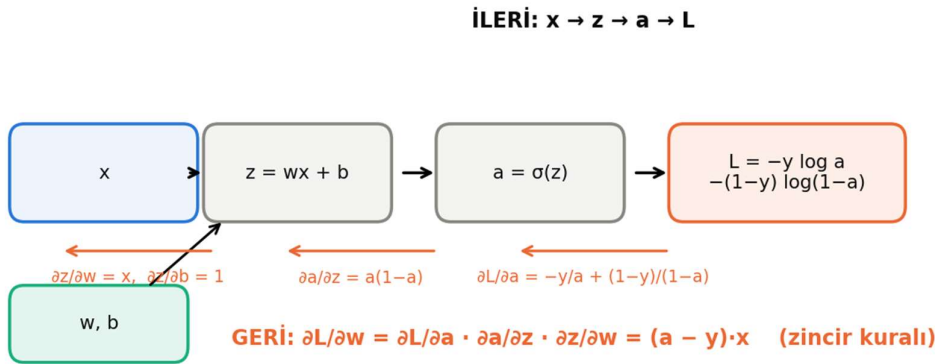
Tablo 10.1 Haftanın ders planı

10.1 Kazanımlar

- Zincir kuralını hesap grafi üzerinde uygular; tek nöron ve iki katmanlı ağ için gradyanları elle türetir.
- Geri yayılımın 'hata sinyali geriye taşıma' fikrini ve her katmandaki formülü açıklar.
- Öğrenme oranı, batch büyüklüğü ve epoch kavramlarını tanımlar; SGD ile Adam'ı karşılaştırır.
- PyTorch tensörleri, autograd, nn.Module ve optimizer ile bir eğitim döngüsü yazar.
- NumPy ile yazdığı gradyanı autograd ile doğrular; kaybolan gradyanı gözlemler.

10.2 Zincir Kuralı ve Hesap Grafi

5. haftada doğrusal regresyonun gradyanını elle türettik; ağda onlarca katman ve binlerce parametre varken bunu nasıl yapacağız? Cevap, bileşik fonksiyonların türevi için zincir kuralıdır: $L = f(g(w))$ ise $dL/dw = f'(g(w)) \cdot g'(w)$. Bir sinir ağı, iç içe geçmiş basit işlemlerin (çarpma, toplama, aktivasyon) uzun bir zinciridir; her işlemin türevi basittir ve zincir kuralı bunları çarparak birleştirir. Bu hesabı düzenlemenin yolu hesap grafidir: düğümler işlemler, oklar veri akışı. İleri geçişte her düğümün çıktısı hesaplanıp saklanır; geri geçişte kayıptan başlayarak her okun üzerinden yerel türev ile 'yukarıdan gelen' gradyan çarpılarak geriye iletilir. Bu düzene geri yayılım (backpropagation) denir; 1986'da Rumelhart, Hinton ve Williams ile yaygınlaşmıştır ve tüm derin öğrenmenin motorudur.



Şekil 10.1 Tek nöron için hesap grafi: ileri geçiş üstte, zincir kuralıyla geri geçiş altta

Örnek 10.1 — Tek nöronun gradyanı (elle)

Soru: Nöron $a = \sigma(wx + b)$, kayıp $L = -[y \log a + (1-y) \log(1-a)]$. $x = 2$, $w = 0.5$, $b = -0.5$, $y = 1$ için $\partial L/\partial w$ ve $\partial L/\partial b$ 'yi bulun; $\eta = 0.1$ ile bir güncelleme yapın.

İleri: $z = 0.5 \cdot 2 - 0.5 = 0.5$; $a = \sigma(0.5) = 0.622$; $L = -\log 0.622 = 0.474$.

Geri, zincir kuralı: $\partial L/\partial a = -y/a + (1-y)/(1-a) = -1/0.622 = -1.607$. $\partial a/\partial z = a(1-a) = 0.622 \times 0.378 = 0.235$.
 $\partial L/\partial z = (-1.607)(0.235) = -0.378$ — kontrol: $(a - y) = 0.622 - 1 = -0.378$ ✓ (sigmoid + log-loss birleşimi her zaman $a - y$ verir; bu yüzden ikili çıktıda bu çift kullanılır). $\partial z/\partial w = x = 2 \rightarrow \partial L/\partial w = -0.378 \times 2 = -0.756$.
 $\partial z/\partial b = 1 \rightarrow \partial L/\partial b = -0.378$.

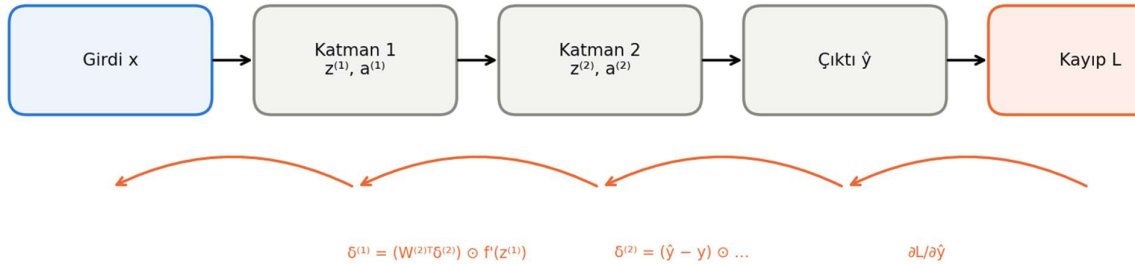
Güncelleme: $w = 0.5 - 0.1(-0.756) = 0.576$; $b = -0.5 - 0.1(-0.378) = -0.462$. Yeni ileri: $z = 0.69$, $a = 0.666$, $L = 0.406 < 0.474$ ✓. Gradyan negatif çıktı çünkü $y = 1$ iken a 'yı büyötmek gerekiyordu; güncelleme z 'yi artırdı.

```
sig = lambda z: 1/(1+np.exp(-z))
x, w, b, y, eta = 2.0, 0.5, -0.5, 1, 0.1
z = w*x + b; a = sig(z); L = -(y*np.log(a) + (1-y)*np.log(1-a))
dz = a - y; dw, db = dz*x, dz
print(f"a={a:.3f} L={L:.3f} dL/dw={dw:.3f} dL/db={db:.3f}")
w, b = w - eta*dw, b - eta*db; a2 = sig(w*x + b); print(f"yeni w={w:.3f} b={b:.3f} a={a2:.3f} L={-np.log(a2):.3f}")
```

10.3 İki Katmanlı Ağda Geri Yayılım

Katmanlar arttığında aynı fikir tekrarlanır. Katman l 'de $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$, $a^{(l)} = f(z^{(l)})$ olsun. 'Hata sinyali' $\delta^{(l)} = \partial L/\partial z^{(l)}$ tanımlansın. Çıktı katmanında (sigmoid + log-loss ya da softmax + çapraz entropi için) $\delta^{(l)} = \hat{y} - y$. Bir önceki katman için zincir kuralı $\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot f'(z^{(l)})$ verir: sonraki katmanın hatası, ağırlıkların transpozuyula geriye taşınır ve aktivasyonun türeviyle eleman eleman ($\odot$) çarpılır. Parametre gradyanları $\partial L/\partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T$ ve $\partial L/\partial b^{(l)} = \delta^{(l)}$. Üç formöl, her derinlikteki ağ için yeter. Aktivasyon türevi ReLU'da 0/1 olduğundan kapalı nöronlar ($z < 0$) hata sinyalini geçirmez; sigmoidde türev ≤ 0.25 olduğundan sinyal her katmanda en az dörtte birine iner — 9. haftadaki kaybolan gradyan tam olarak bu çarpımdır.

İleri yayılım: aktivasyonları hesapla ve SAKLA



Geri yayılım: hata sinyalini (δ) çıktıdan girdiye doğru katman katman taşı; her katmanda $\partial L/\partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T$

Şekil 10.2 Geri yayılım: δ çıktıdan girdiye katman katman taşınır

Örnek 10.2 — 2-2-1 ağda geri yayılım (elle)

Soru: Girdi $x = (1, 2)$. Gizli katman (ReLU): $W^{(1)} = \begin{bmatrix} 0.5 & -1 \\ 1 & 0.5 \end{bmatrix}$, $b^{(1)} = (0, -1)$. Çıktı (sigmoid): $W^{(2)} = \begin{bmatrix} 1 & -1 \end{bmatrix}$, $b^{(2)} = 0$. Gerçek $y = 0$. Tüm gradyanları bulun.

İleri: $z^{(1)} = [0.5 \cdot 1 + (-1) \cdot 2 + 0, 1 \cdot 1 + 0.5 \cdot 2 - 1] = [-1.5, 1.0]$; $a^{(1)} = \text{ReLU} = [0, 1.0]$. $z^{(2)} = 1 \cdot 0 + (-1) \cdot 1 + 0 = -1$; $\hat{y} = \sigma(-1) = 0.269$. $L = -\log(1 - 0.269) = 0.313$.

Geri: $\delta^{(2)} = \hat{y} - y = 0.269$. $\partial L / \partial W^{(2)} = \delta^{(2)} a^{(1)T} = 0.269 \times [0, 1.0] = [0, 0.269]$; $\partial L / \partial b^{(2)} = 0.269$. Gizli katman: $W^{(2)T} \delta^{(2)} = [1, -1] \times 0.269 = [0.269, -0.269]$; $f'(z^{(1)}) = [0, 1]$ (ilk nöron kapalı) $\rightarrow \delta^{(1)} = [0, -0.269]$. $\partial L / \partial W^{(1)} = \delta^{(1)} x^T = [[0, 0], [-0.269, -0.538]]$; $\partial L / \partial b^{(1)} = [0, -0.269]$.

Yorum: İlk gizli nöron kapalı olduğundan onun ağırlıkları hiç güncellenmez (ölü nöron etkisi). İkinci nöronun gradyanı negatif: $y = 0$ iken $\hat{y}$ 'yi düşürmek gerek; $W^{(2)}_2 = -1$ olduğundan a_2 'yi büyütme $\hat{y}$ 'yi düşürür, dolayısıyla z_2 'yi büyütecek yönde (gradyanın tersi) güncelleme yapılır. Defterde aynı hesap NumPy ile ve autograd ile doğrulanır.

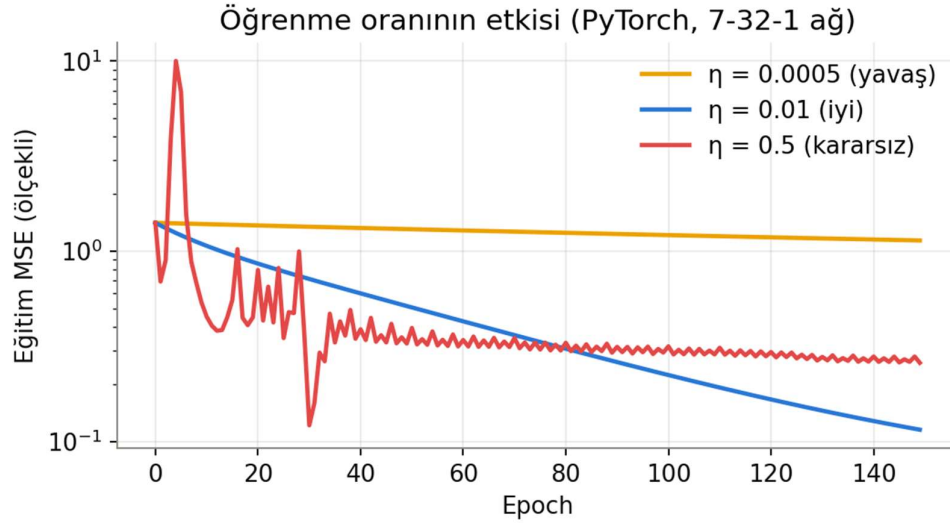
```
relu = lambda z: np.maximum(z, 0)
x = np.array([1., 2.]); W1 = np.array([[.5, -1], [1, .5]]); b1 = np.array([0., -1]); W2 = np.array([1., -1]); b2 = 0.; y = 0
z1 = W1 @ x + b1; a1 = relu(z1); z2 = W2 @ a1 + b2; yp = sig(z2); L = -np.log(1 - yp)
d2 = yp - y; dw2 = d2 * a1; db2 = d2
d1 = (W2 * d2) * (z1 > 0); dw1 = np.outer(d1, x); db1 = d1
print(f"ŷ={yp:.3f} L={L:.3f}"); print("dw2", dw2.round(3), "db2", round(db2, 3)); print("dw1", dw1.round(3)); print("db1", db1.round(3))
```

10.4 Eğitim Döngüsü: Öğrenme Oranı, Batch, Epoch, Optimizasyon

Gradyan hazır olunca öğrenme, 5. haftadaki gradyan inişidir; ama birkaç pratik kavram eklenir. Tüm veriyle her adımda gradyan hesaplamak (batch GD) büyük veride pahalıdır; tek örnekle hesaplamak (stokastik GD) gürültülüdür. Orta yol mini-batch'tir: 32-256 örneklik rastgele parçalarla adım atılır; gradyan gürültülü ama ucuzdur ve gürültü yerel minimumlardan kaçmaya bile yardım eder. Bir epoch, tüm eğitim verisinden bir geçiştir; 10-100 epoch tipiktir. Her epoch başında veri karıştırılır. Öğrenme oranı η en önemli hiperparametredir: küçükse yavaş, büyükse ıraksama; genellikle 10^{-3} civarı başlanır ve zamanla azaltılır (schedule).



Şekil 10.3 Bir eğitim adımı: mini-batch $\rightarrow$ ileri $\rightarrow$ kayıp $\rightarrow$ geri $\rightarrow$ güncelle $\rightarrow$ sıfırla

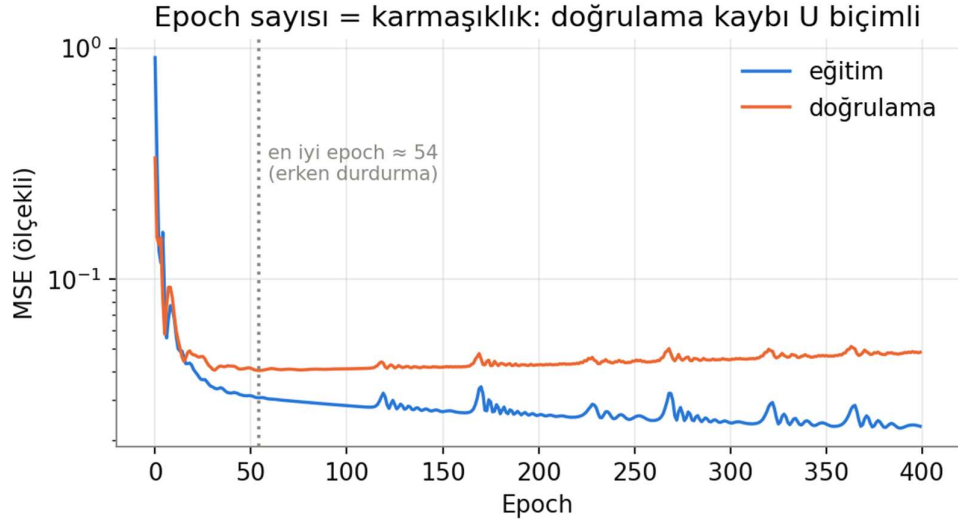


Şekil 10.4 Öğrenme oranının kayıp eğrisine etkisi (dersin verisi, 7-32-1 ağı)

Sade SGD'nin iki iyileştirmesi yaygındır. Momentum, önceki adımların yönünü hatırlayarak zikzağı azaltır ve dar vadilerde hızlanır (bir topun yokuş aşağı ivmelenmesi gibi). Adam, her parametre için ayrı bir uyarlanırlar öğrenme oranı tutar (gradyanın birinci ve ikinci momentlerinin hareketli ortalamaları) ve pratikte 'ilk denemede çalışan' seçimdir: $\text{lr} = 10^{-3}$ ile başlayın. Çok büyük veri ve dikkatli ayar imkânı varsa momentumlu SGD bazen daha iyi genelleştirir.

Kavram	Tanım	Tipik değer	Etkisi
Öğrenme oranı η	Adım büyüklüğü	$10^{-4} - 10^{-2}$	Küçük: yavaş; büyük: ıraksar; en kritik hiperparametre
Batch büyüklüğü	Bir adımda kullanılan örnek	32 – 256	Küçük: gürültülü, düzenleştirici; büyük: hızlı, bellek
Epoch	Tüm veriden bir geçiş	10 – 100	Fazla: aşırı öğrenme → erken durdurma
SGD + momentum	Önceki yönü hatırla	momentum 0.9	Zikzak azalır, hızlanır
Adam	Parametre başına uyarlanırlar η	$\text{lr} 10^{-3}$, $\beta = (0.9, 0.999)$	Ayarsız iyi çalışır; varsayılan seçim
Ağırlık başlangıcı	Rastgele, ölçekli (He/Xavier)	$\sqrt{2/n_{\text{in}}}$	Aktivasyonların patlamasını/sönmesini önler

Tablo 10.2 Eğitim hiperparametreleri



Şekil 10.5 Epoch sayısı bir karmaşıklık eksenidir: doğrulama kaybı dipten sonra artar → erken durdurma

Örnek 10.3 — Adım sayısı hesabı

Soru: 57 600 eğitim örneği, batch 64, 30 epoch. Kaç gradyan adımı atılır? Her adım 5 ms sürerse eğitim ne kadar sürer? Batch 256 olsaydı?

Adım/epoch = $57\,600 / 64 = 900$; toplam $900 \times 30 = 27\,000$ adım; süre $27\,000 \times 5\text{ ms} = 135\text{ s}$. Batch 256: 225 adım/epoch, 6 750 adım; adım süresi 4 kat artmaz (GPU paralel) — diyelim 8 ms → 54 s. Büyük batch daha hızlıdır ama adım sayısı azaldığından aynı sonuca ulaşmak için daha çok epoch ya da daha büyük η gerekebilir; küçük batch'in gradyan gürültüsü de bir tür düzenleştirmedir.

10.5 PyTorch

Tüm bunları elle yazmak öğreticidir ama pratik değildir. PyTorch, (1) NumPy'a benzeyen ama GPU'da çalışabilen tensörler, (2) her işlemi kaydedip `backward()` çağrısıyla tüm gradyanları otomatik hesaplayan autograd, (3) katmanları ve modelleri düzenleyen `nn.Module`, (4) SGD/Adam gibi güncelleme kurallarını uygulayan `torch.optim` ve (5) mini-batch'leri üreten `DataLoader` sunar. Öğrenmenin özü altı satırdır: `zero_grad` → `ileri` → `kayıp` → `backward` → `step` → `tekrar`. Bu döngüyü bir kez yazan, her derin öğrenme modelini yazabilir; 11-13. haftalarda yalnızca model tanımı değişecek.

PyTorch'un beş yapı taşı

torch.Tensor	autograd	nn.Module	torch.optim	DataLoader
NumPy dizisi + GPU + gradyan takibi	<code>loss.backward()</code> tüm türevleri otomatik	Katmanlar, model <code>nn.Linear</code> , <code>nn.ReLU</code>	SGD, Adam <code>step()</code> , <code>zero_grad()</code>	Mini-batch, karıştırma

Şekil 10.6 PyTorch'un beş yapı taşı

Autograd: türevi biz değil, PyTorch alır

```
import torch, torch.nn as nn
# 1) Tensörler ve autograd: Örnek 10.1'i PyTorch'a yaptıralım
w = torch.tensor(0.5, requires_grad=True); b = torch.tensor(-0.5, requires_grad=True)
x = torch.tensor(2.0); y = torch.tensor(1.0)
a = torch.sigmoid(w*x + b)
L = -(y*torch.log(a) + (1-y)*torch.log(1-a))
L.backward() # zincir kuralı otomatik
print(w.grad, b.grad) # tensor(-0.7560) tensor(-0.3780) -> elle
bulduğumuzla aynı
```

Altı satırlık eğitim döngüsü

```
# 2) Model, kayıp, optimizer, eğitim döngüsü – yük tahmini (regresyon)
X_tr = torch.tensor(Xs[tr], dtype=torch.float32); y_tr = torch.tensor(ys[tr],
dtype=torch.float32)[: , None]
model = nn.Sequential(nn.Linear(7, 32), nn.ReLU(), nn.Linear(32, 1)) # 7 özellik -> 32 ReLU ->
1 çıktı
kayip = nn.MSELoss(); opt = torch.optim.Adam(model.parameters(), lr=1e-2)
for epoch in range(200):
    opt.zero_grad() # eski gradyanları sil (birikir!)
    tahmin = model(X_tr) # ileri yayılım
    L = kayip(tahmin, y_tr) # kayıp
    L.backward() # geri yayılım: tüm .grad'lar dolar
    opt.step() # parametreleri güncelle
    if epoch % 50 == 0: print(epoch, L.item())
```

Mini-batch

```
# 3) Mini-batch ile DataLoader
from torch.utils.data import TensorDataset, DataLoader
yukleyici = DataLoader(TensorDataset(X_tr, y_tr), batch_size=64, shuffle=True)
for epoch in range(30):
    for xb, yb in yukleyici: # her epoch'ta 575/64 ≈ 9 adım
        opt.zero_grad(); L = kayip(model(xb), yb); L.backward(); opt.step()
```

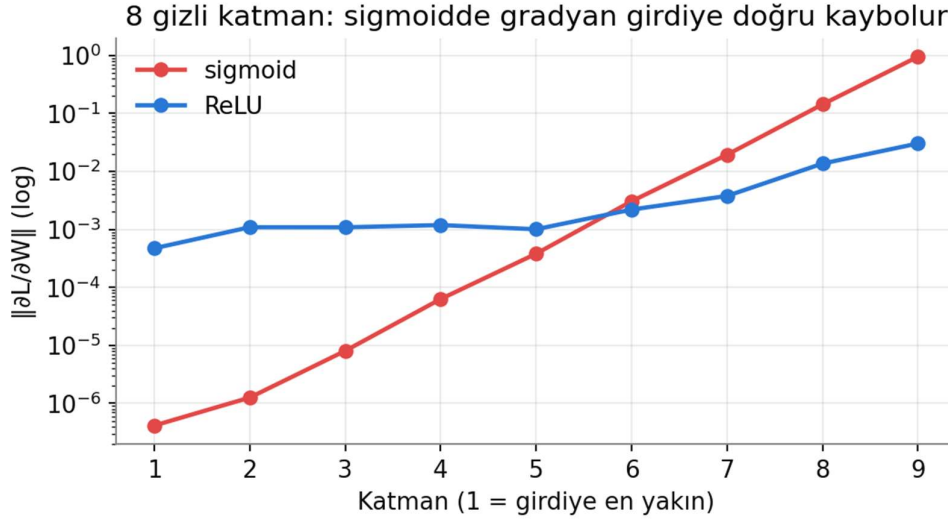
NumPy	PyTorch	Not
np.array([1, 2])	torch.tensor([1., 2.])	dtype float32 tercih edilir
a @ b	a @ b	Aynı
x.shape	x.shape	Aynı
np.exp(x)	torch.exp(x)	Çoğu fonksiyon aynı adla
—	x.requires_grad_(True)	Gradyan takibi aç
—	L.backward(); x.grad	Türevler
—	x.to('cuda')	GPU'ya taşı (Colab: Runtime → GPU)
x_np = x	x.detach().numpy()	Geri dönüş

Tablo 10.3 NumPy–PyTorch karşılağı

Üç klasik hata: ``zero_grad()`` unutulursa gradyanlar her adımda birikir ve eğitim saçmalar — en sık yapılan PyTorch hatası. İkinci sık hata: modeli ``eval()`` moduna almadan doğrulama yapmak (dropout/batchnorm varsa sonuç değişir, 11. hafta). Üçüncü: değerlendirmede ``torch.no_grad()`` kullanmamak (bellek şişer).

10.6 Kaybolan Gradyanı Görmek

9. haftada sigmoidli derin ağıın öğrenemediğini gözlemledik; şimdi nedenini ölçebiliriz: geri yayılımdan sonra her katmanın ağırlık gradyanının normuna bakın. Sigmoidde norm girdiye doğru katman başına yaklaşık dörtte birine iner; sekiz katmanda 10^{-4} - 10^{-5} mertebesine düşer ve ilk katmanlar fiilen öğrenmez. ReLU'da türev 1 olduğundan norm katmanlar boyunca aynı mertebede kalır. Bu grafik, 'neden ReLU' sorusunun sayısal cevabıdır; batch normalization ve artık bağlantılar (ResNet) aynı sorunun diğer çözümleridir (11-12. hafta).



Şekil 10.7 8 gizli katmanlı ağda katman başına gradyan normu

10.7 Sınıf İçi Sorular

1. $L = (a - y)^2$, $a = \sigma(z)$, $z = wx + b$ için $\partial L/\partial w$ 'yi zincir kuralıyla yazın. Sigmoid + MSE neden sigmoid + log-loss'tan kötüdür? (ipucu: $a(1-a)$ çarpanı uygunlukta sıfırlanır)
2. ReLU nöronu bir örnek için kapalıysa ($z < 0$) o örnekten ağırlıklarına gradyan gelir mi? Tüm örnekler için kapalıysa ne olur?
3. 1000 örnek, batch 100, 20 epoch: kaç adım? Batch 1 olsaydı?
4. Öğrenme oranını 10 kat artırınca kayıp NaN oldu. Ne oldu, ne yaparsınız?
5. ``opt.zero_grad()`` satırını silerseniz ne olur? Bir adım için gradyanın değerini tahmin edin.

10.8 Ödev 5 — Geri Yayılım ve PyTorch

- NumPy ile 2-8-1 ağ (ReLU + sigmoid) için ileri ve geri yayılımı fonksiyon olarak yazın; motor arıza verisinde (5 özellik, ölçekli) 500 epoch tam-batch gradyan inisiyle eğitin; kayıp ve test AUC eğrilerini çizin.
- Aynı ağı PyTorch ile kurun; bir mini-batch için NumPy gradyanlarınızla ``grad`` değerlerinin eşleştiğini ($1e-5$ tolerans) gösterin.
- PyTorch ile $\eta \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ ve optimizier $\in \{\text{SGD}, \text{SGD+momentum}, \text{Adam}\}$ için 4×3 kayıp eğrisi ızgarası çizin; hangi kombinasyon en hızlı ve kararlı?
- Batch büyüklüğü 8, 64, 512 için epoch başına süreyi ve 30 epoch sonunda doğrulama kaybını karşılaştırın.
- Rapor: yarım sayfa; eğitim/doğrulama eğrisinden erken durdurma epoch'unu belirleyin ve gerekçelendirin.

Hafta 11 — Derin Öğrenme Pratiği

Bölüm	Süre	İçerik
Saat 1	50 dk	Çok sınıflı sınıflandırma: softmax + çapraz entropi (elle); MNIST; MLP mimarisi; eğitim/doğrulama/test protokolü
Saat 2	50 dk	Aşırı öğrenme ile mücadele: erken durdurma, weight decay, dropout, veri artırma; başlangıç ve batch normalization; öğrenme eğrilerini okumak; hiperparametre stratejisi
Saat 3	50 dk	Uygulama: MNIST'te MLP eğitimi, düzenleme deneyleri, hata analizi (karışıklık matrisi, yanlış örnekler)

Tablo 11.1 Haftanın ders planı

11.1 Kazanımlar

- Softmax ve çapraz entropiyi elle hesaplar; çok sınıflı çıktı katmanını kurar.
- Görüntüyü vektöre düzleştirip MLP ile sınıflandırır; parametre sayısını hesaplar.
- Eğitim/doğrulama eğrilerinden aşırı öğrenmeyi teşhis eder; erken durdurmayı uygular.
- Weight decay ve dropout'un ne yaptığını açıklar ve etkisini deneyle gösterir.
- Ağırlık başlangıcının ve batch normalization'ın derin ağlarda neden gerektiğini bilir.
- Bir derin öğrenme deneyini sistematik yürütür: taban → büyüt → düzenle → hata analizi.

11.2 Çok Sınıflı Sınıflandırma: Softmax ve Çapraz Entropi

İkili sınıflandırmada tek bir sigmoid çıktı yeterliydi. K sınıf için çıktı katmanında K nöron (logit) bulunur ve softmax bunları toplamı 1 olan olasılıklara çevirir: $p_k = \exp(z_k) / \sum_j \exp(z_j)$. Kayıp, log-loss'un genellemesi olan çapraz entropidir: $L = -\log p_y$, yani yalnızca doğru sınıfa verilen olasılığın negatif logaritması. Gradyanı yine zarif çıkar: $\partial L / \partial z_k = p_k - 1[k = y]$; yani softmax + CE çifti, çıktı katmanında 'tahmin – gerçek (one-hot)' hata sinyali verir — 10. haftadaki üç formül aynen çalışır. PyTorch'ta `nn.CrossEntropyLoss` softmax'ı içerir; modelin son katmanı ham logit üretir, softmax ayrıca eklenmez (eklenirse çift softmax hatası).

Örnek 11.1 — Softmax + çapraz entropi ve gradyanı (elle)

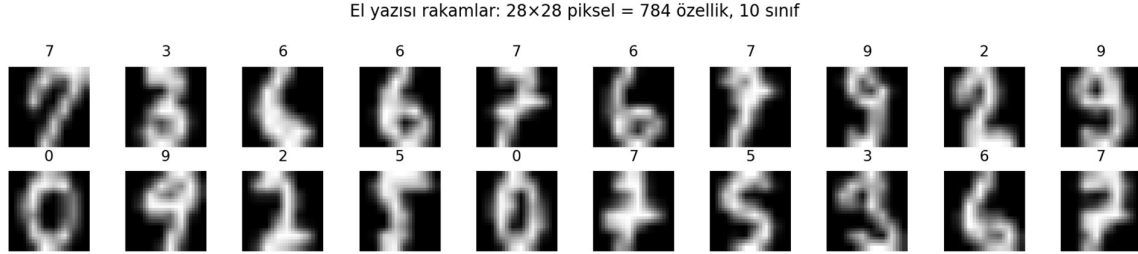
Soru: Üç sınıflı ağın logitleri $z = [1.0, 2.0, 0.5]$, gerçek sınıf $y = 1$ (ikinci). Olasılıkları, kaybı ve $\partial L / \partial z$ 'yi hesaplayın. Gerçek sınıf $y = 2$ olsaydı kayıp?

exp: [2.718, 7.389, 1.649], toplam 11.756 → $p = [0.231, 0.629, 0.140]$. Kayıp = $-\log 0.629 = 0.464$. Gradyan: $p - \text{onehot}(1) = [0.231, -0.371, 0.140]$: doğru sınıfın logiti artırılacak (negatif gradyan), diğerleri azaltılacak, toplamı sıfır. $y = 2$ olsaydı: $-\log 0.140 = 1.966$ — model o sınıfa %14 vermişti, çok cezalanır. Rastgele tahmin (her sınıfa 1/3) kaybı $\log 3 = 1.099$ 'dur; iyi bir modelin ortalama kaybı bunun çok altındadır (MNIST'te ≈ 0.05).

```
z = np.array([1.0, 2.0, 0.5]); p = np.exp(z)/np.exp(z).sum()
print(p.round(3), "kayıp(y=1):", round(-np.log(p[1]), 3), "grad:", (p - np.eye(3)[1]).round(3),
      "kayıp(y=2):", round(-np.log(p[2]), 3))
import torch, torch.nn.functional as F
zt = torch.tensor(z, requires_grad=True); L = F.cross_entropy(zt[None], torch.tensor([1]));
L.backward(); print(L.item(), zt.grad)
```

11.3 MNIST ve MLP

MNIST, derin öğrenmenin 'merhaba dünya'sıdır: 70 000 el yazısı rakam görüntüsü (28×28 gri ton, 0-255), 10 sınıf; 60 000 eğitim, 10 000 test. Bu hafta görüntüyü 784 elemanlı bir vektöre düzleştirip tam bağlı bir ağa (MLP) veriyoruz; 12. haftada aynı problemi görüntünün 2-B yapısını koruyan CNN ile çözeceğiz. Pikseller 0-1 aralığına ölçeklenir (255'e bölme) — 3. haftanın min-max ölçeklemesi. Eğitim kümesinden ayrılan bir doğrulama parçası (ör. %10) hiperparametre ve erken durdurma için kullanılır; test kümesi en sonda bir kez. Colab'da veri torchvision ile indirilir; defterde internet olmayan ortamlar için sklearn'ün 8×8 rakam verisini 28×28'e büyüten bir yedek vardır.



Şekil 11.1 Rakam örnekleri (yerelde yedek veri ile üretildi; Colab'da gerçek MNIST)



$$\text{Parametre: } 784 \cdot 256 + 256 + 256 \cdot 128 + 128 + 128 \cdot 10 + 10 = 235\,146$$

Şekil 11.2 MNIST için MLP: 784 → 256 → 128 → 10, yaklaşık 235 bin parametre

MLP tanımı

```

model = nn.Sequential(
    nn.Flatten(),                # (N, 28, 28) -> (N, 784)
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, 128), nn.ReLU(),
    nn.Linear(128, 10)          # logit; softmax CrossEntropyLoss'un içinde
)
print(sum(p.numel() for p in model.parameters())) # 235146
  
```

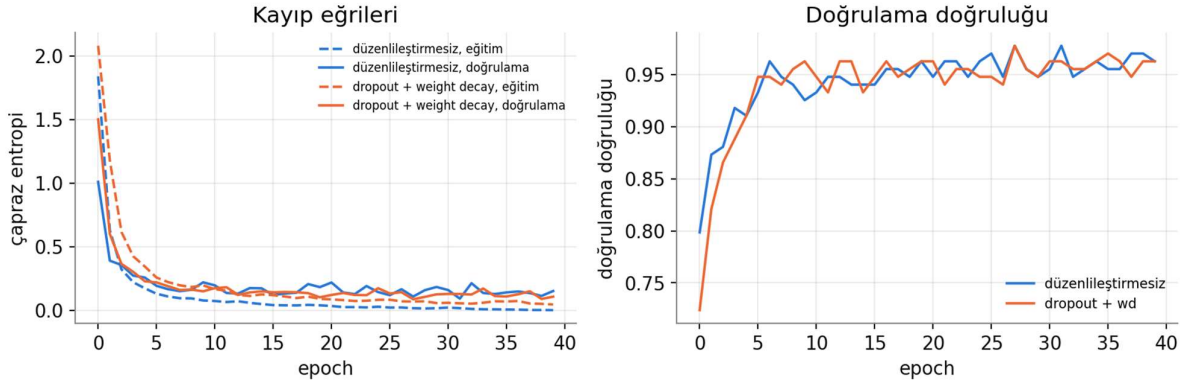
Örnek 11.2 — Kaç parametre, kaç örnek?

Soru: 784-256-128-10 ağı 235 146 parametreye sahip; MNIST eğitim kümesi 54 000 örnek (doğrulama ayrıldıktan sonra). Parametre/örnek oranı nedir? Bu oran aşırı öğrenme anlamına mı gelir?

Oran ≈ 4.4 parametre/örnek. Klasik istatistik bakışıyla (örnek $\gg$ parametre) bu kaygı vericidir; ama derin ağlarda örnek başına 784 sayı (piksel) bilgi vardır ve düzenleme (dropout, weight decay, erken durdurma), SGD'nin gürültüsü ve mimari kısıtlar pratikte aşırı öğrenmeyi frenler. Yine de eğitim doğruluğu %100'e giderken doğrulama %98'de kalır: fark aşırı öğrenmenin işareti ve düzenleme farkı kapatır. Ders mesajı: parametre sayısı değil, eğitim–doğrulama farkı ölçüttür.

11.4 Öğrenme Eğrilerini Okumak

Her epoch sonunda eğitim ve doğrulama kaybını (ve doğruluğunu) kaydedip çizmek, derin öğrenmede en önemli teşhis aracıdır; 4. haftanın tablosu burada aynen geçerlidir. Eğitim kaybı düşmüyorsa: öğrenme oranı yanlış (çok büyükse NaN/salınım, çok küçükse düz), gradyan kaybolmuş (sigmoid derin ağ), ya da veri/etiket hatası (ör. etiketler karışmış). Eğitim düşüyor ama doğrulama erken yükseliyorsa: aşırı öğrenme → erken durdur, düzenleştire, veri artır. İkisi de düşük ama doğrulama eğitimden düşükse: dropout eğitimde açık olduğu için normaldir. Kayıp gürültülüysse: batch küçük ya da η büyük.



Şekil 11.3 Düzenleştirmesiz büyük ağ: eğitim kaybı sıfıra iner, doğrulama yükselir. Dropout + weight decay farkı kapatır

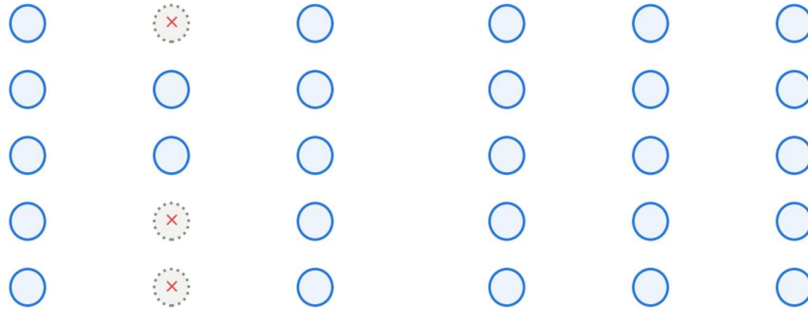
Belirti	Olası neden	Ne yapmalı?
Eğitim kaybı hiç düşmüyor / NaN	η çok büyük; ölçeklenmemiş girdi; hatalı kayıp/etiket	η 'yı 10 kat düşür; girdiyi ölçekle; tek batch'i ezberletmeyi dene (sağlık testi)
Eğitim kaybı çok yavaş düşüyor	η çok küçük; kaybolan gradyan; kötü başlangıç	η 'yı artır; ReLU + He başlangıcı; batch norm; Adam
Eğitim ↓, doğrulama önce ↓ sonra ↑	Aşırı öğrenme	Erken durdurma; dropout; weight decay; veri artırma; küçült
Eğitim ve doğrulama ikisi de yüksek	Eksik öğrenme	Daha büyük ağ; daha uzun eğitim; daha iyi özellik/mimari (CNN)
Doğrulama < eğitim kaybı	Dropout/veri artırma yalnızca eğitimde	Normal; endişelenme
Kayıp zikzaklı	Batch küçük, η büyük	Batch büyüt; η azalt; η schedule

Tablo 11.2 Öğrenme eğrisi teşhis rehberi

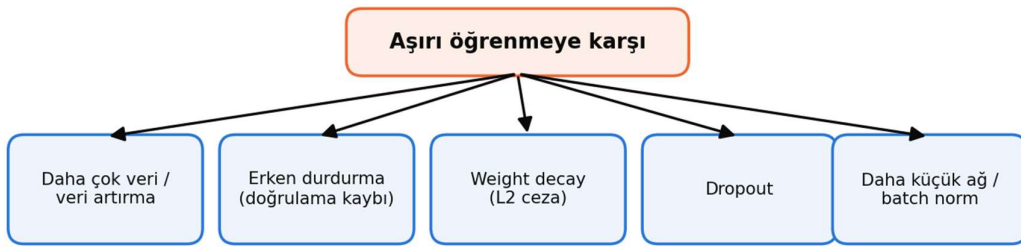
11.5 Düzenleştirme Teknikleri

Derin ağlar milyonlarca parametreyle eğitim verisini ezberleyebilir; düzenleştirme, ağı genelleyen çözümlere yönlendiren tüm tekniklerin ortak adıdır. Erken durdurma en basiti: doğrulama kaybı en düşükken ağırlıkları sakla, sonrası boşa geçen epoch. Weight decay (L2 cezası), 5. haftanın ridge'yle aynıdır: büyük ağırlıklar cezalandırılır, optimizer'a `weight_decay=1e-4` parametresiyle verilir. Dropout (Hinton 2012), eğitimde her adımda rastgele seçilen nöronların (tipik %20-50) çıktısını sıfırlar; ağ hiçbir nörona 'güvenemez', yedekli temsiller öğrenir; testte tüm nöronlar açıktır ve çıktılar ölçeklenir (PyTorch bunu otomatik yapar; `model.eval()` bu yüzden şart). Veri artırma (augmentation), eğitim görüntülerini rastgele döndürerek/kaydırarak/gürültüleyerek sanal olarak çoğaltır; 12. haftada CNN ile önem kazanır. Batch normalization ise hem eğitimi hızlandırır hem hafif düzenleştire.

Eğitimde: her adımda rastgele %50 nöron kapalı **Testte: hepsi açık, çıktılar ölçekli**



Şekil 11.4 Dropout: eğitimde rastgele nöronlar kapalı, testte hepsi açık



Hepsi aynı amaç: eğitim verisini ezberlemek yerine genelleyen çözümü tercih ettirmek

Şekil 11.5 Düzenleştirme araç kutusu

Teknik	Ne yapar?	PyTorch	Tipik değer
Erken durdurma	Doğrulama kaybı dipteyken dur	Elle: en iyi epoch'ta state_dict kaydet	sabır (patience) 5-10 epoch
Weight decay (L2)	Büyük ağırlıkları cezalandır	optim.Adam(..., weight_decay=1e-4)	1e-5 – 1e-3
Dropout	Rastgele nöron kapat	nn.Dropout(p)	p = 0.2 – 0.5 (gizli katman)
Veri artırma	Girdiyi rastgele dönüştür	torchvision.transforms	Döndürme $\pm 10^\circ$, kaydırma %10
Batch norm	Katman girdisini normalize et	nn.BatchNorm1d / 2d	Linear/Conv'dan sonra, aktivasyondan önce
Küçültme	Daha az nöron/katman	—	Doğrulama eğrisine göre

Tablo 11.3 Düzenleştirme teknikleri

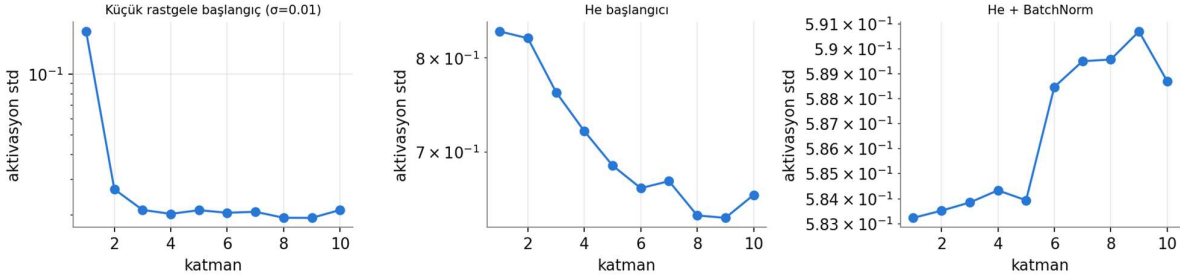
Örnek 11.3 — Dropout ölçeklemesi (elle)

Soru: Bir gizli katmanın 4 nöronunun aktivasyonu [2, 4, 6, 8], dropout $p = 0.5$. Eğitimde 2. ve 4. nöron kapatıldı. 'Ters dropout' ile eğitimdeki çıktı vektörü nedir? Testte ne olur? Beklenen değerler neden eşleşir?

Ters dropout: açık kalan nöronlar $1/(1-p) = 2$ ile çarpılır $\rightarrow [4, 0, 12, 0]$. Testte hiçbir şey kapatılmaz ve ölçekleme yapılmaz $\rightarrow [2, 4, 6, 8]$. Eğitimde her nöronun beklenen çıktısı $(1-p) \cdot a \cdot 1/(1-p) = a$; yani eğitim ve testteki beklenen aktivasyonlar aynı kalır, sonraki katman iki rejimde tutarlı girdi görür. PyTorch bu ters ölçeklemeyi eğitim modunda uygular; `model.eval()` dropout'u kapatır — çağırmaı unutursanız test tahminleri rastgele olur.

11.6 Başlangıç ve Batch Normalization

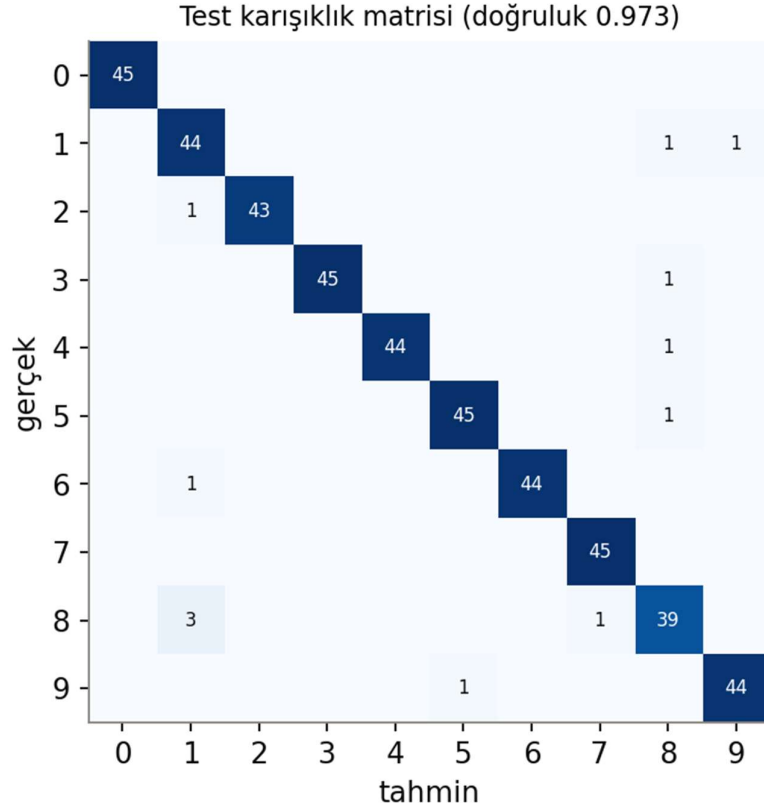
Derin ağda ağırlıkların başlangıç ölçeği kritiktir. Çok küçük başlangıçta aktivasyonlar katmandan katmana sıfıra iner ve gradyanlar kaybolur; çok büyükte patlar. He başlangıcı (ReLU için $N(0, 2/n_{in})$) her katmanın çıktı varyansını girdiyle aynı tutar; PyTorch'un Linear/Conv katmanları benzer bir ölçek kullanır. Batch normalization (Ioffe & Szegedy 2015) daha güçlü bir çözümdür: her katmanın girdisini mini-batch içinde standartlaştırır (ortalama 0, varyans 1) ve öğrenilebilir bir ölçek/kayma ekler. Sonuç: daha büyük öğrenme oranı, daha hızlı yakınsama, başlangıca duyarsızlık ve hafif düzenleme. Testte batch istatistikleri yerine eğitimde biriken hareketli ortalamalar kullanılır — yine `model.eval()` gerekir.



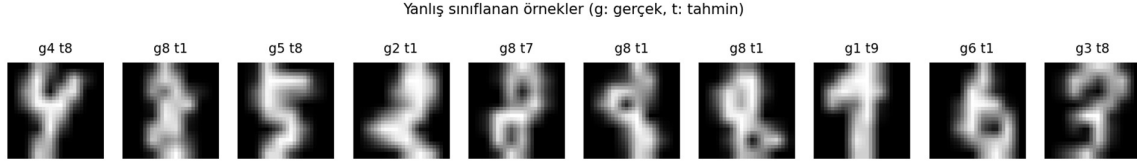
Şekil 11.6 10 katmanlı ağda aktivasyon std'si: küçük başlangıçta söner, He ile korunur, BatchNorm ile sabitlenir

11.7 Hiperparametre Stratejisi

Derin öğrenmede hiperparametre çoktur ve ızgara araması pahalıdır. İşe yarayan sıra şudur: (1) Sağlık testi: tek bir mini-batch'i ezberletebiliyor musunuz? Ezberleyemiyorsa kod hatalıdır. (2) Taban: küçük ağ, Adam $lr = 1e-3$, düzenlemez; doğrulama eğrisine bak. (3) Büyüt: eksik öğrenme varsa ağı büyüt, uzun eğit. (4) Düzenle: aşırı öğrenme varsa dropout/weight decay/veri artırma; erken durdurmayı her zaman kullan. (5) Öğrenme oranı: 3 kat aralıklarla tara ($3e-4, 1e-3, 3e-3$); en etkili hiperparametre budur. (6) Tohum: en az 2-3 farklı tohumla tekrarlayıp sapmayı raporla. (7) Hata analizi: karışıklık matrisi ve yanlış örneklere bak; sistematik hatalar veri ya da özellik sorunudur, model değil.



Şekil 11.7 10 sınıflı karışıklık matrisi: hangi rakamlar karışıyor?



Şekil 11.8 Yanlış sınıflanan örnekler: bazıları insan için de belirsiz

Örnek 11.4 — Erken durdurma kararı

Soru: Doğrulama kaybı epoch'lara göre: 0.52, 0.31, 0.22, 0.17, 0.15, 0.14, 0.14, 0.15, 0.17, 0.19, 0.22. Sabır 3 epoch olan erken durdurma hangi epoch'ta durur ve hangi ağırlıkları kullanır? Eğitim 11 epoch sürseydi ne kaybedilirdi?

En düşük değer 0.14, 6. epoch'ta (7. epoch da 0.14 ama iyileşme yok). 7, 8, 9. epoch'larda 3 ardışık iyileşmesiz epoch → eğitim 9. epoch sonunda durur; kullanılan ağırlıklar 6. epoch'unkilerdir (kaydedilmiş en iyi). 11 epoch'a devam edilseydi son ağırlıklar 0.22 kayıplı olurdu — 6. epoch'tan %57 daha kötü. Erken durdurma, hem zaman kazandırır hem de 'epoch sayısı' hiperparametresini otomatik seçer.

11.8 Uygulama Protokolü

```
# Eğitim döngüsü + erken durdurma iskeleti
en_iyi, sabir, bekleme = float("inf"), 5, 0
for epoch in range(100):
    model.train()
    for xb, yb in yukleyici:
        opt.zero_grad(); L = F.cross_entropy(model(xb), yb); L.backward(); opt.step()
    model.eval()
    with torch.no_grad(): va = F.cross_entropy(model(X_va), y_va).item()
    if va < en_iyi: en_iyi, bekleme = va, 0; torch.save(model.state_dict(), "en_iyi.pt")
    else:
        bekleme += 1
        if bekleme >= sabir: print("erken durdurma, epoch", epoch); break
model.load_state_dict(torch.load("en_iyi.pt"))
```

11.9 Sınıf İçi Sorular

1. $z = [0, 0, 0]$ için softmax? Kayıp? Bu değere 'rastgele tahmin kaybı' denmesinin nedeni?
2. Modelin son katmanına nn.Softmax eklerseniz ve CrossEntropyLoss kullanırsanız ne olur?
3. Dropout $p = 0.5$ ile eğitilmiş modelde model.eval() çağırmadan test yaparsanız doğruluk nasıl değişir?
4. Weight decay ile ridge regresyon arasındaki ilişki nedir?
5. Eğitim doğruluğu %99.9, doğrulama %97.5, test %97.4. Hangisi raporlanır? Fark neyi gösterir?

11.10 Kısa Sınav 3 (örnek) ve Uygulama

1. $z = [2, 1, 0]$, $y = 0$ için softmax olasılıkları ve çapraz entropi. (4 p)
2. Örnek 10.1 tipinde tek nöron gradyanı. (3 p)
3. Erken durdurma ve dropout'un ne yaptığını iki cümleyle. (3 p)
 - Defer: MNIST yükleme (yedekli), MLP taban, öğrenme eğrileri, aşırı öğrenme deneyi, dropout/wd/erken durdurma, η taraması, batch norm, karışıklık matrisi ve yanlış örnekler.
 - Ödev yok (kısa sınav 3). Gelecek hafta: CNN — konvolüsyon, filtreler, havuzlama; PCB kusur / MNIST ile CNN.

Hafta 12 — Evrişimli Sinir Ağları (CNN)

Bölüm	Süre	İçerik
Saat 1	50 dk	Neden MLP görüntüde yetersiz; konvolüsyon (elle hesap, sinyal işleme bağlantısı), filtreler, padding/stride, çıktı boyutu, havuzlama
Saat 2	50 dk	CNN mimarisi ve parametre sayısı; öğrenilen filtreler; klasik mimariler (LeNet → ResNet); veri artırma; transfer öğrenme
Saat 3	50 dk	Uygulama: sentetik PCB kusur veri setinde CNN vs MLP; filtre ve özellik haritalarını görselleştirme; artırma; (Colab) MNIST/CIFAR-10

Tablo 12.1 Haftanın ders planı

12.1 Kazanımlar

- 2-B konvolüsyonu elle hesaplar; padding, stride ve çıktı boyutu formülünü uygular.
- Konvolüsyonun sinyal işlemedeki FIR filtreyle ilişkisini ve CNN'in filtreleri 'öğrenmesini' açıklar.
- Havuzlamanın ne yaptığını ve neden kullanıldığını söyler.
- Küçük bir CNN'in parametre sayısını hesaplar; MLP ile karşılaştırır (yerellik, ağırlık paylaşımı).
- PyTorch ile CNN kurar, eğitir; öğrenilen filtreleri ve özellik haritalarını görselleştirir.
- Transfer öğrenmenin ne zaman ve nasıl kullanıldığını bilir; 1-B konvolüsyonun sinyallere uygulanabileceğini görür.

12.2 Görüntüde MLP Neden Yetersiz?

Geçen hafta 28×28 görüntüyü 784 elemanlı vektöre düzleştirip MLP'ye verdik ve iyi sonuç aldık; ama bu yaklaşım iki şeyi atar. Birincisi uzamsal yapı: komşu piksellerin ilişkili olduğu bilgisi düzleştirmede kaybolur; MLP için 1. ve 2. piksel ile 1. ve 500. piksel eşit uzaklıktadır. İkincisi öteleme değişmezliği: bir rakam bir piksel sağa kayarsa 784 girdinin tamamı değişir ve MLP bunu 'yeni bir örüntü' olarak öğrenmek zorundadır. Üstelik parametre sayısı görüntü boyutuyla patlar: 224×224 renkli görüntüde ilk katman $150\,528$ girdi $\times$ 256 nöron $\approx$ 38 milyon ağırlık. Konvolüsyon üç fikirle bunları çözer: yerellik (her nöron küçük bir pencereye bakar), ağırlık paylaşımı (aynı pencere ağırlıkları her konumda kullanılır) ve hiyerarşi (katmanlar birleşerek kenar $\rightarrow$ köşe $\rightarrow$ parça $\rightarrow$ nesne dedektörleri kurar).

Tam bağlı (MLP) 32×32 girdi $\rightarrow$ 256 nöron $1024 \times 256 = 262\,144$ ağırlık Her piksel her nörona ayrı ağırlıkla bağlı; nesne 1 px kayarsa her şey değişir	Konvolüsyon 32×32 girdi $\rightarrow$ 16 filtre 3×3 $16 \times 9 = 144$ ağırlık Aynı çekirdek her konumda paylaşılır; öteleme değişmezliği, yerel yapı
--	---

Yerellik + ağırlık paylaşımı = 1800 kat az parametre, çok daha iyi genelleme

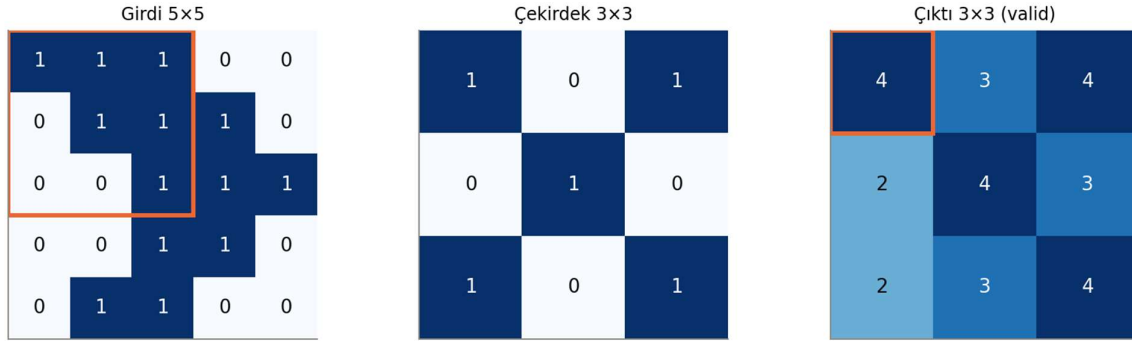
Şekil 12.1 Tam bağlı katmana karşı konvolüsyon: 1800 kat az parametre

12.3 Konvolüsyon

Konvolüsyon, sinyal işlemeden tanıdığınız işlemidir: küçük bir çekirdek (kernel, filtre) girdi üzerinde kaydırılır, her konumda çekirdek ile altındaki pencere eleman eleman çarpılıp toplanır ve tek bir çıktı değeri üretilir. 1-B'de bu tam olarak FIR filtresidir: $y[n] = \sum_k h[k] x[n-k]$. Derin öğrenme kütüphaneleri teknik olarak 'çapraz

korelasyon' (çekirdek çevrilmez) uygular ama adı konvolüsyon kalmıştır; öğrenilen çekirdek için fark önemsizdir. Görüntüde çekirdek 2-B'dir (tipik 3×3 veya 5×5) ve renkli görüntüde derinliği kanal sayısı kadardır (3×3×3). Bir konvolüsyon katmanı çok sayıda çekirdek içerir; her biri bir 'özellik haritası' üretir. Klasik görüntü işlemede bu çekirdekler elle tasarlanırdı (Sobel kenar, Gauss bulanıklaştırma); CNN'de geri yayılımla öğrenilir — 7. haftada elle çıkardığımız özelliklerin görüntü karşılığı otomatik bulunur.

Konvolüsyon (çapraz korelasyon): çekirdek görüntü üzerinde kayar, eleman çarpımı toplanır



Şekil 12.2 5×5 görüntü, 3×3 çekirdek, 3×3 çıktı

Örnek 12.1 — 2-B konvolüsyon elle

Soru: Şekil 12.2'deki 5×5 ikili görüntüye 3×3 çekirdek $K = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$ uygulayın (padding yok, stride 1). Sol üst çıktı değerini ve çıktı boyutunu bulun.

Sol üst 3×3 pencere: $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$. Eleman çarpımı toplamı: $1 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 = 4$. Çıktı boyutu: $(5 - 3)/1 + 1 = 3 \rightarrow 3 \times 3$. Tüm çıktı: $\begin{bmatrix} 4 & 3 & 4 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{bmatrix}$. Çekirdek 'X' biçimini arar; en yüksek yanıtlar (4) X-benzeri desenlerin olduğu konumlardır — bir filtre bir 'desen dedektörüdür'.

```
from scipy.signal import correlate2d
img = np.array([[1,1,1,0,0],[0,1,1,1,0],[0,0,1,1,1],[0,0,1,1,0],[0,1,1,0,0]]); K =
np.array([[1,0,1],[0,1,0],[1,0,1]])
print(correlate2d(img, K, mode="valid"))
import torch, torch.nn.functional as F
print(F.conv2d(torch.tensor(img, dtype=torch.float32)[None, None], torch.tensor(K,
dtype=torch.float32)[None, None])[0, 0])
```

Kavram	Tanım	Etkisi
Çekirdek boyutu k	Pencerenin kenarı (3, 5, 7)	Büyük k: geniş bağlam, çok parametre; modern ağlar 3×3 yığar
Padding p	Kenarlara eklenen sıfır piksel	$p = (k-1)/2$ ('same') ile boyut korunur; kenar bilgisi kaybolmaz
Stride s	Kaydırma adımı	$s = 2$ boyutu yarıya indirir (havuzlama alternatifi)
Kanal sayısı	Filtre sayısı = çıktı derinliği	Her filtre bir özellik haritası; katman ilerledikçe artar (16 $\rightarrow$ 32 $\rightarrow$ 64)
Çıktı boyutu	$\lfloor (n + 2p - k)/s \rfloor + 1$	n: girdi kenarı
Parametre	$k \cdot k \cdot C_{in} \cdot C_{out} + C_{out}$	Görüntü boyutundan BAĞIMSIZ

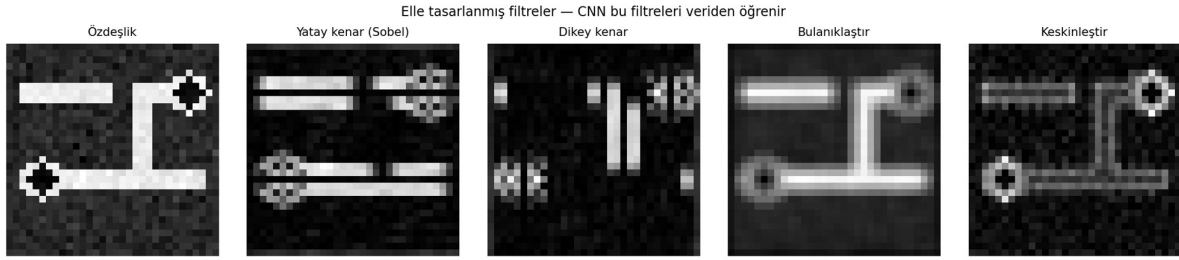
Tablo 12.2 Konvolüsyon katmanının hiperparametreleri

Örnek 12.2 — Çıktı boyutu ve parametre sayısı

Soru: 32×32 gri görüntüye (1 kanal) 16 adet 3×3 filtre, padding 1, stride 1 uygulanıyor; ardından 2×2 max pool; sonra 32 adet 3×3 filtre (padding 1) ve tekrar 2×2 pool. Her aşamanın çıktı boyutunu ve toplam konvolüsyon parametresini bulun.

Conv1: $(32 + 2 - 3)/1 + 1 = 32 \rightarrow 16 \times 32 \times 32$; parametre $3 \cdot 3 \cdot 1 \cdot 16 + 16 = 160$. Pool $\rightarrow 16 \times 16 \times 16$. Conv2: $32 \rightarrow 32 \times 16 \times 16$; parametre $3 \cdot 3 \cdot 16 \cdot 32 + 32 = 4640$. Pool $\rightarrow 32 \times 8 \times 8 = 2048$ değer. Toplam konvolüsyon parametresi 4800 — tam bağlı bir 1024→256 katmanın (262 400) %2'si. Sonra Flatten $\rightarrow$ Linear 2048→64 (131 136) $\rightarrow$ Linear 64→4 (260). Tüm ağ $\approx 136\ 356$ parametre; büyük kısmı ilk tam bağlı katmanda (bu yüzden modern ağlar global average pooling ile onu küçültür).

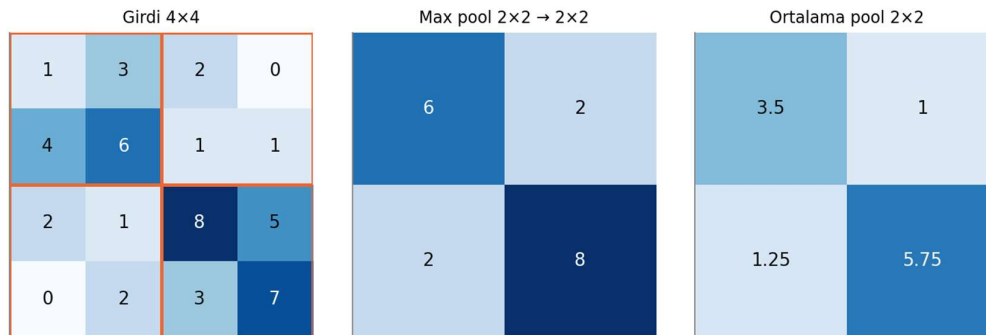
```
import torch.nn as nn
cnn = nn.Sequential(nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                    nn.Flatten(), nn.Linear(32*8*8, 64), nn.ReLU(), nn.Linear(64, 4))
x = torch.zeros(1, 1, 32, 32)
for kat in cnn: x = kat(x); print(f"{kat.__class__.__name__:10s} -> {tuple(x.shape)}")
print("parametre:", sum(p.numel() for p in cnn.parameters()))
```



Şekil 12.3 Elle tasarlanmış klasik filtreler PCB görüntüsünde: CNN bunları ve daha iyilerini veriden öğrenir

12.4 Havuzlama

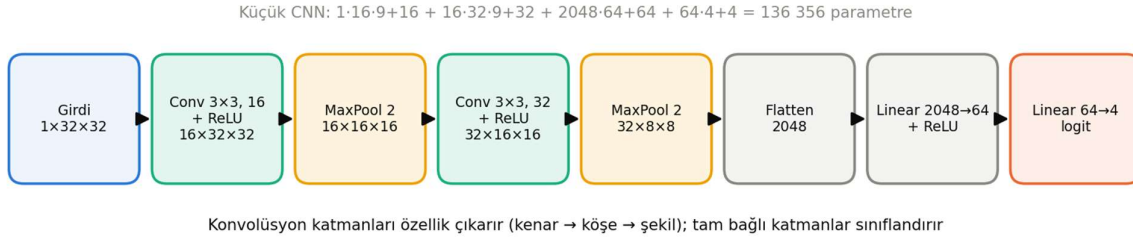
Havuzlama (pooling), özellik haritasını küçültür: 2×2 pencerelerde en büyük (max pool) ya da ortalama değer alınır, boyut yarıya iner. Üç yararı vardır: hesap ve parametre azalır (sonraki katmanların girdisi küçülür), küçük ötelemelere karşı dayanıklılık artar (özellik bir piksel kayınca max aynı kalır) ve alıcı alan (receptive field) büyür — sonraki katmanın 3×3 penceresi orijinal görüntüde 6×6'lık bölgeye bakar. Max pool en yaygındır; modern ağlar bazen havuzlama yerine stride 2 konvolüsyon kullanır. Ağın sonunda global average pooling her özellik haritasını tek sayıya indirir ve büyük tam bağlı katmanı gereksiz kılar.



Şekil 12.4 2×2 max ve ortalama havuzlama

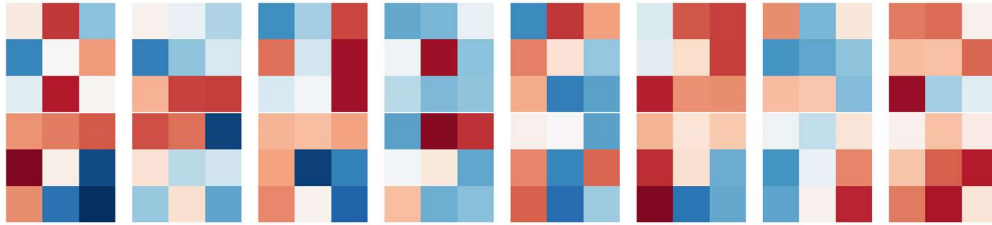
12.5 Bir CNN'in Anatomisi

Tipik bir CNN, [Conv → ReLU → (Conv → ReLU) → Pool] bloklarının tekrarı ve sonunda birkaç tam bağı katmandan oluşur. Katmanlar ilerledikçe uzamsal boyut küçülür (32 → 16 → 8), kanal sayısı artar (16 → 32 → 64): ağ 'nerede' bilgisini 'ne' bilgisine dönüştürür. İlk katman filtreleri kenar ve renk geçişleri, ikinci katman köşe ve doku, üçüncü katman parça (bir delik, bir yol kesişimi), sonraki katmanlar nesne düzeyi desenler öğrenir. Bu hiyerarşi, 9. haftadaki 'derinlik hiyerarşik özellik kurar' iddiasının somut hâlidir. Batch normalization her konvolüsyondan sonra (BatchNorm2d), dropout ise tam bağı katmanlarda kullanılır.



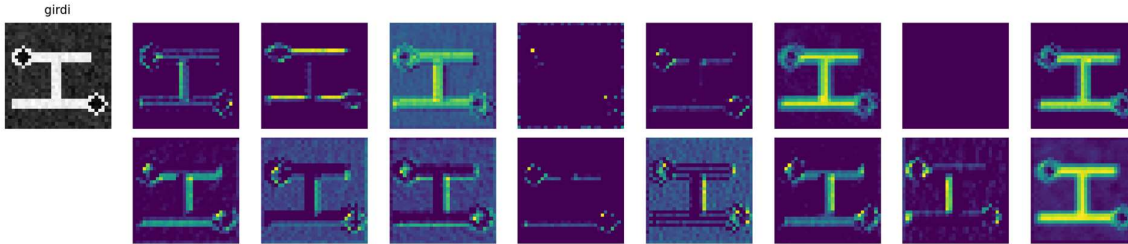
Şekil 12.5 Bu haftaki küçük CNN: 136 356 parametre

İlk katmanın öğrendiği 16 filtre (3x3): kenar ve köşe dedektörleri



Şekil 12.6 PCB verisinde ilk katmanın öğrendiği filtreler

İlk katman özellik haritaları: her filtre görüntünün farklı bir yönünü vurgular

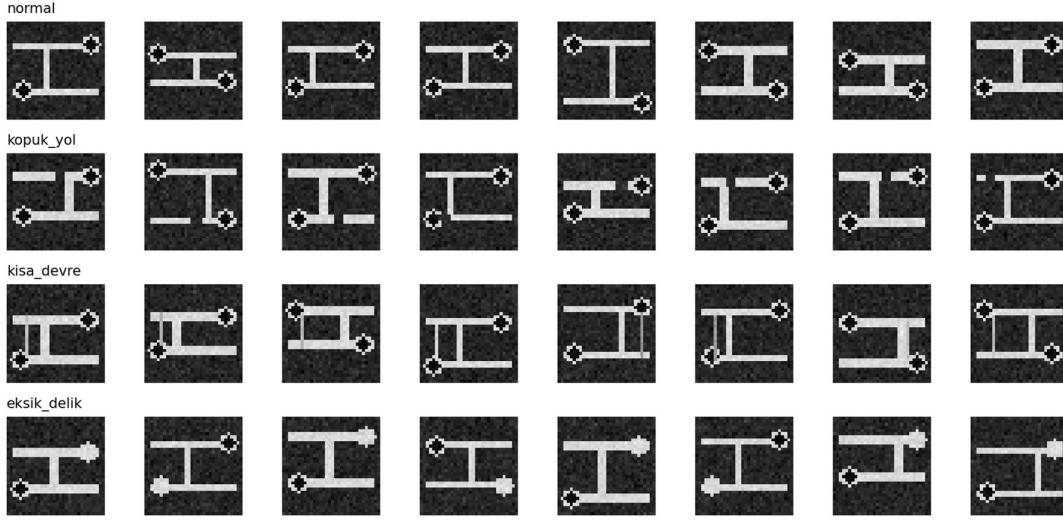


Şekil 12.7 Aynı görüntü için 16 özellik haritası: kimi yatay yolu, kimi delikleri, kimi kenarları vurgular

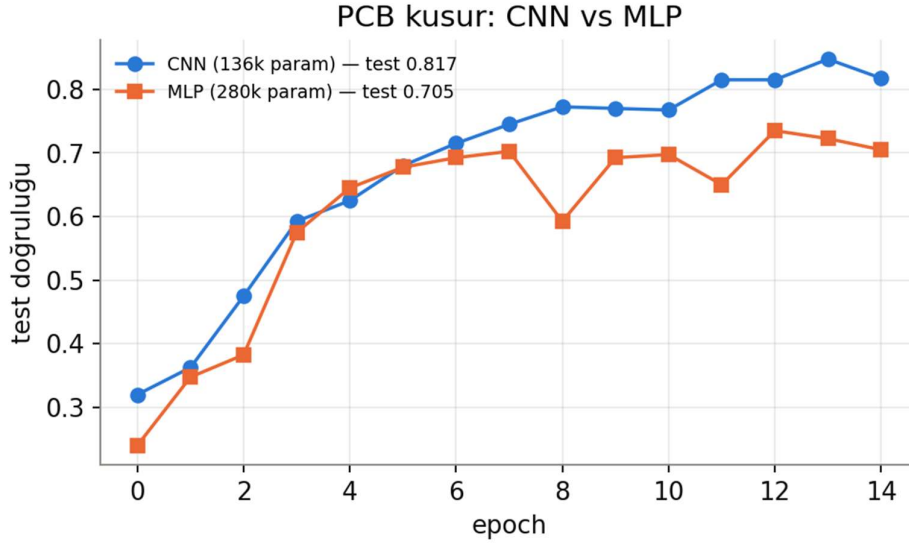
12.6 Uygulama Verisi: PCB Kusur Tespiti

Bu haftanın veri seti pcb_kusur.npz, sentetik olarak üretilmiş 2000 adet 32x32 gri ton PCB yolu görüntüsüdür: normal, kopuk yol (open), kısa devre (short, iki yol arasında ince köprü), eksik delik (pad'de delik yok). Görüntüler rastgele yol konumu, kalınlık, gürültü ve yansıma içerir; %80/20 eğitim/test olarak dosyada hazır. Gerçek üretim hattı verilerinden (AOI kameraları) farklı olarak temiz ve dengeli; ama CNN'in ne öğrendiğini görmek için idealdir. Aynı defter, Colab'da bir bayrağı değiştirerek MNIST veya CIFAR-10 ile de çalışır.

pcb_kusur: 32x32 sentetik PCB yolu görüntüleri, 4 sınıf



Şekil 12.8 Dört sınıftan örnekler



Şekil 12.9 Aynı veride CNN ve MLP: daha az parametreyle daha yüksek doğruluk

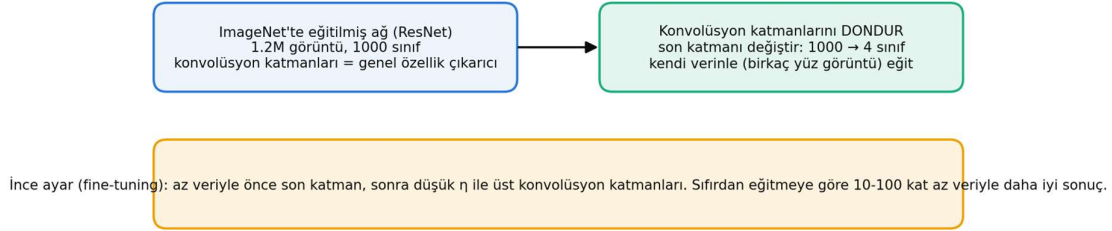
PCB verisi ve CNN — Conv2d girdisi (N, C, H, W) biçiminde olmalı

```
veri = np.load("pcb_kusur.npz"); siniflar = list(veri["siniflar"])
X_tr = torch.tensor(veri["X_train"])[:, None] # (N, 1, 32, 32): kanal boyutu eklenir
y_tr = torch.tensor(veri["y_train"]).long()
cnn = nn.Sequential(nn.Conv2d(1, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
                    nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
                    nn.Flatten(), nn.Dropout(0.3), nn.Linear(32*8*8, 64), nn.ReLU(), nn.Linear(64,
4))
# eğitim döngüsü 10-11. haftalarla aynı: F.cross_entropy, Adam, DataLoader
```

12.7 Veri Artırma ve Transfer Öğrenme

Görüntüde veri artırma çok etkilidir çünkü etiketi değiştirmeyen dönüşümler bellidir: yatay/dikey çevirme, küçük döndürme, kaydırma, parlaklık değişimi, kırpm. PCB gibi simetrik nesnelerde çevirme güvenlidir; rakamlarda yatay çevirme değildir (6 ↔ 9 sorunu). torchvision.transforms bu dönüşümleri eğitim sırasında rastgele uygular; ağ her epoch'ta 'yeni' görüntüler görür.

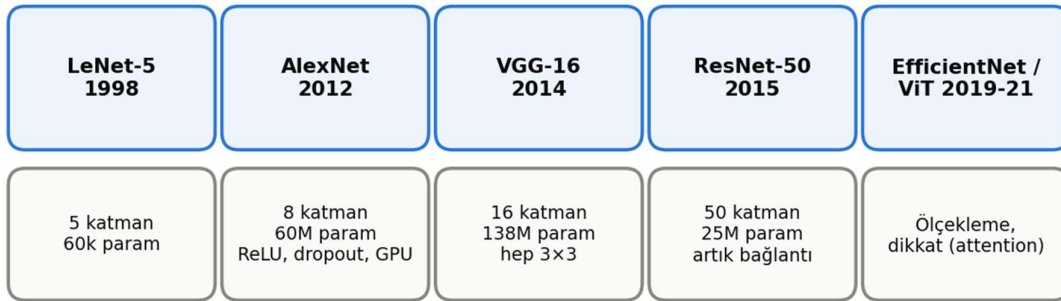
Gerçek projelerde birkaç yüz etiketli görüntüyle sıfırdan CNN eğitmek genellikle başarısızdır. Transfer öğrenme çözümdür: ImageNet'te (1.2 milyon görüntü, 1000 sınıf) eğitilmiş bir ağı (ResNet-18/50, EfficientNet) konvolüsyon katmanları genel amaçlı bir özellik çıkarıcıdır; kenar, doku ve şekil dedektörleri PCB'de de işe yarar. Son sınıflandırma katmanı kendi sınıf sayınıza göre değiştirilir ve önce yalnızca o katman eğitilir (konvolüsyonlar dondurulur); yeterli veri varsa sonra düşük öğrenme oranıyla üst katmanlar da 'ince ayarlanır'. Bu, endüstriyel görüntü uygulamalarında standart yoldur ve genellikle sıfırdan eğitimden hem daha hızlı hem daha doğrudur.



Şekil 12.10 Transfer öğrenme akışı

Transfer öğrenme iskeleti (Colab, internet gerekir)

```
import torchvision
model = torchvision.models.resnet18(weights="IMAGENET1K_V1") # önceden eğitilmiş
for p in model.parameters(): p.requires_grad = False # dondur
model.fc = nn.Linear(model.fc.in_features, 4) # son katman: 1000 -> 4 sınıf
(yalnızca bu eğitilir)
# Girdi 3 kanallı 224x224 beklenir: gri görüntüyü 3 kanala kopyala, yeniden boyutlandır, ImageNet
ortalama/std ile normalize et
```



Şekil 12.11 Klasik mimariler: derinlik artar, artık bağlantılar (ResNet) 50+ katmanı eğitilebilir kılar

Mimari	Yıl	Katman	Parametre	Yenilik
LeNet-5	1998	5	60 k	İlk pratik CNN (posta kodu okuma)
AlexNet	2012	8	60 M	ReLU, dropout, GPU eğitimi; ImageNet devrimi
VGG-16	2014	16	138 M	Yalnızca 3x3 filtre yığını
ResNet	2015	18-152	11-60 M	Artık (skip) bağlantı: çok derin ağ eğitilebilir
MobileNet	2017	28	4 M	Derinlik-ayrılabilir konvolüsyon: gömülü cihaz
EfficientNet	2019	—	5-66 M	Genişlik/derinlik/çözünürlük dengeli ölçekleme
ViT	2020	—	86 M+	Konvolüsyon yerine dikkat (transformer)

Tablo 12.3 Kilometre taşı mimariler

12.8 1-B Konvolüsyon: Sinyaller İçin CNN

Konvolüsyon görüntüye özgü değildir; 7. haftadaki titreşim sinyalleri gibi 1-B diziler için Conv1d kullanılır. Çekirdek bir FIR filtresidir ve ağ, arıza darbelerini yakalayan filtreleri kendisi öğrenir; elle çıkardığımız kurtosis/bant enerjisi özelliklerine gerek kalmaz. Tipik yapı: Conv1d(1, 16, k=16) → ReLU → MaxPool → Conv1d(16, 32, k=8) → ... → global pool → Linear. 13. haftada zaman serisi tahmininde ve dönem projelerinde (EKG, titreşim, güç kalitesi) bu yapı doğrudan kullanılabilir. Defterin sonunda rulman verisiyle bir 1-B CNN örneği vardır.

12.9 Sınıf İçi Sorular

1. 64×64 girdi, 5×5 çekirdek, padding 0, stride 2: çıktı boyutu? Padding 2 olsaydı?
2. 3 kanallı 224×224 görüntüde 64 adet 7×7 filtre: kaç parametre? Görüntü 448×448 olsaydı?
3. Max pool'un öğrenilebilir parametresi var mı? Neden öteleme dayanıklılığı sağlar?
4. Sobel çekirdeğini $\begin{bmatrix} -1, 0, 1 \\ -2, 0, 2 \\ -1, 0, 1 \end{bmatrix}$ dikey bir yola uygularsanız çıktı nerede büyük olur?
5. Rakam tanımda yatay çevirme neden zararlı bir veri artırmadır? PCB'de neden zararsız?

12.10 Ödev 6 — Görüntü Sınıflandırma

- pcb_kusur.npz ile: (a) MLP taban, (b) küçük CNN, (c) BatchNorm + dropout + veri artırmalı CNN; test doğruluğu, makro-F1 ve 4×4 karışıklık matrisi; hangi kusur en zor?
- İlk katman filtrelerini ve bir 'kısa devre' görüntüsünün özellik haritalarını çizin; kısa devre köprüsünü vurgulayan haritayı bulun.
- Sağlamlık: test görüntülerine $\sigma = 0.1$ gürültü ekleyip ve 2 piksel kaydırıp doğruluğu yeniden ölçün; artırmalı model daha dayanıklı mı?
- Colab (GPU): aynı CNN'i MNIST'te 5 epoch eğitin (%99+ beklenir); 11. haftanın MLP'siyle parametre/doğruluk karşılaştırması.
- Bonus: torchvision resnet18 ile PCB'de transfer öğrenme (görüntüleri 3 kanal, 64×64'e büyütün); sıfırdan CNN ile karşılaştırın.
- Teslim: defter + yarım sayfa rapor.

Hafta 13 — Zaman Serileri ve Tekrarlayan Sinir Ağları

Bölüm	Süre	İçerik
Saat 1	50 dk	Zaman serisi nedir; bileşenler (trend, mevsimsellik, gürültü); otokorelasyon; zaman bölmesi ve sızıntı; naif tabanlar; hata ölçütleri (MAE, RMSE, MAPE)
Saat 2	50 dk	Gecikme özellikleri ile 'klasik' tahmin; pencereleme (N, W, F); RNN ve BPTT; kaybolan gradyan; LSTM/GRU hücresi; 1-B CNN alternatifi; tahmin ufku
Saat 3	50 dk	Uygulama: sebeke_yuku.csv ile 1 saat ve 24 saat ileri yük tahmini; naif → ridge → orman → LSTM; artık tabanlı anomali; proje ara teslimi

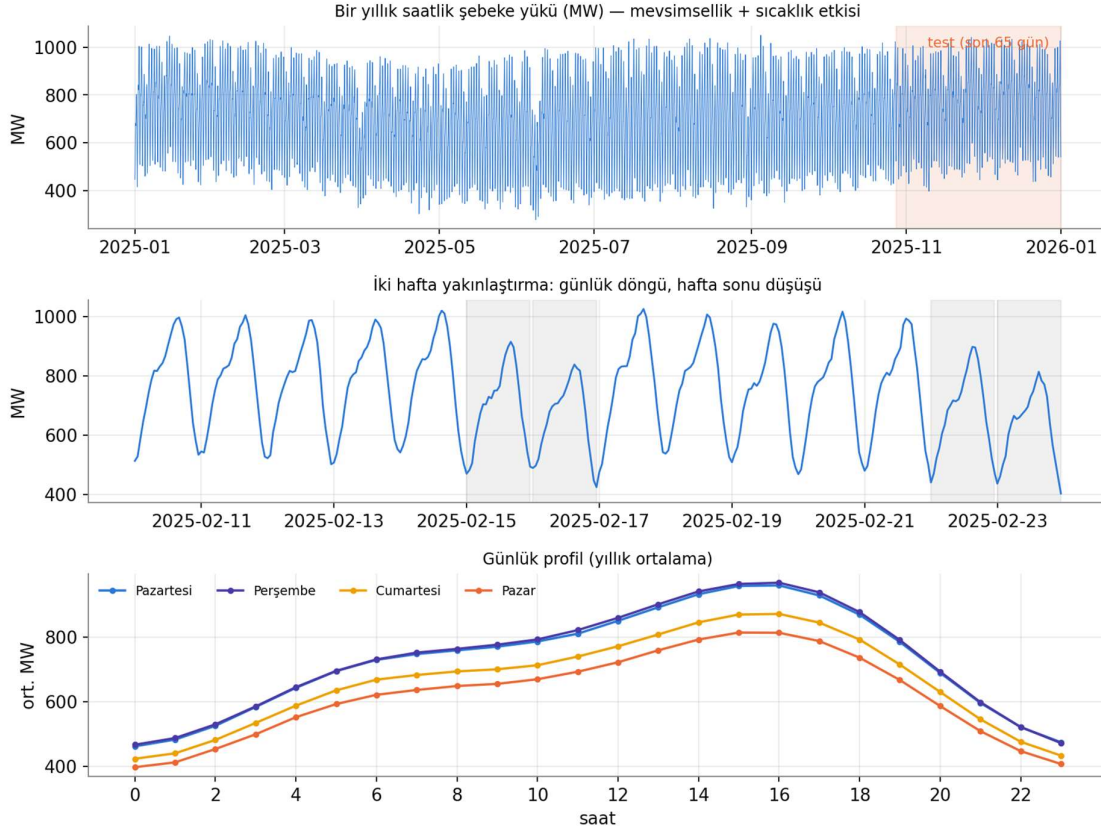
Tablo 13.1 Haftanın ders planı

13.1 Kazanımlar

- Bir zaman serisinin bileşenlerini (mevsimsellik, trend, gürültü) ayırt eder ve otokorelasyon fonksiyonunu yorumlar.
- Zaman serisinde doğru bölme (geçmiş → gelecek) ve genişleyen pencere CV kurar; sızıntı kaynaklarını sayar.
- Naif ve mevsimsel naif tabanları kurup MAE/RMSE/MAPE hesaplar; her modeli tabana karşı raporlar.
- Gecikme (lag) ve takvim özellikleriyle doğrusal/ağaç modelleri eğitir; ufuk (h) kavramını özellik seçimine yansıtır.
- Diziyi (N, W, F) tensörüne pencereler; RNN ileri geçişini elle hesaplar; LSTM kapılarının ne yaptığını ve parametre sayısını bilir.
- PyTorch'ta LSTM ile 1 saat ve 24 saat ileri tahmin yapar; sonuçları klasik modellerle dürüstçe karşılaştırır; artıklardan anomali üretir.

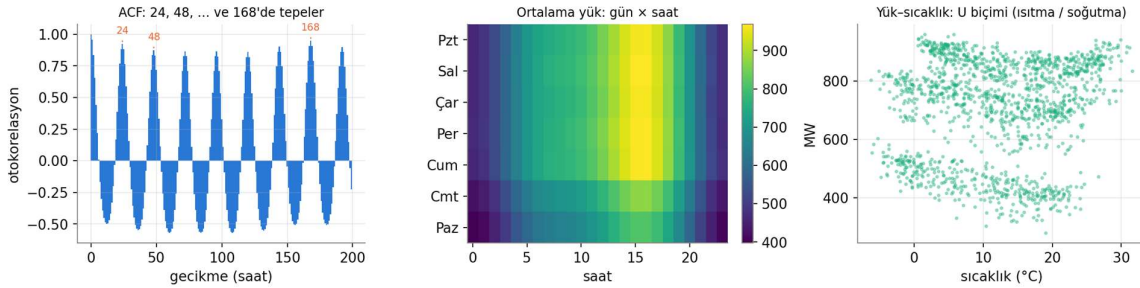
13.2 Zaman Serisi: Bileşenler ve Otokorelasyon

Zaman serisi, aynı büyüklüğün eşit aralıklarla ölçülmüş dizisidir: saatlik şebeke yükü, bir motorun dakikalık sıcaklığı, bir hattın 15 dakikalık gerilimi. Şimdiye kadarki verilerden temel farkı, örneklerin bağımsız olmamasıdır: t anındaki değer $t-1$ 'e çok benzer (kalıcılık) ve $t-24$ ile $t-168$ 'e benzer (günlük ve haftalık mevsimsellik). Bu bağımlılık hem fırsattır — geçmiş, geleceğin en iyi göstergesidir — hem tuzaktır — rastgele bölme, komşu saatleri eğitim ve teste dağıtarak sızıntı yaratır (4. haftadaki k-NN örneği). Bir seriyi klasik ayrıştırma ile üç bileşene bakarak anlarız: trend (yavaş değişen seviye; yıllık ısınma, talep büyümesi), mevsimsellik (sabit periyotlu döngü; gün, hafta, yıl) ve artık/gürültü. Otokorelasyon fonksiyonu (ACF) serinin kendisiyle k adım kaymış hâli arasındaki korelasyonu k 'ya göre çizer; hangi gecikmelerin bilgi taşıdığını (dolayısıyla hangi lag özelliklerinin gerektiğini) doğrudan gösterir.



Şekil 13.1 *sebeke_yuku.csv*: bir yıl (yaz ve kış tepeleri), iki hafta (günlük döngü, gri = hafta sonu) ve gün türüne göre ortalama profil

Bu haftanın veri seti *sebeke_yuku.csv* sentetiktir: 2025 yılı için saatlik bölgesel yük (MW) ve sıcaklık, gün, tatil sütunları. Yük; sabah ve akşam tepeli günlük profil, hafta sonu düşüşü (Cmt %90, Paz %84), resmî tatillerde %18 düşüş, sıcaklıkla U biçimli ilişki (15 °C altında ısıtma, 22 °C üstünde soğutma) ve yavaş değişen bir gürültü bileşeninden üretildi. Gerçek şebeke verisine (örn. EPİAŞ şeffaflık platformu) yapı olarak benzer; farkı, 'doğru cevabı' bildiğimiz için modellerimizin neyi yakalayıp neyi kaçırdığını görebilmemizdir.

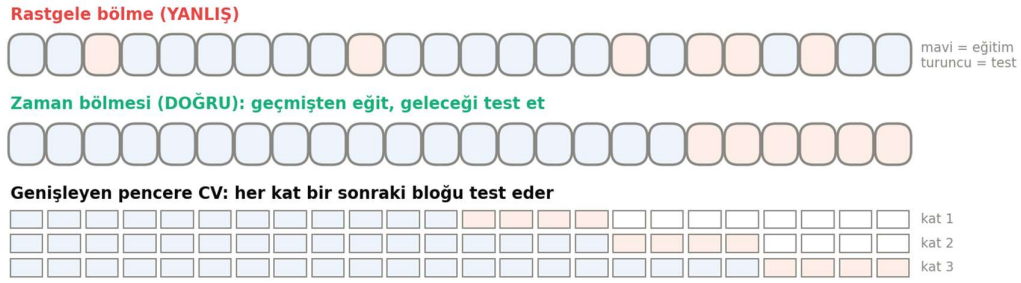


Şekil 13.2 Sol: ACF'de 24, 48, ... ve 168 saatlik tepeler. Orta: gün x saat ısı haritası. Sağ: yük-sıcaklık ilişkisi U biçimli — doğrusal model için sıcaklığı 5. haftadaki gibi $\max(T-22,0)$, $\max(15-T,0)$ olarak dönüştürmek gerekir

13.3 Zaman Bölmesi, Sızıntı ve Naif Tabanlar

Kural basittir: eğitim verisi test verisinden önce gelmelidir. Son 65 günü test ayırdık (24-300 = 7200 saat eğitim, 1560 saat test). Doğrulama için de aynı mantık: eğitimin son haftaları. Daha sağlam bir kestirim için genişleyen pencere çapraz doğrulaması kullanılır (scikit-learn `TimeSeriesSplit`): 1. kat ilk 6 ayla eğitilip 7. ayı test eder, 2. kat ilk 7 ayla 8. ayı... Sızıntı kaynakları zaman serisinde çeşitlidir: (i) rastgele bölme; (ii) tüm seride hesaplanan ölçekleme/ortalama (μ ve σ yalnızca eğitim bölümünden gelmeli); (iii) hedefi içeren kayan pencere ortalaması (`shift(1).rolling(24)` yazılmalı, `rolling(24)` değil); (iv) tahmin anında henüz

bilinmeyen özellikler — $t+24$ 'ü tahmin ederken $t+23$ 'ün yükünü kullanmak. Son madde tahmin ufku (h) kavramını gerektirir: h saat ilerisini tahmin ederken kullanılabilecek en yeni gözlem t 'dir; yani $\text{lag} \geq h$ olmalıdır.



Şekil 13.3 Rastgele bölme komşu saatleri iki tarafa dağıtır (sızıntı). Zaman bölmesi ve genişleyen pencere CV gerçek kullanımı taklit eder

Her tahmin modeli önce naif tabanlarla karşılaştırılmalıdır: naif (yarınki saat = şu anki), mevsimsel naif 24 saat (aynı saat dün) ve 168 saat (aynı saat geçen hafta). Yük gibi güçlü mevsimsel serilerde mevsimsel naif şaşırtıcı derecede iyidir ve 'karmaşık' modellerin çoğu onu ancak %20–40 iyileştirir. Hata ölçütleri: MAE (ortalama mutlak hata, MW), RMSE (büyük hataları daha çok cezalar, MW), MAPE (yüzde; ölçekten bağımsız, sıfıra yakın değerlerde patlar — PV üretimi gecelerinde kullanılmaz). Şebeke işletmesinde gün öncesi yük tahmini için %1–3 MAPE tipik bir hedeftir.

Örnek 13.1 — Naif tabanlar ve ölçütler (elle)

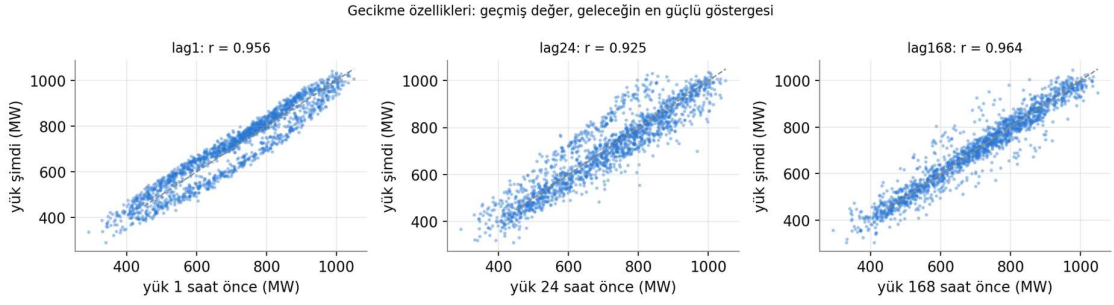
Soru: Beş saatlik gerçek yük [700, 720, 760, 790, 780] MW. Naif tahmin (bir önceki saat) [680, 700, 720, 760, 790] ve bir modelin tahmini [705, 715, 750, 800, 770]. Her ikisi için MAE, RMSE ve MAPE hesaplayın; model tabanı geçiyor mu?

Naif hatalar: [20, 20, 40, 30, -10] → MAE = 24.0; kareler [400, 400, 1600, 900, 100] ortalama 680 → RMSE = 26.1; yüzdeler [2.86, 2.78, 5.26, 3.80, 1.28] → MAPE = 3.20 %. Model hatalar: [-5, 5, 10, -10, 10] → MAE = 8.0; RMSE = $\sqrt{\text{ortalama}[25, 25, 100, 100, 100] = 70} = 8.4$; MAPE = ortalama [0.71, 0.69, 1.32, 1.27, 1.28] = 1.05 %. Model tabanın üçte biri hata yapıyor: beceri (skill) = $1 - 8.0/24.0 = 0.67$. Tabanı raporlamadan '%1 MAPE' söylemek anlamsızdır; bir serinin kendisi %1 oynuyorsa naif de %1 verir.

```
y = np.array([700, 720, 760, 790, 780.]); naif = np.array([680, 700, 720, 760, 790.]); model = np.array([705, 715, 750, 800, 770.])
for ad, t in (("naif", naif), ("model", model)):
    e = y - t; print(f"{ad:6s} MAE {np.abs(e).mean():.1f} RMSE {np.sqrt((e**2).mean()):.1f} MAPE {100*np.mean(np.abs(e)/y):.2f} %")
```

13.4 Gecikme Özellikleriyle Klasik Tahmin

Derin öğrenmeye geçmeden önce, 4–7. haftaların araçlarıyla zaman serisi bir tablo problemine çevrilebilir: her satır bir t anı, sütunlar geçmiş değerler (lag1, lag2, lag3, lag24, lag48, lag168), son 24 saatin ortalaması, takvim (saatin sin/cos'u, hafta sonu, tatil) ve dış değişkenler (sıcaklık). Ardından ridge veya rastgele orman eğitilir. Bu 'özellik mühendisliği + klasik model' yaklaşımı çoğu mühendislik probleminde LSTM ile başa baş ya da daha iyidir, çok daha hızlıdır ve katsayıları okunabilir. ACF, hangi gecikmelerin gerektiğini söyler: 1–3 (kalıcılık), 24 ve 48 (günlük), 168 (haftalık). Ufuk $h > 1$ ise yalnızca $\text{lag} \geq h$ kullanılabılır; 24 saat ileri için lag24, lag48, lag168 ve dünün ortalaması kalır — bilgi azalır, hata büyür (Şekil 13.6).



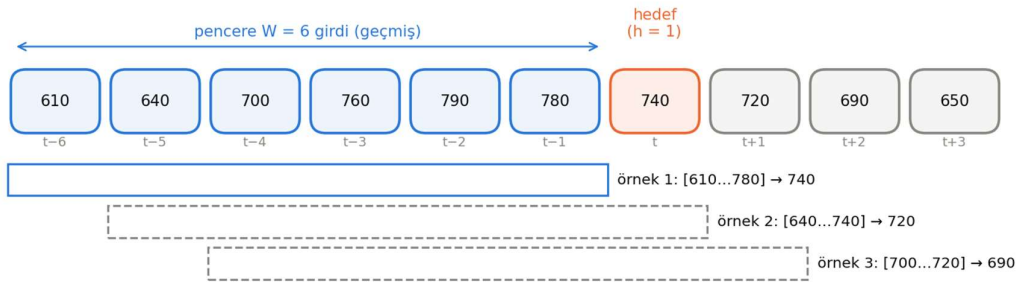
Şekil 13.4 Şimdiki yükün 1, 24 ve 168 saat önceki yüklerle ilişkisi: lag1 en güçlü (kalıcılık), lag168 lag24'ten iyi (hafta sonu etkisi)

Özellik	Ne yakalar	Ufuk kısıtı
lag1, lag2, lag3	Kalıcılık, kısa vadeli dinamik (AR bileşeni)	yalnızca $h \leq 1, 2, 3$
lag24, lag48	Günlük döngü	$h \leq 24$
lag168	Haftalık döngü, hafta sonu	$h \leq 168$
ort24 (shift(h).rolling(24))	Seviye; gürültüyü yumuşatır	kaydırma h olmalı
sin/cos(saat), hafta sonu, tatil	Takvim; gelecekte bilinir	kısıt yok
sıcaklık (tahmini)	Dış etken; U biçimi $\rightarrow$ parçalı dönüşüm	gelecek değeri hava tahmininden

Tablo 13.2 Gecikme ve takvim özellikleri

13.5 Pencereleme: Diziden Tensöre

Sinir ağları sabit biçimli girdi ister. Zaman serisini, W uzunluğunda kayan pencerelerle (N, W, F) tensörüne çeviririz: N örnek, her biri W zaman adımı, her adımda F özellik (yük, saatin sin/cos'u, hafta sonu, sıcaklık $\rightarrow F = 5$). Hedef, pencerenin bittiği andan h adım sonrasındır. Pencere birer adım kaydığında $N = L - W - h + 1$ örnek çıkar; ardışık örnekler büyük oranda örtüşür, bu normaldir. Ölçekleme yine yalnızca eğitim istatistikleriyle yapılır; hedef de ölçeklenir ve tahmin sonra geri çevrilir. W seçimi bir hiperparametredir: günlük döngü için en az 24, haftalık için 168; daha uzun pencere daha çok bellek ve daha yavaş eğitim demektir.



Pencere birer adım kayar $\rightarrow N - W - h + 1$ örnek; tensör biçimi (N, W, F)

Şekil 13.5 $W = 6$ pencere, $h = 1$ hedef; pencere kayarak örnekleri üretir. PyTorch biçimi (N, W, F), batch_first=True

Örnek 13.2 — Pencere sayısı ve tensör biçimi

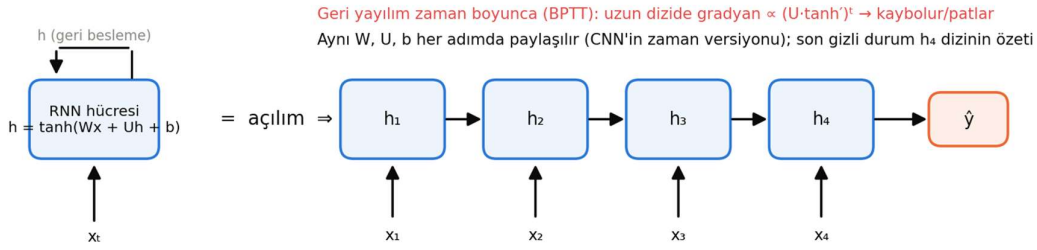
Soru: 7200 saatlik eğitim serisi, $W = 168$, $h = 1$, $F = 5$. Kaç eğitim örneği çıkar, tensör biçimi nedir, bellek (float32) ne kadardır? $h = 24$ olsaydı?

$N = 7200 - 168 - 1 + 1 = 7032$ örnek; biçim (7032, 168, 5); $7032 \cdot 168 \cdot 5 \cdot 4$ bayt = 23.6 MB — küçük. Test bölümünde ilk pencereler eğitimin son 168 saatini kullanır, bu sızıntı değildir (geçmişe bakmak serbest). $h = 24$ için $N = 7200 - 168 - 24 + 1 = 7009$; hedef 24 saat sonraki tek değer (tek çıktı) ya da önümüzdeki 24 saatin tamamı (24 çıktı, 'doğrudan çok adımlı').

```
L, W, h, F = 7200, 168, 1, 5; N = L - W - h + 1
print("N =", N, " biçim =", (N, W, F), " bellek MB =", round(N*W*F*4/1e6, 1), " h=24 →", L - W - 24 + 1)
```

13.6 Tekrarlayan Sinir Ağları (RNN) ve Zaman Boyunca Geri Yayılım

RNN, diziyi adım adım okuyan ve bir gizli durum h taşıyan ağıdır: $h_t = \tanh(W x_t + U h_{t-1} + b)$. Aynı W, U, b her adımda kullanılır — CNN'in uzayda yaptığı ağırlık paylaşımını RNN zamanda yapar; bu yüzden herhangi uzunluktaki diziyi işleyebilir. Son gizli durum h_t dizinin özetidir, üzerine bir doğrusal katman konarak tahmin üretilir. Eğitim, açılmış ağıda sıradan geri yayılımdır (BPTT). Sorun: kayıp t adımından geriye giderken gradyan $(U \cdot \tanh')^t$ ile çarpılır; $|U \cdot \tanh'| < 1$ ise üstel küçülür (kaybolan gradyan, ağı uzak geçmişe öğrenemez), > 1 ise patlar (gradyan kırpması ile bastırılır). Bu, 10. haftadaki derinlik problemi ile aynıdır; burada 'derinlik' dizinin uzunluğudur.



Şekil 13.6 Solda geri beslemeli hücre, sağda zamanda açılmış hâli. Kayıp son adımdan geriye tüm adımlara akar (BPTT)

Örnek 13.3 — RNN ileri geçişi elle

Soru: Tek özellikli girdi, gizli boyut 1: $W = 0.5, U = 0.8, b = 0, h_0 = 0$. Dizi $x = [1, 0, -1]$. Gizli durumları ve çıkış $\hat{y} = 2 \cdot h_3$ 'ü hesaplayın. $U = 0.8$ için gradyanın 10 adım geriye giderken kabaca kaçla çarpıldığını yorumlayın.

$h_1 = \tanh(0.5 \cdot 1 + 0.8 \cdot 0) = \tanh 0.5 = 0.462$. $h_2 = \tanh(0 + 0.8 \cdot 0.462) = \tanh 0.370 = 0.354$. $h_3 = \tanh(-0.5 + 0.8 \cdot 0.354) = \tanh(-0.217) = -0.214$. $\hat{y} = -0.427$. Gadyanın her adımda çarpanı $U \cdot \tanh'(z) \leq 0.8 \cdot 1$; 10 adımda en iyi hâlde $0.8^{10} = 0.107$, $\tanh' < 1$ olduğundan gerçekte çok daha küçük (burada $\approx 0.8^{10} \cdot 0.8 \cdot 0.87 \cdot 0.95 \dots$). Uzak geçmişin etkisi hızla silinir: sade RNN 20–30 adımdan uzun bağımlılıkları pratikte öğrenemez.

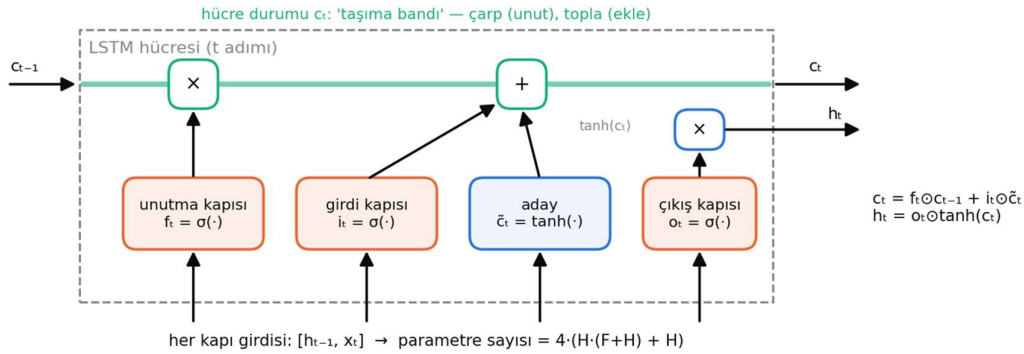
LSTM'nin hücre durumu bu çarpım zincirini toplama zincirine çevirir.

```
W, U, b, h = 0.5, 0.8, 0.0, 0.0
for x in [1, 0, -1]: h = np.tanh(W*x + U*h + b); print(round(h, 3))
print("ŷ =", round(2*h, 3), " 0.8^10 =", round(0.8**10, 3))
```

13.7 LSTM ve GRU

LSTM (Long Short-Term Memory, 1997) gizli duruma ek olarak bir hücre durumu c_t taşır ve üç kapı ile onu yönetir: unutma kapısı f_t eski bilginin ne kadarının kalacağını, girdi kapısı i_t yeni aday bilginin ($\tilde{c}_t$) ne

kadarının ekleneceğini, çıkış kapısı o_t hücre durumunun ne kadarının dışarı (h_t) verileceğini belirler: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $h_t = o_t \odot \tanh(c_t)$. Kapılar sigmoid çıkışlı küçük doğrusal katmanlardır ve $[h_{t-1}, x_t]$ birleşimini girdi alır. Kritik nokta, c_t 'nin toplama ile güncellenmesidir: gradyan çarpım zinciri yerine 'taşıma bandı' üzerinden neredeyse değişmeden geriye akar (ResNet'in artık bağlantısıyla aynı fikir). GRU (2014) iki kapıyla (sıfırlama, güncelleme) aynı işi daha az parametreyle yapar; küçük veride sıklıkla LSTM kadar iyidir. PyTorch'ta `nn.LSTM(F, H, batch_first=True)` (N, W, F) alır, (N, W, H) çıktıları ve son (h, c) durumunu verir; genellikle son adımın çıktısı bir `nn.Linear`'a bağlanır.



Şekil 13.7 LSTM hücresi: hücre durumu (yeşil bant) unutma kapısıyla çarpılır, girdi kapısıyla süzülen adayla toplanır; çıkış kapısı h_t 'yi üretir

Örnek 13.4 — LSTM parametre sayısı

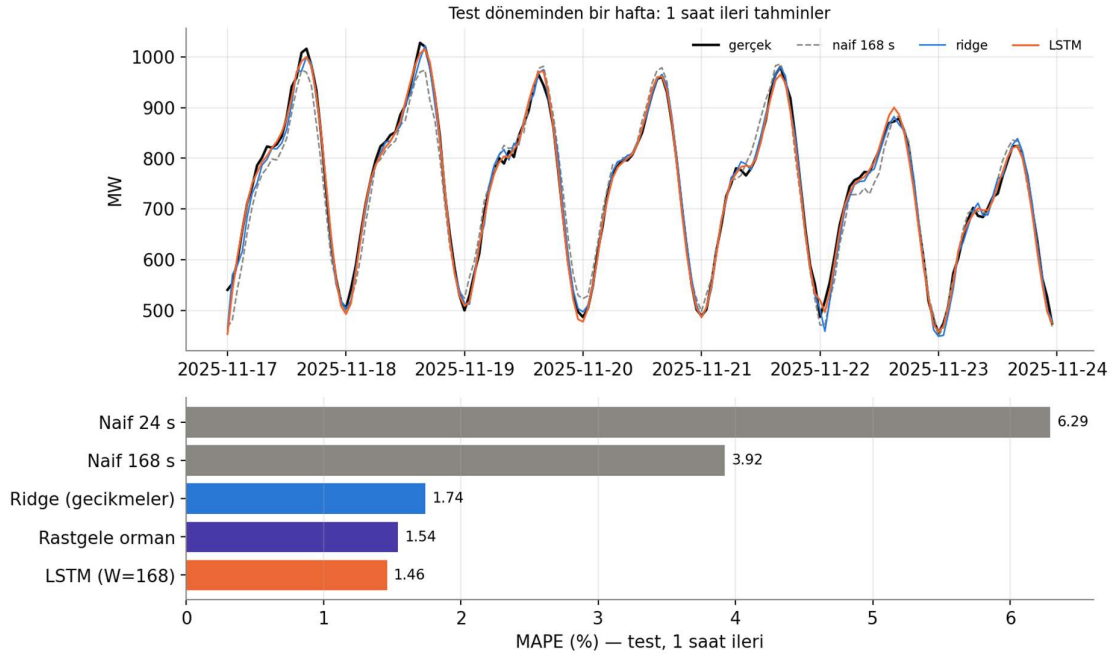
Soru: $F = 5$ özellikli, $H = 32$ gizli boyutlu tek katmanlı LSTM'nin parametre sayısı? Ardından `Linear(32 → 1)`. Aynı H 'de sade RNN ve GRU kaç parametre?

Her kapı/aday için ağırlık $H \times (F+H) +$ sapma $H = 32 \cdot 37 + 32 = 1216$; dört tanesi ($f, i, \tilde{c}, o$) $\rightarrow 4864$. (PyTorch iki ayrı sapma vektörü tutar: b_{ih} ve b_{hh} ; bu yüzden $\text{sum}(p.\text{numel}()) = 4 \cdot (32 \cdot 5 + 32 \cdot 32 + 32 + 32) = 4992$.) Çıkış katmanı 33. Toplam ≈ 5000 : 11. haftanın MLP'sinden (235 k) çok küçük — dizi modelleri ağırlık paylaştığı için kompakttır. Sade RNN tek 'kapı' $\rightarrow 1216$ (PyTorch 1248); GRU üç $\rightarrow 3648$ (PyTorch 3744).

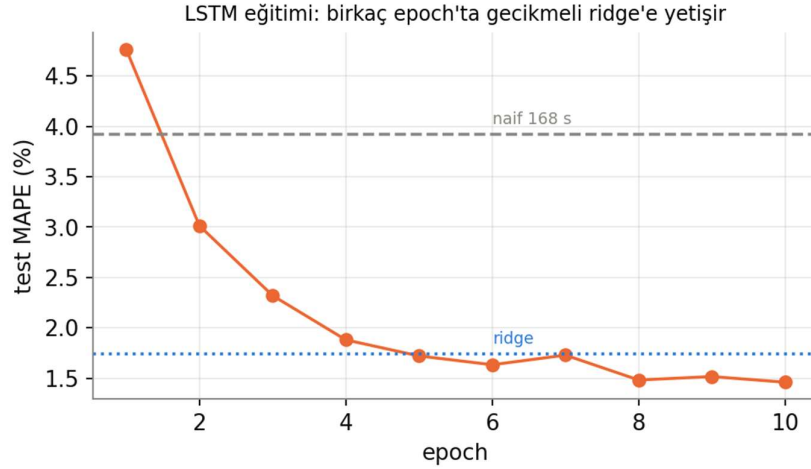
```
import torch.nn as nn
for ad, m in (("RNN", nn.RNN(5, 32)), ("GRU", nn.GRU(5, 32)), ("LSTM", nn.LSTM(5, 32))): print(ad,
sum(p.numel() for p in m.parameters()))
print("elle LSTM:", 4*(32*(5+32) + 32), "PyTorch (çift sapma):", 4*(32*5 + 32*32 + 64))
```

13.8 Uygulama: Yük Tahmini — Naif $\rightarrow$ Ridge $\rightarrow$ Orman $\rightarrow$ LSTM

Defterde aynı test dönemi (son 65 gün) üzerinde dört yaklaşımı karşılaştırıyoruz. Sonuçlar (1 saat ileri, MAPE): mevsimsel naif 24 s $\approx$ %6.3, naif 168 s $\approx$ %3.9, gecikme özellikli ridge $\approx$ %1.7, rastgele orman $\approx$ %1.5, LSTM ($W = 168, H = 32, 10$ epoch, CPU'da ~ 15 s) $\approx$ %1.5. Dersin mesajı iki katlıdır. Birincisi: doğru özelliklerle doğrusal bir model, tabanı 2.5 kat iyileştirir; LSTM'nin ek katkısı bu veride küçüktür — çünkü serinin yapısı (günlük/haftalık döngü, sıcaklık) elle özelliklerle zaten yakalanır. İkincisi: LSTM'nin avantajı özellik tasarımı gerektirmemesi ve uzun/değişken dizilerde, çok değişkenli ve doğrusal olmayan etkileşimlerde (birçok sensör, karmaşık rejimler) ortaya çıkar. Hangisini seçeceğinizi ölçüm belirler: önce taban, sonra gecikmeli doğrusal model, ancak ölçülebilir kazanç varsa derin model.

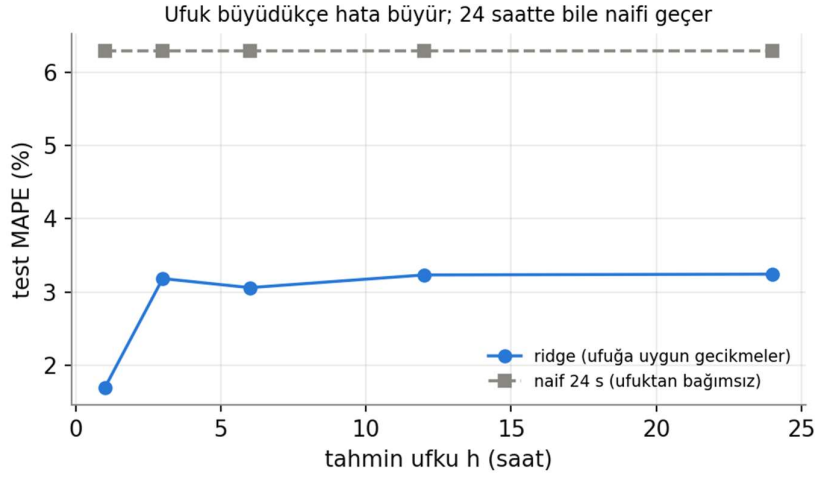


Şekil 13.8 Üst: test döneminden bir hafta, 1 saat ileri tahminler. Alt: modellerin test MAPE'si — naif 168 s güçlü bir taban; ridge ve LSTM benzer



Şekil 13.9 LSTM'nin epoch'lara göre test MAPE'si: birkaç epoch'ta ridge seviyesine iner

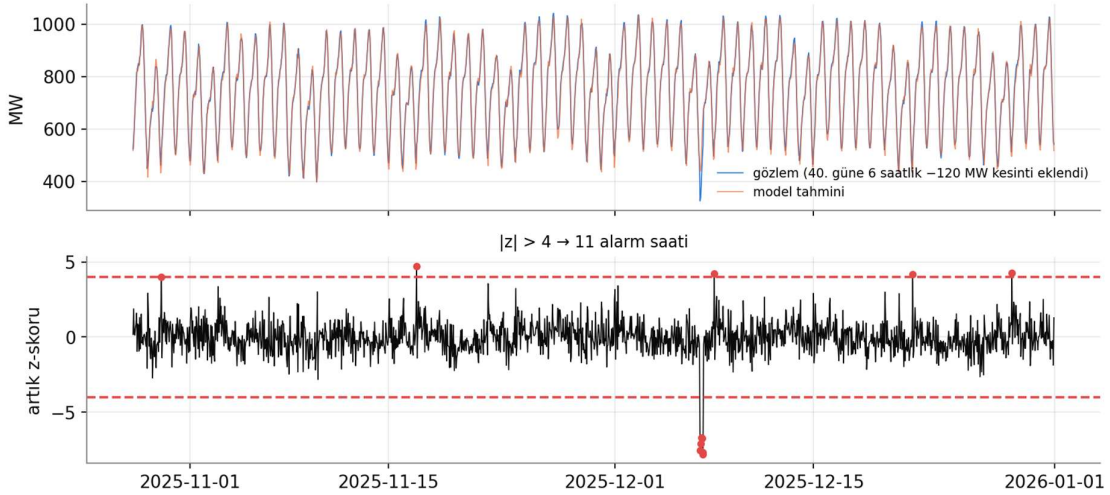
Tahmin ufku büyüdükçe iş zorlaşır: 24 saat ileri için yalnızca lag ≥ 24 kullanılabilir ve ridge MAPE'si %3'e çıkar; yine de naif 24 s'nin (%6.3) yarısıdır. Çok adımlı tahmin için üç strateji vardır: özyinelemeli (1 saat ileri modeli tahminini girdi olarak kullanıp 24 kez uygular — hata birikir), doğrudan (her ufuk için ayrı model ya da 24 çıktı tek model) ve dizi-dizi (encoder–decoder). Defterde 24 çıktı LSTM ile doğrudan strateji denir: ortalama MAPE $\approx$ %3.4, 24. saatte $\approx$ %4.2 — gecikmeli ridge (%3.2) ile aynı düzeyde. Çözümlerde özyinelemeli strateji de (%3.5) benzer çıkar; bu veride fark küçüktür, çünkü takvim ve sıcaklık gelecekte bilinir, seri düzgündür ve 1 saatlik model çok doğrudur. Gürültülü ve rejim değiştiren serilerde özyinelemeli hata birikimi belirgin biçimde büyür.



Şekil 13.10 Ridge'in test MAPE'si ufka göre: uzaklaştıkça bilgi azalır; 24 saatte bile naif tabanın yarıısı

13.9 Artıklardan Anomali Tespiti

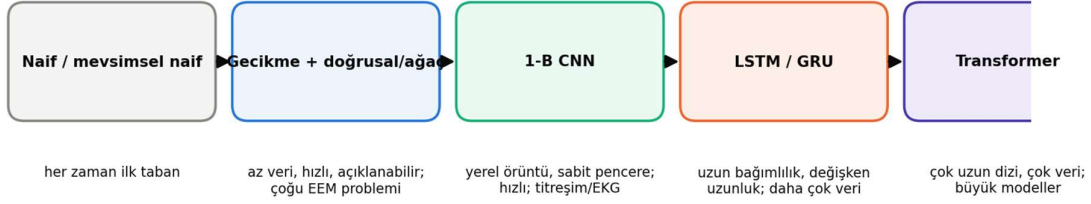
İyi bir tahmin modeli, aynı zamanda bir anomali dedektörüdür: gözlem – tahmin artığı normalde küçük ve düzensizdir; büyük bir sapma (kesinti, sensör arızası, beklenmedik talep) artığı büyütür. Eğitim artıklarının σ 'sı ile z-skoru hesaplanıp $|z| > 3-4$ eşiği alarm üretir. Şekil 13.11'de test dönemine 6 saatlik –120 MW'lık yapay bir kesinti eklenmiştir; ridge modelinin artığı kesintinin 6 saatini de yakalar; ayrıca 24 saat sonra bir 'yankı' alarmı üretir (lag24 özelliği kesintili değeri görür ve tahmini düşürür) ve 1500+ saatte 4 yanlış alarm verir. Eşik, yanlış alarm maliyeti ile kaçırma maliyetine göre seçilir (6. haftanın maliyet tabanlı eşiği). Bu 'model + artık + eşik' kalıbı 7. haftanın sınıflandırma yaklaşımının etiket gerektirmeyen tamamlayıcısıdır ve 14. haftada güvenilirlik başlığında geri gelir.



Şekil 13.11 Üst: gözlem ve tahmin; 40. güne yapay kesinti. Alt: artık z-skoru ve $|z| > 4$ eşiği

13.10 Hangi Model Ne Zaman?

Karmaşıklık ve veri ihtiyacı artar → Önce soldan başla; sağa geçmek için gerekçe (ölçülen kazanç) iste



Şekil 13.12 Zaman serisi modelleri: soldan başla, sağa geçmek için ölçülmüş kazanç iste

Yaklaşım	Güçlü yanı	Zayıf yanı	Tipik EEM kullanımı
Mevsimsel naif	Sıfır maliyet; dürüst taban	Rejim değişimini göremez	Her tahmin raporunda referans
Gecikme + ridge/orman	Hızlı, açıklanabilir, az veri	Özellik tasarımı gerekir; uzun bağımlılık sınırlı	Yük/PV/rüzgâr tahmini, sensör kalibrasyonu
1-B CNN	Yerel örüntü; hızlı; sabit pencere	Uzun bağlam için derin/geniş çekirdek	Titreşim, EKG, güç kalitesi olayları
LSTM/GRU	Değişken uzunluk; uzun bağımlılık; çok değişkenli	Yavaş eğitim; daha çok veri; ayar	Batarya durumu (SoC/SoH), kalan ömür (RUL), konuşma
Transformer	Çok uzun bağlam, paralel eğitim	Veri ve hesap iştahı	Büyük ölçekli tahmin, dil modelleri (14. hafta)

Tablo 13.3 Zaman serisi yaklaşımlarının karşılaştırması

13.11 Proje Ara Teslimi

Bu hafta proje ara teslimi vardır: (1) problem tanımı ve başarı ölçütü (hangi metrik, hangi taban), (2) verinin kaynağı, boyutu, bölme stratejisi (zaman serisi ise zaman bölmesi!), (3) EDA'dan üç şekil, (4) en az bir taban ve bir klasik modelin doğrulama sonucu, (5) kalan iki haftanın planı. Tek sayfalık PDF + çalışan defter. Değerlendirmede 'karmaşık model' değil, tabana karşı dürüst ölçüm ve doğru bölme puan getirir.

13.12 Özet ve Gelecek Hafta

- Zaman serisi = bağımlı örnekler: bölme zamanla, ölçekleme eğitimle, özellikler ufka uygun gecikmeyle.
- Naif tabanlar zorunludur; MAE/RMSE/MAPE'nin farkı ve sınırları bilinmeli.
- Gecikme + takvim + dış değişken + doğrusal/ağaç model: çoğu problemde ilk ciddi aday.
- Pencereleme (N, W, F); RNN ağırlığı zamanda paylaşır; BPTT'de gradyan kaybolur; LSTM hücre durumuyla bunu toplama zincirine çevirir.
- LSTM ile 1 s ve 24 s ileri tahmin; sonuç ridge ile benzer — model seçimi ölçümle yapılır.
- Artık + eşik = etiketsiz anomali tespiti.
- Gelecek hafta: EEM'de yapay zekâ uygulamaları, TinyML ve gömülü çıkarım, büyük dil modelleri, güvenilirlik ve etik.

Ödev: Ödev 7: sebeke_yuku.csv'de (a) TimeSeriesSplit ile 5 katlı genişleyen pencere CV kurup ridge ve orman MAPE'lerini kat kat raporlayın; (b) GRU ve 1-B CNN'i LSTM ile aynı pencerede karşılaştırın (parametre sayısı, eğitim süresi, MAPE); (c) 24 saat ileri için özyinelemeli ve doğrudan stratejileri karşılaştırın. Bonus: 5. haftanın pv_uretim.csv'sinde gündüz saatleri için 1 saat ileri PV tahmini (MAPE yerine MAE kullanın ve nedenini yazın).

Hafta 14 — EEM'de Yapay Zekâ Uygulamaları, Gömülü YZ, Büyük Dil Modelleri, Güvenilirlik ve Etik

Bölüm	Süre	İçerik
Saat 1	50 dk	EEM alt alanlarında YZ uygulama haritası; uçtan uca yaşam döngüsü (MLOps); gömülü YZ / TinyML: kaynak bütçesi, kuantizasyon, budama, damıtma, dağıtım biçimleri
Saat 2	50 dk	Büyük dil modelleri: token, öz-dikkat, transformer, ön eğitim → RLHF, istem, RAG, halüsinasyon; mühendis için kullanım ilkeleri. Güvenilirlik: kalibrasyon, belirsizlik, dağılım kayması, düşmanca örnekler
Saat 3	50 dk	Uygulama: motor_ariza modelini int8'e indirip C dizisine dökme, ONNX; dikkat elle; kalibrasyon ve kayma deneyi; FGSM; adillik raporu. Etik ve düzenleme: AB YZ Yasası, KVKK, model kartı

Tablo 14.1 Haftanın ders planı

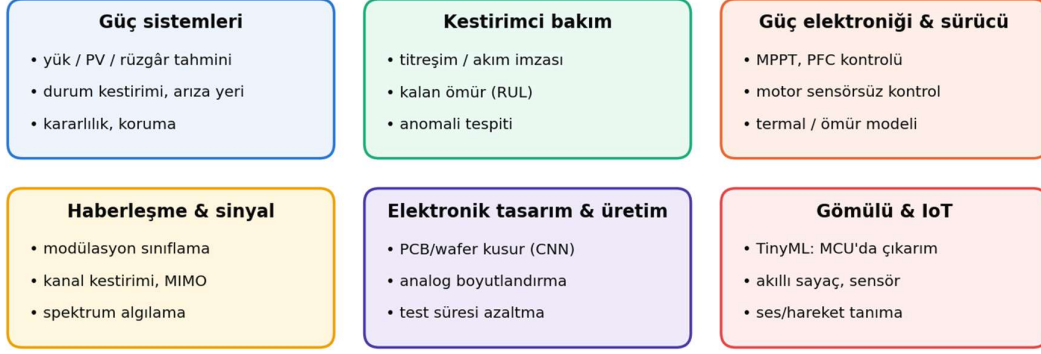
14.1 Kazanımlar

- EEM alt alanlarında hangi YZ aracının hangi problemi çözdüğünü örneklerle eşleştirir.
- Bir modelin eğitim sonrası yaşam döngüsünü (dağıtım, izleme, yeniden eğitim) ve kayma kavramını açıklar.
- Mikrodenetleyici bütçesi (flash, RAM, MAC, enerji) hesaplar; int8 kuantizasyonu elle uygular; budama ve damıtmayı bilir.
- Öz-dikkati elle hesaplar; LLM'lerin nasıl eğitildiğini ve sınırlarını (halüsinasyon, gizlilik) bilir; doğrulayarak kullanır.
- Kalibrasyonu ölçer (güvenilirlik diyagramı, Brier, ECE); belirsizlik kestirimi ve düşmanca örneklere karşı temel önlemleri bilir.
- Adillik, açıklanabilirlik, gizlilik ve düzenleyici çerçeveyi (AB YZ Yasası, KVKK) bir EEM projesine uygular; model kartı yazar.

14.2 EEM'de YZ: Uygulama Haritası

On üç haftada öğrendiğimiz araçların her biri elektrik-elektronik mühendisliğinin bir köşesinde günlük olarak kullanılıyor. Güç sistemlerinde yük, PV ve rüzgâr tahmini (5. ve 13. haftalar) piyasa teklifleri ve şebeke işletmesinin temelidir; durum kestirimi ve arıza yeri tespiti sınıflandırma/regresyonla, kararlılık değerlendirmesi ise simülasyon verisiyle eğitilmiş modellerle hızlandırılır. Kestirimci bakımda titreşim ve akım imzasından arıza sınıflama (7. hafta) ve kalan ömür tahmini (LSTM) endüstri standardı hâline geldi. Güç elektroniğinde MPPT ve motor sürücü kontrolünde sinir ağları doğrusal olmayan haritaları öğrenir; termal ömür modelleri regresyondur. Haberleşmede modülasyon sınıflama ve kanal kestirimi CNN/RNN ile, 5G/6G'de MIMO hüzmeye yönetimi öğrenmeyle yapılır. Elektronik üretiminde PCB/wafer kusur denetimi (12. hafta) görüntü CNN'lerinin en yaygın endüstriyel kullanımıdır. Gömülü sistemlerde ise bu modellerin mikrodnetleyiciye sığdırılması (TinyML) ayrı bir uzmanlık alanıdır — bugünün ilk teknik konusu.

Haftalarda gördüğümüz araçlar — regresyon, sınıflandırma, kümeleme, CNN, LSTM — EEM'in her alt alanında



Ortak örüntü: sensör/sayaç verisi → özellik ya da ham sinyal → model → karar/tahmin → izleme

Şekil 14.1 EEM alt alanlarında YZ uygulamaları ve ders boyunca gördüğümüz karşılıkları

Uygulama	Veri	Yöntem (hafta)	Başarı ölçütü	Dikkat
Gün öncesi yük tahmini	saatlik yük, sıcaklık, takvim	gecikme + ridge/LSTM (5, 13)	MAPE %1–3, naif tabana beceri	zaman bölmesi; tatil/rejim
Rulman arızası sınıflama	titreşim 2048 örnek	özellik + orman / 1-B CNN (7, 12)	makro-F1	aynı makine sızıntısı; sınıf dengesizliği
PCB kusur denetimi	32×32... görüntü	CNN + artırma (12)	sınıf başına geri çağırma	az kusur örneği; aydınlatma kayması
Batarya SoH kestirimi	çevrim gerilim/akım eğrileri	regresyon / LSTM (5, 13)	MAE (%)	yeni hücre kimyası → kayma
Modülasyon sınıflama	IQ örnekleri	1-B CNN (12)	SNR'a göre doğruluk	kanal koşulları eğitimde temsil edilmeli
Akıllı sayaç profil kümeleme	24 saatlik profiller	k-means / PCA (8)	silhouette, iş yorumu	gizlilik (KVKK)

Tablo 14.2 Örnek uygulamalar ve ders karşılıkları

14.3 Modelin Ömrü: Dağıtım, İzleme, Yeniden Eğitim

4. haftanın hattı (problem → veri → taban → model → doğrulama) eğitimin sonunda biter; gerçek projede iki adım daha vardır: dağıtım ve izleme. Dağıtım, modelin karar verdiği yere göre değişir: bir sunucuda web servisi (bulut), fabrika bilgisayarında yerel süreç (edge PC), ya da sensörün yanındaki mikrodenetleyici (TinyML). Her biri farklı gecikme, bant genişliği, enerji ve gizlilik dengesi sunar: bulut esneklik ama bağlantı ve gecikme gerektirir; MCU'da çıkarım milisaniyeler ve milivatlar ister, veri cihazdan çıkmaz. İzleme, modelin üretimde sessizce bozulmasını yakalar: girdi dağılımı kayması (covariate shift — yaz gelir, yeni motor tipi eklenir, sensör yaşlanır) ve hedef ilişkisinin değişmesi (concept drift — bakım politikası değişir, tarife değişir). Şekil 14.8'de sıcaklık dağılımı 12–20 °C kayınca orman modelinin AUC'si (sıralama) neredeyse değişmiyor ama sabit 0.3 eşliğinde F1 0.63'ten 0.39'a düşüyor ve alarm oranı %15'ten %47'ye çıkıyor — model her şeyi 'sıcak, arızalı' görüyor ve kimse hata mesajı almıyor. Bu, AUC'nin tek başına yetersiz olduğunu da gösterir: eşikli karar, kalibrasyon ve alarm oranı izlenmelidir. Basit izleme: girdi özelliklerinin eğitim dağılımıyla karşılaştırılması (ortalama/σ, KS testi), tahmin dağılımı, gecikmeli gelen gerçek etiketlerle

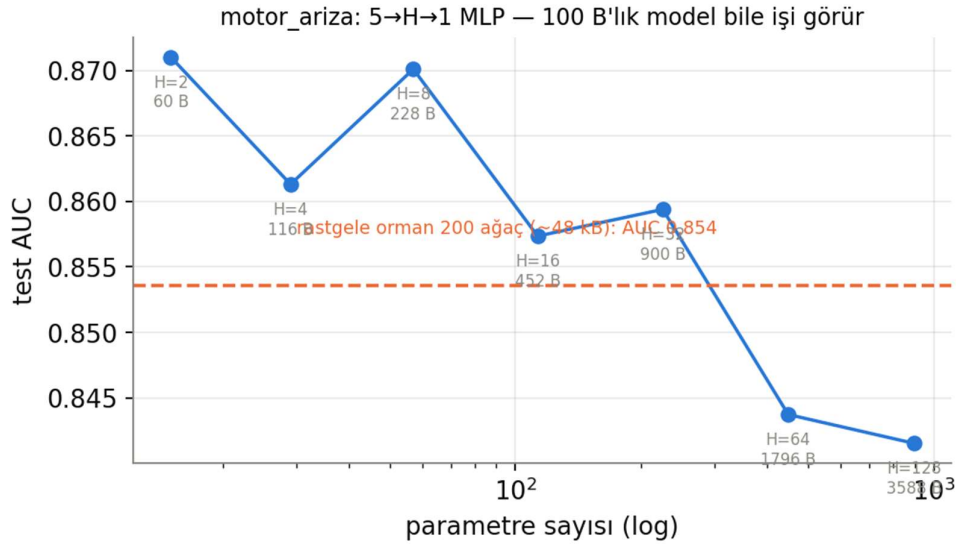
performans; eşik aşıldıca yeniden eğitim. Bu döngü MLOps olarak adlandırılır; sürüm kontrolü (veri + kod + model), tekrar üretilebilirlik (sabit tohum, kayıtlı hiperparametre) ve otomatik testler için omurgasıdır.



Şekil 14.2 Modelin yaşam döngüsü: eğitim sonu bir başlangıçtır; kayma ölçülünce döngü başa döner

14.4 Gömülü YZ / TinyML: Modeli Mikrodenetleyiciye Sığdırmak

Tipik bir Cortex-M4 mikrodenetleyici 100–200 MHz, 256 KB–1 MB flash, 64–256 KB RAM ile çalışır ve pilde yıllarca dayanması beklenir. Bir sinir ağı burada üç bütçeye sığmalıdır: bellek (parametreler flash'ta, ara aktivasyonlar RAM'de), hesap (çarpma-toplama sayısı MAC → gecikme) ve enerji (her MAC ve bellek erişimi birkaç pikojul). Dört teknik bu bütçeyi sağlar. (1) Küçük mimari: motor_ariza için 5→8→1 MLP 57 parametredir; Şekil 14.3 daha büyük ağın AUC'ye katkısı yapmadığını gösteriyor. (2) Kuantizasyon: float32 yerine int8 ağırlık/aktivasyon — 4 kat küçük, tamsayı MAC birimi 2–4 kat hızlı, doğruluk kaybı genellikle < %1 (Örnek 14.1). (3) Budama: küçük ağırlıkları sıfırlayıp seyrek depolama; %50–90 sıfır ile doğruluk çoğu kez korunur (Alıştırma). (4) Damıtma: büyük 'öğretmen' ağı yumuşak olasılıklarıyla küçük 'öğrenci' eğitilir; öğrenci sert etiketlerle eğitildiğinden daha iyi olur. Dağıtım biçimleri: ONNX (çerçeveden bağımsız format, onnxruntime ile PC/edge'de), TensorFlow Lite Micro ve CMSIS-NN (Arm MCU çekirdekleri), ya da defterde yapacağımız gibi ağırlıkları C dizisi olarak dökmek ve çıkarımı 20 satırlık C ile yazmak — küçük MLP'ler için en şeffaf yol.



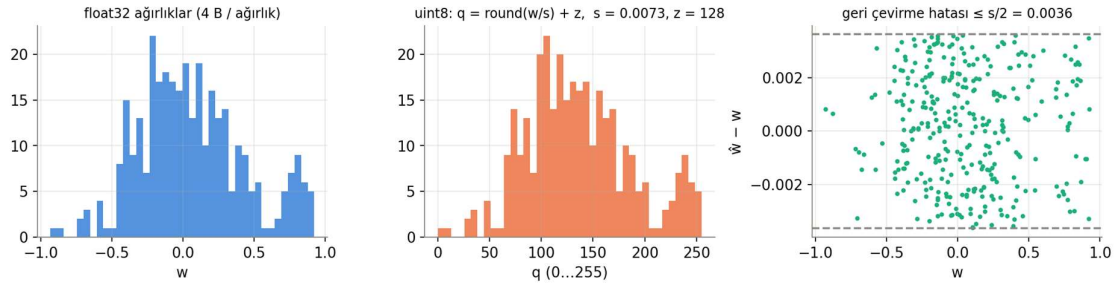
Şekil 14.3 motor_ariza: gizli boyut H'ye göre parametre sayısı ve test AUC — 57 parametre (228 B) yeterli; orman ~50 kB

Örnek 14.1 — int8 kuantizasyon elle

Soru: Bir katmanın ağırlıkları $w = [-0.62, -0.10, 0.05, 0.31, 0.88]$. Asimetrik uint8 kuantizasyon için ölçek s ve sıfır noktası z 'yi bulun; q değerlerini ve geri çevirme hatasını hesaplayın. Bellek kazancı nedir?

$s = (\max - \min)/255 = (0.88 + 0.62)/255 = 0.005882$; $z = \text{round}(-\min/s) = \text{round}(0.62/0.005882) = \text{round}(105.4) = 105$. $q = \text{round}(w/s) + z$: $-0.62 \rightarrow -105.4 + 105 = 0$ (yuvarlama: $-105 + 105 = 0$); $-0.10 \rightarrow -17 + 105 = 88$; $0.05 \rightarrow 8.5 \rightarrow 9$ (banker yuvarlaması 8) $+ 105 = 113/114$; $0.31 \rightarrow 53 + 105 = 158$; $0.88 \rightarrow 150 + 105 = 255$. Geri çevirme $\hat{w} = (q - z) \cdot s$: $[-0.6176, -0.1000, 0.0529, 0.3118, 0.8824]$; hata $\leq s/2 = 0.0029$. Bellek: $5 \cdot 4 = 20 \text{ B} \rightarrow 5 \cdot 1 + (s, z: 4 + 1 \text{ B}) = 10 \text{ B}$; büyük katmanlarda ≈ 4 kat. Simetrik int8 ($z = 0, s = \max|w|/127$) daha basittir ve ağırlıklar için yaygındır; aktivasyonlar için asimetrik (ReLU çıktısı ≥ 0) tercih edilir. Hassas nokta: dağılımdaki tek bir uç değer s'yi büyütüp geri kalan ağırlıkların çözünürlüğünü bozar — bu yüzden kanal başına (per-channel) ölçek kullanılır.

```
w = np.array([-0.62, -0.10, 0.05, 0.31, 0.88]); s = (w.max() - w.min())/255; z = int(round(-w.min()/s))
q = np.clip(np.round(w/s) + z, 0, 255).astype(np.uint8); w_hat = (q.astype(float) - z)*s
print("s =", round(s, 6), "z =", z, "q =", q, "hata =", np.abs(w_hat - w).round(4), "≤ s/2 =", round(s/2, 4))
```



Şekil 14.4 float32 dağılımı 256 seviyeye eşlenir; hata $\leq s/2$. Uç değerler ölçeği büyütür — kanal başına ölçek çözümdür

Örnek 14.2 — MCU bütçesi: bellek, gecikme, enerji

Soru: 12. haftanın küçük CNN'i (136 196 parametre; 32×32 girdi için $\approx 2.4 \text{ M MAC}$) ve motor_ariza MLP'si ($5 \rightarrow 8 \rightarrow 1$: 57 parametre, 48 MAC) için: (a) int8'de flash gereksinimi; (b) 100 MHz, çevrim başına 1 MAC ile gecikme; (c) MAC başına 5 pJ + sabit 2 μJ uyanma ile çıkarım enerjisi; (d) saniyede 1 çıkarım ile 1000 mAh 3 V pilin ömrü (yalnızca çıkarım). Hangisi 256 KB flash'lı MCU'ya sığar?

(a) CNN: $136\,196 \text{ B} \approx 133 \text{ KB}$ (float32'de 532 KB — sığmaz; int8 sığar). MLP: 57 B. (b) CNN: $2.4 \cdot 10^6$ çevrim / $10^8 \text{ Hz} = 24 \text{ ms}$; MLP: $0.48 \mu\text{s}$ (pratikte bellek erişimi ve döngü yükü ile 2–5 kat). (c) CNN: $2.4 \cdot 10^6 \cdot 5 \text{ pJ} = 12 \mu\text{J} + 2 = 14 \mu\text{J}$; MLP: $\approx 2 \mu\text{J}$ (uyanma baskın). (d) Pil enerjisi $1 \text{ Ah} \cdot 3 \text{ V} = 3 \text{ Wh} = 10\,800 \text{ J}$. CNN: $14 \mu\text{J/s} \rightarrow 7.7 \cdot 10^8 \text{ s} \approx 24 \text{ yıl}$; MLP $\approx 170 \text{ yıl}$ — yani çıkarım enerjisi pil ömrünü belirlemez; sensör okuma, radyo ve uyku akımı belirler. TinyML'de asıl kazanç, ham veriyi radyo ile göndermek yerine (tipik 50–100 mJ/paket) yerel karar verip yalnızca alarmı göndermektir: 1000 kat enerji farkı.

```
for ad, P, MAC in (("CNN 12. hafta", 136196, 2.4e6), ("MLP motor", 57, 48)):
    print(f"{ad:14s} flash int8 {P/1024:7.1f} KB gecikme {MAC/1e8*1e3:7.3f} ms enerji {MAC*5e-12*1e6 + 2:6.2f} μJ pil ömrü {10800/(MAC*5e-12 + 2e-6)}/3.15e7:6.0f} yıl")
```

Teknik	Kazanç	Bedel	Ne zaman
Küçük mimari	Her bütçede doğrudan	Kapasite	Her zaman ilk adım — tabana bakın
int8 kuantizasyon	4× bellek, 2–4× hız	< %1 doğruluk (genelde)	Her MCU dağıtımı; kalibrasyon verisi gerekir

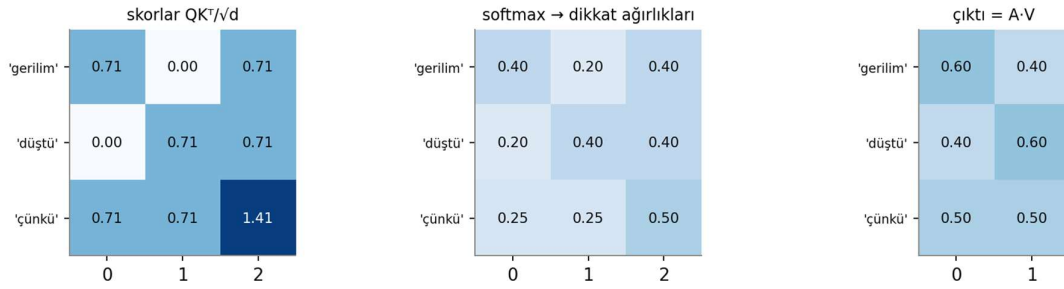
Teknik	Kazanç	Bedel	Ne zaman
Budama	%50–90 daha az ağırlık	Seyrek depolama/donanım desteği	Bellek dar, doğruluk payı var
Damıtma	Küçük model, büyük modelin bilgisi	Ek eğitim	Büyük öğretmen zaten varsa
Özellik mühendisliği + klasik model	Sinir ağı hiç gerekemeyebilir	El emeği	7. hafta: RMS/kurtosis + ağaç MCU'da mükemmel

Tablo 14.3 Modeli küçültme teknikleri

14.5 Büyük Dil Modelleri: Nasıl Çalışır, Nasıl Kullanılır

Büyük dil modelleri (LLM) 11. haftanın sınıflandırıcısının devasa bir versiyonudur: girdi, metnin token'lara (kelime parçaları) bölünmüş hâli; çıktı, sözlükteki ~100 000 token üzerinde bir softmax; kayıp, çapraz entropi; hedef, bir sonraki token. Fark ölçektedir: trilyon token metin, 10^9 – 10^{12} parametre, binlerce GPU. Mimari transformer'dır ve çekirdeği öz-dikkattir: her token, diğer tokenlardan ne kadar bilgi alacağını Q (sorgu), K (anahtar), V (değer) vektörleriyle kendisi hesaplar — $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d}) \cdot V$. RNN'nin aksine tüm tokenlar paralel işlenir ve uzak bağımlılıklar doğrudan kurulur; bedeli dizi uzunluğunun karesiyle büyüyen hesap. Ön eğitimden çıkan model bir 'tamamlayıcı'dır; talimat ayarı (SFT) ve insan geri bildirimle pekiştirmeli öğrenme (RLHF) onu yardımcıya çevirir. Kullanımda istem (prompt) tasarımı, kendi belgelerinizle zenginleştirme (RAG: soruyla ilgili parçaları bulup bağlama ekleme) ve araç çağırısı (hesap makinesi, kod çalıştırma, veri tabanı) modelin sınırlarını genişletir.

Öz-dikkat: her token diğerlerine 'ne kadar bakacağını' kendi hesaplar (Örnek 14.3)



Şekil 14.5 Öz-dikkat üç token için: skorlar, softmax ağırlıkları ve çıktı. 'çünkü' hem 'gerilim'e hem 'düştü'ye bakar

Örnek 14.3 — Öz-dikkat elle

Soru: Üç token, $d = 2$. $Q = K = [[1,0],[0,1],[1,1]]$, $V = [[1,0],[0,1],[0.5,0.5]]$. Üçüncü tokenın dikkat ağırlıklarını ve çıktısını hesaplayın.

Skorlar (3. satır): $q_3 \cdot k_1/\sqrt{2} = 1/1.414 = 0.707$; $q_3 \cdot k_2/\sqrt{2} = 0.707$; $q_3 \cdot k_3/\sqrt{2} = 2/1.414 = 1.414$. Softmax: $\exp \rightarrow [2.03, 2.03, 4.11]$, toplam 8.17 $\rightarrow [0.248, 0.248, 0.503]$. Çıktı = $0.248 \cdot [1,0] + 0.248 \cdot [0,1] + 0.503 \cdot [0.5,0.5] = [0.50, 0.50]$. Üçüncü token kendine yarı, diğer ikisine çeyrek ağırlık verir. Gerçek modellerde $d = 128$, çok başlı (multi-head) dikkat 32–128 baş, 30–100 katman; ama işlem budur. $\sqrt{d}$ ölçeği, skorların büyüüp softmax'ın 'tek sıcak'a dönüşmesini (gradyanın kaybolmasını) önler.

```
Q = K = np.array([[1, 0], [0, 1], [1, 1]]); V = np.array([[1, 0], [0, 1], [.5, .5]])
S = Q @ K.T/np.sqrt(2); A = np.exp(S)/np.exp(S).sum(1, keepdims=True); print(A.round(3)); print((A @ V).round(3))
```

Transformer + öz-dikkat; parametre 10^9 - 10^{12} ; 11. haftanın CE kaybı, 10. haftanın Adam'ı — ölçek farklı, ilkeler aynı



Mühendis için: kod ve doküman yardımı, veri temizleme, literatür özeti — ama her sayısal/teknik iddia DOĞRULANIR (halüsinasyon)

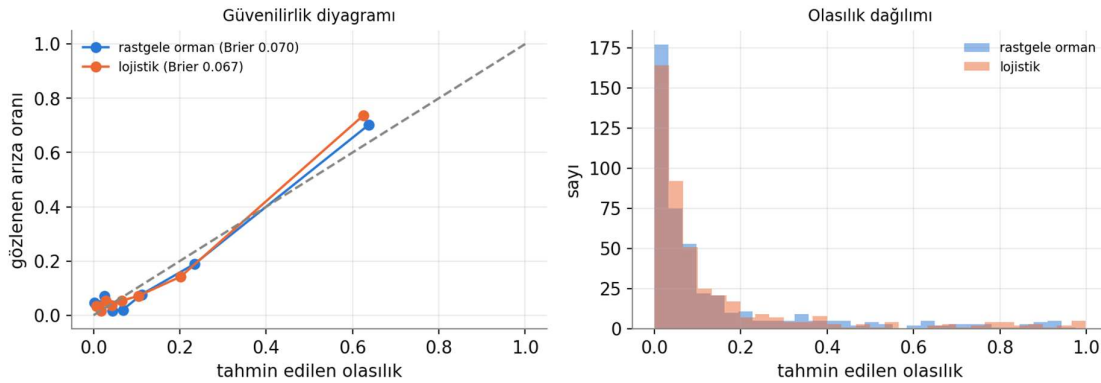
Şirket verisi ve KVKK: hangi veriyi dışarı gönderdiğinizi bilin; yerel/açık modeller (Llama, Qwen, Mistral) bir seçenek

Şekil 14.6 LLM boru hattı: ön eğitim → talimat ayarı → RLHF → kullanım (istem, RAG, araç). İlkeler bu dersin ilkeleridir

Mühendis için kullanım ilkeleri: (1) LLM bir olasılıksal metin üreticisidir, doğruluk garantisi yoktur; sayısal sonuçları, standart referanslarını, formülleri ve kod çıktısını bağımsız doğrulayın (halüsinasyon). (2) En verimli kullanım alanları: kod taslağı ve hata ayıklama, dokümantasyon, veri temizleme betikleri, literatür özetleme, farklı bakış açıları üretme. (3) Gizlilik: şirket verisini, kişisel veriyi (KVKK) ve müşteri bilgisini bulut servisine göndermeden önce politika ve sözleşmeye bakın; açık ağırlıklı modeller (Llama, Qwen, Mistral) yerel çalıştırılabilir. (4) İstem tasarımı: bağlam ver, rol ve çıktı formatı belirt, örnek göster, adım adım düşünmesini iste; ama bunlar doğruluğu garanti etmez. (5) Enerji ve maliyet: bir LLM çağırısı, bir arama sorgusundan onlarca kat daha fazla enerji harcar; eğitim yüz GWh mertebesindedir — enerji mühendisi bu ölçeği bilmelidir.

14.6 Güvenilirlik: Kalibrasyon, Belirsizlik, Kayma, Düşmanca Örnekler

Bir modelin doğruluğu tek başına 'güvenilir' olduğunu göstermez. Kalibrasyon, tahmin edilen olasılıkların gerçek sıklıklarla uyuşmasıdır: 'arıza olasılığı 0.8' dediği motorların %80'i gerçekten arızalanmalıdır. Güvenilirlik diyagramı (Şekil 14.7) bunu kutu kutu çizer; ECE (beklenen kalibrasyon hatası) kutulardaki |sıklık – ortalama tahmin| farklarının ağırlıklı ortalamasıdır; Brier skoru $(p - y)^2$ ortalaması hem ayırıştırma hem kalibrasyonu ölçer. Sinir ağırları ve boosting sıklıkla aşırı güvenlidir; lojistik regresyon doğal olarak kalibredir; düzeltme için Platt ölçekleme (lojistik) ya da izotonik regresyon doğrulama verisiyle uygulanır (CalibratedClassifierCV). Kalibrasyon, 6. haftanın maliyet tabanlı eşik için ön koşuldur: olasılıklar yanlışsa eşik de yanlıştır.



Şekil 14.7 motor_ariza: orman ve lojistik regresyonun güvenilirlik diyagramı ve olasılık dağılımı

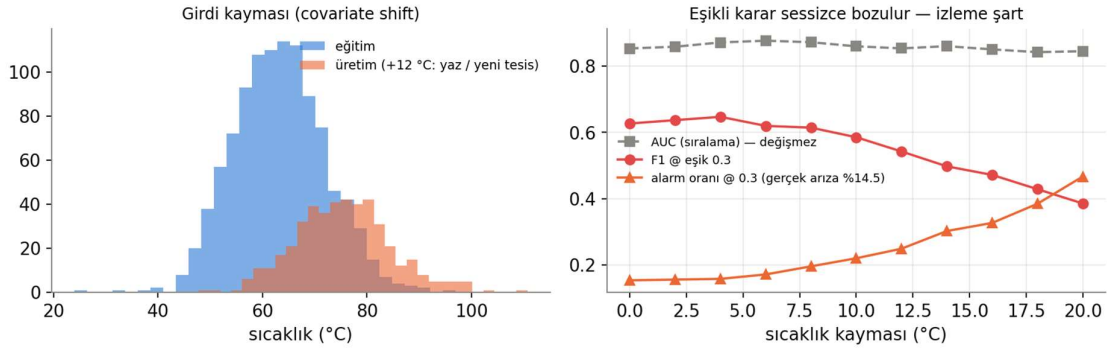
Örnek 14.4 — ECE hesabı

Soru: Doğrulama kümesi 4 olasılık kutusuna ayrıldı. Kutu 1: 400 örnek, ort. tahmin 0.05, gerçek arıza oranı 0.03. Kutu 2: 300, 0.25, 0.20. Kutu 3: 200, 0.55, 0.65. Kutu 4: 100, 0.85, 0.70. ECE'yi hesaplayın ve modelin nerede fazla/az güvenli olduğunu söyleyin.

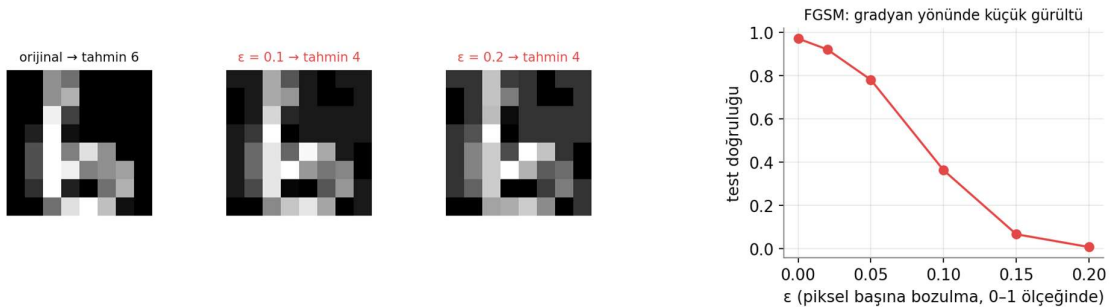
$ECE = \sum (n_k/N) \cdot |sıklık - tahmin| = (400 \cdot 0.02 + 300 \cdot 0.05 + 200 \cdot 0.10 + 100 \cdot 0.15)/1000 = (8 + 15 + 20 + 15)/1000 = 0.058$. Model düşük kutularda hafif aşırı güvenli (0.05 diyor, 0.03 çıkıyor — kabul edilebilir), orta kutuda az güvenli (0.55 diyor, 0.65 çıkıyor), yüksek kutuda belirgin aşırı güvenli (0.85 → 0.70). 6. haftadaki maliyet eşiği 0.3 civarındaysa kutu 2–3 sınırındaki hata karar kalitesini doğrudan etkiler; izotonik düzeltme bu eğriyi köşegene çeker.

```
n = np.array([400, 300, 200, 100]); tah = np.array([.05, .25, .55, .85]); sik = np.array([.03, .20, .65, .70])
print("ECE =", round(np.sum(n*np.abs(sik - tah))/n.sum(), 3))
```

Belirsizlik iki türdür: veriden gelen (aleatorik — ölçüm gürültüsü, azaltılamaz) ve modelden gelen (epistemik — az veri, azaltılabilir). Epistemik belirsizlik için pratik araçlar: farklı tohumlarla eğitilmiş birkaç modelin (topluluk) tahmin dağılımı; MC dropout (test zamanında dropout açık, 20–50 geçişin varyansı); ormanda ağaçlar arası varyans; ve dağıtımsız garanti veren konformal tahmin (doğrulama artıklarının %90 kantili → tahmin aralığı). Girdi eğitim dağılımının dışındaysa (OOD) model bunu bilmez ve güvenle yanlış yapar; eğitim özellik dağılımına uzaklık (Mahalanobis, z-skor) basit bir OOD dedektörüdür. Şekil 14.8 dağılım kaymasını gösteriyor: sıcaklık kayıncı sıralama (AUC) korunsa da eşikli karar bozulur, alarm oranı patlar. Düşmanca örnekler (Şekil 14.9): gradyan yönünde, insanın fark etmediği ϵ büyüklüğünde bozulma (FGSM: $x + \epsilon \cdot \text{sign}(\nabla_x L)$) doğruluğu çökertir. Kamera tabanlı denetim, ses komutu ve biyometrik sistemlerde gerçek bir tehdittir; düşmanca eğitim, girdi ön işleme ve topluluklar kısmi savunmadır. Emniyet-kritik sistemlerde (koruma rölesi, sürücü) YZ modeli klasik bir güvenlik katmanının (sınır kontrolü, fizik tabanlı doğrulama, insan onayı) arkasında çalışmalıdır.



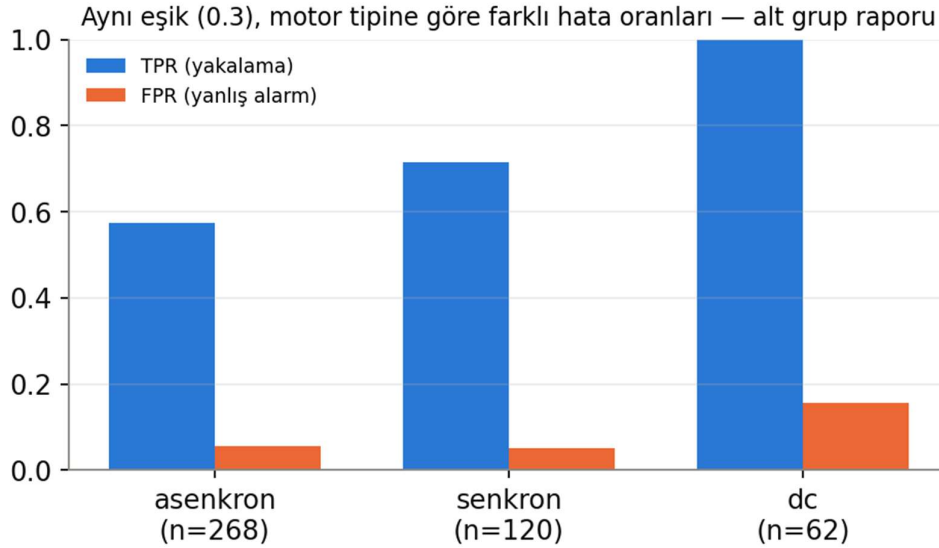
Şekil 14.8 Girdi kayması: eğitim (mavi) ve üretim (turuncu) sıcaklık dağılımı; AUC değişmese de sabit eşikte F1 düşer, alarm oranı %15'ten %47'ye çıkar



Şekil 14.9 FGSM: rakam görüntüsüne $\epsilon \cdot \text{sign}(\text{gradyan})$ eklemek MLP'yi yanıltır; doğruluk ϵ ile çöker

14.7 Etik, Adillik, Gizlilik ve Düzenleme

YZ sistemleri karar verdikçe, kararların kime nasıl dokunduğu mühendisin sorumluluğuna girer. Adillik: aynı model farklı alt gruplarda farklı hata oranları verebilir — Şekil 14.10'da aynı eşik motor tiplerine göre farklı yakalama/yanlış alarm oranı üretiyor; insanlar söz konusu olduğunda (kredi, işe alım, sağlık) bu doğrudan ayrımcılıktır. Çözümün ilk adımı alt grup raporu: performansı gruplara göre ayrı ölçmek; sonra veri dengesi, grup başına eşik ya da adillik kısıtlı eğitim. Açıklanabilirlik: ağaç önemleri, permütasyon önemi, SHAP değerleri, saliency (12. hafta) — bir kararın gerekçesi sorulduğunda 'model öyle dedi' cevap değildir. Gizlilik: akıllı sayaç verisi evde kimin ne zaman olduğunu açığa çıkarır; KVKK ve GDPR kişisel veriyi işleme amacı, süresi ve rızasıyla sınırlar; anonimleştirme, toplulaştırma ve federated learning (veri cihazda kalır, yalnızca model güncellemeleri paylaşılır) araçlardır. Düzenleme: AB Yapay Zekâ Yasası (2024) risk temelli yükümlülükler getirir; elektrik şebekesi gibi kritik altyapıda kullanılan YZ 'yüksek risk' sınıfındadır ve risk yönetimi, veri yönetimi, teknik dokümantasyon, insan gözetimi, doğruluk/sağlamlık gereksinimleri vardır. Emniyet-kritik sistemlerde IEC 61508 / ISO 26262 gibi fonksiyonel güvenlik standartları öğrenen bileşenlerle nasıl bağdaşacağı hâlâ aktif bir konudur. Sürdürülebilirlik: büyük model eğitimi yüz GWh ve tonlarca CO₂e; çıkarım ölçeklendiğinde toplam daha büyük — enerji mühendisleri hem tüketici hem çözüm tarafındadır (şebeke esnekliği, veri merkezi yükü).



Şekil 14.10 Aynı eşik, motor tipine göre farklı TPR/FPR — alt grup raporu her projede olmalı

AB Yapay Zekâ Yasası (2024): risk temelli sınıflar — EEM'in birçok uygulaması 'yüksek risk' katmanına düşer



Şekil 14.11 AB YZ Yasası'nın risk katmanları; kritik altyapı uygulamaları yüksek risk

Belge / ilke	İçerik	EEM projesinde karşılığı
Model kartı	Amaç, veri, metrikler (alt gruplarla), sınırlar, etik değerlendirme	Rulman modeli: hangi makine/rpm aralığı için geçerli, hangi sınıfta zayıf
Veri sayfası (datasheet)	Verinin kaynağı, toplama koşulları, eksikler, lisans	Sensör tipi, örnekleme hızı, etiketleme yöntemi
İnsan gözetimi	Model önerir, insan onaylar; geçersiz kılma yolu	Bakım mühendisi alarmı onaylar; koruma rölesi klasik mantık
Kayıt ve izlenebilirlik	Tahmin, girdi ve sürüm kaydı	Her kararın hangi model sürümüyle verildiği
Alt grup raporu	Grup başına TPR/FPR, MAE	Motor tipi, tesis, mevsim

Tablo 14.4 Sorumlu YZ için belgeler ve pratikler

14.8 Özet ve Gelecek Hafta

- EEM'in her alt alanında ders araçlarının doğrudan karşılığı var; ortak örüntü sensör → model → karar → izleme.
- Modelin ömrü dağıtımla başlar; kayma ölçülmeli, yeniden eğitim planlanmalı.
- TinyML: küçük mimari, int8 kuantizasyon (4×), budama, damıtma; bütçe = flash, RAM, MAC, enerji; asıl kazanç radyo yerine yerel karar.
- LLM = ölçeklenmiş sonraki-token sınıflandırıcısı + öz-dikkat; doğrulayarak kullan, gizliliği gözet.
- Güvenilirlik: kalibrasyon (ECE, Brier), belirsizlik (topluluk, MC dropout, konformal), kayma, düşmanca örnek; emniyet katmanı YZ'nin önünde.
- Etik: alt grup raporu, açıklanabilirlik, KVKK, AB YZ Yasası; model kartı yazın.
- Gelecek hafta: proje sunumları, dönem özeti ve final hazırlığı.

Ödev: Ödev 8 (final projesiyle birlikte): Projenizin modeli için (a) bir model kartı (1 sayfa: amaç, veri, metrikler alt gruplarla, sınırlar, riskler); (b) kalibrasyon eğrisi ve ECE; (c) en az bir kayma senaryosunda performans; (d) MCU/edge'e taşınacaksa bellek–gecikme–enerji bütçesi; (e) projede bir LLM kullandıysanız nasıl doğruladığının kaydı.

Hafta 15 — Proje Sunumları, Dönem Özeti ve Final Hazırlığı

Bölüm	Süre	İçerik
Saat 1	50 dk	Proje sunumları (grup başına 10 dk: 7 dk sunum + 3 dk soru); akran değerlendirme formu
Saat 2	50 dk	Proje sunumları (devam); genel geri bildirim: ortak hatalar, iyi örnekler
Saat 3	50 dk	Dönem özeti: kavram haritası ve yöntem seçim ağacı; formül ve ölçüt özeti; örnek final soruları ve çözümleri; final formatı

Tablo 15.1 Haftanın ders planı

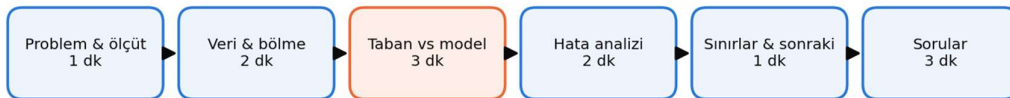
15.1 Kazanımlar

- Bir YZ projesini problem, veri, taban, model, doğrulama, sınırlar başlıklarıyla 10 dakikada sunar ve savunur.
- Başka bir projeyi rubrikle değerlendirir; 'tabanı geçti mi, bölme doğru mu' sorularını sorar.
- Dönem boyunca görülen yöntemleri tek bir seçim ağacında konumlandırır.
- Temel formülleri (kayıp, gradyan, ölçütler, kuantizasyon, dikkat) elle uygular; final formatını ve beklentileri bilir.

15.2 Proje Sunumları

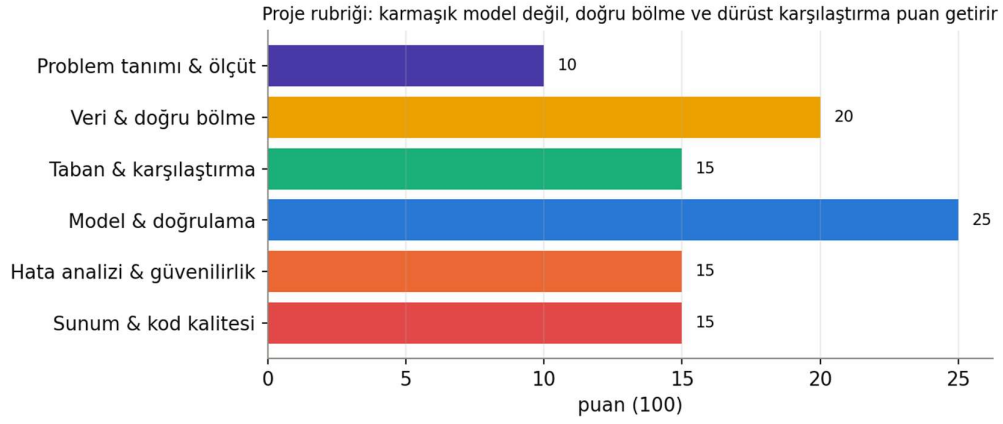
Sunumlar 4. haftada verilen ve 13. haftada ara teslimi yapılan projelerin son hâlidir. Her grup 10 dakika alır: 7 dakika sunum, 3 dakika soru. Sunum yapısı Şekil 15.1'dedir; her dakikanın bir mesajı vardır ve merkezdeki soru şudur: 'Tabanı geçtiniz mi, ne kadar, hangi bölme ile ölçtünüz ve model nerede başarısız oluyor?' Dinleyiciler akran değerlendirme formunu (Tablo 15.2) doldurur; puanın %20'si akran, %80'i öğretim elemanı değerlendirmesidir. Kod ekran görüntüsü yerine sonuç grafiği; 'yapay zekâ kullandık' yerine 'ridge regresyonu naif tabandan %38 daha az MAE verdi, zaman bölmesiyle' cümlesi beklenir.

10 dakikalık proje sunumu — her dakika bir mesaj: 'tabanı geçtik mi, neden, nerede başarısız?' ana soru



Slayt: ≤ 8 . Her sonuç için: ölçüt, taban, bölme yöntemi aynı slaytta. Kod ekran görüntüsü yok — sonuç grafiği var.

Şekil 15.1 10 dakikalık sunum yapısı — 'taban vs model' bölümü merkezdedir



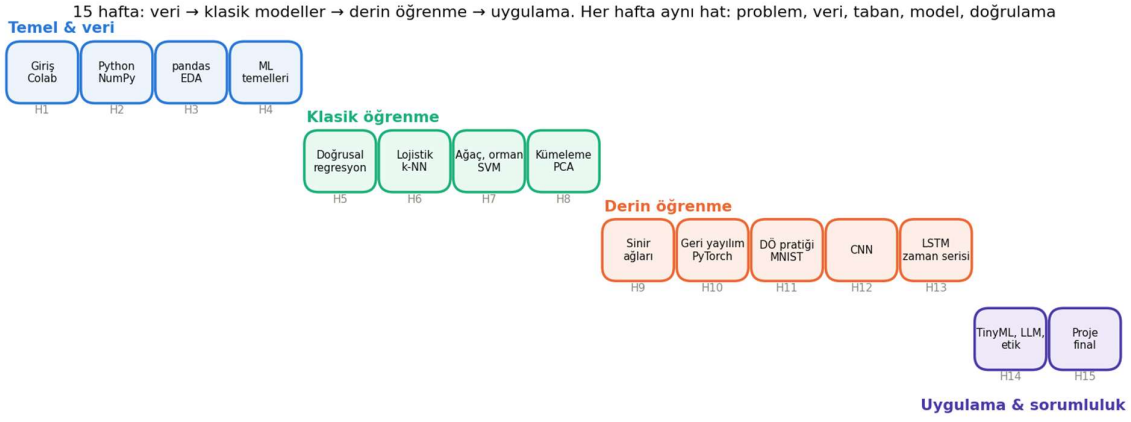
Şekil 15.2 Proje rubriği: 100 puanın dağılımı

Kalem	Puan	Tam puan için	Sık görülen eksik
Problem tanımı & ölçüt	10	Kimin, hangi kararı için; ölçüt ve hedef değer önceden yazılmış	'Doğruluk' seçilmiş ama sınıflar dengesiz
Veri & doğru bölme	20	Kaynak, boyut, temizleme; zaman/grup bölmesi gerekçeli; ölçekleme yalnızca eğitimle	Rastgele bölme ile zaman serisi; test verisiyle ölçekleme
Taban & karşılaştırma	15	Naif/mevsimsel naif/çoğunluk tabanı; en az bir klasik model; aynı test kümesi	Taban yok; farklı bölmelerle karşılaştırma
Model & doğrulama	25	CV ile hiperparametre; test yalnızca bir kez; sonuçlar $\pm$ yayılımla	Test kümesinde ayar; tek tohum tek sayı
Hata analizi & güvenilirlik	15	Karışıklık matrisi / artık grafiği; en kötü örnekler; kalibrasyon veya kayma senaryosu; alt grup	Yalnızca toplam skor
Sunum & kod	15	Yapı, süre, grafik kalitesi; çalışan defter; sabit tohum; okunur fonksiyonlar	Ekran görüntüsü kod; çalışmayan defter

Tablo 15.2 Rubrik ayrıntısı ve ekran formu başlıkları

15.3 Dönem Özeti: Kavram Haritası

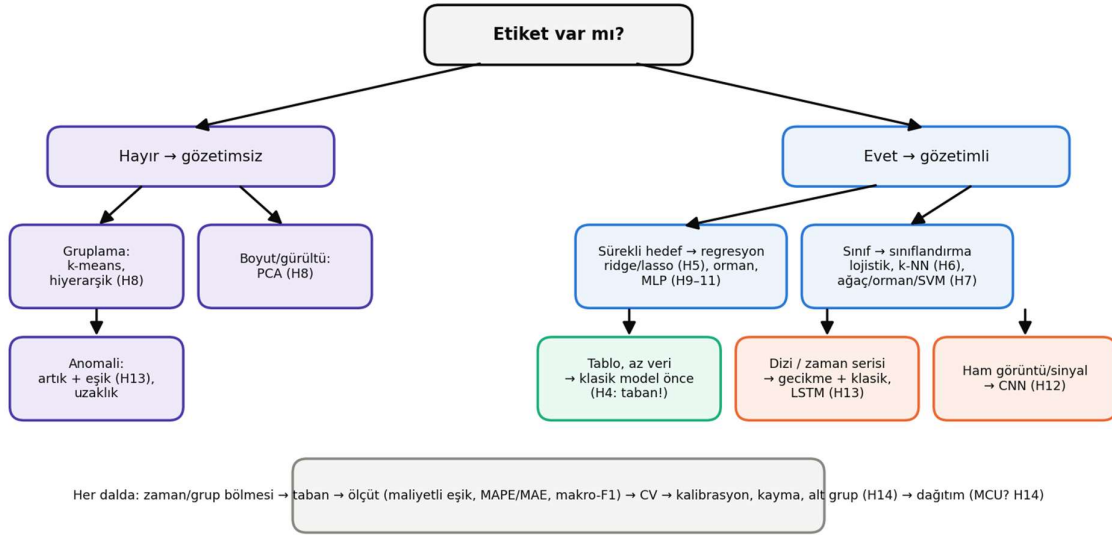
On beş haftayı dört bloğa ayırmak mümkündür (Şekil 15.3). Birinci blok araçları ve veriyi kurdu: Colab, Python/NumPy, pandas ve keşifsel analiz; sonra 4. haftada dersin omurgası olan hattı öğrendik — problem, veri, bölme, taban, model, doğrulama — ve sızıntı, aşırı öğrenme, ölçüt kavramlarını. İkinci blok klasik gözetimli ve gözetimsiz öğrenmeydi: doğrusal ve lojistik regresyon (kayıp ve gradyan iniş elle), k-NN, karar ağaçları ve ormanlar, SVM, k-means, hiyerarşik kümeleme ve PCA — hepsi EEM verilerinde (PV üretimi, motor arızası, rulman titreşimi, abone profilleri). Üçüncü blok derin öğrenmeydi: perceptron'dan çok katmanlı ağa, geri yayılımın üç formülüne, PyTorch'un autograd'ına, eğitim pratiğine (erken durdurma, dropout, batch norm), CNN'lere (PCB kusur) ve LSTM'lere (yük tahmini). Dördüncü blok modeli dünyaya taşıdı: TinyML ve kuantizasyon, LLM'lerin çalışma ilkesi, kalibrasyon ve kayma, adillik ve düzenleme. Bu blokların ortak omurgası hiç değişmedi: dürüst bölme, taban, ölçüt, hata analizi.



Şekil 15.3 Dönem haritası: dört blok, on beş hafta

15.4 Yöntem Seçim Ağacı

Bir mühendis yeni bir problemle karşılaştığında Şekil 15.4'teki soruları sırayla sorar. Etiket yoksa gözetimsiz araçlar: gruplama için k-means/hiyerarşik, boyut ve gürültü için PCA, anomali için artık ya da uzaklık tabanlı eşik. Etiket varsa hedef türü (sürekli → regresyon, sınıf → sınıflandırma) ve veri türü (tablo, görüntü/sinyal, dizi) modeli belirler. Tablo verisinde ve az veride klasik modeller (ridge, orman) ilk adaydır; ham görüntü ve sinyalde CNN; dizilerde önce gecikme özellikli klasik model, gerekirse LSTM. Hangi dalda olursak olalım aynı kontrol listesi geçerlidir: bölme türü (zaman, grup), taban, ölçüt (dengesiz sınıfta makro-F1 / maliyetli eşik; sifıra yakın hedefte MAE), çapraz doğrulama, kalibrasyon ve kayma değerlendirme, alt grup raporu, dağıtım bütçesi.



Şekil 15.4 Yöntem seçim ağacı ve her dalda geçerli ortak kontrol listesi

Problem	İlk taban	İlk ciddi model	Ölçüt	Tuzak
Yük/PV tahmini (H5, 13)	mevsimsel naif	gecikme + ridge	MAPE / MAE, beceri	rastgele bölme; sıcaklık U biçimi
Motor arızası (H6)	çoğunluk / rastgele	lojistik, orman	maliyetli eşik, AUC, recall	%14 pozitif → 'doğruluk' yanılır
Rulman sınıfı (H7)	çoğunluk	özellik + orman	makro-F1	aynı makine iki tarafta

Problem	İlk taban	İlk ciddi model	Ölçüt	Tuzak
Abone profilleri (H8)	—	k-means (k: silhouette)	iş yorumu, ARI (etiket varsa)	ölçeklenmemiş özellik
PCB kusur (H12)	çoğunluk	küçük CNN + artırma	sınıf başına recall	az kusur; aydınlatma kayması
Rakam (H11)	çoğunluk (%10)	MLP → CNN	doğruluk, karışıklık	test kümesinde ayar

Tablo 15.3 Dersin veri setleri: taban, model, ölçüt ve tuzak

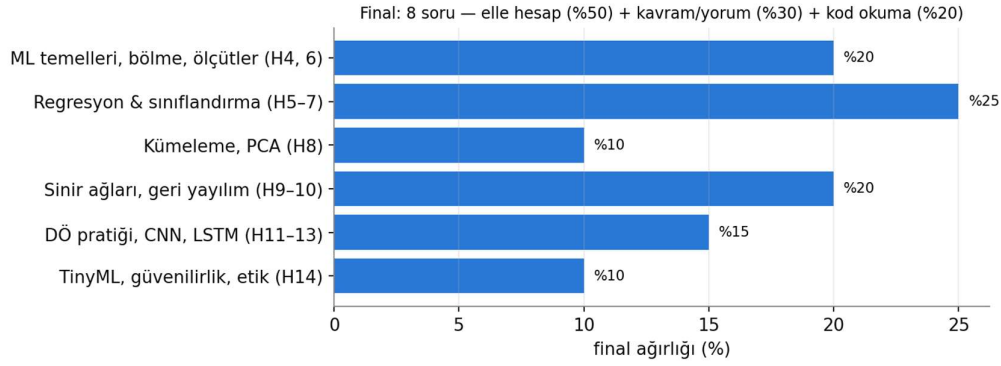
15.5 Formül ve Ölçüt Özeti

Konu	Formül	Hafta
MSE ve gradyanı	$L = (1/n)\sum(\hat{y}-y)^2$; $\partial L/\partial w = (2/n)\sum(\hat{y}-y)x$; $w \leftarrow w - \eta \nabla L$	5
Normal denklem / ridge	$w = (X^T X)^{-1} X^T y$; ridge: $(X^T X + \lambda I)^{-1} X^T y$	5
Sigmoid, log-loss	$\sigma(z) = 1/(1+e^{-z})$; $L = -[y \log p + (1-y) \log(1-p)]$; $\partial L/\partial z = p - y$	6
Precision / recall / F1	$P = TP/(TP+FP)$; $R = TP/(TP+FN)$; $F1 = 2PR/(P+R)$; makro-F1 = sınıf ortalaması	6, 7
Gini	$G = 1 - \sum p_k^2$; bölme kazancı = $G_{\text{ebeveyn}} - \text{ağırlıklı } G_{\text{çocuk}}$	7
k-means amaç	$\sum_i \ x_i - \mu_c(i)\ ^2$; silhouette $s = (b-a)/\max(a,b)$	8
PCA	kovaryans özvektörleri; açıklanan varyans = $\lambda_k/\sum \lambda$	8
Geri yayılım	$\delta_L = \partial L/\partial z_L$; $\delta_l = (W_{l+1}^T \delta_{l+1}) \odot f'(z_l)$; $\partial L/\partial W_l = \delta_l a_{l-1}^T$	10
Softmax + CE	$p_k = e^{z_k}/\sum e^{z_j}$; $L = -\log p_y$; $\partial L/\partial z_k = p_k - 1[k=y]$	11
Konvolüsyon çıktı boyutu	$\lfloor (n + 2p - k)/s \rfloor + 1$; parametre = $k \cdot k \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$	12
LSTM parametre	$4 \cdot (H \cdot (F+H) + H)$ (PyTorch: +H sapma)	13
MAPE / beceri	$100 \cdot \text{ort} y - \hat{y} / y $; beceri = $1 - \text{MAE}_{\text{model}}/\text{MAE}_{\text{naif}}$	13
Kuantizasyon	$s = (\max - \min)/255$; $z = \text{round}(-\min/s)$; $q = \text{round}(w/s) + z$	14
Dikkat	$\text{softmax}(QK^T/\sqrt{d}) \cdot V$	14
ECE	$\sum (n_k/N) \cdot sıklık_k - tahmin_k $	14

Tablo 15.4 Final için formül özeti

15.6 Final Sınavı: Format ve Örnek Sorular

Final 120 dakika, 8 soru, kapalı kitap; bir A4 formül kâğıdı (elle yazılmış, iki yüz) serbesttir. Soruların yaklaşık yarısı elle hesap (dersteki örneklerin birebir türü: gradyan adımı, karışıklık matrisinden ölçüt, Gini, geri yayılım, konvolüsyon boyutu, kuantizasyon, dikkat, ECE), %30'u kavram ve yorum (bir senaryo verilir: bölme nasıl olmalı, hangi ölçüt, hangi tuzak), %20'si kod okuma (kısa scikit-learn/PyTorch parçası: ne yapar, hata nerede — örneğin test verisiyle fit edilen StandardScaler, softmax'tan sonra CrossEntropyLoss, shift'siz rolling ortalama). Şekil 15.5 konu ağırlıklarını gösterir. Aşağıdaki örnek sorular final zorluğundadır; çözümleri defterde ve bu notların sonundadır.



Şekil 15.5 Final konu ağırlıkları ve soru türleri

Örnek Final Sorusu 1 — Gradyan adımı ve ölçek (H5)

Soru: Tek özellikli doğrusal model $\hat{y} = wx + b$, veri $(x, y) = (2, 5), (4, 9)$, $w = 1$, $b = 0$, $\eta = 0.05$. Bir gradyan iniş adımından sonra w ve b nedir? x 'ler 100 kat büyük olsaydı aynı η ile ne olurdu?

$\hat{y} = [2, 4]$; hata $\hat{y} - y = [-3, -5]$. $\partial L / \partial w = (2/n) \sum (\hat{y} - y)x = (2/2)(-3 \cdot 2 + -5 \cdot 4) = -26$; $\partial L / \partial b = (2/2)(-3 - 5) = -8$.
 $w \leftarrow 1 + 0.05 \cdot 26 = 2.3$; $b \leftarrow 0 + 0.05 \cdot 8 = 0.4$. x 100 kat büyükse gradyan $\approx 10^4$ kat büyür (x^2 ile), aynı η ile w sıçrar ve ıraksar — ölçekleme (StandardScaler) ya da η 'yı 10^4 kat küçültme gerekir; bu, 5. haftada 'ölçekleme neden şart' sorusunun cevabıdır.

Örnek Final Sorusu 2 — Karışıklık matrisi ve maliyetli eşik (H6)

Soru: 1000 motor, 150 arızalı. Eşik 0.5'te TP = 90, FP = 40. Eşik 0.3'te TP = 130, FP = 150. Kaçırılan arıza 5000 ₺, yanlış alarm 300 ₺. Hangi eşik? Precision, recall ve toplam maliyeti hesaplayın.

Eşik 0.5: FN = 60; $P = 90/130 = 0.692$; $R = 90/150 = 0.600$; maliyet = $60 \cdot 5000 + 40 \cdot 300 = 312\ 000$ ₺. Eşik 0.3: FN = 20; $P = 130/280 = 0.464$; $R = 0.867$; maliyet = $20 \cdot 5000 + 150 \cdot 300 = 145\ 000$ ₺. Düşük eşik precision'ı düşürse de toplam maliyeti yarıya indirir $\rightarrow$ 0.3. Genel kural: eşik = maliyet oranına göre; $p^* \approx C_{FP} / (C_{FP} + C_{FN}) = 300/5300 \approx 0.057$ bile denenebilir (kalibre olasılık şartıyla — H14).

Örnek Final Sorusu 3 — Geri yayılım (H10)

Soru: Ağ: $x = 1 \rightarrow z_1 = w_1x = 0.5 \cdot 1$, $a_1 = \text{ReLU}(z_1) \rightarrow z_2 = w_2a_1 = (-2) \cdot a_1$, $\hat{y} = z_2$; $L = (\hat{y} - y)^2$ ile $y = 1$. $\partial L / \partial w_2$ ve $\partial L / \partial w_1$ 'i hesaplayın; $w_1 = -0.5$ olsaydı?

İleri: $z_1 = 0.5$, $a_1 = 0.5$, $\hat{y} = -1$, $L = 4$. $\delta_2 = \partial L / \partial z_2 = 2(\hat{y} - y) = -4$. $\partial L / \partial w_2 = \delta_2 \cdot a_1 = -2$. $\delta_1 = w_2 \cdot \delta_2 \cdot \text{ReLU}'(z_1) = (-2)(-4)(1) = 8$; $\partial L / \partial w_1 = \delta_1 \cdot x = 8$. $w_1 = -0.5$ ise $z_1 = -0.5$, $\text{ReLU}' = 0 \rightarrow \delta_1 = 0$, $\partial L / \partial w_1 = 0$: 'ölü ReLU' — nöron öğrenemez; Leaky ReLU ya da He başlangıcı bu yüzden.

Örnek Final Sorusu 4 — CNN boyutu ve parametre (H12)

Soru: $1 \times 64 \times 64$ girdi $\rightarrow \text{Conv}(16, k=5, p=2, s=1) \rightarrow \text{MaxPool } 2 \rightarrow \text{Conv}(32, k=3, p=1, s=2) \rightarrow$ global ortalama havuz $\rightarrow \text{Linear}(32 \rightarrow 4)$. Her katmanın çıktı boyutunu ve toplam parametre sayısını bulun.

Conv1: $(64 + 4 - 5)/1 + 1 = 64 \rightarrow 16 \times 64 \times 64$; parametre $5 \cdot 5 \cdot 1 \cdot 16 + 16 = 416$. Pool $\rightarrow 16 \times 32 \times 32$. Conv2: $[(32 + 2 - 3)/2] + 1 = 16 \rightarrow 32 \times 16 \times 16$; parametre $3 \cdot 3 \cdot 16 \cdot 32 + 32 = 4640$. GAP $\rightarrow 32$. Linear: $32 \cdot 4 + 4 =$

132. Toplam 5188 — int8'de 5.1 KB, 256 KB flash'lı MCU'ya rahat sığar; MAC $\approx 64 \cdot 64 \cdot 25 \cdot 16 + 16 \cdot 16 \cdot 9 \cdot 16 \cdot 32 \approx 1.64 \text{ M} + 1.18 \text{ M} \approx 2.8 \text{ M} \rightarrow 100 \text{ MHz'de } \sim 28 \text{ ms}$.

Örnek Final Sorusu 5 — Zaman serisi senaryosu (H13)

Soru: Bir öğrenci saatlik PV üretimini tahmin ediyor: veriyi rastgele %80/20 böldü, tüm seride StandardScaler fit etti, özellik olarak 'aynı saatin son 24 saatlik ortalaması'nı rolling(24).mean() ile hesapladı, MAPE %2 elde etti. Üç hatayı bulun ve düzeltin; MAPE neden uygun değil?

(1) Rastgele bölme komşu saatleri iki tarafa dağıtır $\rightarrow$ zaman bölmesi. (2) Ölçekleyici tüm seriyle $\rightarrow$ yalnızca eğitimle fit. (3) rolling(24) pencereye şimdiki değeri (hedefi) dahil eder — center=True ile geleceği bile $\rightarrow$ shift(h).rolling(24). Sonuç iyimser; gerçek performans muhtemelen çok daha kötü. PV geceleri sıfır $\rightarrow$ MAPE paydası sıfıra gider, yüzde patlar; MAE ya da kurulu güce göre nMAE kullanılmalı; gündüz saatleri ayrı raporlanmalı.

Örnek Final Sorusu 6 — Kod okuma (H4, 11)

Soru: Aşağıdaki parçadaki iki hatayı bulun.

```
sc = StandardScaler().fit(X) # X: tüm veri
Xtr, Xte, ytr, yte = train_test_split(sc.transform(X), y, test_size=0.2)
model = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 3), nn.Softmax(dim=1))
loss = nn.CrossEntropyLoss()(model(torch.tensor(Xtr)), torch.tensor(ytr))
```

(1) Ölçekleyici test verisini görmüş (sızıntı) $\rightarrow$ önce böl, sonra yalnızca Xtr ile fit. (2) CrossEntropyLoss softmax'ı içerir; modelin sonunda Softmax olunca 'çift softmax' — kayıp yumuşar, gradyan küçülür, eğitim yavaşlar. Son katman ham logit vermeli. (Bonus: dengesiz sınıf varsa stratify=y.)

15.7 Dersten Sonra: Yol Haritası

Bu ders bir temel attı; devamı için üç yön öneririz. (1) Pratik: Kaggle ya da açık veri (EPİAŞ şeffaflık, NASA rulman, UCI) üzerinde ders hattını bağımsız uygulamak — her projede taban, bölme, ölçüt. (2) Derinleşme: derin öğrenme için 'Dive into Deep Learning' (d2l.ai, ücretsiz), klasik ML için 'An Introduction to Statistical Learning' (ücretsiz PDF); PyTorch resmî öğreticileri. (3) EEM'e özel: kestirimci bakım (ISO 13373 titreşim analizi + ML), güç sistemlerinde ML (IEEE PES öğreticileri), TinyML (tinymml.org, Edge Impulse). Ve en önemlisi: kendi laboratuvar/sensör verinizle küçük bir uçtan uca proje — veri toplamaktan MCU'ya kadar.

15.8 Ders Özeti — Tek Sayfa

- Yapay zekâ = veriden karar; makine öğrenmesi = hatayı azaltarak parametre öğrenme; derin öğrenme = katmanlı temsil öğrenme.
- Hat: problem $\rightarrow$ veri $\rightarrow$ bölme $\rightarrow$ taban $\rightarrow$ model $\rightarrow$ doğrulama $\rightarrow$ dağıtım $\rightarrow$ izleme. Sızıntı ve aşırı öğrenme her adımda pusuda.
- Klasik modeller çoğu EEM problemini çözer; derin modeller ham görüntü/sinyal/diziye ve çok veriye değer katar.

- Ölçütü problem seçer: dengesiz sınıfta makro-F1/maliyetli eşik; sıfıra yakın hedefte MAE; tahminde tabana beceri.
- Model bir olasılık üretir: kalibrasyon, belirsizlik, kayma ve alt grup raporu olmadan güvenilir değildir.
- Mühendisin sorumluluğu modelle bitmez: emniyet katmanı, gizlilik, düzenleme, dokümantasyon (model kartı).

Not: Final formül kâğıdı: A4, iki yüz, elle yazılmış. Tablo 15.4'teki formüller + kendi notlarınız. Hesap makinesi serbest; bilgisayar yok. Örnek soruların defterdeki Python çözümleri, elle çözümünüzü kontrol etmek içindir.